

Programs Offered

Post Graduate Programmes (PG)

- Master of Business Administration
- Master of Computer Applications
- Master of Commerce (Financial Management / Financial Technology)
- Master of Arts (Journalism and Mass Communication)
- Master of Arts (Economics)
- Master of Arts (Public Policy and Governance)
- Master of Social Work
- Master of Arts (English)
- Master of Science (Information Technology) (ODL)
- Master of Science (Environmental Science) (ODL)

Diploma Programmes

- Post Graduate Diploma (Management)
- Post Graduate Diploma (Logistics)
- Post Graduate Diploma (Machine Learning and Artificial Intelligence)
- Post Graduate Diploma (Data Science)

Undergraduate Programmes (UG)

- Bachelor of Business Administration
- Bachelor of Computer Applications
- Bachelor of Commerce
- Bachelor of Arts (Journalism and Mass Communication)
- Bachelor of Arts (General / Political Science / Economics / English / Sociology)
- Bachelor of Social Work
- Bachelor of Science (Information Technology) (ODL)

Software Testing and Reliability

Software Testing and Reliability

Amity University Online



AMITY UNIVERSITY | DIRECTORATE OF
DISTANCE & ONLINE EDUCATION

Amity Helpline: 1800-102-3434 (Toll-free), 0120-4614200

For Distance Learning Programmes: dladmissions@amity.edu | www.amity.edu/addoe

For Online Learning programmes: elearning@amity.edu | www.amityonline.com

Product code



81030112000748

AMITY



AMITY UNIVERSITY | DIRECTORATE OF
DISTANCE & ONLINE EDUCATION

Software Testing and Reliability

(c) Amity University Online



AMITY | DIRECTORATE OF DISTANCE &
UNIVERSITY | ONLINE EDUCATION

© Amity University Press

All Rights Reserved

No parts of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise without the prior permission of the publisher.

SLM & Learning Resources Committee

Chairman : Prof. Abhinash Kumar

Members : Dr. Divya Bansal
Dr. Coral J Barboza
Dr. Monica Rose
Dr. Winnie Sharma

Member Secretary : Ms. Rita Naskar

(c) Amity University Online

Contents

Page No.

Module - 1: Fundamentals of Testing

01

- 1.1.1 Human and Errors, Defects, Faults, Failures
- 1.1.2 Defect Rate and Reliability
- 1.1.3 Defect Prevention, Reduction and Containment
- 1.1.4 Testing
- 1.2.1 Debugging
- 1.2.2 Software Quality, Requirement Behaviour and Correctness
- 1.2.3 Fundamentals of Test Process
- 1.2.4 Psychology of Testing
- 1.3.1 General Principles of Testing
- 1.3.2 Test Metrics
- 1.3.3 Agile Methodology
- 1.3.4 Impact on Testing
- 1.4.1 Test Levels
- 1.4.2 Unit Testing
- 1.4.3 Component, Module
- 1.4.4 Integration Testing - Big Bang and Bottom up Integration
- 1.5.1 Top Down Integration and Mixed Integration
- 1.5.2 System Testing
- 1.5.3 Acceptance Testing
- 1.5.4 Generic Testing

Module - 2: Approaches to Testing

64

- 2.1.1 Static Testing, Structured Group Examinations
- 2.1.2 Static Analysis, Control flow & Data flow
- 2.1.3 Determining Metrics, Dynamic Testing
- 2.1.4 Black Box Testing:
- 2.2.1 Equivalence Class Partitioning
- 2.2.2 Examples on Equivalence Class Partitioning
- 2.2.3 Boundary Value Analysis
- 2.2.4 Examples on Boundary Value Analysis
- 2.3.1 State Transition Test, Cause Effect Graphing
- 2.3.2 Decision Table Technique and Used Case Testing
- 2.3.3 White Box Testing
- 2.3.4 Statement Coverage
- 2.4.1 Branch Coverage
- 2.4.2 Decision Coverage
- 2.4.3 Examples on Decision Coverage
- 2.4.4 MCC Coverage

- 2.5.1 MC/DC Coverage
- 2.5.2 System integration, Deployment testing, Beta Testing
- 2.5.3 Scalability testing, Reliability , Stress Testing
- 2.5.4 Acceptance testing: Acceptance criteria, test case selection and execution

Module - 3: Test Management

168

- 3.1.1 Test Organization
- 3.1.2 Test teams
- 3.1.3 Tasks and Qualifications
- 3.1.4 Test Planning
- 3.2.1 Quality Assurance Plan
- 3.2.2 Test Plan
- 3.2.3 Prioritization Plan
- 3.2.4 Test Exit Criteria, Cost and economy Aspects
- 3.3.1 Test Strategies
- 3.3.2 Preventive versus Reactive Approach
- 3.3.3 Analytical versus heuristic Approach
- 3.3.4 Test Activity Management
- 3.4.1 Incident Management
- 3.4.2 Configuration Management - Need and Identification
- 3.4.3 Configuration Control
- 3.4.4 Test Progress Monitoring and Control
- 3.5.1 Performance Testing
- 3.5.2 Load Testing
- 3.5.3 Stress testing
- 3.5.4 Security Testing

Module - 4: Software Reliability, Reliability Growth Models

240

- 4.1.1 Defining Software Reliability
- 4.1.2 Software Reliability Attributes and Specification
- 4.1.3 Concept of Introduction to Measurement and Inspection Process
- 4.1.4 Basics of Reliability Theory
- 4.2.1 Software Reliability Problem
- 4.2.2 Modeling Process
- 4.2.3 Software Reliability Models (SRGM)
- 4.2.4 preliminary Concepts of Reliability Engineering, Parameter Estimation
- 4.3.1 Model Validation.
- 4.3.2 Execution Time Models
- 4.3.3 Calendar Time Models
- 4.3.4 Erlang Model
- 4.4.1 Modeling Fault Dependency and Debugging Time Lag
- 4.4.2 Testing Effort Dependent Modeling
- 4.4.3 Distributed Environment, Imperfect Debugging

- 4.4.4 Testing-Domain Models
- 4.5.1 Change-Point
- 4.5.2 Unified Approach Concepts
- 4.5.3 Artificial Neural Networks based SRGM
- 4.5.4 Introduction to Discrete SRGM

Module - 5: Software Reliability Allocation Models, Software Verification, Validation & Testing 302

- 5.1.1 Software Reliability Allocation Models
- 5.1.2 Criteria for Model Evaluation
- 5.1.3 Optimal Reliability Allocation
- 5.1.4 Quality Planning
- 5.2.1 Quality Control
- 5.2.2 Quality Improvement Process
- 5.2.3 Evolution of Software Quality Assurance
- 5.2.4 Major SQA Activities
- 5.3.1 Major SQA Issues
- 5.3.2 Zero Defect Software
- 5.3.3 Verification
- 5.3.4 Validation
- 5.4.1 Evolutionary Nature of Verification and Validation
- 5.4.2 Impracticality of Testing all Data and Paths
- 5.4.3 Proof of Correctness
- 5.4.4 Testing
- 5.5.1 Functional, Structural and Error-Oriented Analysis and Testing
- 5.5.2 Static Testing Tools
- 5.5.3 Dynamic Testing Tools
- 5.5.4 Characteristics of Modern Testing Tools

(c) Amity University Online

Module - 1: Fundamentals of Testing

Notes

Structure:

- 1.1.1 Human and Errors, Defects, Faults, Failures
- 1.1.2 Defect Rate and Reliability
- 1.1.3 Defect Prevention, Reduction and Containment
- 1.1.4 Testing
- 1.2.1 Debugging
- 1.2.2 Software Quality, Requirement Behaviour and Correctness
- 1.2.3 Fundamentals of Test Process
- 1.2.4 Psychology of Testing
- 1.3.1 General Principles of Testing
- 1.3.2 Test Metrics
- 1.3.3 Agile Methodology
- 1.3.4 Impact on Testing
- 1.4.1 Test Levels
- 1.4.2 Unit Testing
- 1.4.3 Component, Module
- 1.4.4 Integration Testing - Big Bang and Bottom up Integration
- 1.5.1 Top Down Integration and Mixed Integration
- 1.5.2 System Testing
- 1.5.3 Acceptance Testing
- 1.5.4 Generic Testing

Objectives:

After studying this chapter, you will be able to:

- Explain the meaning of Fundamentals of testing
- Discuss the Human and errors
- Explain the Defects, Faults, Failures, Defect Rate and Reliability
- Discuss the Test Levels
- Understand the Testing and Debugging

Introduction

Testing is the process of assessing a system or its component(s) with the goal of determining whether or not it meets the set requirements.

Notes

Testing is the process of running a system to find any gaps, faults, or missing requirements that aren't covered by the real requirements.

This course will teach you the fundamentals of software testing, including kinds, methodologies, levels, and other terms.

Why to Learn Software Testing?

In the IT industry, large companies have a team with responsibilities to evaluate the developed software in context of the given requirements. Moreover, developers also conduct testing which is called **Unit Testing**. In most cases, the following professionals are involved in testing a system within their respective capacities –

- Software Tester
- Software Developer
- Project Lead/Manager
- End User

People that test software are given different titles depending on their expertise and skills, such as Software Tester, Software Quality Assurance Engineer, QA Analyst, and so on.

Applications of Software Testing

- **Cost Effective Development** - In many ways, early testing saves both time and money; yet, cutting costs without testing may result in a software application with an incorrect design, leaving the product unusable.
- **Product Improvement** - Testing is never a time-consuming activity during the SDLC phases. Diagnosing and correcting mistakes discovered via proper testing, on the other hand, is a time-consuming but worthwhile exercise.
- **Test Automation** - Test automation decreases testing time, however it cannot be started at any point during the software development process. When the software has been manually tested and is somewhat reliable, test automation should begin. Furthermore, test automation cannot be employed if requirements are constantly changing.
- **Quality Check** - Software testing helps in determining following set of properties of any software such as
 - Functionality
 - Reliability
 - Usability
 - Efficiency
 - Maintainability
 - Portability

Audience

This lesson is for software testing experts who want to learn more about the Testing Framework, including its different types, methods, and levels. This course

contains all of the necessary ingredients to begin the software testing process, from which you can progress to greater degrees of proficiency.

Notes

Prerequisites

You should have a basic understanding of the software development life cycle before starting this training (SDLC). You should also have a fundamental understanding of software programming in any programming language.

1.1.1 Human and Errors, Defects, Faults, Failures

Human and Errors

Error:

The error is a human mistake. During the various stages of software development, any member of the IT team can make mistakes. The requirements may have been misunderstood or misread by the BA (business analyst). It's possible that the client gave insufficient or inaccurate information. It's possible that the architect is to blame for the software's problem. The logical faults in the programmes could have been created by the developers. Lethargy, deadline pressure, unclear or insufficient requirements, assumptions, complexity of requirements, and a variety of other factors cause team members to make mistakes.

Errors are a part of our daily life. Humans make errors in their thoughts, actions, and in the products that might result from their actions. Errors occur almost everywhere. For example, humans make errors in speech, in medical prescription, in surgery, in driving, in observation, in sports, and, certainly in love and software development. Table 1.1 provides examples of human errors. The consequences of human errors vary significantly. An error might be insignificant in that it leads to a gentle friendly smile, such as when a slip of tongue occurs. Or, an error may lead to a catastrophe, such as when an operator fails to recognize that a relief valve on the pressurizer was stuck open and this resulted in a disastrous radiation leak. To determine whether or not there are any errors in our thought, actions, and the products generated, we resort to the process of testing. The primary goal of testing is to determine if the thoughts, actions, and products are as desired, i.e. they conform to the requirements. Testing of thoughts is usually designed to determine if a concept or method has been understood satisfactorily. Testing of actions is designed to check if a skill that results in the actions has been acquired satisfactorily. Testing of a product is designed to check if the product behaves as desired. Note that both syntax and semantic errors arise during programming. Given that most modern compilers are able to detect syntactic errors, testing focuses on semantic errors, also known as faults, that cause the program under test to behave incorrectly.

EXAMPLE 1.1. An instructor administers a test to determine how well the students have understood what the instructor wanted to convey. A tennis coach administers a test to determine how well the understudy makes a serve. A software developer tests the program developed to determine if it behaves as desired. In each of these three cases there is an attempt by a tester to determine if the human thoughts, actions, and products behave as desired. Behaviour that deviates from the desirable is possibly due to an error.

Notes

Table 1.1: Examples of errors in various fields of human endeavour.

Area	Error
Hearing	Spoken: He has a garage for repairing <i>foreign</i> cars. Heard: He has a garage for repairing <i>falling</i> cars.
Medicine	Incorrect antibiotic prescribed.
Music performance	Incorrect note played.
Numerical analysis	Incorrect algorithm for matrix inversion.
Observation	Operator fails to recognize that a relief valve is stuck open.
Software	Operator used: $\neq$, correct operator: $>$. Identifier used: <code>new_line</code> , correct identifier: <code>next_line</code> . Expression used: $a \wedge (b \vee c)$, correct expression: $(a \wedge b) \vee c$. Data conversion from 64-bit floating point to 16-bit integer not protected (resulting in a software exception).
Speech	Spoken: <i>waple malnut</i> , intent: <i>maple walnut</i> . Spoken: <i>We need a new refrigerator</i> , intent: <i>We need a new washing machine</i> .
Sports	Incorrect call by the referee in a tennis match.
Writing	Written: What kind of <i>pans</i> did you use? Intent: What kind of <i>pants</i> did you use?

Defects, Faults, and Failures

Defect: A defect is a result of the error. A defect is another phrase for a defect. Defect is sometimes used interchangeably with bug. Though there is no evident distinction between the two terms. A bug is a fault in the development environment, while a defect is a fault in the test environment. A flaw happens when the behaviour of the actual software developed by the developer differs from the client's expectations.

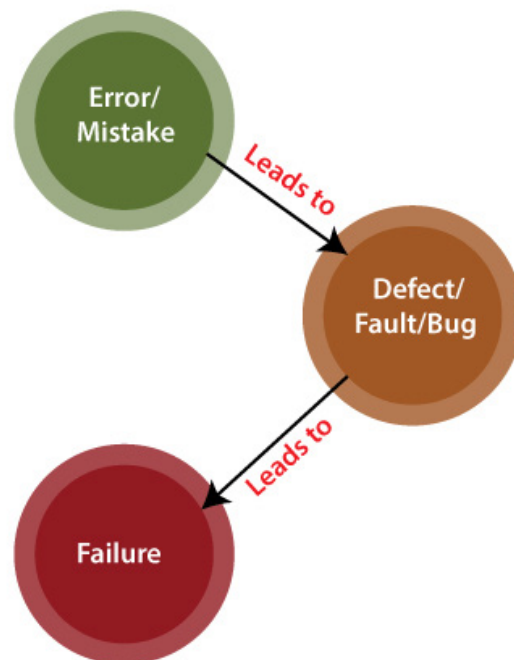
Failure: A failure is a consequence of defect. When software fails to perform in a real-world setting, it is said to be a failure. Hardware system flaws are also to blame for the incident. The failure may be caused by the environment in which the software is expected to perform. Magnetism, electronic fields, radiations, pollutions, chemical reactions, and other factors can all affect the environment.

Errors lead to defects, and defects lead to software failure. Software testing assesses the software's quality.

Bug: In software testing, a bug is the informal name of defects, which means that software or application is not working as per the requirement. When we have some coding error, it leads a program to its breakdown, which is known as a bug. The test engineers use the terminology **Bug**.

If a **QA (Quality Analyst)** detect a bug, they can reproduce the bug and record it with the help of the **bug report template**.

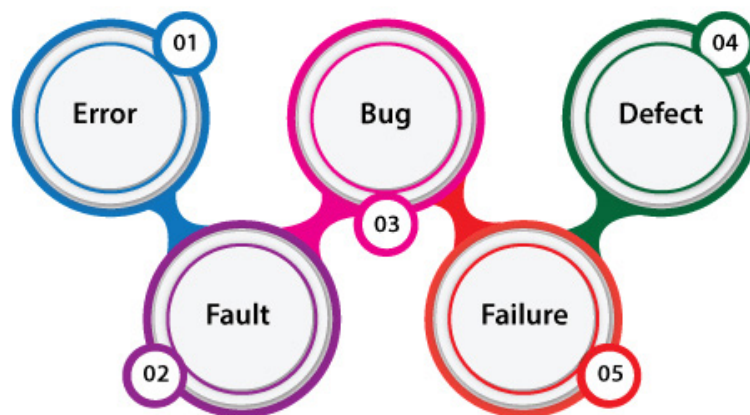
Notes



Fault: The fault may occur in software because it has not added the code for fault tolerance, making an application act up.

A fault may happen in a program because of the following reasons:

- Lack of resources
- An invalid step
- Inappropriate data definition



We have listed some of the vital differences between bug, defect, error, fault, and failure in the below table.

Notes

Comparison basis	Bug	Defect	Error	Fault	Failure
Definition	It is an informal name specified to the defect.	The Defect is the difference between the actual outcomes and expected outputs.	An Error is a mistake made in the code; that's why we cannot execute or compile code.	The Fault is a state that causes the software to fail to accomplish its essential function.	If the software has lots of defects, it leads to failure or causes failure.
Raised by	The Test Engineers submit the bug.	The Testers identify the defect. And it was also solved by the developer in the development phase or stage.	The Developers and automation test engineers raise the error.	Human mistakes cause fault.	The failure finds by the manual test engineer through the development cycle.
Different types	Different type of bugs are as follows: Logic bugs Algorithmic bugs Resource bugs	Different type of Defects are as follows: Based on priority: High Medium Low And based on the severity: Critical Major Minor Trivial	Different type of Error is as below: Syntactic Error User interface error Flow control error Error handling error Calculation error Hardware error Testing Error	Different type of Fault are as follows: Business Logic Faults Functional and Logical Faults Faulty GUI Performance Faults Security Faults Software/hardware fault	-----

Notes

Comparison basis	Bug	Defect	Error	Fault	Failure
Reasons behind	Following are reasons which may cause the bugs: Missing coding Wrong coding Extra coding	The below reason leads to the defects: Giving incorrect and wrong inputs. Dilemmas and errors in the outside behaviour and inside structure and design. An error in coding or logic affects the software and causes it to breakdown or the failure.	The reasons for having an error are as follows: Errors in the code. The Mistake of some values. If a developer is unable to compile or run a program successfully. Confusions and issues in programming. Invalid login, loop, and syntax. Inconsistency between actual and expected outcomes. Blunders in design or requirement actions. Misperception in understanding the requirements of the application.	The reasons behind the fault are as follows: A Fault may occur by an improper step in the initial stage, process, or data definition. Inconsistency or issue in the program. An irregularity or loophole in the software that leads the software to perform improperly.	Following are some of the most important reasons behind the failure: Environmental condition System usage Users Human error
Way to prevent the reasons	Following are the way to stop the bugs: Test-driven development. Offer programming language support. Adjusting, advanced, and operative development procedures. Evaluating the code systematically.	With the help of the following, we can prevent the Defects: Implementing several innovative programming methods. Use of primary and correct software development techniques. Peer review It is executing consistent code reviews to evaluate its quality and correctness.	Below are ways to prevent the Errors: Enhance the software quality with system review and programming. Detect the issues and prepare a suitable mitigation plan. Validate the fixes and verify their quality and precision.	The fault can be prevented with the help of the following: Peer review. Assess the functional necessities of the software. Execute the detailed code analysis. Verify the correctness of software design and programming.	The way to prevent failure are as follows: Confirm re-testing. Review the requirements and revisit the specifications. Implement current protective techniques. Categorize and evaluate errors and issues.

1.1.2 Defect Rate and Reliability

Reliability: While developing a software or an application, there are several characteristics and aspects of the software, authenticated by the software engineers.

Notes

To deliver a quality product to the end-users is one of the essential features of software reliability that is tested and verified by the team with the help of the assistance of reliability testing.

In this section, we are going to explore and discuss the following topics related to **reliability testing**, which help us understand the necessity of reliability testing and product consistency in software testing.

- **What is Reliability Testing?**
- **The goal of Reliability Testing**
- **Characteristics of Reliability Testing**
- **Types of Reliability Testing**
- **Approaches of reliability testing**

Before going deep into all the above mention topics, firstly, we will understand the definition of reliability testing.

What is Reliability Testing?

Another essential **software testing** technique is **reliability testing**. The team implements reliability testing to guarantee the software or an application is performing and functioning reliably in each environmental condition with a particular period.

The implementation of reliability testing makes sure that the software product is defect-free and reliable for its planned objective.

In simple words, we can say that it verifies whether the software can achieve a failure-free operation for a specific period in a technological environment or not.

In general terms, Reliability signifies **yielding the same**; in other words, **reliable** means **something is dependable**, and both will provide the same outcome each time. And the same is true in the case of **Reliability testing**.

Reliability is described as the **probability of defect-free software operation** for a particular time in a specific environment. Mainly it is implemented to guarantee that the software is consistent.

The end-users think that the data shown is correct, and the software will continuously operate appropriately. In the above situations, the need for reliability testing comes into the illustration.

The function of defining that the system has been thoroughly verified and tested is the true meaning of software reliability.

The purpose of Reliability Testing

Following are some of the important objective of performing the reliability testing:

- To test the working of software under a specific condition is the significant purpose of Reliability testing.
- The purpose of executing the reliability testing is to determine the number of defects in the specific period.
- To determine the perceptual structure of monotonous breakdown.

- To find the nasty presence of the software.
- To identify the leading cause of failure.

Characterises of Reliability Testing

Some of the significance of reliability testing which can further understand with the assistance of its various features are as discussed below:

- The reliability testing guarantee that the software meets the client's along with customer's reliability necessities.
- It ensures that the software is performing without any faults and is working as per its planned determination.
- It can be described as the ability of the software to work precisely under the detailed conditions.
- It is a significant surface of reliability engineering programs.
- The data is collected from several phases **of software development** at the time of reliability testing.
- Before the product is delivered to the end-users, reliability testing helps us to identify defects and faults in the software product.
- It is executed at several software development levels to enhance the reliability and dependability of the product.

Types of Reliability Testing

In software testing, reliability testing categorizes into three types which are discussed as below:

- **Feature Testing**
- **Regression Testing**
- **Load Testing**

Feature Testing

The first type of reliability testing is **feature testing**. The feature testing concludes the suitability, which implies that it checks if it executes as expected for its intended use.

The main objective of feature testing is to test the attribute and the functionality of the software product, the correctness of the system and to verify if there are no defects identified throughout the beta testing.

In fulfilment of this testing, we will check if the application follows specific criteria such as **standards, rules**, etc.

Some of the below characteristics are checked during the execution of feature testing:

- All function needs to be done at least once by the team.
- Each function must implement completely.
- The team should also check the proper implementation of each operation.
- The communication between two or more function have to be reduced.

Notes

Note: Apart from this, the feature testing verifies some security and compliance. As we already understand, security testing is related to the inhibition of illegal access to the application, either purposely or accidentally.

Regression Testing: Another part of reliability testing is **Regression testing**. The regression testing is mainly used to check whether any new bugs or defects have been declared because of earlier bugs' fixing.

In simple words, we can say that the Regression testing can be implemented on a new build when there is an essential change in the original functionality. The implementation of regression testing guarantees that the code still works even when the changes are happening.

Regression means **Re-test** those parts of the application, which are unchanged.

In regression testing, verification of **software quality** and the new components' accuracy added to the software product. And the identification of bugs or defects that are generated in the system after the execution of changes.

Load Testing

The next type of Reliability testing is **Load testing**. This test is concluded to confirm the functionality of the software under the circumstances of maximum workload.

Load testing is testing where we check an application's performance by applying some load, which is either less than or equal to the desired load.

In load testing, the term load implies that when **N-number** of users using the application simultaneously or sending the request to the server at a time.

This type of testing will help us to detect an application's maximum operating capacity and any **blockages** or bottlenecks.

How to evaluate the Reliability Testing

The reliability testing can be evaluated in terms of *Mean time between failures* (MTBF).

- **Mean Time Between Failures (MTBF):**

The MTBF is combines **Mean time to failure (MTTF)** and **Mean time to Repair (MTTR)**.

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

Here,

- **Mean Time To Failure (MTTF)**
- The mean time to failure (MTTF) is the time between two consecutive failures.
- **Mean Time To Repair (MTTR)**
- Mean time to repair (MTTR) is the time taken to fix the failures.

How to Perform Reliability Testing

To perform any software testing, we need proper planning and management. The execution of Reliability Testing is a bit costlier as compared to other types of software testing.

Notes

The implementation of reliability testing includes **data for the test environment, test schedule, test points, etc.**

The test engineer has to retain the following aspects in order to start the implementation of reliability testing:

- The test engineer needs to use the test results to drive decisions.
- They need to determine the reliability goals.
- They need to plan and execute tests.
- The test engineer needs to cultivate the effective profile.

The reliability testing contains some essential constraints, which are as follows:

- Measurement of time of bug-free operation.
- Possibility of bug-free operation.
- The environment in which it is performed.

The analysis of **reliability testing** can be divided into **three parts**, which are as discussed below:

- **Modelling**
- **Measurement**
- **Improvement**

For our better understanding, let us discuss them one by one in details:

Part1: Modelling

We can achieve a meaningful outcome by applying suitable models. In reliability testing, the **Software Modelling Technique** can be divided into two subcategories, which are as below:

- **Prediction Modelling**
- **Estimation Modelling**

The **prediction modelling and estimation modelling** have some significant differences, which we understand from the below table:

Issues	Prediction Models	Estimation Models
When used in Development Cycle	Generally, the prediction model is developed before the development or testing stages.	Generally, the estimation models are developed at the later stage of the Software Development Life Cycle (SDLC).
Data Reference	The prediction models use historical data.	The estimation models use existing data from software development.
Time Frame	Prediction models will predict consistency in the future.	The estimation models will predict the reliability for both the present time or in the future time.

Notes

Part2: Measurement

In software testing, we cannot directly measure the Software reliability. Therefore, some other connected aspects are considered in order to evaluate the software reliability.

The **Software Reliability Measurement** are divided into four types, which are as follows:

1. **Product Metrics**
2. **Fault and Failure Metrics**
3. **Process Metrics**
4. **Project Management Metrics**

Let see a detailed information of the software reliability measurement parts:

1. Product Metrics

The **Product metrics** is the first part of software reliability measurement, which are the combination of four different types of metrics:

- **Complexity**
- **Function point Metric**
- **Software size**
- **Test Coverage Metrics**

Complexity

- The representation of complexity is significant as it is directly related to software reliability.
- It is a process of defining a program's control structure's difficulty by simplifying the code into a graphical illustration.

Function point Metric

- Another metric that comes under the product metric is the **Function point Metric**, which is mainly used for determining the functionality of Software Development.
- The function point metric will take the count of inputs, outputs, master files, etc.
- It calculates those components which are delivered to the user and is independent of the programming language.

Software size

The next part of the product metric is **software size**. In this Metric, we can use the Line of Code (LOC) for measuring the size of the software as it is an in-built initial approach.

Test Coverage Metrics

- The test coverage metrics approximate the fault and reliability by implementing the full test of software products.

2. Fault and Failure Metrics

In fault and failure metrics, we used to check whether the system is entirely bug-free or not. Both the types of faults were identified at the time of the testing process, which means before delivering the product.

And to accomplish the objective, we need to check and summarized the failure reported by users after delivery.

3. Process Metrics

The process metrics are used to evaluate, display, and enhance the software's reliability and quality. As we already knew that the quality of the product is directly related to the process.

4. Project Management Metrics

The last part of software reliability measurement is **project management metrics**. We know that good management can accomplish advanced reliability by using a better risk management process, development process, configuration management process, etc.

Part 3: Improvement

The next part of software reliability testing is **Improvement**. The Improvement is entirely depending upon the difficulties that happened in the application or system or the software features.

The method of Improvement will also vary as per the complexity of the software module. The **time and budget** are the two main constraints that will limit the attempts are put into the software reliability improvement.

As we know that, so far, there is no good approach to overcome the complexity problem of software.

Approaches of Reliability Testing

In software testing, we will execute the reliability testing to make sure that all the bugs and faults present in the system are detected and fixed by the specified team before deployed to the user site.

Sometimes it is not easy to evaluate the precise reliability. But it can be assessed with the help of several methods. Some of the most commonly used approaches of reliability testing are as discussed below:

- **Test-Retest Reliability**
- **Parallel Forms Reliability**
- **Decision Consistency**

Test-Retest Reliability

- The first approach of reliability testing is **Test-retest reliability**. This type of reliability method explains how a test can give stable, reliable scores within time.

Notes

- In order to test the constancy and the reliability of the score, it is performed in a short period. In simple words, we can say that the team tests and **retests** the software over several testing techniques to confirm its reliability and constancy.

Parallel Forms Reliability

- The second method of software reliability testing is **parallel forms reliability**.
- In this particular approach, the software is tested by two distinct groups to test how equally the two function forms.
- By using this method, the team can validate the reliability of the outcome through several forms.

Decision Consistency

- The last approach of reliability testing is **decision consistency**.
- In this method, the output from the first two approaches (test-retest reliability and parallel forms reliability) are tested during the decision consistency. And concludes their success and failure.

1.1.3 Defect Prevention, Reduction and Containment

Defect Prevention

Quality Assurance is the term that is commonly used to address the testing teams in IT projects.

Technicalities aside, Quality assurance activities are not just targeted at defect identification (which is finding defects after they have happened. This simply is testing or Quality control) but also include defect prevention (making sure the defects do not happen in the first place or the defects are removed/reduced before making their way into the software product).

QA= QC (defect identification) + Defect prevention

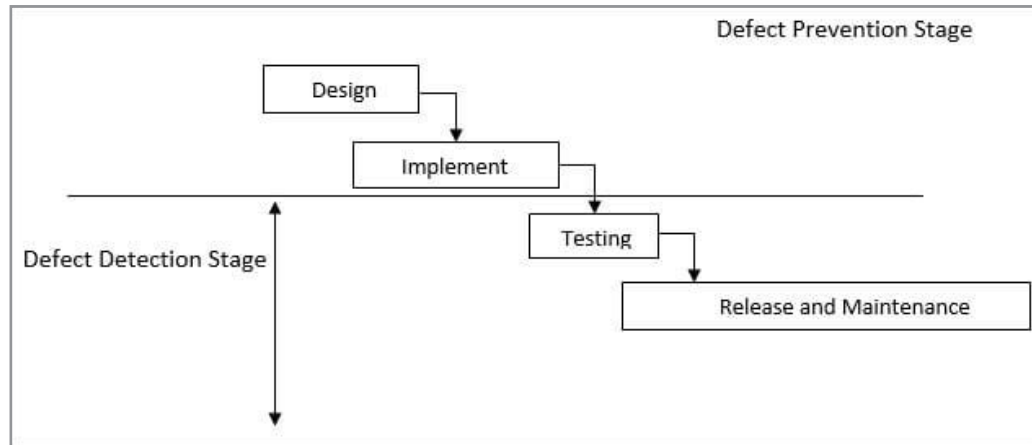
Although this sounds fairly simple, there is less emphasis or direction available on how or what exactly are defect prevention tasks.

The truth of the matter is, defects found during the testing phase or worse after release are costlier to find & fix and might cause a loss of trust on the brand. Hence, the earlier the prevention measures are taken, the better. Besides, defect prevention also helps companies achieve the highest CMMI (Capability Maturity Model Integration) Level.

Defect Prevention

Defect Prevention is a crucial step or activity in any software development process and as can be seen from the below diagram is pretty much half of our testing tasks:

Notes



In brief, the following are the defect prevention responsibilities for testers in each of the below stages:

#1) Requirement Specification Review:

After understanding customer's requirements prepare your requirement's gist.

A review is important at this step- the First level of review should be within the team, followed by another level of external review (by a dev or BA or client) to make sure that all the perspectives are in sync.

#2) Design Review:

Design stage can be considered a strategy stage of sorts and going through it will ensure that the QA team understands the pros and cons of each strategy.

This kind of critical walkthrough will help unearth any problems with the said strategies and fix them before going further. This can be considered a feasibility study for the strategy (or strategies).

#3) Code Review:

There is not a lot for testers to directly get involved in this phase, but the review does go on here too. Developers carry out code inspections, walkthroughs and reviews before they unit and integration test the application.

Defect Prevention Methods and Techniques

Some traditional and common methods that have been in use since a long time for defect prevention are listed below;

#1) Review and Inspection: This method includes the review by an individual team member (self-checking), peer reviews and inspection of all work products.

=> For more information on how this is carried out, please check our [Test Documentation Reviews](#) article.

#2) Walkthrough: This is more or less like a review but it's mostly related to comparing the system to the prototype which will give a better idea regarding the correctness and/or the look-and-feel of the system.

Notes

#3) Defect Logging and Documentation: This method provides some key information, arguments/parameters that can be used to support analysing defects.

#4) Root Cause Analysis: Root cause analysis includes two major approaches:

I) Pareto Analysis:

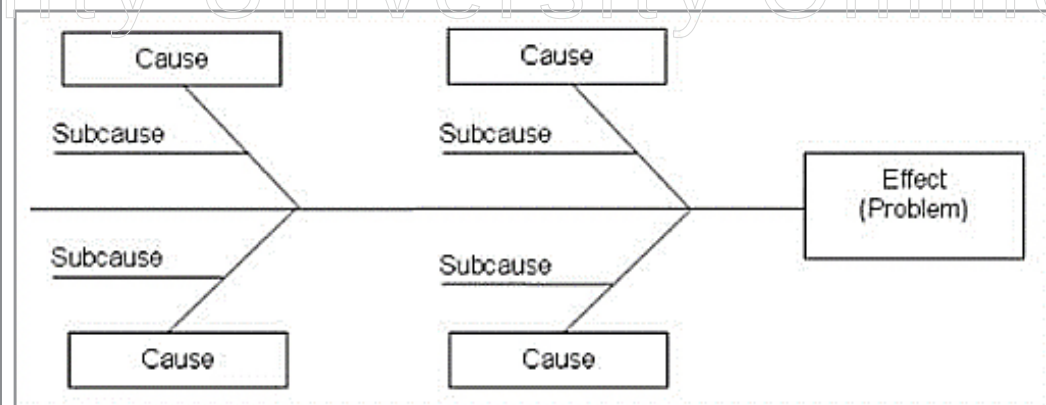
Pareto analysis is a formal and simple technique which helps prioritize the order of problem resolution for maximum impact. It states that 80% of the problem arises due to 20% reasons.

Therefore, the problems once identified are prioritized according to frequency and a detailed statistics based analysis is performed as to find which 20% of the reasons attributed to the 80% problems. By simply focusing on those 20% reasons and eliminating those, results are guaranteed while optimizing the extent of work involved.

II) Fishbone Analysis:

Also known as **Ishikawa Analysis** this method is a more visual root cause analysis technique. There are no statistics involved as this method is based on team-wide brainstorming. The following diagram helps understand this better.

The problem is first written on the rightmost side and on the horizontal line that passes through it, the various causes are listed. The branch that has the most cause-subclause bones (or lines/branches) is the problem that is most serious and that is to be worked towards elimination. This technique is also sometimes called cause and effect analysis.



TMM Level and Defect Handling by Test Organization

#1) TMM (Testing Maturity Model) is based on CMM i.e.; Capability Maturity Model.

#2) Defect Prevention involves many staff members and their collaborative effort at various stages which is the reason why it plays a prominent role in TMM level 5. e.g.; If a defect occurs frequently in any test case or procedure, the organization might allocate a group of staff members to analyze the defect and develop the plan containing actions for changes in the process with the problem.

#3) Some of the benefits of the defect prevention program are:

- The staff gets motivated and are more aware

Notes

- Customers satisfaction
- Increased reliability, manageability and predictability
- Enhanced continuous process improvement

Team roles and responsibilities

Three critical groups are involved in the process of defect prevention:

Manager's Role:

- For the success of any defect prevention program management must be strongly supportive.
- The support can be in the form of resources, training, and tools which are required to implement the plan successfully.
- Management should define the appropriate policy and should make some cultural changes if necessary.
- Managers are supposed to promote discussions, distribution of common defect list and changes in the process.

Tester's Role:

- Testers maintain the defect database which includes the collection of defect data.
- Defect data must be updated at regular intervals and defect info should be kept current at all time.
- To plan the implementation of change

Client's Role:

- The client plays a comparatively small or limited role but their commitment towards quality is critical.

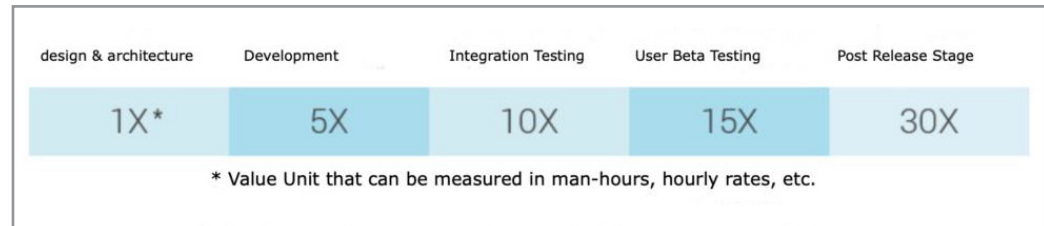
Reduction and Containment

1.4.1 Reduction 8 main step

1. Start testing your product as early as possible

One of the most common mistakes is to start testing the product at a later stage when it is almost ready for release. The earlier your QA team syncs up with the development team, the lower the chance of missing an error in production. In addition, an error identified at an early stage of development will always cost cheaper than one found after the final release. Way cheaper. Our experience shows that **the cost of bug-fixing at the later stages can be 30 times more expensive than bug-fixing at the prototype stage.**

Notes



A real-life 8allocate's client case

One of our client Dedicated QA Teams had to test a server application that collected and processed a large array of user data. The client's in-house developers, however, refused to submit it for testing to our hosted QA team, saying the app has a lot of known issues and suggesting that testing could be done at the final stage. The Client who was fully accountable for project management and delivery agreed with developers and let them build the product without interim testing. As a result, they spent 4 months working on the new app's version without having tested the previous version.

When the outsourced QA team started testing the application with real user data, it turned out that the selected database architecture could not cope with such data loads. A development team of five people had to re-plan everything, rewrite the design and architecture of the application, and the QA team had to test it from scratch. As a result, the app was released with a 3-month delay, which cost the client twice as much as they had planned initially.

The bottom line: the urge to save 150 man-hours on software testing and push it to the release stage led to the loss of 1,100 man-hours and doubled the cost of testing for the Client.

2. Use API virtualization

API virtualization mimics system behaviour and implies that no real software development environment is required to simulate realistic behaviour of application. By using API virtualization tools such as *Smartbear*, *Wiremock*, *Microfocus Data Simulation*, and so on, QA engineers, front-end developers and system integrators can start doing their part of work as soon as developers produce an initial specification (Swagger, RAML, WSDL, etc.) early on at the software design stage.

Using API virtualization can help QA managers get ahead of the game by creating functional and performance tests against virtual APIs, and, thus, reduce the overall cost of using 3rd party APIs. Testing against the real 3rd party APIs can involve fees beyond the established budget. When you virtualize external APIs, you don't have to pay for anything and, as a result, you won't likely get an unexpected explosion in cost at the end of the billing cycle with your service provider.

3. Never save on analytics

Let's assume you've listened to my advice and decided to implement the first method. **At what stage of your product development is it most reasonable to lash out on testing and hire a QA team?** The answer is – at the application architecture planning stage, or at least at the stage of discovery and collection of business and technical requirements.

Notes

Without proper analytics, you risk launching a product that nobody needs. This happens when you do not consider the peculiarities of each of your target audiences.

Do you remember a “horror” case about a running app that was tailored specifically to Asian users? However, none of the users was ever able to install the app on their smartphone! Why? Because of an error made at the stage of requirements gathering – the discovery and all feasibility hypotheses were made using the iPhone and Samsung phones which aren’t as popular in Asia as Huawei and OPPO.

The bottom line: no matter if you involve an external QA service provider or choose to test your software product internally, user testing and finding bugs at the support stage costs ten times more. And your reputation risks will always outweigh financial losses in this situation.

4. Define testing goals and expectations

Do you know how to tell a good test manager from a bad one? The good ones immediately start bombarding you with questions in order to understand exactly what product quality you’re looking to receive. We call it the collection of expectations. The quality of your software depends on certain building blocks. An experienced test manager will want to know what kind of building blocks to put in the foundation of your product’s quality.

When the client comes to allocate and says: “I want it to work perfectly well,” we begin by asking what they mean by “perfectly well”. Once we understand it, we know how to make it work to client expectations. So, asking the client/product owner as many product related questions as possible can help you get a better picture and visualize a clear roadmap of what needs to be tested, when, and how.

Having collected client expectations, we set up SMART goals, decompose them, define tasks and KPIs based on them. As a result, we clearly define:

- Types of testing;
- Testing milestones and due dates;
- Line-up;
- Possible risks.

All of this is needed to deliver the expected level of product quality and eliminate any guesswork.

Sometimes, however, incidents occur. We once worked with the client who needed a robust InsureTech solution. As usual, we held a comprehensive discovery session, conducted a thorough analysis and made a proposal. Due to the complexity of functional requirements and large size of modules planned, we suggested that regression testing be fully automated, as regression testing was the most laborious and time-consuming QA activity. Despite our recommendations, the client chose not to automate regression testing, because, as they believed they had a strong internal team of manual testers and wanted to make use of them. Unfortunately, all our further arguments and estimations were ignored, too. We could not get the client’s buy-in and for three months we’d conducted regression tests manually, which, in fact, proved highly unprofitable for the customer.

Notes

At the end of the day, the client put the whole project on hold, spent 2 months analysing it, and returned to us with a proposal to use the regression test automation plan that we'd presented 5 months ago.

The bottom line: besides understanding the project mission, goals and the client's vision, you need to be able to argumentatively prove your point of view, especially if you're 100% sure you're right. The client's failure to follow your suggestions can result in financial losses down the road, so if you can't get their buy-in from the very beginning, it sometimes makes sense to reject such a client.

5. Automate testing wisely

To be able to save money with automation, the product must be stable. Even if it changes dynamically, it doesn't mean you don't need automation. The use of tools and special apps is already sort of automation. For example, you can help automate routine operations that eat up most of your testers' time. Say, create a utility tool that collects the necessary configurations.

A real-life 8allocate's client case

One of our clients wanted to release code every day instead of twice a week. At the same time, the full manual regression testing took two to three days to complete. And since 80% of the product's functionality was pretty stable, it was decided to automate it. For this purpose, we developed four automation scenarios and conducted a comparative analysis of their effectiveness. We used 8 indicators for this task:

1. The number of test cases per scenario;
2. Automation (man-days) – the number of resources per one automated scenario (excluding test data for each test case);
3. Single Test Case automation (man-hrs) – the cost of automating a single test case;
4. Manual testing (man-days) – the cost of manual testing of the scenario as a whole;
5. Results investigation (man-hrs) – the cost of researching the results of the AutoTest run;
6. Execution times – the number of test runs required for the whole project development. This number reflects the expected number of runs, taking into account the stability of the functionality, the required schedule of test runs (regularity of obtaining information), etc.;
7. Automation efficacy (%) – the effectiveness of test automation as a percentage of time saved. Given the computation errors, test automation can be considered effective if its efficacy exceeds 150%;
8. Saved time (man-days) – the number of man-days saved during the entire project.

Two automation scenarios passed the final selection. Their use allowed us to reduce the regression from two days to a couple of hours for running autotests, and save two hours for the remaining manual checks, which were unprofitable to automate.

The bottom line: test automation can either reduce QA labour costs tenfold, or it can drain the budget completely. Without the ability to analyse and calculate ROI, you always risk giving up on automation.

6. Use junior resources where possible

Still struggling to automate your software testing? Or can't afford to hire professional QA engineers due to their high cost on the local jobs market? Hiring junior resources for a small yet reasonable salary can be a good idea! Yet, there're nuances you need to understand: a child can find a bug in his favorite online game and break the app with monkey testing, but will he still be able to find all of the bugs? That's pretty unlikely. While newbies can be useful for some manual test tasks, you need to have a real QA master on your team to give tasks in a language rookies will understand and mentor them as the project goes.

At **8allocate**, we run one of Ukraine's leading Coding Academies that has just been recognized as Ukraine's Top 10 extracurricular school. It graduates 4,000-6,000 IT specialists every year. This allows us to source the best junior talent to assist your senior staff on projects (with non business critical tasks, of course). As the practice shows, it takes us up to a month to onboard a rookie and "plug" them into your project as testers. They normally cost very little compared to mid-level resources and can be nurtured internally with your senior's help.

The bottom line: using newbies labour for QA proves highly cost-effective. The junior talent is able to bring real benefits to your software project starting from the week two, and help reduce the team adaptation time by 4 times (from 160 to 40 hours, on average).

An experienced mentor will help you grow and develop a stellar QA team within a very short timeframe, which will help you save costs on using several mid-level and senior talent on your project.

7. Always choose quality over quantity

You do not want to pay through the nose for hiring many experienced QA guys who command high salaries but can quickly get tired and bored at working on a mainstream testing project. On the other hand, you don't want your QA team to be composed of 50 low-cost and inexperienced juniors.

While your gut feeling is important, still try to use good BAs to help you assess the amount of QA tasks, the number of people needed to get the job done, and identify their skills and qualifications. Be pragmatic and use less expensive offshore talent to increase bottom line savings.

A real-life 8allocate's client case

Not so long ago, we were approached by a client who had 12 testers in their in-house team and wanted to extend it offshore to increase the bandwidth and do more work for the same budget. They had the following roles in their team: a Test Manager, one senior, one middle and 9 junior testers. As a first step, we conducted an audit of the client's internal testing processes and identified around 1,000 (!) unnecessary tasks

Notes

such as regression of functionality that wasn't upgraded, etc. It helped us figure out how to optimize the tests.

Our analysis showed that the project would only need a Test Manager, three junior automation testers (whom we could supply from our Coding Academy best-achieving graduates' pool), one senior QA engineer and one middle tester. So the client retained just a Test Manager and a senior QA guy in-house, and hired a mid-level tester and replaced 9 junior testers with 3 same-level offshore employees in Ukraine. The team was curtailed by half, but only gained from it.

The bottom line: our client's in-house QA team audit allowed us to reduce the cost of pre-release testing by 50%, achieve a 5-fold increase in the client's ROI for QA (as a result of automation), and reduce pre-release testing time by 23 man-hours.

8. Outsource your QA function to an expert provider

According to a recently published *CEE Software Development Report 2019*, **17% of all software projects outsourced to Ukraine relate to QA and testing**. This number increases from year to year due to some obvious reasons:

1) Lack of QA talent in North America, Europe, and globally

As reported by Forbes, **84% of enterprise executives say the lack of digital QA talent is a huge issue!** If large companies face this challenge, it's obvious that startups and SMEs feel these shortcomings even more.

2) The high cost of senior and mid-level talent

According to *Indeed.com*, in NYC, the average mid-level manual QA engineer costs \$85,000 per annum; in London – \$88,000; in Ukraine, you can get a QA tech lead for as low as \$33,600 a year. This salary discrepancy explains a lot!

Also, time to hire a QA resource in Ukraine is less than 30 days, while in North America it can take up to 60 days and more. In this highly dynamic world speed is crucial, so accelerating your time to hire will help speed up your time to market accordingly.

3) Lack of internal QA expertise

If you're a small non-IT company from the USA or UK, the chance is slim you can develop your software product onshore/in-house without inflating your budget. You simply won't be able to launch an in-house team without making substantial investments in the office space and infrastructure, i.e. increasing your operating expenses.

By outsourcing QA, you can access untapped pools and, thus, source good yet more cost-effective QA specialists to save on cost arbitrage and to benefit from the speed of testing and development in general.

1.4.2

Because of the large size of most software systems, the defect reduction activities can only reduce the number of faults to a fairly low level but not completely eliminate

Notes

them. So some other means need to be used to prevent failures by breaking the casual relations between these faults and the resulting failures, thus “tolerating” these faults, or “to contain” the failures by reducing the resulting damage. This way of dealing with defects is called “defect containment”.

DCE or total defect containment is a way of finding the effectiveness of the working team of resource and development department of any organisation.

Explanation:

- This is a **very effective way** of saying the **effectiveness of the development Team** by calculating the the **percentage of effectiveness** of the team.
- This is done by taking into **account the number of defect steam** found or before release of a certain product.
- Then dividing this value with the **total defects that is found out** before and after the release of the same product.
- Hence the **percentage will show how effective a team of development** is.

1.1.4 Testing

We are going to understand the difference between **Testing** and **Debugging**. Both the terminologies are the integral parts of SDLC as both are used at the different phase of **Software Development Life Cycle** and gives distinct types of outcomes.

At the time of development and after the outcome of any application or the software product established in any programming language, both **Testing and Debugging** play a vital role in finding and removing mistakes.

Note: Both Testing and Debugging are two words that seem to share a similar meaning but intensively different from one another.

They have quite an equivalent function, but they are diverse in terms of **designs, requirements, benefits, and performance**.

Therefore, it is required for us to understand the **differences between testing and debugging** properly that will support us in receiving better software development outcomes.

Before we see the difference between **testing and debugging**, we will discuss the **in-detail** evaluation of **testing and debugging**, which will help us distinguish both of them appropriately.

Software Testing:

Software testing is a process of identifying defects in the software product. It is performed to validate the behavior of the software or the application compared to requirements.

In other words, we can say that the testing is a collection of techniques to determine the accuracy of the application under the predefined specification but, it cannot identify all the defects of the software.

Notes

Each software or application needs to be tested before delivering to the clients and checks whether the particular software or the application is working fine as per the given requirements.

1.2.1 Debugging

Debugging

As opposed to Testing, Debugging is the action where the development team or a developer implements after receiving the test report related to the bugs in the software from the testing team.

In the software development process, debugging includes detecting and modifying code errors in a software

Program.

In debugging process, the developer needs to identify the reason behind the particular **bug** or defect, which is carried out by analyzing the coding rigorously.

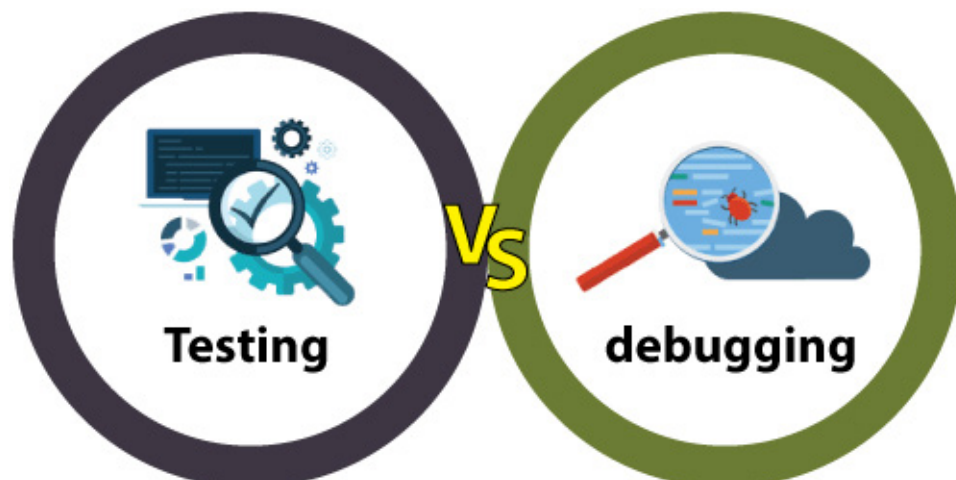
The developer changes the code and then rechecks whether the defect has been deleted whenever the bug or error is found.

Once the debugging is successfully finished, the application is again sent back to the test engineers, who remain in the process of testing.

The debugging process allows us an earlier finding of an error and makes software development stress-free and easy.

Now, based on features and technique of practice, we can distinguish between **Testing and Debugging**.

Testing Vs. Debugging



In the below table, we have listed some of the significant difference between testing and debugging:

Notes

S.no	Testing	Debugging
1.	It is the implementation of the software with the intent of identifying the defects	The process of fixing and resolving the defects is known as debugging.
2.	Testing can be performed either manually or with the help of some automation tools.	The debugging process cannot be automated.
3.	A group of test engineers executes testing, and sometimes it can be performed by the developers.	Debugging is done by the developer or the programmer.
4.	The test engineers perform manual and automated test cases on the application, and if they detect any bug or error, they can report back to the development team for fixing.	The developers will find, evaluates, and removes the software errors.
5.	Programming knowledge is not required to perform the testing process.	Without having an understanding of the programming language, we cannot proceed with the debugging process.
6.	Once the coding phase is done, we proceed with the testing process.	After the implementation of the test case, we can start the Debugging process.
7.	Software Testing includes two or more activities such as validation and verification of the software.	Debugging tries to match indication with cause, hence leading to the error correction.
8.	It is built on different testing levels such as Unit Testing, Integration Testing, System Testing, etc.	It is built on different kinds of bugs because there is no such level of debugging is possible.
9.	Software testing is the presentation of defects.	It is a logical procedure.
10.	Software testing is the vital phase of SDLC (Software Development Life Cycle).	It is not a part of SDLC because it occurs as a subset of testing.
11.	Some advantages of software testing are as below: - It can easily understand by the new test engineers or the beginner. - The test engineer can interact with software as a real end-user to check the usability and user interface issues. - It is used to test dynamically altering GUI designs. - Testing is a cost-effective and time-saving process. - Software testing delivers a consistence software. - It will help us to execute the root cause analysis that will enhance the software's productivity. - The testing process also helps detect and fixing the bugs before the software becomes active, which significantly reduces the risk of failure.	Some advantages of debugging process are as follows: - It supports the developer in minimizing the data. - If the perform the debugging, we can report the error condition directly. - During the debugging process, the developer can avoid complex one-use testing code that helps the developer save time and energy. - Debugging delivers maximum useful information of data structures and allows its informal understanding.

Notes

S.no	Testing	Debugging
12.	Software testing contains various type of testing methods, which are as follow: • Black-box testing • White-box testing • Grey-box testing And some other type of testing types is as below: • Unit testing • Integration Testing • System Testing • Stress Testing • Performance Testing • Compatibility Testing • Beta Testing • Alpha Testing • Smoke Testing • Regression Testing • User Acceptance Testing and so on.	Debugging involves a various type of approaches, which are as follows: • Induction • Brute Force • Deduction
13.	The testing team can subcontract to the outside team as well.	Debugging cannot be subcontracted to an outside team because the inside development team only does it.
14.	We can plan, design, and implement the testing process.	As compared to the testing process, the debugging process cannot be forced.

1.2.2 Software Quality, Requirement Behaviour and Correctness

Software Quality

Software quality product is determined by its suitability for its intended use. That is, a high-quality product does exactly what its users expect. The fitness of use of software products is usually stated in terms of meeting the requirements outlined in the SRS document. For many items, such as an automobile, a table fan, a grinding machine, and so on, “fitness of purpose” is a reasonable interpretation of quality. However, “fitness of purpose” is not a totally satisfactory definition of quality for software products.

Example: Consider a software programme that is functionally correct. That is, it completes all of the tasks outlined in the SRS document. However, the user interface is nearly unusable. We cannot consider it a quality product, even if it is functionally correct.

The modern view of a quality associated with a software product several quality methods such as the following:

Portability: If a software device can be configured to run in a variety of operating system contexts, on different machines, with other software items, and so on, it is said to be portable.

Notes

Usability: A software product's usability is improved if different types of users can easily access the product's features.

Reusability: If distinct components of a software product may be readily reused to construct other products, it has great reusability.

Correctness: If various requirements listed in the SRS document have been correctly implemented, a software product is correct.

Maintainability: A software product is maintainable if bugs can be easily fixed as they arise, new tasks can be easily added to the product, and the product's features can be easily changed, among other things.

Software Quality Management System

A quality management system is the principal methods used by organizations to provide that the products they develop have the desired quality.

A quality system subsists of the following:

Managerial Structure and Individual Responsibilities: The entire organisation is responsible for a quality system. Every organisation, on the other hand, has multiple quality departments that undertake diverse quality system functions. The arrangement's quality system should have the endorsement of the senior management. Some employees will take the quality system seriously if the quality system is not supported at a high level in the firm.

Quality System Activities: The quality system activities encompass the following:

Auditing of projects

Review of the quality system

Development of standards, methods, and guidelines, etc.

Production of documents for the top management summarizing the effectiveness of the quality system in the organization.

Evolution of Quality Management System:

Quality systems have increasingly evolved over the last five decades. Before World War II, the usual function to produce quality products was to inspect the finished products to remove defective devices. Since that time, quality systems of organizations have undergone through four steps of evolution, as shown in the fig. The first product inspection task gave method to quality control (QC).

Quality control focuses on not only finding and removing defective items, but also on determining the reasons of the flaws. As a result, quality control attempts to remedy the causes of faults rather than simply rejecting the items. The creation of quality assurance methods was the next major advancement in quality methods.

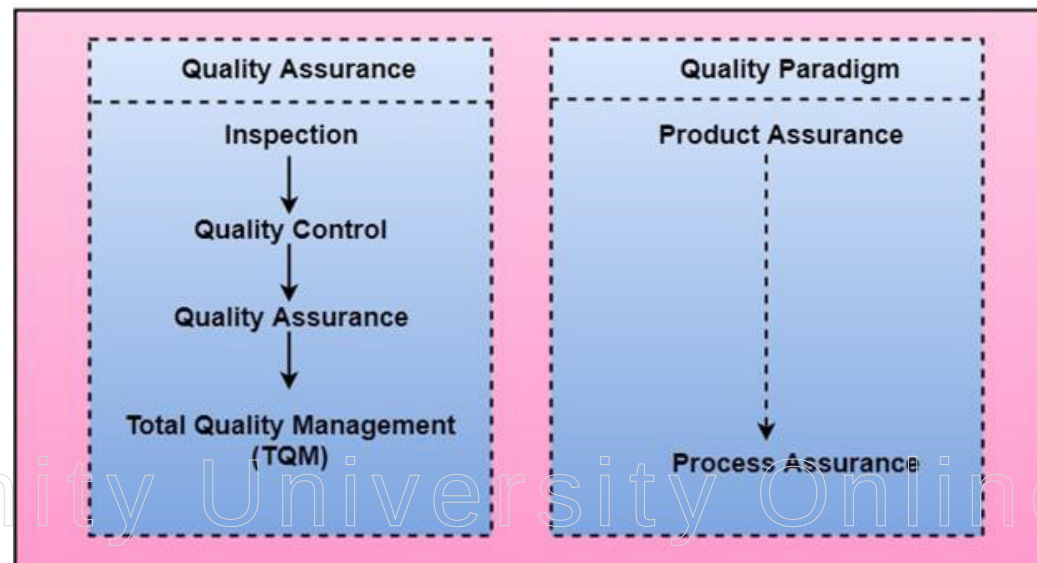
The core principle of modern quality assurance is that if an organization's processes are correct and strictly followed, the products must be of high quality. The new quality functions provide instructions for identifying, specifying, analysing, and improving the manufacturing process.

Notes

Total quality management (TQM) asserts that an organization's procedures must be continually improved through process measurements. TQM goes beyond quality assurance and seeks for continuous process improvement. TQM goes beyond documenting processes to re-designing them to make them more efficient. Business Process Reengineering is a term associated with TQM (BPR).

The goal of BPR is to reengineer the way business is done in a company. As a result of the discussion above, it can be concluded that the quality paradigm has shifted from product assurance to process assurance over time, as seen in fig.

Evolution of quality system and corresponding shift in the quality paradigm



Requirement Behaviour and Correctness

Behavioural Testing is a testing of the external behaviour of the program, also known as black box testing. It is usually a functional testing.

Techniques used in Black box testing

- Equivalence Class
- Boundary Value Analysis
- Domain Tests
- Orthogonal Arrays
- Decision Tables
- State Models
- Exploratory Testing
- All-pairs testing

Correctness

Correctness from software engineering perspective can be defined as the adherence to the specifications that determine how users can interact with the software and how the software should behave when it is used correctly.

If the software behaves incorrectly, it might take considerable amount of time to achieve the task or sometimes it is impossible to achieve it.

Important rules:

Below are some of the important rules for effective programming which are consequences of the program correctness theory.

- Defining the problem completely.
- Develop the algorithm and then the program logic.
- Reuse the proved models as much as possible.
- Prove the correctness of algorithms during the design phase.
- Developers should pay attention to the clarity and simplicity of your program.
- Verifying each part of a program as soon as it is developed.

1.2.3 Fundamentals of Test Process

Testing is a process rather than a single activity. This process starts from test planning then designing **test cases**, preparing for execution and evaluating status till the test closure. So, we can divide the activities within the fundamental test process into the following basic steps:

- 1) Planning and Control
- 2) Analysis and Design
- 3) Implementation and Execution
- 4) Evaluating exit criteria and Reporting
- 5) Test Closure activities

1) Planning and Control:

Test planning has following major tasks:

- i. To determine the scope and **risks** and identify the objectives of testing.
- ii. To determine the test approach.
- iii. To implement the test policy and/or the **test strategy**. (Test strategy is an outline that describes the testing portion of the **software development cycle**. It is created to inform PM, testers and developers about some key issues of the testing process. This includes the testing objectives, method of testing, total time and resources required for the project and the testing environments.).
- iv. To determine the required test resources like people, test environments, PCs, etc.
- v. To schedule test analysis and design tasks, test implementation, execution and evaluation.
- vi. To determine the **Exit criteria**, we need to set criteria such as **Coverage criteria**. (Coverage criteria are the percentage of statements in the software that must be executed during testing. This will help us track whether we are completing test activities correctly. They will show us which tasks and checks

Notes

we must complete for a particular level of testing before we can say that testing is finished.)

Test control has the following major tasks:

- i. To measure and analyse the results of reviews and testing.
- ii. To monitor and document progress, **test coverage** and exit criteria.
- iii. To provide information on testing.
- iv. To initiate corrective actions.
- v. To make decisions.

2) Analysis and Design:

Test analysis and Test Design has the following major tasks:

- i. To review the **test basis**. (The test basis is the information we need in order to start the test analysis and create our own test cases. Basically it's a documentation on which test cases are based, such as requirements, design specifications, product risk analysis, architecture and interfaces. We can use the test basis documents to understand what the system should do once built.)
- ii. To identify test conditions.
- iii. To design the tests.
- iv. To evaluate testability of the requirements and system.
- v. To design the test environment set-up and identify and required infrastructure and tools.

3) Implementation and Execution:

During test implementation and execution, we take the test conditions into **test cases** and procedures and other **testware** such as scripts for automation, the test environment and any other test infrastructure. (Test cases is a set of conditions under which a tester will determine whether an application is working correctly or not.) (Testware is a term for all utilities that serve in combination for testing a software like scripts, the test environment and any other test infrastructure for later reuse.)

Test implementation has the following major task:

- i. To develop and prioritize our test cases by using techniques and create **test data** for those tests. (In order to test a software application you need to enter some data for testing most of the features. Any such specifically identified data which is used in tests is known as test data.) We also write some instructions for carrying out the tests which is known as **test procedures**. We may also need to automate some tests using **test harness** and automated tests scripts. (A test harness is a collection of software and test data for testing a program unit by running it under different conditions and monitoring its behaviour and outputs.)
- ii. To create test suites from the test cases for efficient test execution. (Test suite is a collection of test cases that are used to test a software program to

Notes

show that it has some specified set of behaviours. A test suite often contains detailed instructions and information for each collection of test cases on the system configuration to be used during testing. Test suites are used to group similar test cases together.)

- iii. To implement and verify the environment.

Test execution has the following major task:

- i. To execute test suites and individual test cases following the test procedures.
- ii. To re-execute the tests that previously failed in order to confirm a fix. This is known as **confirmation testing or re-testing**.
- iii. To log the outcome of the test execution and record the identities and versions of the software under tests. The **test log** is used for the audit trail. (A test log is nothing but, what are the test cases that we executed, in what order we executed, who executed that test cases and what is the status of the test case (pass/fail). These descriptions are documented and called as test log.).
- iv. To Compare actual results with expected results.
- v. Where there are differences between actual and expected results, it report discrepancies as Incidents.

4) Evaluating Exit criteria and Reporting:

Based on the risk assessment of the project we will set the criteria for each test level against which we will measure the “enough testing”. These criteria vary from project to project and are known as **exit criteria**.

Exit criteria come into picture, when:

- Maximum test cases are executed with certain pass percentage.
- Bug rate falls below certain level.
- When achieved the deadlines.

Evaluating exit criteria has the following major tasks:

- i. To check the test logs against the exit criteria specified in test planning.
- ii. To assess if more test are needed or if the exit criteria specified should be changed.
- iii. To write a test summary report for stakeholders.

5) Test Closure activities:

Test closure activities are done when software is delivered. The testing can be closed for the other reasons also like:

- When all the information has been gathered which are needed for the testing.
- When a project is cancelled.
- When some target is achieved.
- When a maintenance release or update is done.

Notes

Test closure activities have the following major tasks:

- i. To check which planned deliverables are actually delivered and to ensure that all incident reports have been resolved.
- ii. To finalize and archive test ware such as scripts, test environments, etc. for later reuse.
- iii. To handover the test ware to the maintenance organization. They will give support to the software.
- iv. To evaluate how the testing went and learn lessons for future releases and projects.

1.2.4 Psychology of Testing

Psychology of testing is a type of testing which fully depends on the mindset of developers and tester. When we are building the software, we working positively towards the software never think about negative things. The mindset should be different while testing and reviewing developing software. With the correct mindset, the programmer can test their own code. At a certain point independence tester often makes the tester more **effective to finding defects.**

- Levels of Independence Tester in Psychology of testing
- Tests designed by the person who wrote the software under test
- Tests designed by another person
- Tests designed by a person from a different organizational group or test specialists
- Tests designed by a person from a different organization or company

In **Psychology of testing**, People and projects are driven by objectives. While developing the software, programmer writing code to build software, that time Programmers test their code then pass through the next phase but as we already mentioned, It is difficult to find our own mistakes. That's why independence tester highly recommendable. Because they are testing specialists or professional testers. In Psychology of testing communication also matters, tester and test lead need good interpersonal skill to communicate. In Psychology of testing Tester and others can improve communication by

- Starting collaboration instead of battles.
- Everyone should have a common goal of the quality software.
- Improve mutual understanding while working.
- Confirm that the other person has understood what you have said.

A software tester is any human being who tests the software. So, what do you answer if anybody asks you 'Does software testing need a specialized skill'? It looks very simple to answer this question, isn't it? But of course not. Software Testing is not as easy as you might think of. Needless to say, It's one of the most important and difficult tasks in the entire gamut of software development life cycle process.

Human beings reaction in this complex world of happenings varies widely with respect to situations, surroundings, emotions, need, requirements, time frame, money,

Notes

visualization, belief, education, knowledge, expertise, intuition and so on. Such complex is the nature of human being and certainly there's no exception at work place too. Therefore, it's cynical to say that the job being done (software testing) by a software tester is simple & complete.

Nevertheless, the quality of the job done by the software tester is directly proportional to his or her psychological maturity and profoundness acquired, adopted and developed with age and experience.

"The psychology of the persons carrying out the testing will have an impact on the testing process."

Let's examine the psychology of the person under discussion (tester) by describing the definition of software testing under three circumstances.

1. "Software testing is the process to prove that the software works correctly"

The above definition sounds good. If the person under discussion (software tester) is the one who has developed the software he/she will try to use the above definition. This person's intentions would mostly revolve around the point to prove the software works. He/She will only give those inputs for which correct results are obtained.

This above explanation is the typical psychology of testing by a software developer.

2. "Testing is the process to prove that the software doesn't work"

This definition sounds very good especially if the aim of the tester is to prove that the software doesn't work for what it's supposed to do. This type of psychology would bring out the most of the defects in the software. A software tester should therefore have such a psychology to push the software beyond its boundaries.

Nevertheless, this definition involves a practical difficulty – If you question "how many days does the software should be tested to conclude that the software works perfectly?", perhaps the answer would again recur to be a question in itself.

It's never a right conclusion to infer that there are no bugs in the software if one says that "the software has no bugs even after testing for more than a week or a month or an year" nor it is wise to say that the software is completely reliable.

Testing does not guarantee defect less or zero bug software at all times but can only reduce or minimize known defects because the very nature of a bug can be volatile. If the above definition is to be strictly implemented, then perhaps the most Operating Systems & other Commercial Software packages which we use today would not have been in the market yet. If so, then the above definition would seem to be unrealistic.

3. "Testing is the process to detect the defects and minimize the risks associated with the residual defects"

This definition appears to sound realistic. Practically, if at any point, the software product development starts, the testing process should start and keep in track the number of bugs being detected while correcting them.

Notes

At some stage of a planned testing, there would be a stage where no bugs are identified after many days or weeks or sometimes months of testing which statistically allows you to conclude that the software is “good enough” to be released into the market. i.e there may still exist some bugs undetected, but the risk associated with the residual defects is not very high or are tolerable.

The decision to release a software may thus vary based on many other factors like business value, time constraint, client requirements & satisfaction, company’s quality policies etc.

From the above three definitions, we can understand that the psychology of a software tester plays a vital role throughout the software development cycle & the quality processes of software testing.

“The objective of testing is to uncover as many bugs as possible. The testing has to be done without any emotional attachment to the software. If someone points out the bugs, it is only to improve the quality of the software rather than to find fault with you.”

Role and Characteristics of a Software Tester

Software Development & Software Testing go hand in hand. Both aim at meeting the predefined requirements and purpose. Both are highly creative jobs in nature but the general outlook towards these two individuals is psychological rather than distinctive classification.

If success is perceived as achieving something constructive or creative, like the job of developing a software to work, software testing in contrast is perceived as destructive job or negative job by majority of the masses. Nevertheless, software testing itself has to be perceived as an equally creative job on par with the development job, but this perception can only be practically possible with a different outlook and mindset.

Software Testers require technical skills similar to their development counterparts, but software testers need to acquire other skills, as well.

- Keen Observation
- Detective Skills
- Destructive Creativity
- Understanding Product as integration of its parts
- Customer Oriented Perspective
- Cynical but Affable Attitude
- Organized, Flexible & Patience at job
- Objective & Neutral attitude

Keen Observation – The software tester must possess the qualities of an ‘eye for detail’. A keen observation is the prime quality any software tester must possess. Not all bugs are visible clearly to the naked eye in a software. With keen observation, the tester can easily identify or detect many critical bugs. Observing the software for established parameters like ‘look & feel’ of GUI, incorrect data representation, user friendliness etc needs this type of characteristics.

Notes

Detective Skills – Ideally the software under development would be documented before, after and throughout the development process. Unfortunately, there is every chance of not updating the documentation (specification, defect reports etc) due to time and resource constraints.

The software under testing would be completely described in a well-defined set library of functions and design specifications in documents called specs or SRS etc. which needs constant update. The tester should therefore apply his knowledge of rationalization in knowing about the product from formal system like system specifications, design & functional specifications. From this information, the tester should look for researching more analytical information through other sources called non-formal sources of information like developers, support personnel, bugs and related product documents, reviews of related and (seemingly) unrelated documents.

A tester should therefore possess the quality of a 'detective' to explore the product under test, more rationally.

Destructive Creativity – The Tester need to develop destructive skills – means skills to perturb and crash the software workflow and its functionality. In other words, the tester should 'not hesitate to break the software' fearing to buy it or being empathetic to the creation of developers. In software testing, boundaries are meant to be crossed not obeyed.

A creative oriented but destructive approach is necessary while testing a software by the tester to make the software evolve more robustly and reliably.

"The Software under test should be tested like a Mercedes-Benz car. "

Understanding the Product as an integration of its parts – The software (read as product) is a culmination of lines of code interacting with data through user interface and database.

It is an integration of separate group of code interacting with other groups assembled together to function as a whole product. The developers might be working on a respective piece of code module focusing more on those modules under work, at a time.

It is no wonder if the developer sometimes may not even know the complete workflow of the product and not necessary too. Whereas, in the case of a tester, being the rider of the software under test, should understand and know about the complete specifications (operational and workflow requirements) of the product.

The tester may not be the best expert on any one module of the software but definitely he/she should gain expertise on the overall operation of the product. In fact, the tester should possess a 'Systems' view of the product because they are the only people to see and test the complete functionality of interdependent modules and compatibility between the software and hardware specifications laid down by the client before it is sent to the customer.

Customer Oriented Perspective – Testers need to possess a customer oriented perspective. Customers (read as users) need not be as wise as software engineers or testers. As the number of computer users are growing every day (need not be as wise as engineers or testers), the end-users may neither be comfortable nor tolerate any

Notes

bugs, explanations, or frequent upgrades to fix the bugs. As any customer would simply be interested in consuming the product (software) by deriving its value the software tester must adopt a customer-oriented perspective while testing the software product.

Hence the tester must develop his abilities to suffice himself/herself in the place of the customer, testing the product as a mere end-user.

Cynical but Affable Attitude – Irrespective of the nature of the project, the tester needs to be tenacious in questioning even the smallest ambiguity until it is proved. In other words, the tester must chant the words “In God we Trust- Everything Else gets Tested” throughout the testing cycle.

There may arise some situations during the course of testing – large number of bugs might be encountered unusually which might compel to further delay in shipping of the product. This may lead to heat up the relation between the testers and other development teams. The tester should balance this relationship not at the cost of the bugs but by convincing and upholding their intentions to “assault the software but not the software developers”.

Organized, Flexible and Patience at Job: Software testers must remember the fact that as the world is shrinking since the advent of internet and web, so, change is very dynamic and no exception for modern software. Not all the tests planned are performed completely and some tests dependent on other tests has to be blocked for later testing.

This needs an organized approach by the tester in attempting to phase out the bugs. Sometimes, significant tests have to be rerun which would change the fundamental functionality of the software. The tester should therefore have patience to retest the planned bugs and any new bugs that may arise.

It is even more important to the testers to be patient and keep prepared in the event of a dynamic development and test model. Development keeps changing continuously as requirements and technology keeps changing rapidly and so should testing. The testers must take these changes into account and plan to perform tests while maintaining the control of test environment to ensure valid test results.

Objective and Neutral Attitude – Nobody would like to hear and believe bad news, right? Well, testers are sometimes viewed as messengers of bad news in a software project team. No wonder, how good the tester is (meaning very negative) and brilliant in doing his job (identifying bugs-no one likes to do it but most human beings are taken for granted to be naturally very good at it, at least from childhood), he/she might always be a messenger of communicating the bad part of the software, which, the creators (developers) doesn't like it.

“Nobody who builds the houses likes to have them burned.”

The tester must be able to deal with the situation where he/she is blamed for doing his/her job (detecting bugs) too good. The tester's jobs must be appreciated and the bugs should be welcomed by the development team because every potential bug found by the tester would mean a reduction of one bug that would potentially might have been encountered by the customer.

Irrespective of the negative aspects of doing such a highly skilful job, the role of a tester is to report honestly every known bug encountered in the product with always an objective and a neutral attitude.

1.3.1 General Principles of Testing

Testing is a crucial element of software development. It can also be a complex activity to structure correctly, and in a way that supports maximum efficiency. Because of this complexity, it is always helpful to review processes and guidelines to ensure you are following best practice, and a great place to start is with the ISTQB (International Software Testing Qualifications Board), who list **seven fundamental principles of testing**.

These are the principles that have been collated and established by the ISTQB as testing and software development has evolved over the years, and are recognised as the absolute core of testing.

This is part of the reason why we are proud to say all our testers at Box UK are ISTQB qualified!

If you're involved with any aspect of software testing it's worthwhile to fully review and comprehend these standards, checking that you are following them within your organisation and teams, as they will help you achieve high standards of quality and give your clients confidence that their software is production-ready.

The seven principles of testing

1. *Testing shows the presence of defects, not their absence*

We test software to discover issues, so that they can be fixed before they are deployed to live environments – this enables us to have confidence that our systems are working. However, this testing process does not confirm that any software is completely correct and completely devoid of issues. Testing helps greatly reduce the number of undiscovered defects hiding in software, but finding and resolving these issues is not itself proof that the software or system is 100% issue-free. This concept should always be accepted by teams, and effort should be made to manage client expectations.

It is important to remember however that while testing shows the presence of bugs and not their absence, thorough testing will give everyone confidence that the software will not fail. Having a comprehensive test strategy that includes thorough test plans, reports and statistics along with testing release plans can all help with this; reassuring clients as to testing progress and providing confidence that the right areas are being tested.

Additionally, ongoing monitoring and testing after systems have gone into production is vital. Thinking forward to potential issues that could arise is another good way to help mitigate against any future problems, for example considering load testing if a site is launching a new marketing campaign, so you can be confident the software will withstand any anticipated larger volumes of traffic.

Notes

2. *Exhaustive testing is impossible*

As much as we would like to believe or wish it true(!), it is absolutely impossible to test EVERYTHING – all combinations of inputs and preconditions – and you could also argue that attempting to do so is not an efficient use of time and budget. However, one of the skills of testing is assessing risks and planning your tests around these – you can then cover vast areas, while making sure you are testing the most important functions. With careful planning and assessment, your test coverage can remain excellent and enable that necessary confidence in your software, without requiring that you test every single line of code.

3. *Early testing saves time and money*

Testing early is fundamentally important in the software lifecycle. This could even mean testing requirements before coding has started, for example – amending issues at this stage is a lot easier and cheaper than doing so right at the end of the product's lifecycle, by which time whole areas of functionality might need to be re-written, leading to overruns and missed deadlines.

Involving testing early is also a fundamental Agile principle, which sees testing as an activity throughout, rather than a phase (which in a traditional waterfall approach would be at the end) because it enables quick and timely continuous feedback loops. When a team encounters hurdles or impediments, early feedback is one of the best ways to overcome these, and testers are essential for this. Consider the tester as the 'information provider' – a valuable role to play.

Essentially, testing early can even help you prevent defects in the first place!

4. *Defects cluster together*

This is the idea that certain components or modules of software usually contain the most number of issues, or are responsible for most operational failures. Testing therefore, should be focused on these areas (proportionally to the expected – and later observed – defect density of these areas). The Pareto principle of 80:20 can be applied – 80 percent of defects are due to 20 percent of code!

This is particularly the case with large and complex systems, but defect density can vary for a range of reasons. Issues are not evenly distributed throughout the whole system, and the more complicated a component, or the more third-party dependencies there are, the more likely it is that there will be defects. Inheriting legacy code, and developing new features in certain components that are undergoing frequent changes and are therefore more volatile, can also cause defect clustering.

Knowing this could prove to be very valuable for your testing; if we find one defect in a particular module/area there is a strong chance of discovering many more there. Identifying the more complex components, or areas that have more dependencies or are changing the most, for example, can help you concentrate your testing on these crucial risk areas.

Notes

5. *Beware of the pesticide paradox*

This is based on the theory that when you use pesticide repeatedly on crops, insects will eventually build up an immunity, rendering it ineffective. Similarly with testing, if the same tests are run continuously then – while they might confirm the software is working – eventually they will fail to find new issues. It is important to keep reviewing your tests and modifying or adding to your scenarios to help prevent the pesticide paradox from occurring – maybe using varying methods of testing techniques, methods and approaches in parallel.

6. *Testing is context dependent*

Testing is ALL about the context. The methods and types of testing carried out can completely depend on the context of the software or systems – for example, an e-commerce website can require different types of testing and approaches to an API application, or a database reporting application. What you are testing will always affect your approach.

7. *Absence-of-errors is a fallacy*

If your software or system is unusable (or does not fulfil users' wishes) then it does not matter how many defects are found and fixed – it is still unusable. So in this sense, it is irrelevant how issue- or error-free your system is; if the usability is so poor users are unable to navigate, or/and it does not match business requirements then it has failed, despite having few bugs.

It is important, therefore, to run tests that are relevant to the system's requirements. You should also be testing your software with users – this can be done against early prototypes (at the usability testing phase), to gather feedback that can be used to ensure and improve usability. Remember, just because there might be a low number of issues, it does not mean your software is shippable – meeting client expectations and requirements are just as important as ensuring quality.

1.3.2 Test Metrics

Software Testing Metrics

Software Testing Metrics are the quantitative measures used to estimate the progress, quality, productivity and health of the software testing process. The goal of software testing metrics is to improve the efficiency and effectiveness in the software testing process and to help make better decisions for further testing process by providing reliable data about the testing process.

A Metric defines in quantitative terms the degree to which a system, system component, or process possesses a given attribute. The ideal example to understand metrics would be a weekly mileage of a car compared to its ideal mileage recommended by the manufacturer.

Software testing metrics or software test measurement is the quantitative indication of extent, capacity, dimension, amount or size of some attribute of a process or product.

Notes

Example for software test measurement: Total number of defects

Why Test Metrics are Important?

“We cannot improve what we cannot measure” and Test Metrics helps us to do exactly the same.

- Take decision for next phase of activities
- Evidence of the claim or prediction
- Understand the type of improvement required
- Take decision on process or technology change

Types of Test Metrics

- **Process Metrics:** It can be used to improve the process efficiency of the SDLC (Software Development Life Cycle)
- **Product Metrics:** It deals with the quality of the software product
- **Project Metrics:** It can be used to measure the efficiency of a project team or any testing tools being used by the team members

Identification of correct testing metrics is very important. Few things need to be considered before identifying the test metrics

- Fix the target audience for the metric preparation
- Define the goal for metrics
- Introduce all the relevant metrics based on project needs
- Analyze the cost benefits aspect of each metrics and the project lifecycle phase in which it results in the maximum output

Manual Test Metrics

In Software Engineering, Manual test metrics are classified into two classes

- **Base Metrics**
- **Calculated Metrics**

Base metrics is the raw data collected by Test Analyst during the test case development and execution (**# of test cases executed, # of test cases**). While calculated metrics are derived from the data collected in base metrics. Calculated metrics is usually followed by the test manager for test reporting purpose (**% Complete, % Test Coverage**).

Depending on the project or business model some of the important metrics are

- Test case execution productivity metrics
- Test case preparation productivity metrics
- Defect metrics
- Defects by priority
- Defects by severity
- Defect slippage ratio

Test Metrics Life Cycle

Different stages of Metrics life cycle	Steps during each stage
Analysis	- Identification of the Metrics - Define the identified QA Metrics
Communicate	- Explain the need for metric to stakeholder and testing team - Educate the testing team about the data points to need to be captured for processing the metric
Evaluation	- Capture and verify the data - Calculating the metrics value using the data captured
Report	- Develop the report with an effective conclusion - Distribute the report to the stakeholder and respective representative - Take feedback from stakeholder

How to calculate Test Metric

Sr#	Steps to test metrics	Example
1	Identify the key software testing processes to be measured	Testing progress tracking process
2	In this Step, the tester uses the data as a baseline to define the metrics	The number of test cases planned to be executed per day
3	Determination of the information to be followed, a frequency of tracking and the person responsible	The actual test execution per day will be captured by the test manager at the end of the day
4	Effective calculation, management, and interpretation of the defined metrics	The actual test cases executed per day
5	Identify the areas of improvement depending on the interpretation of defined metrics	The <u>Test Case</u> execution falls below the goal set, we need to investigate the reason and suggest the improvement measures

Example of Test Metric

To understand how to calculate the test metrics, we will see an example of a percentage test case executed.

To obtain the execution status of the test cases in percentage, we use the formula.

Percentage test cases executed= (No of test cases executed/ Total no of test cases written) X 100

Notes

Notes

Likewise, you can calculate for other parameters like **test cases not executed**, **test cases passed**, **test cases failed**, **test cases blocked**, etc.

Test Metrics Glossary

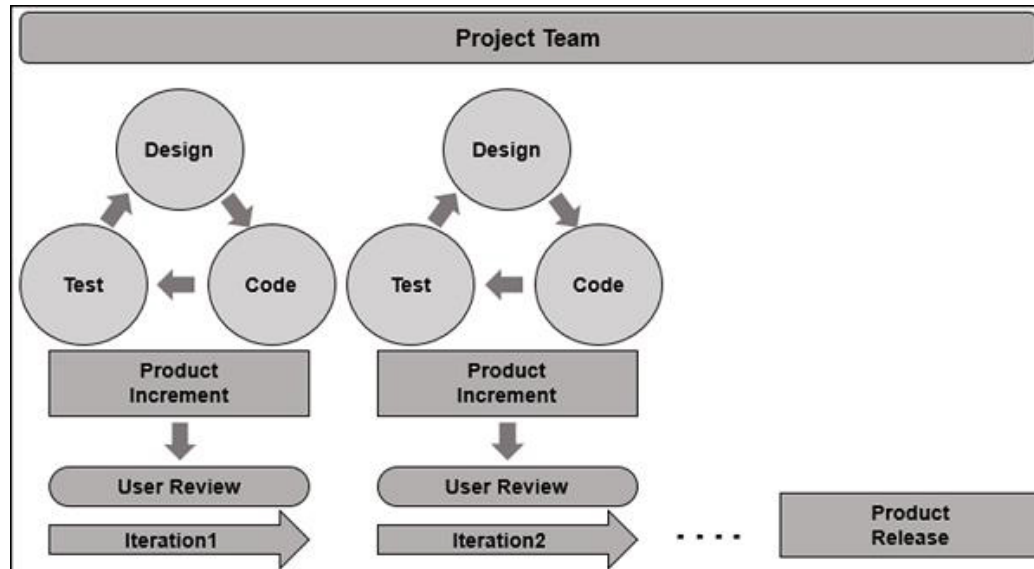
- **Rework Effort Ratio** = (Actual rework efforts spent in that phase/ total actual efforts spent in that phase) X 100
- **Requirement Creep** = (Total number of requirements added/No of initial requirements)X100
- **Schedule Variance** = (Actual Date of Delivery – Planned Date of Delivery)
- **Cost of finding a defect in testing** = (Total effort spent on testing/ defects found in testing)
- **Schedule slippage** = (Actual end date – Estimated end date) / (Planned End Date – Planned Start Date) X 100
- **Passed Test Cases Percentage** = (Number of Passed Tests/Total number of tests executed) X 100
- **Failed Test Cases Percentage** = (Number of Failed Tests/Total number of tests executed) X 100
- **Blocked Test Cases Percentage** = (Number of Blocked Tests/Total number of tests executed) X 100
- **Fixed Defects Percentage** = (Defects Fixed/Defects Reported) X 100
- **Accepted Defects Percentage** = (Defects Accepted as Valid by Dev Team / Total Defects Reported) X 100
- **Defects Deferred Percentage** = (Defects deferred for future releases /Total Defects Reported) X 100
- **Critical Defects Percentage** = (Critical Defects / Total Defects Reported) X 100
- **Average time for a development team to repair defects** = (Total time taken for bugfixes/Number of bugs)
- **Number of tests run per time period** = Number of tests run/Total time
- **Test design efficiency** = Number of tests designed /Total time
- **Test review efficiency** = Number of tests reviewed /Total time
- **Bug find rate or Number of defects per test hour** = Total number of defects/Total number of test hours

1.3.3 Agile Methodology

Agile is an iterative development methodology, where the entire project team participates in all the activities. The requirements evolve as the iterations progress, through collaboration between the customer and the self-organizing teams. As Coding and Testing are done interactively and incrementally, during the course of development, the end-product would be of quality and ensures customer requirements.

Every iteration result in an integrated working product increment and is delivered for User Acceptance Testing. The customer feedback thus obtained would be an input to the next / subsequent Iterations.

Notes



Continuous Integration, Continuous Quality

Continuous Integration is the key for Agile Development success. Integrate frequently, at least daily such that you are ready for a release as and when required. Testing in Agile becomes an essential component of all the phases of the development, ensuring continuous quality of the product. Constant feedback from everyone involved in the project adds to the quality of the product.

In Agile, communication is given utmost importance and the customer requests are received as and when necessary. This gives the satisfaction to the customer that all the inputs are considered and working quality product is available throughout the development.

1.3.4 Impact on Testing

Agile Methodologies

There are several Agile Methodologies that support Agile Development. The Agile Methodologies include –

Scrum

Scrum is an Agile development method that emphasizes on team-centric approach. It advocates participation of the entire team in all the project development activities.

XP

eXtreme Programming is customer-centric and focuses on constantly changing requirements. With frequent releases and customer feedback, the end-product will be of quality meeting customer requirements that are made clearer during the process.

Crystal

Crystal is based on chartering, cyclic delivery and wrap up.

- Chartering involves forming a development team, carrying out a preliminary feasibility analysis, arriving at an initial plan and the development methodology.

Notes

- Cyclic Delivery with two or more delivery cycles focuses on the development phase and final integrated product delivery.
- During Wrap up, deployment into the user environment, post-deployment reviews and reflections are performed.

FDD

Feature Driven Development (FDD) involves designing and building features. The difference between FDD and other Agile Development Methodologies is that the features are developed in specific and short phases separately.

DSDM

Dynamic Software Development Method (DSDM) is based on Rapid Application Development (RAD) and is aligned to the Agile Framework. DSDM focuses on frequent delivery of the product, involving users actively and empowering the teams to make quick decisions.

Lean Software Development

In Lean Software Development, focus is on eliminating waste and giving value to the customer. This results in rapid development and product of value.

Waste includes partially done work, irrelevant work, features that are not used by the customer, defects, etc. that add to delays in delivery.

The **Lean Principles** are –

- Eliminate Waste
- Amplify Learning
- Delay Commitment
- Empower the Team
- Deliver Fast
- Build Integrity in
- See the Whole

Kanban

Kanban focuses on managing work with an emphasis on just-in-time (JIT) delivery, while not overloading the team members. The tasks are displayed for all the participants to see and for the Team Members to pull work from a queue.

Kanban is based on –

- Kanban Board (Visual and Persistent across the Development)
- Work-in-progress (WIP) Limit
- Lead Time

Agile Testing Methodologies

The testing practices are well defined for every project, whether Agile or not, to deliver quality products. Traditional Testing principles are quite often used in Agile Testing. One of them is Early Testing that focuses on –

Notes

- Writing Test Cases to express the behavior of the system.
- Early Defect Prevention, detection and removal.
- Ensuring that the right test types are run at the right time and as part of the right test level.

In all the Agile Methodologies we discussed, Agile Testing in itself is a Methodology. In all the approaches, Test Cases are written before Coding.

In this tutorial, we will focus on Scrum as the Agile Testing Methodology.

The other commonly used Agile Testing Methodologies are –

- **Test-Driven Development (TDD)** – Test-Driven Development (TDD) is based on coding guided by tests.
- **Acceptance Test-Driven Development (ATDD)** – Acceptance Test-Driven Development (ATDD) is based on communication between the customers, developers and testers and driven by pre-defined Acceptance Criteria and Acceptance Test Cases.
- **Behaviour-Driven Development (BDD)** – In Behaviour-Driven Development (BDD) testing is based on the expected behaviour of the software being developed.

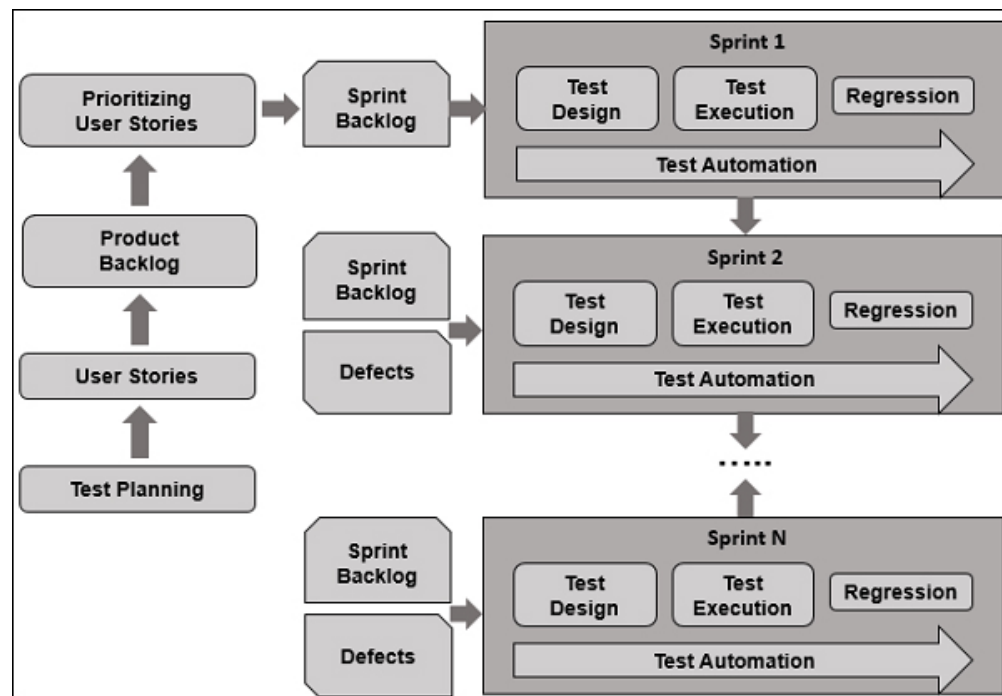
Agile Testing Lifecycle

In Scrum, the Testing activities include –

- Contributing to User Stories based on the expected behaviour of the System depicted as Test Cases
- Release Planning based on Test Effort and Defects
- Sprint Planning based on User Stories and Defects
- Sprint Execution with Continuous Testing
- Regression Testing after the completion of Sprint
- Reporting Test Results
- Automation Testing

Notes

Testing is iterative and sprints based as depicted in the diagram given below –



Agile Test Methodology: 5 Examples

Tests come first in Agile development. When you create a user story, you need to define the acceptance criteria. These drives testing and validation of the user stories.

It doesn't matter which Agile test methodology you use — Scrum, XP, Kanban. The following are typically used for Agile testing methods.

Test-Driven Development (TDD)

Test-driven development (TDD) starts with tests. This type of development begins by discussing what you want to test and then creating a user story. So, you start by writing a unit test. Then you write the user story. Finally, you write the code until the unit test passes.

TDD is typically used on unit and component tests — which can be done with automated testing tools. TDD makes sure the features are working as they should be.

Acceptance Test-Driven Development (ATDD)

Acceptance test-driven development (ATDD) is similar. But ATDD starts with customer input on functionality. This type of development begins by discussing how the product will be used. So, you write a user acceptance test (UAT). And then you write the code until it passes the test.

ATDD is typically used for acceptance tests. It verifies that the product functions as users would expect.

Behavior-Driven Development (BDD)

Behaviour-driven development often stems from TDD and ATDD. In behaviour-driven development, the purpose of development needs to be tied to a business

outcome. So, you'll have a user story — but the user story needs to answer why (in business terms) this feature is being developed. And in BDD, tests are included in user stories as scenarios or specifications.

BDD is also used for acceptance tests. It verifies that the product functions are necessary for the desired business outcome.

Exploratory Testing

Exploratory testing is a style of testing that lets testers follow their intuition — rather than a predefined path. It's typically manual. You record what you're doing and save it as a test. And you figure out what exactly it is that you're testing as you go.

Exploratory testing is typically used to find hidden risks within a product. These would be bugs that are missed in functional tests done in TDD.

Session-Based Testing

Session-based testing has some similarities to exploratory testing. But there's a little more structure in session-based testing. Instead of figuring out what you're testing as you go, you start with a mission in mind.

Session-based testing is also used to find hidden bugs within a project.

Test Strategy for the Agile Tester

Testing in traditional development typically includes a test plan. But a well-documented test plan is not common in Agile development. Instead, Agile testers need to be flexible and ready to respond to shifts in requirements.

So, there needs to be an Agile test strategy rather than an Agile test plan.

There are many ways to outline your Agile test strategy. You may simply outline the strategy in a document. You might create a test matrix. Or, you might use a Kanban board.

No matter which Agile test methodology you take, your strategy should include:

- Purpose (defined by the user story).
- Objectives (test cases).
- Scope (what needs to be tested).
- Methods (how tests will be run).

Creating an Agile test strategy is easy when you're using the right testing tools — such as [Helix ALM](#).

Here's how you create an Agile test strategy in Helix ALM:

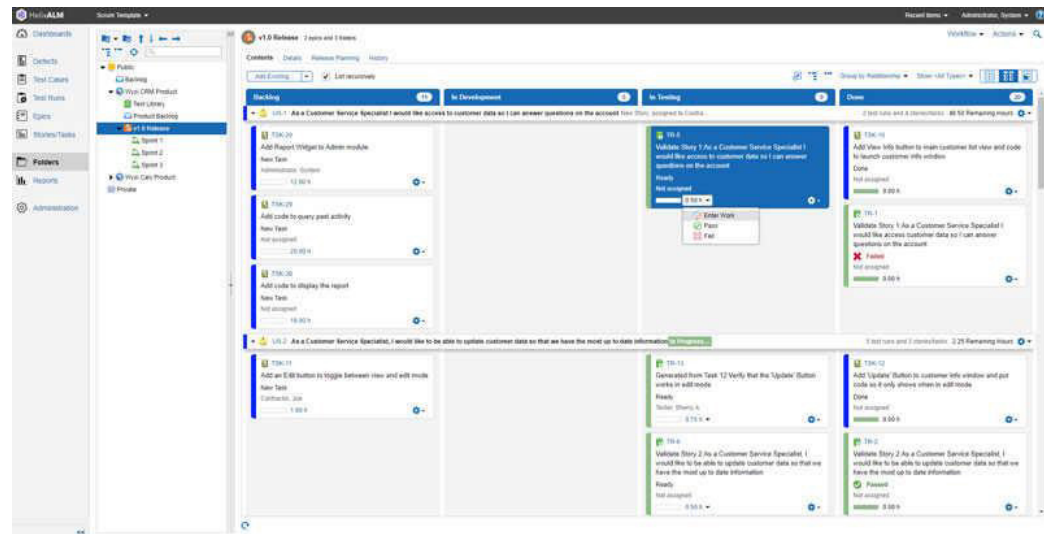
- Start with a purpose.
- Decide what you want to test.
- Write a user story, including acceptance criteria (your definition of done).
- Create a test case from that user story (automatically).
- Generate a test run to validate the user story.

Notes

An Agile Test Plan Example

Here's an example of an Agile test strategy in Helix ALM.

The backlog includes user stories — the items that need to be developed and tested. These are prioritized in order of importance. And this example shows development and testing in progress — and tests that have been completed.



Benefits of Agile Test Methodology

Agile testing improves product quality and enables development teams to release software on shorter cycles. And effective test case management helps Agile testers.

You'll find and fix errors faster. So, you'll lower the risk of finding a bug at the very end of testing — and missing a deadline.

You'll make customers happy by delivering regular releases. So, you'll improve customer retention rates.

And you'll manage the scope of each release. That helps you prioritize features for each iteration and deliver the most important ones first.

Of course, there will be some challenges with Agile testing. But the future of development is Agile. It's time for testers to embrace it.

Transition to Agile Testing

Make the transition to Agile testing easier. Download "Agile Development Methodologies for Testers".

1.4.1 Test Levels

Tests are categorised according to where they are added in the SDLC or the level of detail they include.

Unit testing, integration testing, system testing, and acceptance testing are the four tiers of testing in general. Levels of testing are used to make software testing more methodical and to quickly discover all feasible test cases at a given level.

Notes

For software testing, there are a variety of testing levels that can be used to examine behaviour and performance. These testing levels are intended to identify any gaps in the development lifecycle and to reconcile the various stages. Requirement collecting, analysis, design, coding or execution, testing, and deployment are all defined phases in SDLC models. All of these processes are subjected to various levels of software testing.

Levels of Testing

There are mainly four **Levels of Testing** in software testing:

1. **Unit Testing:** checks if software components are fulfilling functionalities or not.
2. **Integration Testing:** checks the data flow from one module to other modules.
3. **System Testing:** evaluates both functional and non-functional needs for the testing.
4. **Acceptance Testing:** checks the requirements of a specification or contract are met as per its delivery.

Each of these testing levels has a specific purpose. These testing level provide value to the software development lifecycle.

- Other Types of Testing:
- Regression Testing
- Buddy Testing
- Alpha Testing
- Beta Testing

1.4.2 Unit Testing

A Unit is a smallest testable portion of system or application which can be compiled, liked, loaded, and executed. This kind of testing helps to test each module separately.

The aim is to test each part of the software by separating it. It checks that component are fulfilling functionalities or not. This kind of testing is performed by developers.

The likelihood of successful transformation increases when the cost of the transition to individuals who must change is minimal. Resistance may occur as a result of the need for a wide range of new skills or disruptions in operations and work habits. A clear migration strategy developed by a skilled change agent with the assistance of the business sponsor is a critical component in increasing the certainty of transition costs.

Unit testing is a type of software testing in which individual software components are tested.

During the development of an application, unit testing of software products is performed. A single component can be a single function or a single method. The developer is usually the one who performs unit testing.

Unit testing is the initial level of testing performed before integration testing in the SDLC or V Model. Unit testing is a type of testing that is typically carried out by

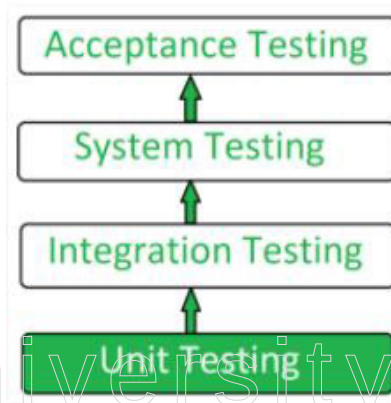
Notes

developers. Because developers are hesitant to test, quality assurance engineers also perform unit testing.

Objective of Unit Testing

The goal of unit testing is to:

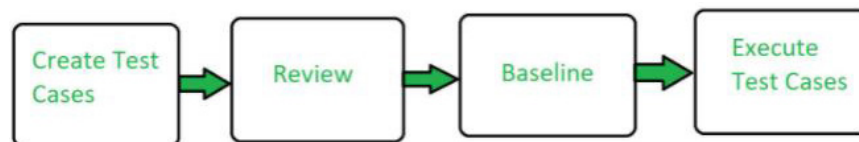
- To isolate a code section.
- To ensure that the code is correct.
- To put each function and technique to the test.
- To save money by fixing bugs early in the development cycle.
- To aid developers in comprehending the code base and allowing them to make changes rapidly.
- To aid with code reuse.



Types of Unit Testing:

There are 2 type of Unit Testing: Manual, and Automated.

Workflow of Unit Testing



Unit Testing Tools:

Here are some commonly used Unit Testing tools:

1. Jtest
2. Junit
3. NUnit
4. EMMA
5. PHPUnit

Advantages of Unit Testing:

- Unit testing enables developers to obtain a basic understanding of the unit API by learning what functionality a unit provides and how to use it.

- Unit testing allows the programmer to fine-tune the code and ensure that the module functions correctly.
- Unit testing allows you to test portions of a project without having to wait for others to finish.

1.4.3 Component, Module

Component

Component testing is a sort of software testing that examines the usability of individual components. Each component is also subjected to a behavioural examination in addition to the usability test. Each component must be in an autonomous and controllable condition in order to execute this type of testing. Each component of the software should be understandable to the average user.

Assume there are five components in a software application. Each component is individually tested by the tester as part of the development cycle before it is subjected to integration testing. It helps save time by detecting faults at an early stage in the cycle. For this type of testing, test structure tools or debugging tools are utilised since it is done by programmers on their own code with the help of an IDE. Defects discovered during component testing are repaired as soon as possible without keeping track of them.

Component testing is critical in identifying the problem. Component testing is done before integration testing to check that each component of the application is working properly and according to the requirements.

Component Testing's Goal

The goal of component testing is to:

- To test the system's input and output behaviour.
- To ensure that each component is usable.
- To evaluate the software's user-friendliness.
- To determine the state of each system component.

Module

A module is a section of a programme or a software component that contains one or more routines. A programme is made up of one or more independently built modules. An enterprise-level software application may have numerous discrete modules, each of which serves a distinct and distinct business process.

Modules make a programmer's job easier by allowing them to concentrate on only one aspect of the software application's functionality. Interfaces are often used to integrate modules into a programme (software).

1.4.4 Integration Testing - Big Bang and Bottom up Integration

Integration means combining. For Example, In this testing phase, different software modules are combined and tested as a group to make sure that integrated system is ready for system testing.

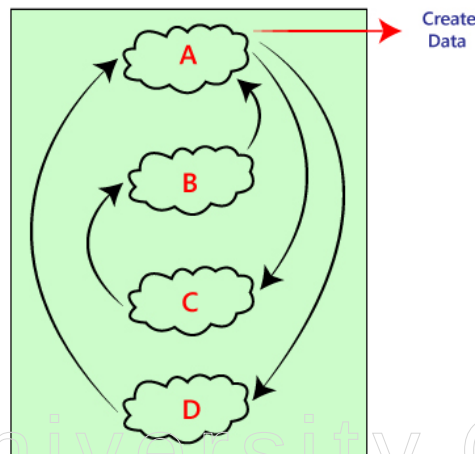
Notes

Integrating testing checks the data flow from one module to other modules. This kind of testing is performed by testers.

Big Bang Method

In this approach, testing is done via integration of all modules at once. It is convenient for small software systems, if used for large software systems identification of defects is difficult.

Since this testing can be done after completion of all modules due to that testing team has less time for execution of this process so that internally linked interfaces and high-risk critical modules can be missed easily.



Advantages:

- It is convenient for small size software systems.

Disadvantages:

- Identification of defects is difficult because finding the error where it came from is a problem, and we don't know the source of the bug.
- Small modules missed easily.
- Time provided for testing is very less.
- We may miss to test some of the interfaces.

Let us see examples for our better understanding of the non-incremental integrating testing or big bang method:

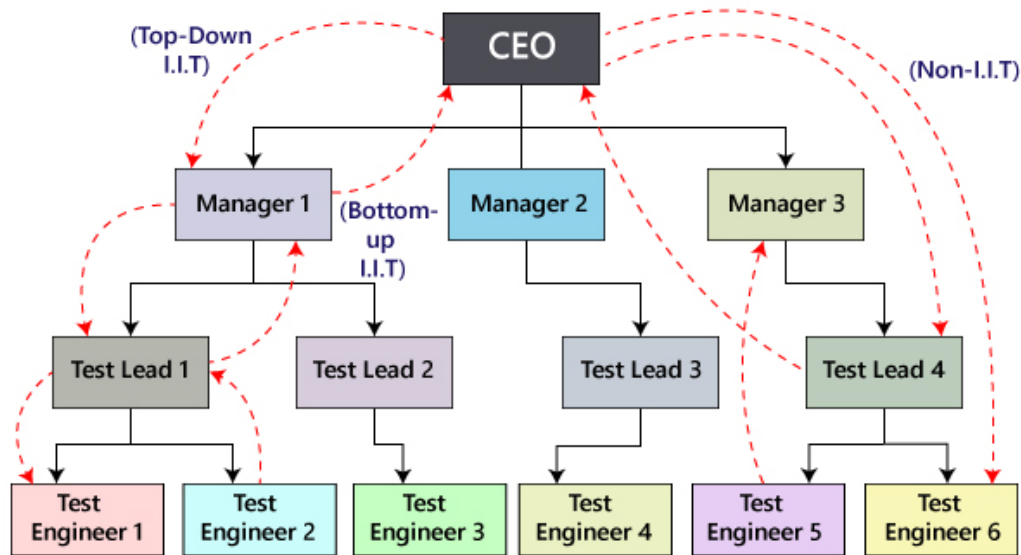
Example1 In the below example, the development team develops the application and sends it to the CEO of the testing team. Then the CEO will log in to the application and generate the username and password and send a mail to the manager. After that, the CEO will tell them to start testing the application.

Then the manager manages the username and the password and produces a username and password and sends it to the **test leads**. And the **test leads** will send it to the **test engineers** for further testing purposes. This order from the CEO to the test engineer is **top-down incremental integrating testing**.

Notes

In the same way, when the test engineers are done with testing, they send a report to the **test leads**, who then submit a report to the **manager**, and the manager will send a report to the **CEO**. This process is known as **Bottom-up incremental integration testing** as we can see in the below image:

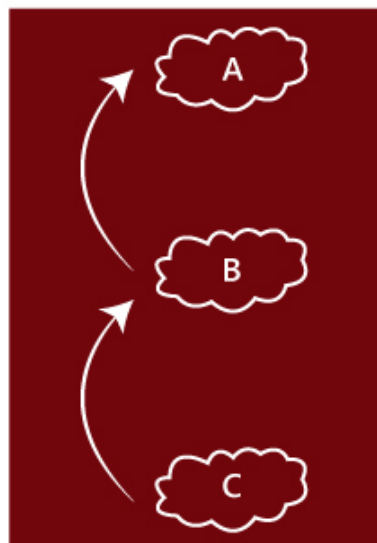
Note: The combination incremental integration testing (I.I.T) and non-incremental integration testing is known as sandwich testing.



Bottom-Up Method

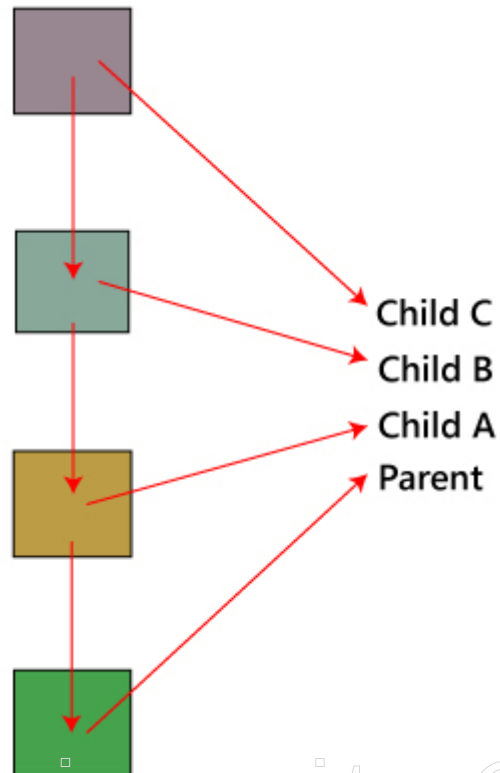
The bottom to up testing strategy deals with the process in which lower level modules are tested with higher level modules until the successful completion of testing of all the modules. Top level critical modules are tested at last, so it may cause a defect. Or we can say that we will be adding the modules from **bottom to the top** and check the data flow in the same order.

Bottom-up Approach



Notes

In the bottom-up method, we will ensure that the modules we are adding **are the parent of the previous one** as we can see in the below image:



Advantages

- Identification of defect is easy.
- Do not need to wait for the development of all the modules as it saves time.

Disadvantages

- Critical modules are tested last due to which the defects can occur.
- There is no possibility of an early prototype.

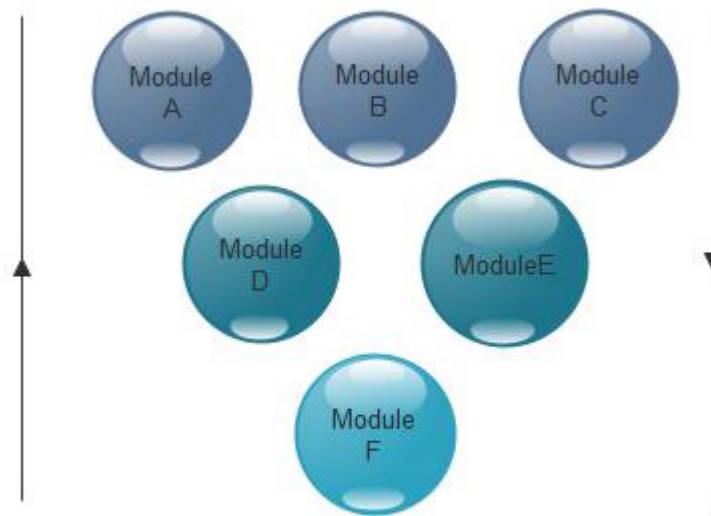
In this, we have one addition approach which is known as **hybrid testing**.

1.5.1 Top Down Integration and Mixed Integration

Hybrid Testing Method

In this approach, both **Top-Down** and **Bottom-Up** approaches are combined for testing. In this process, top-level modules are tested with lower level modules and lower level modules tested with high-level modules simultaneously. There is less possibility of occurrence of defect because each module interface is tested.

Notes



Hybrid Method

Advantages

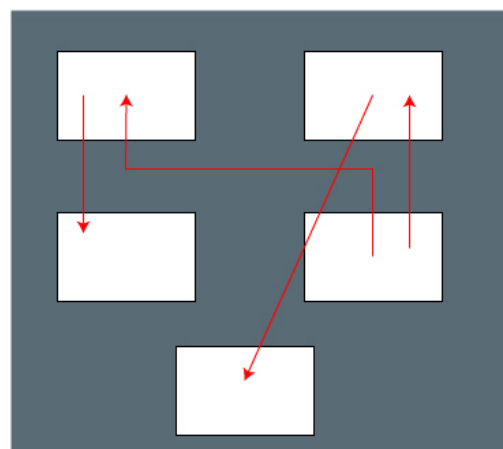
- The hybrid method provides features of both Bottom Up and Top-Down methods.
- It is most time reducing method.
- It provides complete testing of all modules.

Disadvantages

- This method needs a higher level of concentration as the process carried out in both directions simultaneously.
- Complicated method.

Non- incremental integration testing

We will go for this method, when the data flow is very complex and when it is difficult to find who is a parent and who is a child. And in such case, we will create the data in any module bang on all other existing modules and check if the data is present. Hence, it is also known as the **Big bang method**.

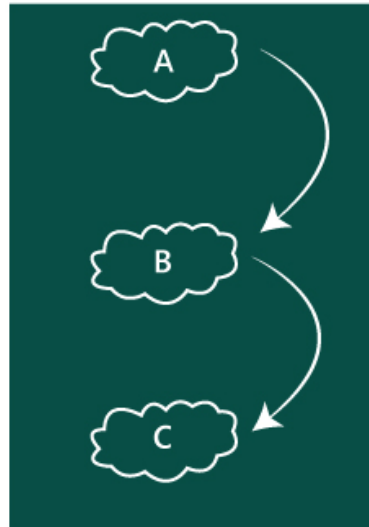


Notes

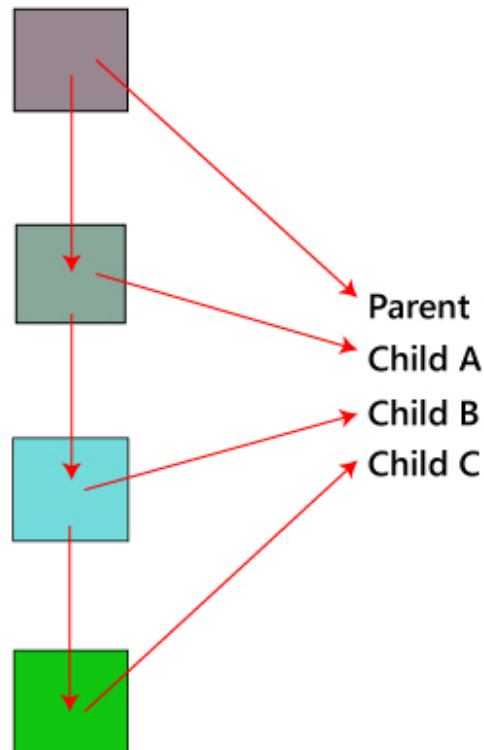
Top-Down Approach

The top-down testing strategy deals with the process in which higher level modules are tested with lower-level modules until the successful completion of testing of all the modules. Major design flaws can be detected and fixed early because critical modules tested first. In this type of method, we will add the modules incrementally or one by one and check the data flow in the same order.

Top-Down Approach



In the top-down approach, we will be ensuring that the module we are adding is the **child of the previous one** like **Child C is a child of Child B** and so on as we can see in the below image:



Advantages:

- Identification of defect is difficult.
- An early prototype is possible.

Disadvantages:

- Due to the high number of stubs, it gets quite complicated.
- Lower level modules are tested inadequately.
- Critical Modules are tested first so that fewer chances of defects.

1.5.2 System Testing

System testing is performed on a complete, integrated system. It allows checking system's compliance as per the requirements. It tests the overall interaction of components. It involves load, performance, reliability and security testing.

System testing most often the final test to verify that the system meets the specification. It evaluates both functional and non-functional need for the testing.

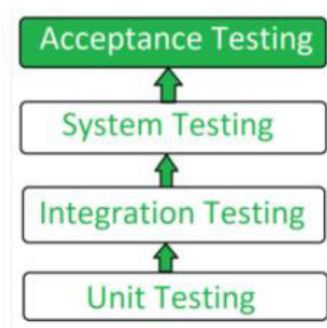
1.5.3 Acceptance Testing

Acceptance testing is a test conducted to find if the requirements of a specification or contract are met as per its delivery. Acceptance testing is basically done by the user or customer. However, other stockholders can be involved in this process.

Acceptance Testing is a type of software testing in which the acceptability of a system is assessed. The main goal of this test is to determine whether the system complies with the business requirements and whether it is suitable for delivery.

It is a type of formal testing that is done in accordance with user demands, requirements, and business processes to establish whether a system meets the acceptance criteria and to let users, customers, or other authorised organisations to decide whether or not to accept the system.

After System Testing and before making the system accessible for actual use, Acceptance Testing is the final phase of software testing.

**Types of Acceptance Testing:**

1. User Acceptance Testing (UAT): User acceptance testing is performed to see if a product is working properly for the user. Specific needs that are often used by consumers are primarily selected for testing. End-User Testing is another name for this.

Notes

Notes

2. **Business Acceptance Testing (BAT):** BAT is a method of determining whether or not a product fits the business's aims and objectives. BAT focuses primarily on company profits, which are difficult to achieve owing to changing market conditions and new technology, therefore the existing implementation may need to be updated, resulting in additional costs.

3. **Contract Acceptance Testing (CAT):** The acceptance test must be done within a set period after the product becomes live, and it must pass all of the acceptance use cases, according to the CAT.

A contract known as a Service Level Agreement (SLA) comprises provisions under which payment will be given only if the Product services meet all of the requirements, indicating that the contract has been met.

This contract may be signed prior to the product's launch. There should be a well-defined contract in terms of the testing length, areas of testing, conditions on difficulties that arise later in the process, and payments, among other things.

4. **Regulations Acceptance Testing (RAT):** RAT is used to see if a product is in violation of the norms and regulations set forth by the government of the country in which it is being released. This may be unintentional, but it will have a bad influence on the company.

In general, any product or application that is to be published on the market must comply with RAT, as each country or region has its own set of rules and regulations established by its regulatory organisations. If any country's or region's rules and regulations are broken, the product will not be released in that country or location. Only the product's vendors will be held directly liable if the product is released despite the breach.

5. **Operational Acceptance Testing (OAT):** Operational Acceptance Testing (OAT) is a non-functional testing method for determining a product's operational readiness. It primarily entails recovery, compatibility, maintainability, and reliability testing.

Before the product is released to production, OAT ensures its stability.

6. **Alpha Testing:** A specialised testers team known as alpha testers uses alpha testing to determine the product in a development testing environment.

7. **Beta Testing:** Beta testing is a method of evaluating a product by exposing it to real end-users, who are known as beta testers, in their environment. The users' feedback is gathered, and the flaws are corrected. This also aids in the enhancement of the product to provide a rich user experience.

Use of Acceptance Testing:

- To locate flaws that were overlooked during the functional testing phase.
- The quality of the product's development.
- A product is what the buyers genuinely require.
- The performance of the product and the user experience can both benefit from feedback.
- Reduce or eliminate difficulties that arise during production.

1.5.4 Generic Testing

“STBoX 3.0 provides a generic software testing process that enables project managers, test managers, and quality managers to plan, create, execute, and monitor testing in any project, suited to the project’s needs.”

The following are the essential features of the generic software testing process:

- Supports testing by enforcing a common way of working to design tests, document test execution, and log defects
- Centralizes information in a consistent way in a central repository
- Prevents data loss
- Provides project intelligence by delivering clear metrics related to test activities (test design, test execution) and the quality of the test object (e.g., defects raised).

Check your Understanding

1. A set of activities that ensure that software correctly implements a specific function.
 - a) verification
 - b) testing
 - c) implementation
 - d) validation
2. Validation is computer based.
 - a) True
 - b) False
3. _____ is done in the development phase by the debuggers.
 - a) Coding
 - b) Testing
 - c) Debugging
 - d) Implementation
4. Locating or identifying the bugs is known as _____.
 - a) Design
 - b) Testing
 - c) Debugging
 - d) Coding
5. Which defines the role of software?
 - a) System design
 - b) Design
 - c) System engineering
 - d) Implementation

Notes

Notes

6. What do you call testing individual components?
 - a) system testing
 - b) unit testing
 - c) validation testing
 - d) black box testing
7. A testing strategy that test the application as a whole.
 - a) Requirement Gathering
 - b) Verification testing
 - c) Validation testing
 - d) System testing
8. A testing strategy that tests the application in the context of an entire system.
 - a) System
 - b) Validation
 - c) Unit
 - d) Gray box
9. A _____ is tested to ensure that information properly flows into and out of the system.
 - a) module interface
 - b) local data structure
 - c) boundary conditions
 - d) paths
10. A testing conducted at the developer's site under validation testing.
 - a) alpha
 - b) gamma
 - c) lambda
 - d) unit

Summary

- After seeing all the significant differences between **bug, defect, error, fault, and failure**, we can say that the several issues and inconsistencies found throughout software are linked and dependent on each other.
- All the above terminology affects and change different parts of the software and differ from one another massively. However, all these differences between **bug, defect, errors, faults, and failures** slow down the software's excellence and performance.
- Defect Prevention plays a major and crucial role in software development process. It helps manage the quality of the software product in a "sooner and cheaper" manner with the help of the techniques listed above.

Notes

- It ensures that the problems get resolved early on without even making it to the application. It considers root cause finding as its primary means of identifying and eventually removing issues.
- To maintain the quality of software is the responsibility of the core management and entire team including project lead, client, and every team member.
- Quality is a multifaceted concept, and, therefore, its price may vary depending on the depth of this term's understanding by the product owner. In software development, there are many ways to save money at the expense of quality, but there are not many options for improving quality without overpaying. Compliance with the above methods helped us earn the reputation of a reliable outsourcing partner (which can be verified on Clutch) who cares about each client's budget and always seeks ways to optimize and save it without compromise on quality.
- Applying these thoughtful principles to your testing can help you become more efficient and focused, and can even help improve the quality of your overall testing strategy. Additionally, sometimes by applying one principle you will find others naturally fall into place. For example, testing early can help mitigate the "absence of errors" fallacy – if testers are involved at a requirements level, you can help ensure the software will match client requirements/expectations.
- Combine all these points, and you can really achieve maximum efficiency by utilising your time and endeavours effectively and economically.
- A level of software testing is a process where every unit or component of a software/system is tested.
- The primary goal of system testing is to evaluate the system's compliance with the specified needs.
- In Software Engineering, four main levels of testing are Unit Testing, Integration Testing, System Testing and Acceptance Testing.

Activity

1. With the help of a presentation state the importance of testing in software testing process.

Questions & Exercises

1. What is Human and errors?
2. Explain these terms Defects, Faults, Failures, Defect Rate and Reliability.
3. What do you understand by Defect Prevention also discuss Reduction and Containment.
4. What is Testing and Debugging?
5. Explain Software Quality.
6. What is requirement behavior and correctness
7. Explain Agile Methodology and Its Impact on testing,
8. Discuss the Test Levels: Unit, Component, Module, Integration, System, Acceptance, Generic

Notes

Glossary

- **Debugging:** The process in which developers identify, diagnose, and fix errors found. See also bug and defect.
- **Bug:** A slang term for fault, defect, or error. Originally used to describe actual insects causing malfunctions in mechanical devices that predate computers. The International Software Testing Qualifications Board (ISTQB) glossary explains that “a human being can make an error (mistake), which produces a defect (fault, bug) in the program code, or in a document. If a defect in code is executed, the system may fail to do what it should do (or do something it shouldn't), causing a failure. Defects in software, systems or documents may result in failures, but not all defects do so.” See also debugging.
- **Agile development:** A development method that emphasizes working in short iterations. Automated testing is often used. Requirements and solutions evolve through close collaboration between team members that represent both the client and supplier.
- **Acceptance testing:** The final test level. Conducted by users with the purpose to accept or reject the system before release.

Further Readings

- 1) Software Engineering, Sixth Edition, 2001, Ian Sommerville; Pearson Education.
- 2) Software Engineering – A Practitioner's Approach, Roger S. Pressman; McGraw-Hill International Edition.
- 3) An Integrated approach to Software Engineering, Pankaj Jalote; Narcosis Publishing House.
- 4) Software Quality, 1997, Mordechai Ben-Menachem and Garry S. Marliss; Thomson Learning.
- 5) Emden, D. and Blakeman, K. (1990). Going Automated. London: ASLIB.
- 6) Gates, H. (1985). Directory of Library and Information Retrieval Software for Microcomputers. Brook-field: Gower Pub. Co.
- 7) Levert, V.M. (1985). Library Specific Microcomputer Software. Library Software Review. 4 (May-June), 3, 5-38.
- 8) Nolan, J.M. (1984). Micro Software Evaluation. Torrance CA: Nolan Information Services. Poole, B.J. (1995). Education for an Information Age. Wisconsin: WCB Brown & Benchmark.
- 9) Rowley, Jennifer (1998). The Electronic Library, 4th Edition of Computers for Libraries. London: Library Association Publishing.
- 10) Tenopir, Carol (1984), Identification and Evaluation of Software for Micro-computerbased in-house Databases. Information Technology, and Libraries. March, 21-34.

Check Your Understanding – Answers

- 1) a
- 2) a
- 3) c
- 4) b
- 5) c
- 6) b
- 7) d
- 8) a
- 9) a
- 10) a

Notes

(c) Amity University Online

Module - 2: Approaches to Testing

Structure:

- 2.1.1 Static Testing, Structured Group Examinations
- 2.1.2 Static Analysis, Control flow & Data flow
- 2.1.3 Determining Metrics, Dynamic Testing
- 2.1.4 Black Box Testing:
- 2.2.1 Equivalence Class Partitioning
- 2.2.2 Examples on Equivalence Class Partitioning
- 2.2.3 Boundary Value Analysis
- 2.2.4 Examples on Boundary Value Analysis
- 2.3.1 State Transition Test, Cause Effect Graphing
- 2.3.2 Decision Table Technique and Used Case Testing
- 2.3.3 White Box Testing
- 2.3.4 Statement Coverage
- 2.4.1 Branch Coverage
- 2.4.2 Decision Coverage
- 2.4.3 Examples on Decision Coverage
- 2.4.4 MCC Coverage
- 2.5.1 MC/DC Coverage
- 2.5.2 System integration, Deployment testing, Beta Testing
- 2.5.3 Scalability testing, Reliability , Stress Testing
- 2.5.4 Acceptance testing: Acceptance criteria, test case selection and execution

Objectives:

After studying this chapter, you will be able to:

- Explain the meaning of Approaches to
- Discuss the Static Testing
- Explain the Structured Group Examinations
- Discuss the White Box Testing
- Discuss the Black Box Testing
- Understand the Acceptance testing

Introduction

We know the difference between Unit Testing and Integration Testing. But sometimes, we may mix the Unit Test with Integration Testing. In this article mainly

Notes

I'll focus on white box testing. If we know the unit testing principles and best practice technique, then it will be easy to maintain the quality of the application. It is also most important to know, how to write the test cases using Behavior Driven Development (BDD) during white-box testing.

White box testing vs. black box approach

- White box approach: Statement coverage; Decision coverage; Condition coverage; Path coverage.
- Black box approach: Field level check; Field level validation; User interface check; Function level check.

The purpose of including a variety of testing approaches into your development process is to ensure that your software can run in a variety of situations and on a variety of platforms. These are usually divided into functional and non-functional testing. Functional testing entails comparing the application to the company's requirements. It encompasses all test methods aimed at ensuring that each component of software operates as intended, based on use cases provided by the design team or business analyst. These tests are normally carried out in the following order:

- Unit testing
- Integration testing
- System testing
- Acceptance testing

Non-functional testing methods incorporate all test types focused on the operational aspects of a piece of software. These include:

- Performance testing
- Security testing
- Usability testing
- Compatibility testing

The key to releasing high quality software that can be easily adopted by your end users is to build a robust testing framework that implements both functional and non-functional software testing methodologies.

Unit Testing

The initial level of testing is unit testing, which is frequently carried out by the developers themselves. It is the process of ensuring that specific components of a piece of software are functional and work as intended at the code level. In a test-driven environment, developers will often write and execute the tests before passing the programme or feature to the test team. Manual unit testing is possible, but automating the process will shorten delivery cycles and increase test coverage. Debugging will be easier as a result of unit testing since flaws will be detected earlier in the testing process and will take less time to fix than if they were discovered later. Advanced testers and developers can use Test Left to shift left with the fastest test automation tool included in any IDE.

Notes

Integration Testing

After each unit has been extensively tested, it is combined with other units to form modules or components that execute certain jobs or activities. These are then put through integration testing as a group to ensure that the entire application behaves as expected (i.e., the interactions between units are seamless). User scenarios, such as logging into an application or opening files, are frequently used to structure these tests. Integrated tests are usually made up of a combination of automated functional and manual tests and can be performed by either developers or independent testers.

System Testing

System testing is a black box testing approach for evaluating the entire and integrated system to ensure that it meets stated requirements. Before the product is sent into production, the functionality of the software is tested from beginning to end by a distinct testing team from the development team.

Acceptance Testing

Acceptance testing comes after functional testing and is used to determine whether the final piece of software is ready to be delivered. It entails ensuring that the product fits all of the original business criteria as well as the needs of the end user. This necessitates internal and external testing, which means you'll need to get it into the hands of your end users as well as your QA team for beta testing. Beta testing is essential for obtaining genuine input from future customers and for addressing any remaining usability issues.

Performance Testing

Performance testing is a type of non-functional testing that determines how an application will operate under different circumstances. The idea is to see how responsive it is and how stable it is in real-world scenarios. There are four different forms of performance testing:

- **Load testing** is the process of putting increasing amounts of simulated demand on your software, application, or website to verify whether or not it can handle what it's designed to handle.
- **Stress testing** takes this a step further and is used to gauge how your software will respond at or beyond its peak load. The goal of stress testing is to overload the application on purpose until it breaks by applying both realistic and unrealistic load scenarios. With stress testing, you'll be able to find the failure point of your piece of software.
- **Endurance testing**, also known as soak testing, is used to analyze the behavior of an application under a specific amount of simulated load over longer amounts of time. The goal is to understand how your system will behave under sustained use, making it a longer process than load or stress testing (which are designed to end after a few hours). A critical piece of endurance testing is that it helps uncover memory leaks.
- **Spike testing** is a type of load test used to determine how your software will respond to substantially larger bursts of concurrent user or system activity over

varying amounts of time. Ideally, this will help you understand what will happen when the load is suddenly and drastically increased.

Notes

Security Testing

With the advent of cloud-based testing platforms and cyberattacks, the security of data used and stored in software is becoming increasingly important. Security testing is a non-functional software testing technique for determining whether a system's information and data are secure. By probing the application for flaws, the goal is to uncover gaps and security hazards in the system that could result in unauthorised access to or loss of information. There are several different variations of this testing procedure, each of which aims to check six basic security principles:

1. Integrity
2. Confidentiality
3. Authentication
4. Authorization
5. Availability
6. Non-repudiation

Usability Testing

Usability testing is a type of testing that evaluates an application's usability from the perspective of the end user, and it's frequently done during the system or acceptance testing stages. The purpose is to verify whether an application's apparent design and aesthetics suit the planned workflow for various tasks, such as logging into it. Usability testing is a wonderful approach for teams to see if certain functions, or the entire system, are easy to use.

Compatibility Testing

Compatibility testing determines how well a programme or piece of software will perform in various contexts. It's used to see if your product works with a variety of operating systems, platforms, browsers, and resolution settings. The goal is to ensure that your software's functionality is supported consistently in every environment that your end customers are likely to utilise.

Testing With Test Complete

Our powerful automated GUI testing tool, Test Complete, excels at compatibility and integration testing. It enables testing experts to speed up delivery cycles and enhance software quality by assisting QA teams in creating and running tests across desktop, mobile, and online apps. Test Complete includes built-in support for a variety of test environments, performance testing tool connectors, and developer-friendly SCM support, allowing you to seamlessly integrate it into your development process. Using Test Complete, you can create a robust testing framework that incorporates a wide range of software testing approaches.

Notes

2.1.1 Static Testing, Structured Group Examinations

Static Testing

We are going to understand **Static testing**, which is used to check the application without executing the code. And we also learn about **static Testing, why we use static Testing, how to perform it, a different technique for static Testing, advantages of static testing, and various Static Testing tools.**

Introduction to Static Testing

Static testing is a verification method for testing an application without having to implement the code. It's also a cost-effective method.

We will perform static testing in the early stages of development to avoid errors because it is easier to identify the origins of faults and rectify them.

In other words, we can say that **Static testing** can be done manually or with the help of tools to improve the quality of the application by finding the error at the early stage of development; that is also called the **verification process**.

We can do some of the following important activities while performing static testing:

- **Business requirement review**
- **Design review**
- **Code walkthroughs**
- **The test documentation review**

Note: Static testing is performed in the white box testing phase, where the developer checks every line of the code before giving it to the Test Engineer.

Static testing also helps us to identify those errors which may not be found by Dynamic Testing.

Why do we need to perform Static Testing?

- We can perform static testing to fulfill the below aspects:
- We can use static testing to improve the development productivity.
- If we performed static testing on an application, we could find the detects in the earlier stages and easily fix them.
- The usage of static testing will decrease the testing cost, development timescales, and time.

What are the different features we can test in Static Testing?

We can test the various testing activities in Static Testing, which are as follows:

- BRD [Business Requirements Document]
- Functional or system Requirements
- Unit Use Cases
- Prototype
- Prototype Specification Document

Notes

- Test Data
- DB Fields Dictionary Spreadsheet
- Documentation/Training Guides/ User Manual
- Test Cases/Test Plan Strategy Document
- Traceability Matrix Document
- Performance Test Scripts/Automation

When we performed Static Testing?

To perform static testing, we need to follow the below steps:

Step1: To review the design of the application entirely, we will perform the **inspection process**.

Step2: After that, we will use a checklist for each document under review to make sure that all reviews are covered completely.

We can also implement several activities while performing static testing, which are discussed in the following table:

Activities	Explanation
Architecture Review	• The architecture review activities contain all business-level processes such as network diagram, load balancing, server locations, protocol definitions, test equipment, database accessibility, etc.
Use Cases Requirements Validation	• It is used to authenticates all the end-user actions along with associated input and output. • If the use case is more comprehensive and detailed, we can make more precise and inclusive test cases.
Functional Requirements Validation	• The functional requirement validation activity is used to make sure that all necessary elements identify correctly. • And it also took care of the software, interface listings, network requirements, hardware, and database functionality.
Field Dictionary Validation	• In the field dictionary validation, we will test each field in the user interface specified to create field-level validation test cases. • And we can check the fields for error messages, minimum or maximum length, list values, etc.
Prototype/ Screen Mockup Validation	• The prototype validation activity contains the authentication of requirements and uses cases.

Why we need Static Testing?

We required Static testing whenever we encounter the following situation while testing an application or the software:

Notes

- **Dynamic Testing is time-consuming**
- **Flaws at earlier stages/identification of Bugs**
- **Dynamic Testing is expensive**
- **Increased size of the software**

Dynamic Testing is time-consuming

Static testing is required to test the programme because dynamic testing takes time, despite the fact that dynamic testing discovers the bug and provides some information about it.

Flaws at earlier stages/identification of Bugs

We cannot totally rely on dynamic testing when developing software because it detects faults or defects in the application/software at a later stage, requiring the programmer's time and effort to remedy the bugs.

Dynamic Testing is expensive

Because dynamic testing is more expensive than static testing, we must undertake static testing on the software product. Because the test cases were built in the early phases, including them in dynamic testing is costly.

We also need to maintain the test case's implementation and validation, which takes a lot of work from the test engineers.

Increased size of the software

Every time we test the software, we increase the size of the software product, which we can't handle due to the decline in code coverage productivity.

That is why, earlier in the software development life cycle, static testing is required to eliminate bugs or faults.

Objectives of Static testing

The main objectives of performing static testing is as below:

- Static testing will decrease the flaws in production.
- Static testing will identify, anticipate and fix the bugs at the earliest possible time.
- It is used to save both time and cost.
- It is used to identify defects in the early stage of SDLC, where we can fix them easily.

Some useful points for Successful Static Testing Process

The following guidelines help us to perform a successful static testing process in Software testing.

- We can train participants with examples.
- The testing cost and time can decrease if we delete the major delays in test execution.

Notes

- We can retain the process formal as per the project culture.
- We can make emphasis only on things that matter.
- As we know that a software walkthrough and review are usually merged into peer reviews; therefore, we can plan explicitly and track the review activities.
- We can resolve the client's problems.

Static Testing Techniques

Static testing techniques offer a great way to enhance the quality and efficiency of software development. The Static testing technique can be done in two ways, which are as follows:

- **Review**
- **Static Analysis**

Review

In static testing, the **review** is a technique or a process implemented to find the possible bugs in the application. We can easily identify and eliminate faults and defects in the various supporting documents such as SRS [**Software Requirements Specifications**] in the review process.

In other words, we can say that a **review** in Static Testing is that where all the team members will understand about the project's progress.

In static testing, **reviews** can be divided into **four different parts**, which are as follows:

- **Informal reviews**
- **Walkthroughs**
- **Technical/peer review**
- **Inspections**



Notes

Let's understand them in detail one by one:

- **Informal reviews**
- In **informal review**, the document designer places the contents in front of viewers, and everyone gives their view; therefore, bugs are acknowledged in the early stage.
- **Walkthrough** Generally, the **walkthrough review** is used to performed by a skilled person or expert to verify the bugs. Therefore, there might not be problem in the development or testing phase.
- **Peer review**
- In **Peer review**, we can check one another's documents to find and resolve the bugs, which is generally done in a team.
- **Inspection** In review, the **inspection** is essentially verifying the document by the higher authority, **for example**, the **verification of SRS [software requirement specifications] document**.

Static Analysis

Another Static Testing technique is **static analysis**, which is used to contain the assessment of the code quality, which is established by developers.

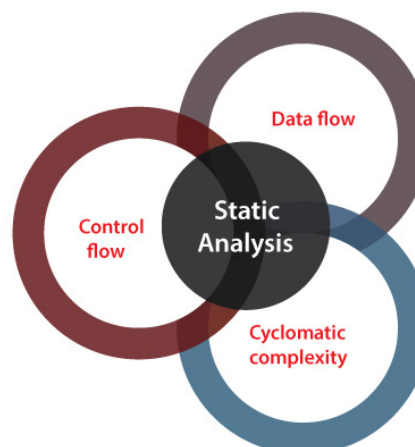
We can use the different tools to perform the code's analysis and evaluation of the same.

In other words, we can say that developers' developed code is analysed with some tools for structural bugs, which might cause the defects.

The **static analysis** will also help us to identify the below errors:

- **Dead code**
- **Unused variables**
- **Endless loops**
- **Incorrect syntax**
- **Variable with undefined value**

In static testing, **Static Analysis** can be further classified into **three parts**, which are as discuss below:



Notes

Data Flow: In static analysis, the data flow is connected to the stream processing.

Control Flow: Generally, the control flow is used to specify how the commands or instructions are implemented.

Cyclomatic Complexity: It is the measurement of the program's complexity, which is mostly linked to the number of independent paths in the control flow graph of the program.

Tools used for Static Testing

In static testing, we have several tools in the market, but here we are discussing the most commonly used tools, which are as follow:

- **CheckStyle**
- **SourceMeter**
- **Soot**

CheckStyle

It is a development tool that is used to help the developers write Java code, which follows a coding standard. The **CheckStyle tool** automates the process of checking Java code.

It is a highly configured tool, which is made to support almost any coding standard. The **Google Java Style, Sun code conventions** are those configuration files, which is supported by Check Style.

Feature of CheckStyle

Following are the most common features of CheckStyle:

- It can check various characteristics of our source code.
- The CheckStyle code has the capability to verify the code layout and formatting issues.
- It can also help to identify the method design problems, class design problems.

SourceMeter

It is an advanced tool for the specific static source code analysis of various programming languages such as **C/C++, C#, Java, Python, and RPG projects**.

With the **SourceMeter tool's** help, we can easily identify the vulnerable spots of a system under development from the source code.

The free version with partial functionality of SourceMeter can be accessible for all programming languages.

In SourceMeter, we can use the output of the analysis, the quality of the analysed source code to enhance and developed both the short and long term in a directed way.

Notes



Feature of SourceMeter

The most commonly used features of the SourceMeter tool are as follows:

- It provides the most precise coding error detection.
- The SourceMeter tool will provide a deep static code analysis.
- It improved the user interface with the help of third-party integration.
- It will provide platform-independent command-line tools.

Soot

It is a **Java optimization framework**, which means that it is a framework for analyzing and transforming Java and Android applications where we can test the following aspects:

- Named module and modular jar files.
- Automatic modules, which means the modules are repeatedly created from jars in the module-path.
- Exploded modules
- Resolving modules in Soot's

And a Soot can also produce possibly transformed code in the various output formats such as **Android bytecode, Java bytecode Jasmin, and Jimple**.

Advantages of Static Testing

The advantages of static testing are as follows:

- **Improved Product quality**
- Static testing will enhance the product quality because it identifies the flaws or bugs in the initial stage of software development.
- **Improved the efficiency of Dynamic testing**
- The usage of Static testing will improve Dynamic Testing efficiency because the code gets cleaner and better after executing Static Testing.
- As we understood above, static Testing needs some efforts and time to generate and keep good quality test cases.
- **Reduced SDLC cost**
- The Static Testing reduced the SDLC cost because it identifies the bugs in the earlier stages of **the software development life cycle**. So, it needs less hard work and time to change the product and fix them.
- **Immediate evaluation & feedback**

Notes

- The static testing provides us immediate evaluation and feedback of the software during each phase while developing the software product.
- **Exact location of bug is traced**
- When we perform the static testing, we can easily identify the bugs' exact location compared to the dynamic Testing.

Overview

In the Static testing section, we have learned the following topics:

- Static testing is used to identify the faults in the early stage of the Software development cycle [SDLC].
- We have understood that Static Testing is not a replacement for dynamic Testing because both testing identify different bug types.
- We have understood the objective of Static Testing.
- In static testing, the reviews are the productive approach to test the application because the reviews help identify the bugs and recognize the design flaws, missing requirements, and non-maintainable code, etc.
- We have understood several static testing tools, which help us enhance testing performance for the software product.

Structured Group Examinations

A structured walkthrough is a static testing technique used by a group of peers to review and debate the technical components of the software development process in an orderly manner. A structured walkthrough's major goal is to uncover problems in order to improve the product's quality.

Structured walkthroughs are rarely utilised for technical conversations or to explore remedies to issues that have been discovered. As previously stated, the goal is to detect rather than repair problems. After the tour is completed, the author of the output is in charge of resolving any difficulties.

Benefits:

- Saves time and money by detecting and correcting faults early in the product lifecycle.
- This adds value by allowing reviewers with a variety of technical backgrounds and experience to contribute.
- It informs the project management team about the development process's progress.
- It raises awareness of various development or maintenance approaches that can help participants advance professionally.

Structured Walkthrough Participants:

- Author - The Author of the document under review.
- Presenter - The presenter usually develops the agenda for the walkthrough and presents the output being reviewed.

Notes

- Moderator - The moderator facilitates the walkthrough session, ensures the walkthrough agenda is followed, and encourages all the reviewers to participate.
- Reviewers - The reviewers evaluate the document under test to determine if it is technically accurate.
- Scribe - The scribe is the recorder of the structured walkthrough outcomes who records the issues identified and any other technical comments, suggestions, and unresolved questions.

2.1.2 Static Analysis, Control flow & Data flow

Static Analysis

Under pressure development teams can benefit from static analysis. On-time delivery of high-quality releases was required. Coding and compliance requirements must be met. And making mistakes isn't an option.

That is why static analysis tools are used by development teams. Static analysis and the advantages of utilising a static code analysis tool are discussed in this article.

"Static analysis is best described as a method of debugging that involves automatically inspecting source code prior to the execution of a programme."

Static Code Analysis?

Static code analysis is a method of debugging that involves reviewing source code prior to running a programme. It is accomplished by comparing a set of code against a set (or several sets) of coding rules.

Along with source code analysis, the terms static code analysis and static analysis are frequently interchanged.

This type of investigation looks for flaws in source code that could lead to vulnerabilities. This can, of course, be accomplished by manual code reviews. However, utilising automated tools is far more efficient.

Static analysis is frequently used to ensure that code requirements, such as MISRA, are followed. It's also frequently utilised to meet industry requirements, such as ISO 26262.

Static Analysis vs Dynamic Analysis

So, what's the difference between static analysis and dynamic analysis?

Both types detect defects. The big difference is *where* they find defects in the development lifecycle.

Static analysis identifies defects *before* you run a program (e.g., between coding and unit testing).

After you run a programme, dynamic analysis detects flaws (e.g., during unit testing). Some coding flaws, however, may go undetected during unit testing. As a result, static code analysis can detect errors that dynamic testing may overlook.

Notes

Developers typically employ static analysis tools as part of the development and component testing process. The crucial point is that the tool, not the code (or other artefact), is executed, and the source code we're interested in is used as input data to the tool.

Developers are the primary users of these tools.

Static analysis tools are a natural extension of compiler technology, and some compilers even include static analysis capabilities. Before investing in a more advanced static analysis tool, it's a good idea to see what's available from existing compilers or development environments.

Static analysis can be used for things other than software code, such as static analysis of requirements or static analysis of websites (for example, to assess for proper use of accessibility tags or the following of HTML standards).

Static code analysis tools can aid developers in comprehending the structure of their code and can also be used to enforce coding standards.

Features or characteristics of static analysis tools are:

- To calculate metrics such as cyclomatic complexity or nesting levels (which can help to identify where more testing may be needed due to increased risk).
- To enforce coding standards.
- To analyze structures and dependencies.
- Help in code understanding.
- To identify anomalies or defects in the code.

Benefits of Static Analysis Tools

There are several benefits of static code analysis tools — especially if you need to comply with an industry standard.

The best static code analysis tools offer speed, depth, and accuracy.

Speed

It takes time for developers to do manual code reviews. Automated tools are much faster.

Static code checking addresses problems early on. And it pinpoints exactly where the error is in the code. So, you'll be able to fix those errors faster. Plus, coding errors found earlier are less costly to fix.

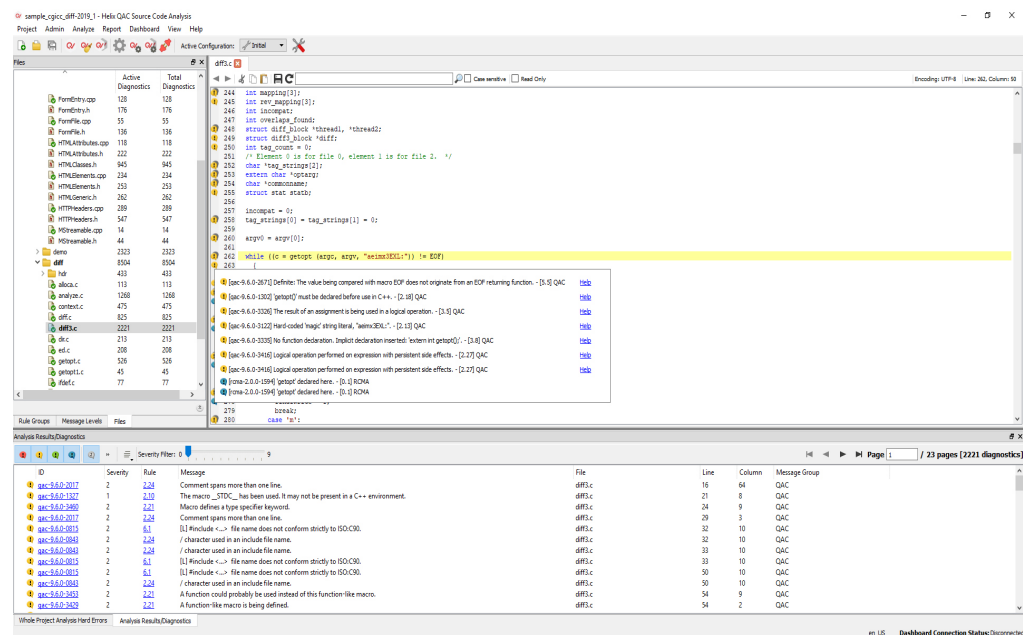
Depth

Testing can't cover every possible code execution path. But a static code analyser can.

It checks the code as you work on your build. You'll get an in-depth analysis of where there might be potential problems in your code, based on the rules you've applied.

Notes

Here's an example of in-depth code analysis in Helix QAC.



An example of code analysis in Helix QAC.

Accuracy

Manual code reviews are prone to human error. Automated tools are not.

They scan every line of code to identify potential problems. This helps you ensure the highest-quality code is in place — before testing begins. After all, when you're complying with a coding standard, quality is critical.

Control flow and Data flow

Data Flow Testing is a type of structural testing. It is a method that is used to find the test paths of a program according to the locations of definitions and uses of variables in the program. It has nothing to do with data flow diagrams.

It is concerned with:

- Statements where variables receive values,
- Statements where these values are used or referenced.

To illustrate the approach of data flow testing, assume that each statement in the program assigned a unique statement number. For a statement number S- $DEF(S) = \{X \mid \text{statement } S \text{ contains the definition of } X\}$

$USE(S) = \{X \mid \text{statement } S \text{ contains the use of } X\}$

If a statement is a loop or if condition then its DEF set is empty and USE set is based on the condition of statement s.

Data Flow Testing uses the control flow graph to find the situations that can interrupt the flow of the program. Reference or define anomalies in the flow of the data are detected at the time of associations between values and variables. These anomalies are:

Notes

- A variable is defined but not used or referenced,
- A variable is used but never defined,
- A variable is defined twice before it is used

Advantages of Data Flow Testing:

Data Flow Testing is used to find the following issues-

- To find a variable that is used but never defined,
- To find a variable that is defined but never used,
- To find a variable that is defined multiple times before it is use,
- Deallocating a variable before it is used.

Disadvantages of Data Flow Testing

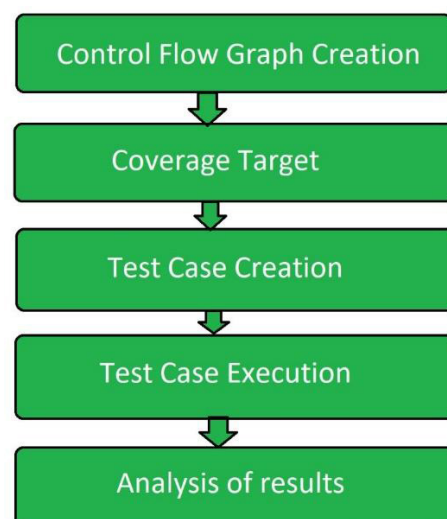
- Time consuming and costly process
- Requires knowledge of programming languages

Control flow testing is a type of software testing that uses program's control flow as a model. Control flow testing is a structural testing strategy. This testing technique comes under white box testing. For the type of control flow testing, all the structure, design, code and implementation of the software should be known to the testing team.

This type of testing method is often used by developers to test their own code and own implementation as the design, code and the implementation is better known to the developers. This testing method is implemented with the intention to test the logic of the code so that the user requirements can be fulfilled. Its main application is to relate the small programs and segments of the larger programs.

Control Flow Testing Process:

Following are the steps involved into the process of control flow testing:



- **Control Flow Graph Creation:**

From the given source code, a control flow graph is created either manually or by using the software.

Notes

- **Coverage Target:**

A coverage target is defined over the control flow graph that includes nodes, edges, paths, branches etc.

- **Test Case Creation:**

Test cases are created using control flow graphs to cover the defined coverage target.

- **Test Case Execution:**

After the creation of test cases over coverage target, further test cases are executed.

- **Analysis:**

Analyze the result and find out whether the program is error free or has some defects.

Control Flow Graph:

Control Flow Graph is a graphical representation of control flow or computation that is done during the execution of the program. Control flow graphs are mostly used in static analysis as well as compiler applications, as they can accurately represent the flow inside of a program unit. Control flow graph was originally developed by Frances E. Allen.

Cyclomatic Complexity:

Cyclomatic Complexity is the quantitative measure of the number of linearly independent paths in it. It is a software metric used to describe the complexity of a program. It is computed using the Control Flow Graph of the program.

$$M = E - N + 2P$$

Advantages of Control flow testing:

- It detects almost half of the defects that are determined during the unit testing.
- It also determines almost one-third of the defects of the whole program.
- It can be performed manually or automated as the control flow graph that is used can be made by hand or by using software also.

Disadvantages of Control flow testing:

- It is difficult to find missing paths if program and the model are done by same person.
- Unlikely to find spurious features.

2.1.3 Determining Metrics, Dynamic Testing

Determining Metrics

Software Testing metrics are quantitative steps taken to evaluate the software testing process's quality, performance, and progress. This helps us to accumulate reliable data about the software testing process and enhance its efficiency. This

will allow developers to make proactive and precise decisions for upcoming testing procedures.

Notes

What is a metric in software testing metrics?

A Metric is a degree to which a system or its components retains a given attribute. Testers don't define a metric just for the sake of documentation. It serves greater purposes in software testing. For example, developers can apply a metric to assume the time it takes to develop software. It can also be assigned to determine the numbers of new features and modifications, etc., added to the software.

Importance of Software Testing Metrics

As mentioned, test metrics are crucial to measuring the quality and performance of the software. With proper software testing metrics, developers can–

- Determine what types of improvements are required to deliver a defect-free, quality software
- Make sound decisions about the subsequent testing phases, such as scheduling upcoming projects as well as estimating the overall cost of those projects
- Evaluate the current technology or process and check whether it needs further modifications

Types of software testing metrics

There are three types of software testing metrics–

- **Process Metrics:** Process metrics define the characteristics and execution of a project. These characteristics are essential to the improvement and maintenance of the process in the SDLC (Software Development Life Cycle).
- **Product Metrics:** Product metrics define the size, design, performance, quality, and complexity of a product. By using these characteristics, developers can enhance their software development quality.
- **Project Metrics:** Project Metrics determine the overall quality of a project. It is used to calculate costs, productivity, defects and estimate the resource and deliverables of a project.

It is incredibly vital to identify the correct testing metrics for the process. Few factors to consider–

- Choose your target audiences wisely before preparing the metrics
- Define the goal behind designing the metrics
- Prepare metrics by considering the specific requirements of the project
- Evaluate the financial gain behind each metrics
- Pair the metrics with the project lifestyle phase that delivers optimum output

Software testing can be further divided into manual and automated testing.

In manual testing, the test is performed by QA analysts in a step-by-step process. Meanwhile, in automated testing, tests are executed with the help of test automation

Notes

frameworks, tools, and software. Both manual and automated testing has its strength and weakness.

Manual testing is a slow process, but it allows testers to handles complex scenarios.

The most significant advantage of automated testing is that it enables testers to run more testing in less time, covering a substantial level of permutations, which is nearly impossible to calculate manually.

Types of Manual Test Metrics

Manual Test Metrics are of two types–

Base Metrics

Base metrics are data collected by analysts during test case development and execution. These metrics are submitted to test leads and project managers by preparing a project status report. It is quantified by using calculated metrics –

- Number of test cases
- Number of test cases executed

Calculated Metrics

Calculated metrics are derived using data from base metrics. The test lead gathers these data and converts them to more meaningful information for tracking the progress of projects at the module level, tester level, etc.

It comprises a significant part of SDLC and empowers developers to make vital improvements in software.

Most used Metrics

Below are the types of metrics, popularly used by developers and testers

- **Defect metrics:** This metric allows developers to understand the various quality aspects of software, including functionality, performance, installation stability, usability, compatibility, etc.
- **Defects finding rate:** It is used to identify the pattern of defects during a specific timeframe
- **Defect severity:** It enables the developer to understand how the defect is going to impact the quality of the software.
- **Defect cause:** It is used to understand the root cause of the defect.
- **Test Coverage:** It defines how many test cases are assigned to the program. This metric ensures the testing is conducted to its full completion. It further aids in checking the code flow and test functionalities.
- **Defect fixing time:** It determines the amount of time it takes to resolve a defect
- **Test case efficiency:** It tells the efficiency rate of test cases in finding defects
- **Schedule adherence:** Its primary motive is to figure out the time difference between the planned schedule and the actual time of executing a schedule.

Test Metrics Life Cycle

The life cycle of test metrics consists of four stages–

- **Analysis:** In this stage, developers identify the required metrics and define them.
- **Communicate:** Once metrics are identified, developers have to explain their importance to stakeholders and the testing team.
- **Evaluation:** This stage includes quantifying and verifying the data. Then testers have to use the data to calculate the value of the metric.
- **Report:** Once the evaluation process is finished, the development team needs to create a report including a detailed summary of the conclusion. Then the report is distributed among stakeholders and relevant representatives. The stakeholders then give their feedback after reading the information carefully.

Dynamic Testing

Dynamic Testing is a kind of software testing technique using which the dynamic behaviour of the code is analysed.

For Performing dynamic, testing the software should be compiled and executed and parameters such as memory usage, CPU usage, response time and overall performance of the software are analysed.

Dynamic testing involves testing the software for the input values and output values are analysed. Dynamic testing is the Validation part of Verification and Validation.

Dynamic Testing Techniques

The Dynamic Testing Techniques are broadly classified into two categories. They are:

- Functional Testing
- Non-Functional Testing

Levels of Dynamic Testing

There are various levels of Dynamic Testing Techniques. They are:

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

Dynamic testing is one of the most important parts of Software testing, which is used to analyse the code's dynamic behaviour.

The dynamic testing is working with the software by giving input values and verifying if the output is expected by implementing a specific test case that can be done manually or with an automation process.

The dynamic testing can be done when the code is executed in the run time environment. It is a **validation process** where functional testing [unit, integration,

Notes

Notes

system, and user acceptance testing] and non-functional testing [Performance, usability, compatibility, recovery and security testing] are performed.

Is a **verification** process, whereas **dynamic testing** is a **validation** process, and together they help us to deliver a cost-effective quality Software product?

Why do we need to perform Dynamic Testing?

We can easily understand how to implement dynamic testing during the STLC (Software Testing Life Cycle).

if we consider the characteristics accessible by dynamic testing.

Using dynamic testing, the team can verify the software's critical features, but some of those can be left without any assessment. And they can also affect the functioning, reliability, and performance of the software product.

Hence, we can perform **Dynamic testing** to fulfil the various below aspects:

- We will perform dynamic testing to check whether the application or software is working fine during and after installing the application without any error.
- We can perform dynamic testing to verify the efficient behavior of the software.
- The software should be compiled and run if we want to perform dynamic testing.
- Generally, Dynamic Testing is implemented to define the dynamic behavior of code.
- The team implements the code to test the software application's performance in a run-time environment during the dynamic testing process.
- It makes sure that the concurrency of the software application with the customer's potentials, needs and the end-user.
- It is an operative technique to measure the effect of several environmental stresses on the software application like **network, hardware**

Characteristic of Dynamic Testing

For understanding the fundamental of the **software testing techniques**, we have to learn their attribute and several other components. Hence, following are some of the important characteristics of **dynamic testing**:

- It is implemented throughout the **validation stage** of software testing.
- Dynamic Testing is done by performing the program.
- Both functional and non-functional testing include in dynamic testing.
- In Dynamic testing, we can easily identify the bugs for the particular software.
- It helps the team in validating the reliability of the software application.
- Unlike static testing, the team implements the software's code to get expected outputs in dynamic testing.
- Dynamic testing is performed directly on the software application as compare to other testing techniques.
- Dynamic testing is a more formal testing approach for different testing activities such as **test execution, coverage consideration, reporting and test case identification.**

Dynamic testing Process

Generally, dynamic testing follows a set process when the approach and test implementation performances are decided, and the team can move to execute the different testing activities.

With the help of this process, the team can find any irregularity from the approaches and strategies and help us display all the testing steps.

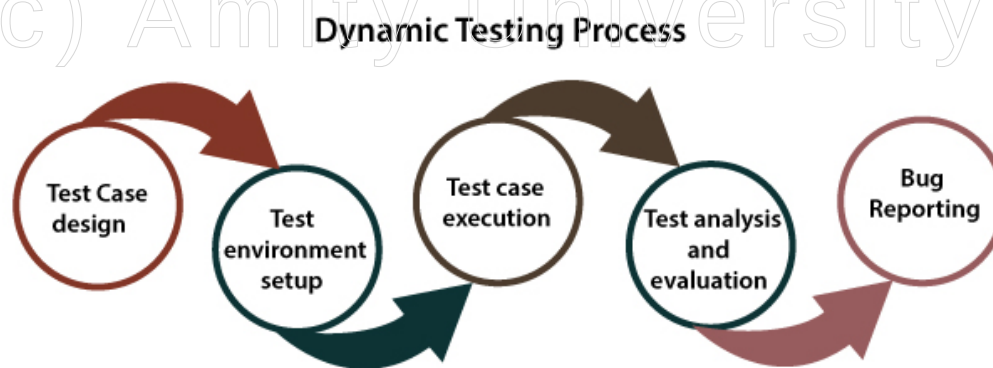
In the **STLC**, the process of Dynamic Testing involves different functions. And all the functions in the dynamic testing process rely on the conclusion of the earlier task in the testing process.

The Dynamic testing process will complete in the following steps:

- **Test case design**
- **Test environment step-up**
- **Test case execution**
- **Test analysis and evaluation**
- **Bug Reporting**

The actual Dynamic Testing Process begins from Test Case Design in the software testing

life cycle. Now, we discuss each step one by one to get complete knowledge of the dynamic testing process.



Step1: Test Case Design

In the first step of the dynamic testing process, the teams will design the test cases. Here, we are creating those test cases that depend on the requirements and scope of testing established before the start of the project.

In this step, we can **originate the test conditions, obtain the test cases, extract the coverage items, and identify those features that need to be tested.**

Step2: Environment Setup

In the **test environment phase**, we will make sure that the testing environment should always be parallel to the production environment because the testing is implemented directly on the software product.

Notes

Notes

In this step, the dynamic testing process's main objective is to install the test environment, which helps us succeed in the test machines.

Step3: Test Execution

Once we successfully install the test environment, we will execute those test cases prepared in the primary stage of the dynamic testing process.

Step4: Analysis & Evaluation

After executing the test cases, we will analyse and evaluate the outcomes derived from the testing. And we will compare those outcomes with the expected results.

If expected and actual results are not the same according to executing, we will consider those test cases as fail, and log the Bug in the bug repository.

Step5: Bug Reporting

After analysing the test cases, we will be reported and recorded any bugs or defects between the actual result and expected result to the concerned person. And the concerned person will make sure that the issue has been solved and delivering a quality product.

Example of Dynamic Testing

Let us take one sample example where we understand how dynamic testing will work.

So, for this, we will understand the login module of any application, such as **www.Twitter.com**.

Suppose we want to create one new account with a secure password, so we need to follow some pre-defined rules in the **password field**.

And the password should have **eight characters long, capital letters and at least one special character**.

If we are testing this functionality, we would take all the input conditions to test this and then verify the output.

We can also put the non-working constraints, such as input a **4-character password**, and validate if there is an error occurred or not.

Types of Dynamic testing

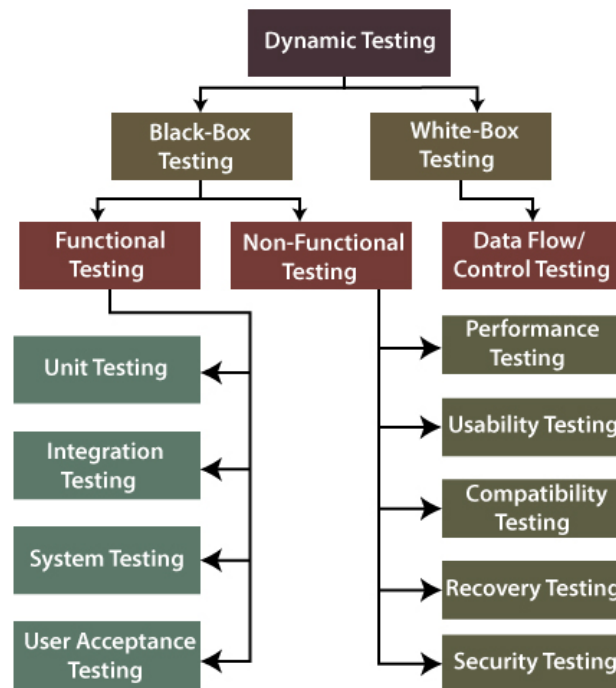
Dynamic testing divided into two different testing approach, which are as follows:

- **White-box testing**
- **Black-box testing**

Both the testing techniques will help us execute the dynamic testing process efficiently as they play an important role in verify the performance and quality of the software.

Let's understand them one by one in detail and also see the below diagram of it:

Notes



2.1.4 Black Box Testing:

Black Box Testing is a software testing method in which the functionalities of software applications are tested without having knowledge of internal code structure, implementation details and internal paths. Black Box Testing mainly focuses on input and output of software applications and it is entirely based on software requirements and specifications. It is also known as Behavioural Testing.

The above Black-Box can be any software system you want to test. For Example, an operating system like Windows, a website like Google, a database like Oracle or even your own custom application. Under Black Box Testing, you can test these applications by just focusing on the inputs and outputs without knowing their internal code implementation. Consider the following video tutorial-

How to do Blackbox Testing

Here are the generic steps followed to carry out any type of Black Box Testing.

- Initially, the requirements and specifications of the system are examined.
- Tester chooses valid inputs (positive test scenario) to check whether SUT processes them correctly. Also, some invalid inputs (negative test scenario) are chosen to verify that the SUT is able to detect them.
- Tester determines expected outputs for all those inputs.
- Software tester constructs test cases with the selected inputs.
- The test cases are executed.
- Software tester compares the actual outputs with the expected outputs.
- Defects if any are fixed and re-tested.

Notes

Types of Black Box Testing

There are many types of Black Box Testing but the following are the prominent ones –

- **Functional testing** – This black box testing type is related to the functional requirements of a system; it is done by software testers.
- **Non-functional testing** – This type of black box testing is not related to testing of specific functionality, but non-functional requirements such as performance, scalability, usability.
- **Regression testing** – Regression Testing is done after code fixes, upgrades or any other system maintenance to check the new code has not affected the existing code.

Tools used for Black Box Testing:

Tools used for Black box testing largely depends on the type of black box testing you are doing.

- For Functional/ Regression Tests you can use – QTP, Selenium
- For Non-Functional Tests, you can use – LoadRunner, Jmeter

Black Box Testing Techniques

Following are the prominent Test Strategy amongst the many used in Black box Testing

- **Equivalence Class Testing:** It is used to minimize the number of possible test cases to an optimum level while maintains reasonable test coverage.
- **Boundary Value Testing:** Boundary value testing is focused on the values at boundaries. This technique determines whether a certain range of values are acceptable by the system or not. It is very useful in reducing the number of test cases. It is most suitable for the systems where an input is within certain ranges.
- **Decision Table Testing:** A decision table puts causes and their effects in a matrix. There is a unique combination in each column.

Comparison of Black Box and White Box Testing:

Black Box Testing	White Box Testing
the main focus of black box testing is on the validation of your functional requirements.	White Box Testing (Unit Testing) validates internal structure and working of your software code
Black box testing gives abstraction from code and focuses on testing effort on the software system behavior.	To conduct White Box Testing, knowledge of underlying programming language is essential. Current day software systems use a variety of programming languages and technologies and its not possible to know all of them.
Black box testing facilitates testing communication amongst modules	White box testing does not facilitate testing communication amongst modules

Black Box Testing and Software Development Life Cycle (SDLC)

Black box testing has its own life cycle called Software Testing Life Cycle (STLC) and it is relative to every stage of Software Development Life Cycle of Software Engineering.

- **Requirement** – This is the initial stage of SDLC and in this stage, a requirement is gathered. Software testers also take part in this stage.
- **Test Planning & Analysis** – Testing Types applicable to the project are determined. A Test Plan is created which determines possible project risks and their mitigation.
- **Design** – In this stage Test cases/scripts are created on the basis of software requirement documents
- **Test Execution**– In this stage Test Cases prepared are executed. Bugs if any are fixed and re-tested.

Techniques Used in Black Box Testing

Decision Table Technique	Decision Table Technique is a systematic approach where various input combinations and their respective system behavior are captured in a tabular form. It is appropriate for the functions that have a logical relationship between two and more than two inputs.
Boundary Value Technique	Boundary Value Technique is used to test boundary values, boundary values are those that contain the upper and lower limit of a variable. It tests, while entering boundary value whether the software is producing correct output or not.
State Transition Technique	State Transition Technique is used to capture the behavior of the software application when different input values are given to the same function. This applies to those types of applications that provide the specific number of attempts to access the application.
All-pair Testing Technique	All-pair testing Technique is used to test all the possible discrete combinations of values. This combinational method is used for testing the application that uses checkbox input, radio button input, list box, text box, etc.
Cause-Effect Technique	Cause-Effect Technique underlines the relationship between a given result and all the factors affecting the result. It is based on a collection of requirements.
Equivalence Partitioning Technique	Equivalence partitioning is a technique of software testing in which input data divided into partitions of valid and invalid values, and it is mandatory that all partitions must exhibit the same behavior.
Error Guessing Technique	Error guessing is a technique in which there is no specific method for identifying the error. It is based on the experience of the test analyst, where the tester uses the experience to guess the problematic areas of the software.
Use Case Technique	Use case Technique used to identify the test cases from the beginning to the end of the system as per the usage of the system. By using this technique, the test team creates a test scenario that can exercise the entire software based on the functionality of each function from start to end.

Notes

Notes

2.2.1 Equivalence Class Partitioning

Equivalence partitioning is a technique of software testing in which input data is divided into partitions of valid and invalid values, and it is mandatory that all partitions must exhibit the same behaviour. If a condition of one partition is true, then the condition of another equal partition must also be true, and if a condition of one partition is false, then the condition of another equal partition must also be false. The principle of equivalence partitioning is, test cases should be designed to cover each partition at least once. Each value of every equal partition must exhibit the same behaviour as other.

The equivalence partitions are derived from requirements and specifications of the software. The advantage of this approach is, it helps to reduce the time of testing due to a smaller number of test cases from infinite to finite. It is applicable at all levels of the testing process.

2.2.2 Examples on Equivalence Class Partitioning

Assume that there is a function of a software application that accepts a particular number of digits, not greater and less than that particular number. For example, an OTP number which contains only six digits, less or more than six digits will not be accepted, and the application will redirect the user to the error page.

1. OTP Number = 6 digits

INVALID	INVALID	VALID	VALID
1 Test case	2 Test case	3 Test case	
DIGITS >=7	DIGITS <=5	DIGITS = 6	DIGITS = 6
93847262	9845	456234	451483

Let's see one more example.

A function of the software application accepts a 10 digit mobile number.

2. Mobile number = 10 digits

Notes

INVALID 1 Test case	INVALID 2 Test case	VALID 3 Test case	VALID
DIGITS >=11	DIGITS <=9	DIGITS = 10	DIGITS =10
93847262219	984543985	9991456234	9893451483

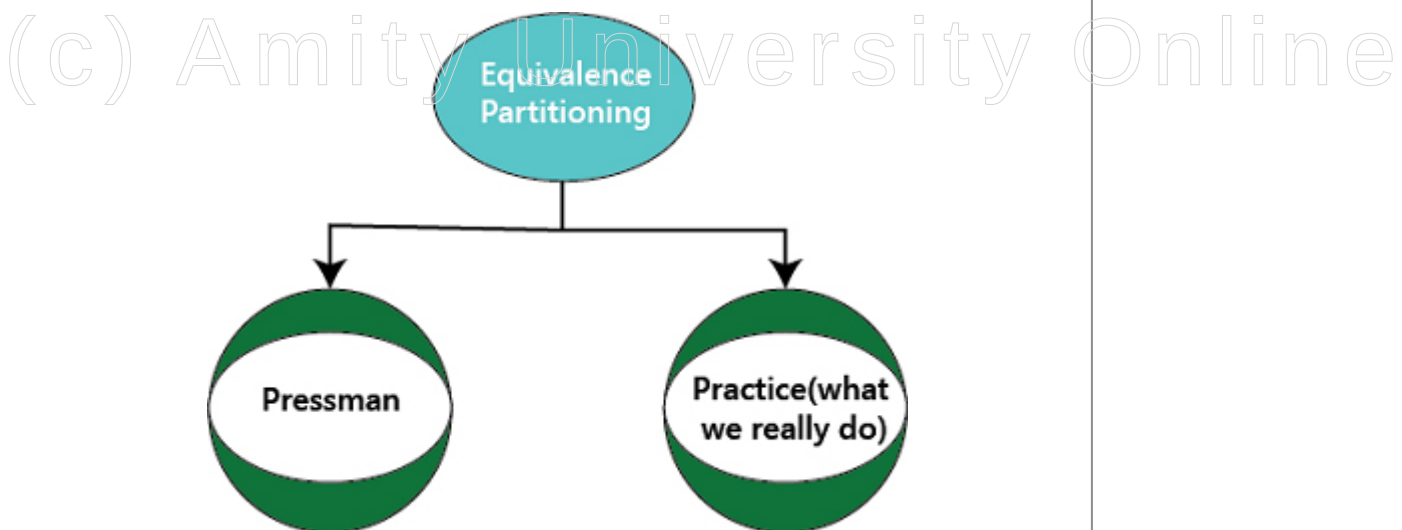
In both examples, we can see that there is a partition of two equally valid and invalid partitions, on applying valid value such as OTP of six digits in the first example and mobile number of 10 digits in the second example, both valid partitions behave same, i.e. redirected to the next page.

Another two partitions contain invalid values such as 5 or less than 5 and 7 or more than 7 digits in the first example and 9 or less than 9 and 11 or more than 11 digits in the second example, and on applying these invalid values, both invalid partitions behave same, i.e. redirected to the error page.

We can see in the example, there are only three test cases for each example and that is also the principal of equivalence partitioning which states that this method intended to reduce the number of test cases.

How we perform equivalence partitioning

We can perform equivalence partitioning in two ways which are as follows:



Let us see how pressman and general practice approaches are going to use in different conditions:

Condition 1

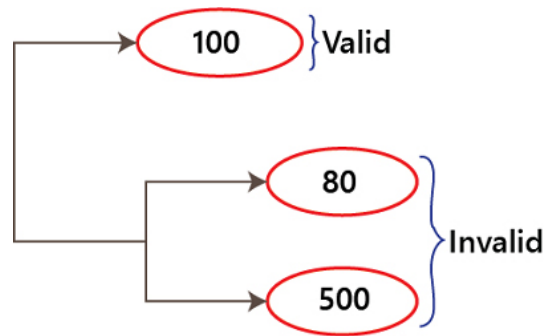
If the requirement is **a range of values**, then derive the test case for **one valid** and **two invalid** inputs.

Here, the **Range of values** implies that whenever we want to identify the range values, we go for equivalence partitioning to achieve the minimum test coverage. And after that, we go for error guessing to achieve maximum test coverage.

Notes

According to pressman:

For example, the Amount of test field accepts a Range (100-400) of values:

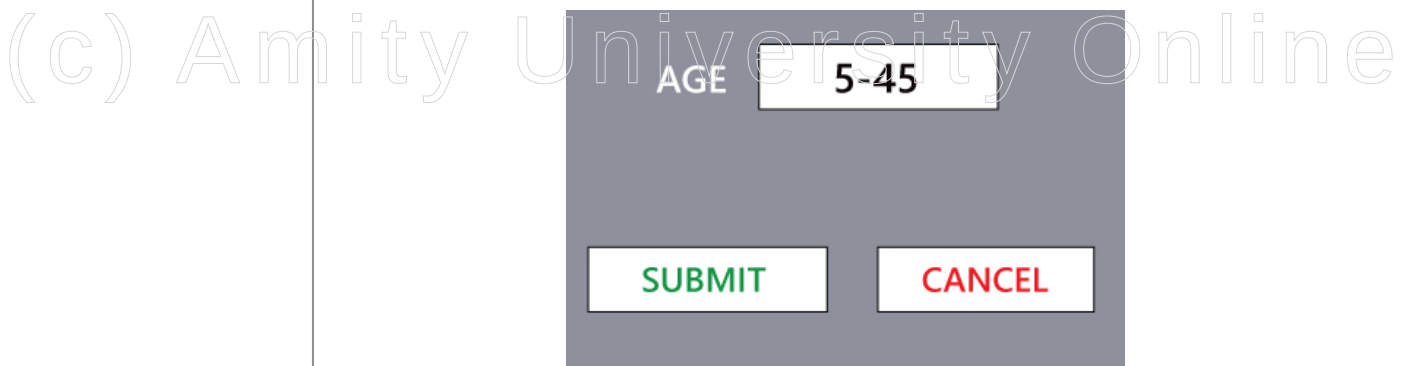


According to General Practice method:

Whenever the requirement is Range + criteria, then divide the Range into the internals and check for all these values.

For example:

In the below image, the pressman technique is enough to test for an age text field for one valid and two invalids. But, if we have the condition for insurance of ten years and above are required and multiple policies for various age groups in the age text field, then we need to use the practice method.



Condition2

If the requirement is a **set of values**, then derive the test case for **one valid** and **two invalid** inputs.

Here, **Set of values** implies that whenever we have to test a set of values, we go for **one positive** and **two negative** inputs, then we moved for error guessing, and we also need to verify that all the sets of values are as per the requirement.

Example 1

Based on the Pressman Method

If the Amount Transfer is (100000-700000)

Then for, 1 lakh → Accept

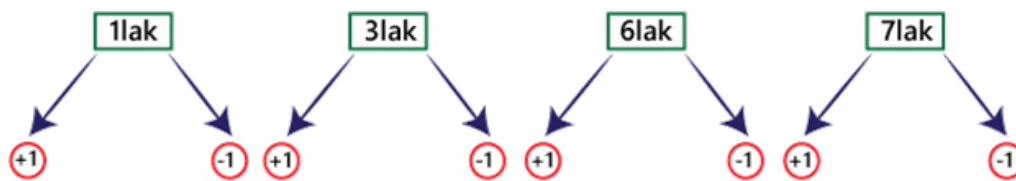
And according to General Practice method

The Range + Percentage given to 1 lakh - 7 lakh

Like: 1lak - 3lak → 5.60%

3lak - 6lak → 3.66%

6lak - 7lak → Free



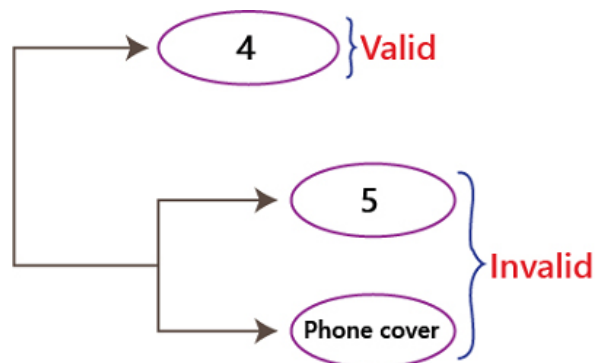
If we have things like loans, we should go for the general practice approach and separate the stuff into the intervals to achieve the minimum test coverage.

Example 2

if we are doing online shopping, mobile phone product, and the different **Product ID -1,4,7,9**

Here, 1 → phone covers 4 → earphones 7 → charger 9 → Screen guard

And if we give the product id as 4, it will be accepted, and it is one valid value, and if we provide the product id as **5 and phone cover**, it will not be accepted as per the requirement, and these are the two invalid values.

**Notes**

Notes

Condition 3

If the requirement id **Boolean (true/false)**, then derive the test case for both true/false values.

The Boolean value can be true and false for the radio button, checkboxes.

For example

Serial no	Description	Input	Expected	Note
1	Select valid	NA	True	---
2	Select invalid	NA	False	Values can be change based according to the requirement.
3	Do not select	NA	Do not select anything, error message should be displayed	We cannot go for next question
4	Select both	NA	We can select any radio button	Only one radio button can be selected at a time.

Note:

In **Practice** method, we will follow the below process:

Here, we are testing the application by deriving the below inputs values:

80	500	1000	2000	3000	4000	5000	6000	7000
----	-----	------	------	------	------	------	------	------

Let us see one program for our better understanding.

1. If (amount < **500** or > 7000)
2. {
3. Error Message

Notes

4. }
5. If (amount is between 500 & 3000)
6. {
7. deduct 2%
8. }
9. if (amount > 3000)
10. {
11. deduct 3%
12. }

When the pressman technique is used, the first two conditions are tested, but if we use the practice method, all three conditions are covered.

We don't need to use the practice approach for all applications. Sometime we will use the pressman method also.

But, if the application has much precision, then we go for the practice method.

If we want to use the practice method, it should follow the below aspects:

- It should be product-specific
- It should be case-specific
- The number of divisions depends on the precision (2% and 3 % deduction)

Advantages and disadvantages of Equivalence Partitioning technique

Following are pros and cons of equivalence partitioning technique:

Advantages	disadvantages
It is process-oriented	All necessary inputs may not cover.
We can achieve the Minimum test coverage	This technique will not consider the condition for boundary value analysis.
It helps to decrease the general test execution time and also reduce the set of test data.	The test engineer might assume that the output for all data set is right, which leads to the problem during the testing process.

2.2.3 Boundary Value Analysis

Boundary value analysis is one of the widely used case design technique for black box testing. It is used to test boundary values because the input values near the boundary have higher chances of error.

Whenever we do the testing by boundary value analysis, the tester focuses on, while entering boundary value whether the software is producing correct output or not.

Boundary values are those that contain the upper and lower limit of a variable. Assume that, age is a variable of any function, and its minimum value is 18 and the maximum value is 30, both 18 and 30 will be considered as boundary values.

Notes

The basic assumption of boundary value analysis is, the test cases that are created using boundary values are most likely to cause an error.

There is 18 and 30 are the boundary values that's why tester pays more attention to these values, but this doesn't mean that the middle values like 19, 20, 21, 27, 29 are ignored. Test cases are developed for each and every value of the range.

Name Enter Your Name

Age Between 18 to 30

Adhar Number of 12 Digits

Address Enter Your Address

Testing of boundary values is done by making valid and invalid partitions. Invalid partitions are tested because testing of output in adverse condition is also essential.

2.2.4 Examples on Boundary Value Analysis

Let's understand via practical:

Imagine, there is a function that accepts a number between 18 to 30, where 18 is the minimum and 30 is the maximum value of valid partition, the other values of this partition are 19, 20, 21, 22, 23, 24, 25, 26, 27, 28 and 29. The invalid partition consists of the numbers which are less than 18 such as 12, 14, 15, 16 and 17, and more than 30 such as 31, 32, 34, 36 and 40. Tester develops test cases for both valid and invalid partitions to capture the behavior of the system on different input conditions.

12 14 15 16 17 **18** 20 22 24 25 26 28 **30** 31 32 34 36 38 40

-----|-----|-----

Invalid Partition Valid Partition Invalid Partition

Invalid test cases	Valid test cases	Invalid test cases
11, 13, 14, 15, 16, 17	18, 19, 24, 27, 28, 30	31, 32, 36, 37, 38, 39

The software system will be passed in the test if it accepts a valid number and gives the desired output, if it is not, then it is unsuccessful. In another scenario, the

software system should not accept invalid numbers, and if the entered number is invalid, then it should display error message.

If the software which is under test, follows all the testing guidelines and specifications then it is sent to the releasing team otherwise to the development team to fix the defects.

2.3.1 State Transition Test, Cause Effect Graphing

State Transition Test

The general meaning of state transition is, different forms of the same situation, and according to the meaning, the state transition method does the same. It is used to capture the behavior of the software application when different input values are given to the same function.

We all use the ATMs, when we withdraw money from it, it displays account details at last. Now we again do another transaction, then it again displays account details, but the details displayed after the second transaction are different from the first transaction, but both details are displayed by using the same function of the ATM. So the same function was used here but each time the output was different, this is called state transition. In the case of testing of a software application, this method tests whether the function is following state transition specifications on entering different inputs.

This applies to those types of applications that provide the specific number of attempts to access the application such as the login function of an application which gets locked after the specified number of incorrect attempts. Let's see in detail, in the login function we use email and password, it gives a specific number of attempts to access the application, after crossing the maximum number of attempts it gets locked with an error message.

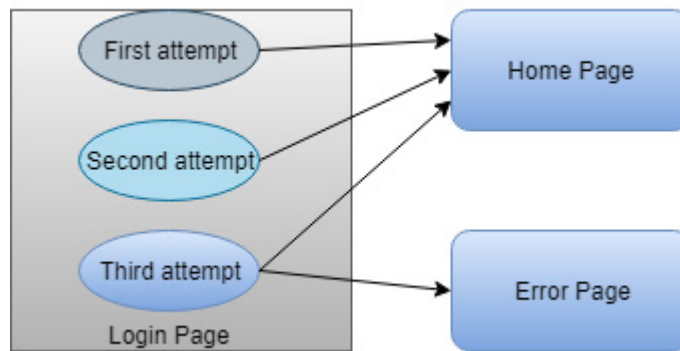
The image shows a login form with a purple background. At the top, it says "Login Here" in bold black text. Below this, there are two input fields. The first is labeled "Email" in bold black text, and the input field contains the placeholder text "Enter Your Email". The second is labeled "Password" in bold black text, and the input field contains the placeholder text "10 characters". At the bottom of the form, there are two buttons: a blue button labeled "Back" and a blue button labeled "Login".

Let see in the diagram:

There is a login function of an application which provides a maximum three number of attempts, and after exceeding three attempts, it will be directed to an error page.

Notes

Notes



State transition table

STATE	LOGIN	VALIDATION	REDIRECTED
S1	First Attempt	Invalid	S2
S2	Second Attempt	Invalid	S3
S3	Third Attempt	Invalid	S5
S4	Home Page		
S5	Error Page		

In the above state transition table, we see that state S1 denotes first login attempt. When the first attempt is invalid, the user will be directed to the second attempt (state S2). If the second attempt is also invalid, then the user will be directed to the third attempt (state S3). Now if the third and last attempt is invalid, then the user will be directed to the error page (state S5).

But if the third attempt is valid, then it will be directed to the homepage (state S4).

Let's see state transition table if third attempt is valid:

STATE	LOGIN	VALIDATION	REDIRECTED
S1	First Attempt	Invalid	S2
S2	Second Attempt	Invalid	S3
S3	Third Attempt	Valid	S4
S4	Home Page		
S5	Error Page		

By using the above state transition table we can perform testing of any software application. We can make a state transition table by determining desired output, and then exercise the software system to examine whether it is giving desired output or not.

Cause Effect Graphing

Cause-effect graph comes under the black box testing technique which underlines the relationship between a given result and all the factors affecting the result. It is used to write dynamic test cases.

Notes

The dynamic test cases are used when code works dynamically based on user input. For example, while using email account, on entering valid email, the system accepts it but, when you enter invalid email, it throws an error message. In this technique, the input conditions are assigned with causes and the result of these input conditions with effects.

Cause-Effect graph technique is based on a collection of requirements and used to determine minimum possible test cases which can cover a maximum test area of the software.

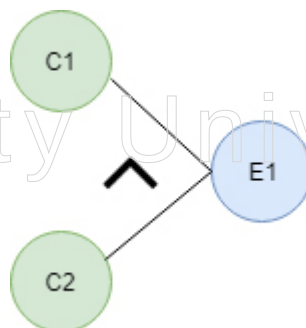
The main advantage of cause-effect graph testing is, it reduces the time of test execution and cost.

This technique aims to reduce the number of test cases but still covers all necessary test cases with maximum coverage to achieve the desired application quality.

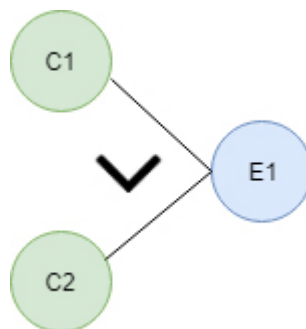
Cause-Effect graph technique converts the requirements specification into a logical relationship between the input and output conditions by using logical operators like AND, OR and NOT.

Notations used in the Cause-Effect Graph

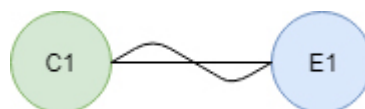
AND - E1 is an effect and C1 and C2 are the causes. If both C1 and C2 are true, then effect E1 will be true.



OR - If any cause from C1 and C2 is true, then effect E1 will be true.

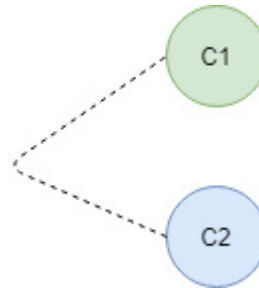


NOT - If cause C1 is false, then effect E1 will be true.



Mutually Exclusive - When only one cause is true.

Notes



Let's try to understand this technique with some examples:

Situation:

The character in column 1 should be either A or B and in the column 2 should be a digit. If both columns contain appropriate values, then update is made. If the input of column 1 is incorrect, i.e. neither A nor B, then message X will be displayed. If the input in column 2 is incorrect, i.e. input is not a digit, then message Y will be displayed.

- A file must be updated, if the character in the first column is either "A" or "B" and in the second column it should be a digit.
- If the value in the first column is incorrect (the character is neither A nor B) then message X will be displayed.
- If the value in the second column is incorrect (the character is not a digit) then message Y will be displayed.

<i>Column 1</i>	<i>Column 2</i>
<i>Correct value - A or B</i>	<i>Correct value- Any digit</i>
<i>Incorrect value - Any character except A or B</i>	<i>Incorrect value- Any character except digit</i>

Now, we are going to make a Cause-Effect graph for the above situation:

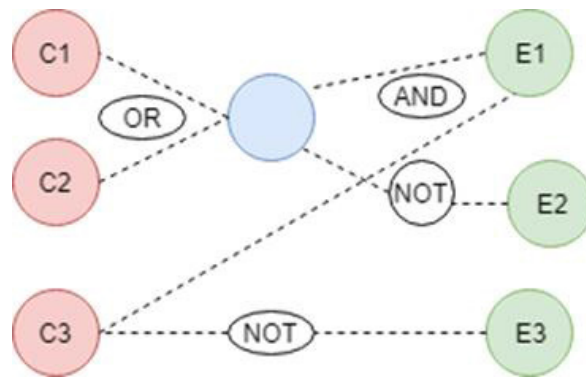
Causes are:

- C1 - Character in column 1 is A
- C2 - Character in column 1 is B
- C3 - Character in column 2 is digit!

Effects:

- E1 - Update made (C1 OR C2) AND C3
- E2 - Displays Message X (NOT C1 AND NOT C2)
- E3 - Displays Message Y (NOT C3)

Where AND, OR, NOT are the logical gates.



Effect E1- Update made- The logic for the existence of effect E1 is «**(C1 OR C2) AND C3**». For **C1 OR C2**, any one from C1 and C2 should be true. For logic **AND C3** (Character in column 2 should be a digit), C3 must be true. In other words, for the existence of effect E1 (Update made) any one from C1 and C2 but the C3 must be true. We can see in graph cause C1 and C2 are connected through OR logic and effect E1 is connected with AND logic.

Effect E2 - Displays Message X - The logic for the existence of effect E2 is «**NOT C1 AND NOT C2**» that means both C1 (Character in column 1 should be A) and C2 (Character in column 1 should be B) should be false. In other words, for the existence of effect E2 the character in column 1 should not be either A or B. We can see in the graph, **C1 OR C2** is connected through NOT logic with effect E2.

Effect E3 - Displays Message Y- The logic for the existence of effect E3 is «**NOT C3**» that means cause C3 (Character in column 2 is a digit) should be false. In other words, for the existence of effect E3, the character in column 2 should not be a digit. We can see in the graph, **C3** is connected through NOT logic with effect E3.

So, it is the cause-effect graph for the given situation. A tester needs to convert causes and effects into logical statements and then design cause-effect graph. If function gives output (effect) according to the input (cause) so, it is considered as defect free, and if not doing so, then it is sent to the development team for the correction.

2.3.2 Decision Table Technique and Used Case Testing

Decision Table Technique

A Decision Table is a table that shows the relationship between inputs and rules, cases, and test conditions. It's an extremely useful tool for both complicated software testing and requirements management. The decision table allows testers to examine all conceivable combinations of requirements for testing and to immediately discover any conditions that were overlooked. True(T) and False(F) values are used to signify the criteria.

Decision table testing is a type of software testing that examines how a system responds to various input combinations. This is a methodical methodology in which the various input combinations and the accompanying system behaviour (Output) are

Notes

Notes

tabulated. That's why it's also known as a Cause-Effect table, because it captures both causes and effects for improved test coverage.

Example - How to make Decision Base Table for Login Screen

Let's create a decision table for a login screen.

The condition is simple: the user will be redirected to the homepage if they give the right username and password. An error warning will appear if any of the input is incorrect.

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username (T/F)	F	T	F	T
Password (T/F)	F	F	T	T
Output (E/H)	E	E	E	H

Legend:

T – Correct username/password

F – Wrong username/password

E – Error message is displayed

H – Home screen is displayed

Interpretation:

Case 1 – Username and password both were wrong. The user is shown an error message.

Case 2 – Username was correct, but the password was wrong. The user is shown an error message.

Case 3 – Username was wrong, but the password was correct. The user is shown an error message.

Case 4 – Username and password both were correct, and the user navigated to homepage

We can generate two situations while converting this to a test case.

Notes

- Enter the correct username and password and click Login; the expected consequence is that the user will be directed to the homepage.

And one from the scenario below.

- If the user types in the wrong username and password and then clicks Login, the user should receive an error message.
- When you provide the proper username and incorrect password and click Login, the user should receive an error message.
- If the user types the wrong username and password and then clicks Login, the user should receive an error message.
- Because they are both essentially testing the same rule.

Importance

Decision table testing is important because it allows you to test multiple combinations of circumstances and ensures that complicated business logic is thoroughly tested. Decision table testing provides good coverage and the representation is simple, making it easy to read and apply for testing the behaviour of a vast number of inputs where system behaviour vary with each set of input.

Boundary value and analogous partition are two more similar strategies used in Software Engineering to provide better coverage. They're utilised when a system's behaviour is consistent over a wide number of inputs. However, in a system with varied system behaviour for each set of input values, the boundary value and equivalent partitioning techniques are ineffective in guaranteeing good test coverage.

Decision table testing is a useful alternative in this case. This technique ensures adequate coverage, and the representation is straightforward, making it simple to understand and apply.

Since it is simple to grasp and covers all possible combinations, this table can be used as a reference for requirements and functionality development.

As the number of inputs grows, the relevance of this strategy becomes apparent. 2^n , where n is the number of inputs, equals the number of possible combinations. The number of combinations for $n = 10$, which is fairly typical in web-based testing with large input forms, is 1024. Obviously, you cannot test all conceivable combinations, but you can choose a large subset of them using the decision-based testing technique.

Advantages of Decision Table Testing

- When the system behaviour varies depending on the input and is not consistent across a range of inputs, neither equivalent partitioning nor boundary value analysis can help, but a decision table can.
- The representation is simple enough to be simply understood and may be utilised for both development and business.
- This table will aid in the creation of effective combinations and can improve testing coverage.
- Any complicated business situation can readily be transformed into a decision table.

Notes

- This strategy can provide coverage when we are aiming for 100 percent coverage, which is usually the case when the number of input combinations is modest.

Disadvantages of Decision Table Testing

- The biggest drawback is that as the number of inputs grows, the table becomes more complicated.

Used Case Testing

In testing, a Use Case is a brief description of a specific actor or user's use of the software programme. User actions and the software application's response to those actions are used to create use cases. It is commonly used in the development of system or acceptability test scenarios.

Use Case Testing is a software testing technique that aids in the identification of test cases that span the full system from beginning to end on a transaction by transaction basis. Interactions between users and software applications are called test cases. Use case testing identifies holes in software applications that would otherwise go undetected if individual software components were tested.

Example

An actor is represented by "A" and a system is represented by "S" in a use-case. As illustrated below, we develop a Use for a Web Application's login capability.

Main Success Scenario	Step	Description
A:Actor S:System	1	A: Enter Agent Name & Password
	2	S: Validate Password
	3	S: Allow Account Access
Extensions	2a	<u>Password not valid</u> S: Display Message and ask for re-try 4 times
	2b	<u>Password not valid 4 times</u> S: Close Application

- Consider the first stage of an end-to-end scenario for our web application's login capability, when the Actor enters his or her email and password.
- The password will be validated by the system in the following step.

Notes

- After that, if the password is accurate, access is given.
- This use case could be expanded upon. If the password is invalid, the system will display a notification and prompt you to try again four times.
- If the password is invalid four times, the IP address will be banned.
- Here, we'll put each extension's success scenario and one situation to the test.

2.3.3 White Box Testing

White Box Testing is software testing technique in which internal structure, design and coding of software are tested to verify flow of input-output and to improve design, usability and security. In white box testing, code is visible to testers so it is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing and Glass box testing.

It is one of two parts of the Box Testing approach to software testing. Its counterpart, Blackbox testing, involves testing from an external or end-user type perspective. On the other hand, White box testing in software engineering is based on the inner workings of an application and revolves around internal testing.

The term "WhiteBox" was used because of the see-through box concept. The clear box or WhiteBox name symbolizes the ability to see through the software's outer shell (or "box") into its inner workings. Likewise, the "black box" in "Black Box Testing" symbolizes not being able to see the inner workings of the software so that only the end-user experience can be tested.

The box testing approach of software testing consists of black box testing and white box testing. We are discussing here white box testing which also known as glass box is **testing, structural testing, clear box testing, open box testing and transparent box testing**. It tests internal coding and infrastructure of a software focus on checking of predefined inputs against expected and desired outputs. It is based on inner workings of an application and revolves around internal structure testing. In this type of testing programming skills are required to design test cases. The primary goal of white box testing is to focus on the flow of inputs and outputs through the software and strengthening the security of the software.

The term 'white box' is used because of the internal perspective of the system. The clear box or white box or transparent box name denote the ability to see through the software's outer shell into its inner workings.

Developers do white box testing. In this, the developer will test every line of the code of the program. The developers perform the White-box testing and then send the application or the software to the testing team, where they will perform the black box testing and verify the application along with the requirements and identify the bugs and sends it to the developer.

The developer fixes the bugs and does one round of white box testing and sends it to the testing team. Here, fixing the bugs implies that the bug is deleted, and the particular feature is working fine on the application.

Here, the test engineers will not include in fixing the defects for the following reasons:

Notes

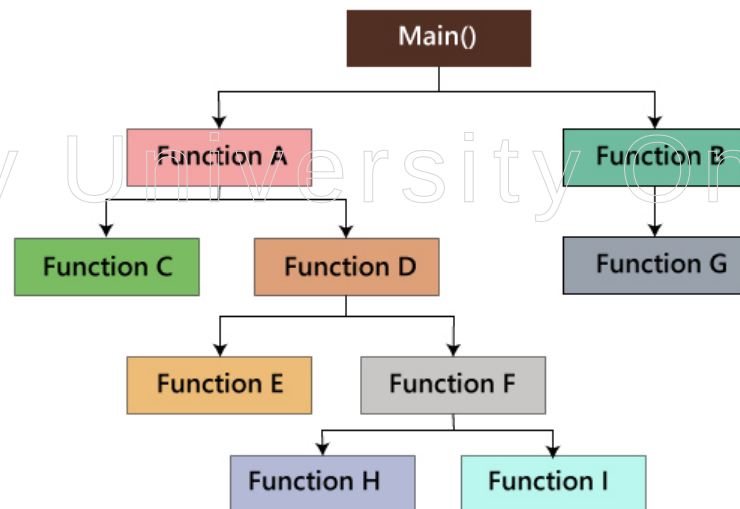
- Fixing the bug might interrupt the other features. Therefore, the test engineer should always find the bugs, and developers should still be doing the bug fixes.
- If the test engineers spend most of the time fixing the defects, then they may be unable to find the other bugs in the application.

The white box testing contains various tests, which are as follows:

- Path testing
- Loop testing
- Condition testing
- Testing based on the memory perspective
- Test performance of the program

Path testing

In the path testing, we will write the flow graphs and test all independent paths. Here writing the flow graph implies that flow graphs are representing the flow of the program and also show how every program is added with one another as we can see in the below image:



And test all the independent paths implies that suppose a path from main() to function G, first set the parameters and test if the program is correct in that particular path, and in the same way test all other paths and fix the bugs.

Loop testing

In the loop testing, we will test the loops such as while, for, and do-while, etc. and also check for ending condition if working correctly and if the size of the conditions is enough.

For example: we have one program where the developers have given about 50,000 loops.

- {
- while(50,000)
-

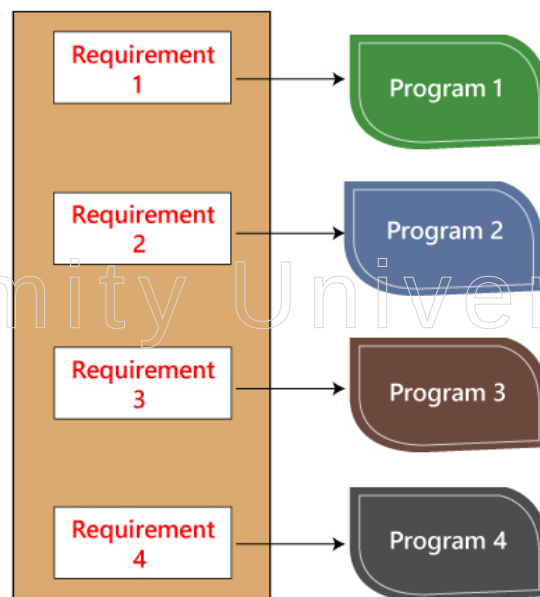
Notes

-
• }

We cannot test this program manually for all the 50,000 loops cycle. So we write a small program that helps for all 50,000 cycles, as we can see in the below program, that test P is written in the similar language as the source code program, and this is known as a Unit test. And it is written by the developers only.

- Test P
- {
•
• }

As we can see in the below image that, we have various requirements such as 1, 2, 3, 4. And then, the developer writes the programs such as program 1,2,3,4 for the parallel conditions. Here the application contains the 100s line of codes.



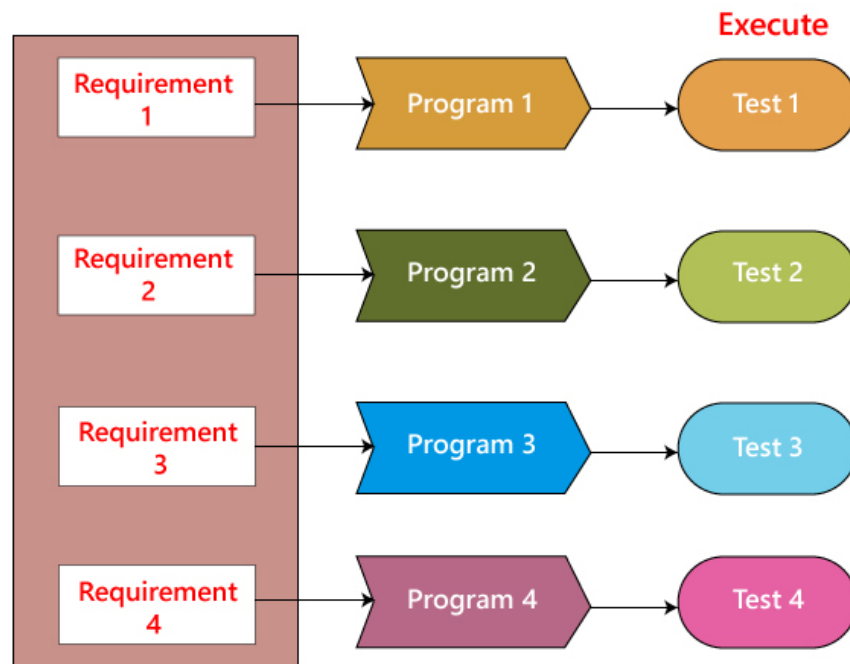
The developer will do the white box testing, and they will test all the five programs line by line of code to find the bug. If they found any bug in any of the programs, they will correct it. And they again have to test the system then this process contains lots of time and effort and slows down the product release time.

Now, suppose we have another case, where the clients want to modify the requirements, then the developer will do the required changes and test all four program again, which take lots of time and efforts.

These issues can be resolved in the following ways:

In this, we will write test for a similar program where the developer writes these test code in the related language as the source code. Then they execute these test code, which is also known as **unit test programs**. These test programs linked to the main program and implemented as programs.

Notes



Therefore, if there is any requirement of modification or bug in the code, then the developer makes the adjustment both in the main program and the test program and then executes the test program.

Condition testing

In this, we will test all logical conditions for both **true** and **false** values; that is, we will verify for both **if** and **else** condition.

For example:

- if(condition) - true
- {
-
-
-
- }
- else - false
- {
-
-
-
- }

The above program will work fine for both the conditions, which means that if the condition is accurate, and then else should be false and conversely.

Testing based on the memory (size) perspective

The size of the code is increasing for the following reasons:

- **The reuse of code is not there:** let us take one example, where we have four programs of the same application, and the first ten lines of the program are similar. We can write these ten lines as a discrete function, and it should be accessible by the above four programs as well. And also, if any bug is there, we can modify the line of code in the function rather than the entire code.
- The **developers use the logic** that might be modified. If one programmer writes code and the file size is up to 250kb, then another programmer could write a similar code using the different logic, and the file size is up to 100kb.
- The **developer declares so many functions and variables** that might never be used in any portion of the code. Therefore, the size of the program will increase.

For example,

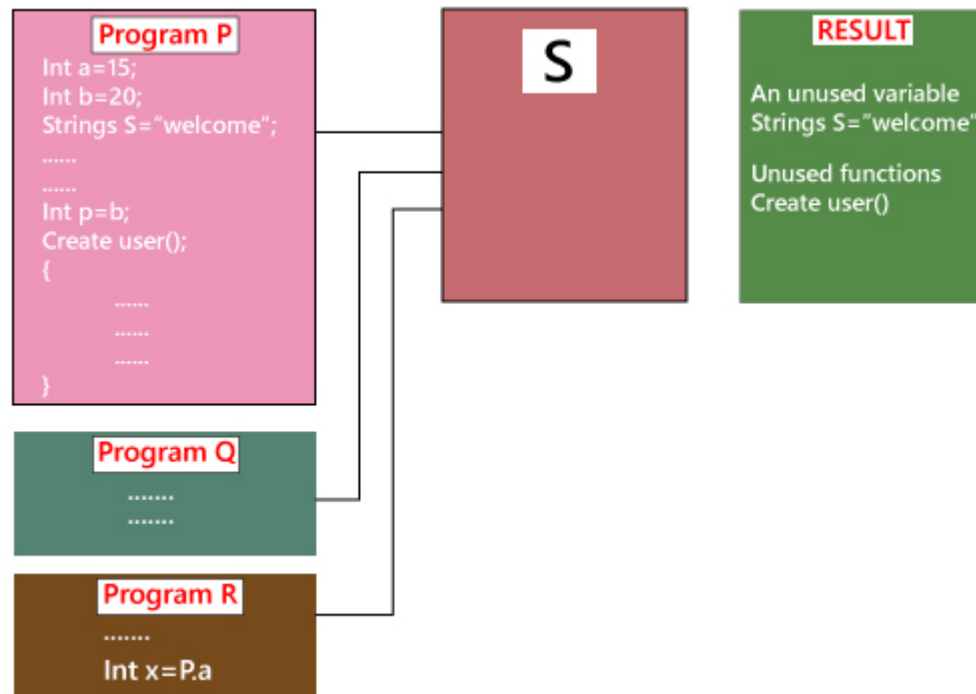
- Int a=15;
- Int b=20;
- String S= "Welcome";
-
-
-(c) Amity University Online
- 200's line of code
- }

In the above code, we can see that the **integer a** has never been called anywhere in the program, and also the function **Create user** has never been called anywhere in the code. Therefore, it leads us to memory consumption.

We cannot remember this type of mistake manually by verifying the code because of the large code. So, we have a built-in tool, which helps us to test the needless variables and functions. And, here we have the tool called **Rational purify**.

Notes

Notes



Suppose we have three programs such as Program P, Q, and R, which provides the input to S. And S goes into the programs and verifies the unused variables and then gives the outcome. After that, the developers will click on several results and call or remove the unnecessary function and the variables.

This tool is only used for the C programming language and C++ programming language; for another language, we have other related tools available in the market.

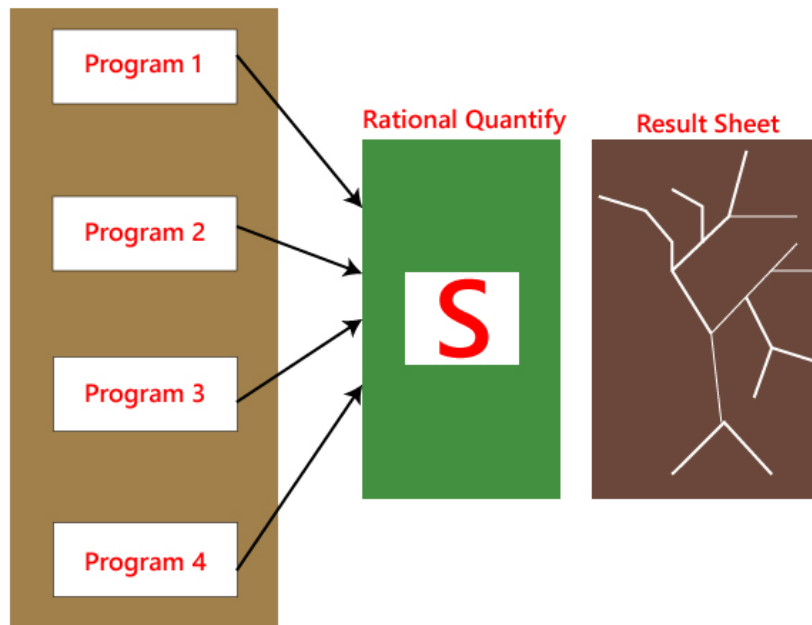
- The developer does not use the available in-built functions; instead, they write the full features using their logic. Therefore, it leads us to waste of time and also postpone the product releases.

Test the performance (Speed, response time) of the program

The application could be slow for the following reasons:

- When logic is used.
- For the conditional cases, we will use **or & and** adequately.
- Switch case, which means we cannot use **nested if**, instead of using a switch case.

Notes

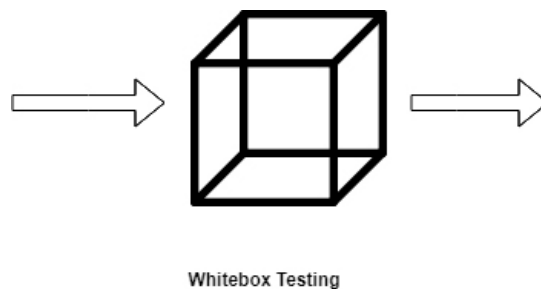


As we know that the developer is performing white box testing, they understand that the code is running slow, or the performance of the program is also getting deliberate. And the developer cannot go manually over the program and verify which line of the code is slowing the program.

To recover with this condition, we have a tool called **Rational Quantify**, which resolves these kinds of issues automatically. Once the entire code is ready, the rational quantify tool will go through the code and execute it. And we can see the outcome in the result sheet in the form of thick and thin lines.

Here, the thick line specifies which section of code is time-consuming. When we double-click on the thick line, the tool will take us to that line or piece of code automatically, which is also displayed in a different color. We can change that code and again and use this tool. When the order of lines is all thin, we know that the presentation of the program has enhanced. And the developers will perform the white box testing automatically because it saves time rather than performing manually.

Test cases for white box testing are derived from the design phase of the software development lifecycle. Data flow testing, control flow testing, path testing, branch testing, statement and decision coverage all these techniques used by white box testing as a guideline to create an error-free software.



White box testing follows some working steps to make testing manageable and easy to understand what the next task to do. There are some basic steps to perform white box testing.

Notes

Generic steps of white box testing

- Design all test scenarios, test cases and prioritize them according to high priority number.
- This step involves the study of code at runtime to examine the resource utilization, not accessed areas of the code, time taken by various methods and operations and so on.
- In this step testing of internal subroutines takes place. Internal subroutines such as nonpublic methods, interfaces are able to handle all types of data appropriately or not.
- This step focuses on testing of control statements like loops and conditional statements to check the efficiency and accuracy for different data inputs.
- In the last step white box testing includes security testing to check all possible security loopholes by looking at how the code handles security.

Reasons for white box testing

- It identifies internal security holes.
- To check the way of input inside the code.
- Check the functionality of conditional loops.
- To test function, object, and statement at an individual level.

Advantages of White box testing

- White box testing optimizes code so hidden errors can be identified.
- Test cases of white box testing can be easily automated.
- This testing is more thorough than other testing approaches as it covers all code paths.
- It can be started in the SDLC phase even without GUI.

Disadvantages of White box testing

- White box testing is too much time consuming when it comes to large-scale programming applications.
- White box testing is much expensive and complex.
- It can lead to production error because it is not detailed by the developers.
- White box testing needs professional programmers who have a detailed knowledge and understanding of programming language and implementation.

Techniques Used in White Box Testing

Data Flow Testing	Data flow testing is a group of testing strategies that examines the control flow of programs in order to explore the sequence of variables according to the sequence of events.
-------------------	--

Notes

Control Flow Testing	Control flow testing determines the execution order of statements or instructions of the program through a control structure. The control structure of a program is used to develop a test case for the program. In this technique, a particular part of a large program is selected by the tester to set the testing path. Test cases represented by the control graph of the program.
Branch Testing	Branch coverage technique is used to cover all branches of the control flow graph. It covers all the possible outcomes (true and false) of each condition of decision point at least once.
Statement Testing	Statement coverage technique is used to design white box test cases. This technique involves execution of all statements of the source code at least once. It is used to calculate the total number of executed statements in the source code, out of total statements present in the source code.
Decision Testing	This technique reports true and false outcomes of Boolean expressions. Whenever there is a possibility of two or more outcomes from the statements like do while statement, if statement and case statement (Control flow statements), it is considered as decision point because there are two outcomes either true or false.

Difference between white-box testing and black-box testing

Following are the significant differences between white box testing and black box testing:

White-box testing	Black box testing
The developers can perform white box testing.	The test engineers perform the black box testing.
To perform WBT, we should have an understanding of the programming languages.	To perform BBT, there is no need to have an understanding of the programming languages.
In this, we will look into the source code and test the logic of the code.	In this, we will verify the functionality of the application based on the requirement specification.
In this, the developer should know about the internal design of the code.	In this, there is no need to know about the internal design of the code.

2.3.4 Statement Coverage

Statement coverage is one of the widely used software testing. It comes under white box testing.

Statement coverage technique is used to design white box test cases. This technique involves execution of all statements of the source code at least once. It is

Notes

used to calculate the total number of executed statements in the source code out of total statements present in the source code.

Statement coverage derives scenario of test cases under the white box testing process which is based upon the structure of the code.

$$\text{Statement coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} * 100$$

In white box testing, concentration of the tester is on the working of internal source code and flow chart or flow graph of the code.

Generally, in the internal source code, there is a wide variety of elements like operators, methods, arrays, looping, control statements, exception handlers, etc. Based on the input given to the program, some code statements are executed and some may not be executed. The goal of statement coverage technique is to cover all the possible executing statements and path lines in the code.

Let's understand the process of calculating statement coverage by an example:

Here, we are taking source code to create two different scenarios according to input values to check the percentage of statement coverage for each scenario.

Source Code Structure:

- Take input of two values like a=0 and b=1.
 - Find the sum of these two values.
 - If the sum is greater than 0, then print "This is the positive result."
 - If the sum is less than 0, then print "This is the negative result."
1. input (int a, int b)
 2. {
 3. Function to print sum of these integer values (sum = a+b)
 4. If (sum>0)
 5. {
 6. Print (This is positive result)
 7. } **else**
 8. {
 9. Print (This is negative result)
 10. }
 11. }

So, this is the basic structure of the program, and that is the task it is going to do.

Now, let's see the two different scenarios and calculation of the percentage of Statement Coverage for given source code.

Scenario 1:**If a = 5, b = 4**

- print (int a, int b) {
- int sum = a+b;
- if (sum>0)
- print ("This is a positive result")
- else
- print ("This is negative result")
- }

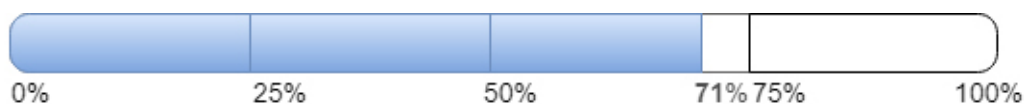
In scenario 1, we can see the value of sum will be 9 that is greater than 0 and as per the condition result will be **"This is a positive result."** The statements highlighted in yellow color are executed statements of this scenario.

To calculate statement coverage of the first scenario, take the total number of statements that is 7 and the number of used statements that is 5.

1. Total number of statements = 7
2. Number of executed statements = 5

$$\text{Statement coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} * 100$$

- Statement coverage = $5/7 * 100$
- = $500/7$
- = 71%



Likewise, in scenario 2,

Scenario 2:**If A = -2, B = -7**

- print (int a, int b) {
- int sum = a+b;
- if (sum>0)
- print ("This is a positive result")
- else
- print ("This is negative result")
- }

In scenario 2, we can see the value of sum will be -9 that is less than 0 and as per the condition, result will be **"This is a negative result."** The statements highlighted in yellow color are executed statements of this scenario.

Notes

Notes

To calculate statement coverage of the first scenario, take the total number of statements that is 7 and the number of used statements that is 6.

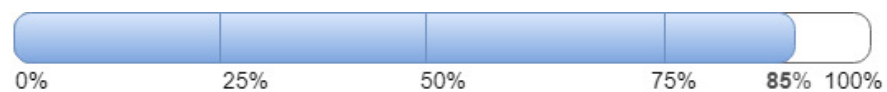
Total number of statements = 7

Number of executed statements = 6

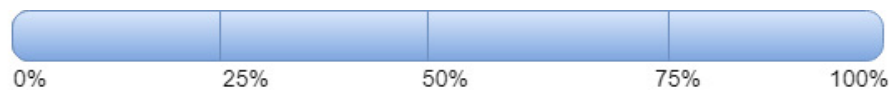
$$\text{Statement coverage} = \frac{\text{Number of executed statements}}{\text{Total number of statements}} * 100$$

- Statement coverage = $6/7 * 100$

- = $600/7$
- = 85%



But, we can see all the statements are covered in both scenario and we can consider that the overall statement coverage is 100%.



So, the statement coverage technique covers dead code, unused code, and branches.

2.4.1 Branch Coverage

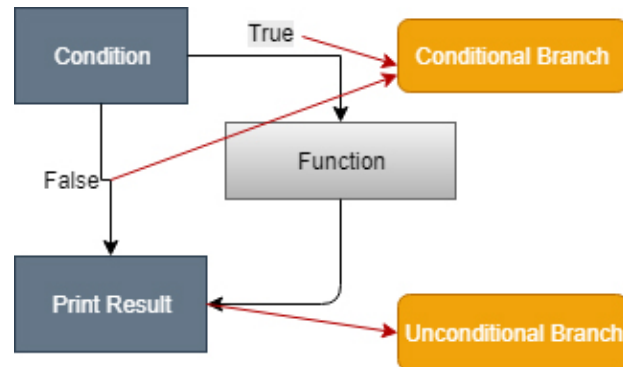
Branch coverage technique is used to cover all branches of the control flow graph. It covers all the possible outcomes (true and false) of each condition of decision point at least once. Branch coverage technique is a white box testing technique that ensures that every branch of each decision point must be executed.

However, branch coverage technique and decision coverage technique are very similar, but there is a key difference between the two. Decision coverage technique covers all branches of each decision point whereas branch testing covers all branches of every decision point of the code.

In other words, branch coverage follows decision point and branch coverage edges. Many different metrics can be used to find branch coverage and decision coverage, but some of the most basic metrics are: finding the percentage of program and paths of execution during the execution of the program.

Like decision coverage, it also uses a control flow graph to calculate the number of branches.

Difference between JDK, JRE, and JVM



How to calculate Branch coverage?

There are several methods to calculate Branch coverage, but pathfinding is the most common method.

In this method, the number of paths of executed branches is used to calculate Branch coverage. Branch coverage technique can be used as the alternative of decision coverage. Somewhere, it is not defined as an individual technique, but it is distinct from decision coverage and essential to test all branches of the control flow graph.

Let's understand it with an example:

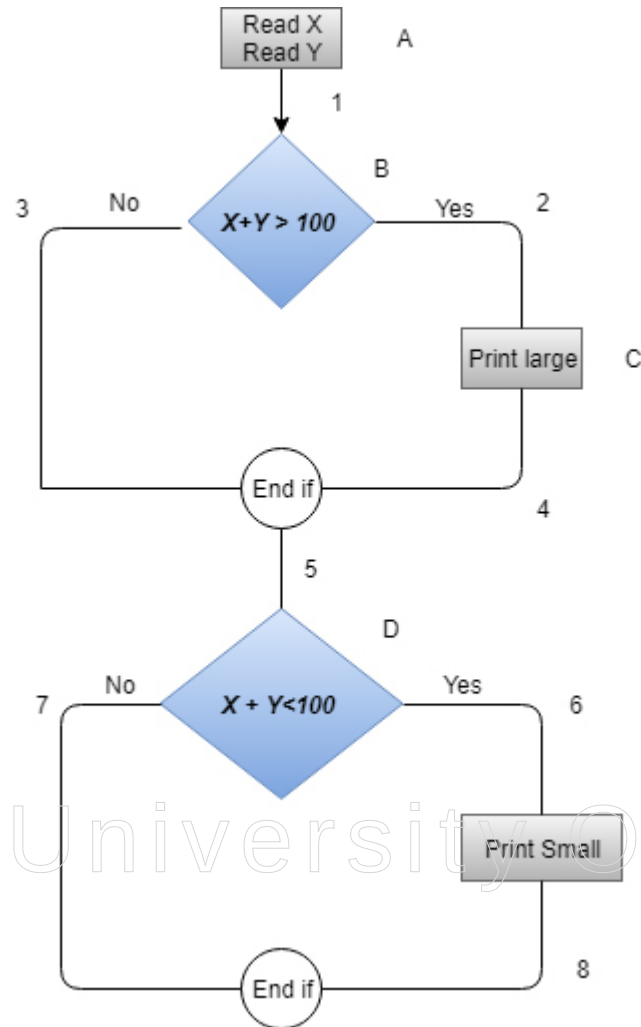
1. Read X
2. Read Y
3. IF X+Y > 100 THEN
4. Print "Large"
5. ENDIF
6. If X + Y < 100 THEN
7. Print "Small"
8. ENDIF

This is the basic code structure where we took two variables X and Y and two conditions. If the first condition is true, then print "Large" and if it is false, then go to the next condition. If the second condition is true, then print "Small."

Notes

Notes

Control flow graph of code structure



In the above diagram, control flow graph of code is depicted. In the first case traversing through “Yes” decision, the path is **A1-B2-C4-D6-E8**, and the number of covered edges is 1, 2, 4, 5, 6 and 8 but edges 3 and 7 are not covered in this path. To cover these edges, we have to traverse through “No” decision. In the case of “No” decision the path is A1-B3-5-D7, and the number of covered edges is 3 and 7. So by traveling through these two paths, all branches have covered.

Path 1 - A1-B2-C4-D6-E8

Path 2 - A1-B3-5-D7

Branch Coverage (BC) = Number of paths

=2

Notes

Case	Covered Branches	Path	Branch coverage
Yes	1, 2, 4, 5, 6, 8	A1-B2-C4-D6-E8	2
No	3,7	A1-B3-5-D7	

2.4.2 Decision Coverage

Decision coverage technique covers all branches of each decision point whereas branch testing covers all branches of every decision point of the code.

Decision coverage, also known as branch coverage, is a testing technique that ensures that each conceivable branch from each decision point is tested at least once, ensuring that all reachable code is performed.

That is to say, every decision is made in both true and erroneous ways. It aids in verifying all of the code's branches, ensuring that none of them lead to anomalous application behaviour.

White box testing uses the decision coverage technique to give Boolean values decision coverage. This method displays the true and false results of Boolean expressions. When there is a chance of two or more outcomes from statements such as do while, if, and case (Control flow statements), it is referred to as a decision point since there are only two possible outcomes: true or false.

Using a control flow graph or chart, decision coverage covers all possible outcomes of each and every Boolean condition in the code.

$$\text{Decision Coverage} = \frac{\text{Number of Decision Outcomes Exercised}}{\text{Total Number of Decision Outcomes}} * 100$$

It's difficult to achieve 100% coverage with this method because expressions can be tricky at times. As a result, there are a variety of ways to report decision coverage. All of these strategies are quite similar to decision coverage in that they cover the most essential combinations. The benefit of these strategies is that they improve control flow sensitivity.

2.4.3 Examples on Decision Coverage

The number of decision coverage can be found as follows.

Let's have a look at an example.

Consider the following code for implementing the decision coverage technique:

Notes

```
Test (int a)
{
  If(a>4)
  a=a*3
  Print (a)
}
```

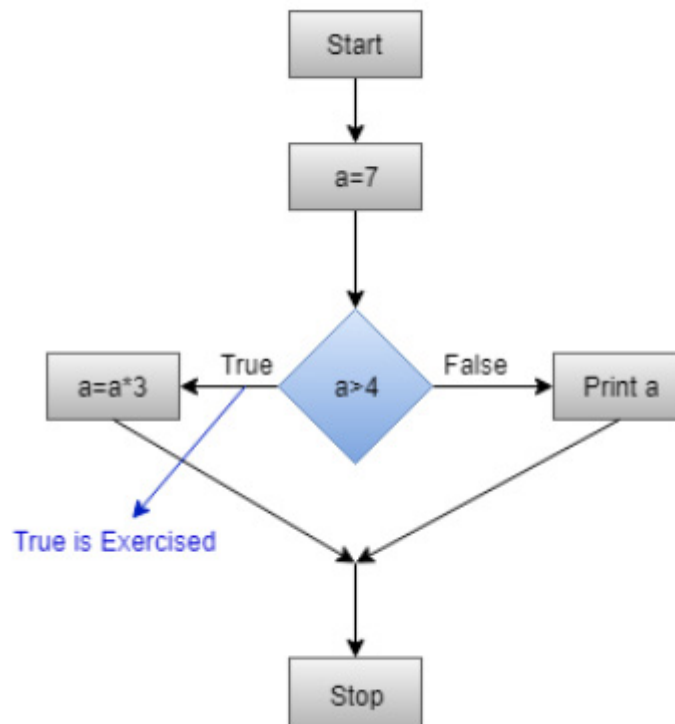
Scenario 1:

Value of a is 7 (a=7)

```
Test (int a=7)
{ if (a>4)
  a=a*3
  print (a)
}
```

The code highlighted in yellow is executed code. The outcome of this code is "True" if condition (a>4) is checked.

Control flow graph when the value of a is 7.



Calculation of Decision Coverage percent:

Notes

$$\text{Decision Coverage} = \frac{\text{Number of Decision Outcomes Exercised}}{\text{Total Number of Decision Outcomes}} * 100$$

Decision Coverage = $\frac{1}{2} * 100$ (Only "True" is exercised)

= $100/2$

= 50

Decision Coverage is 50%

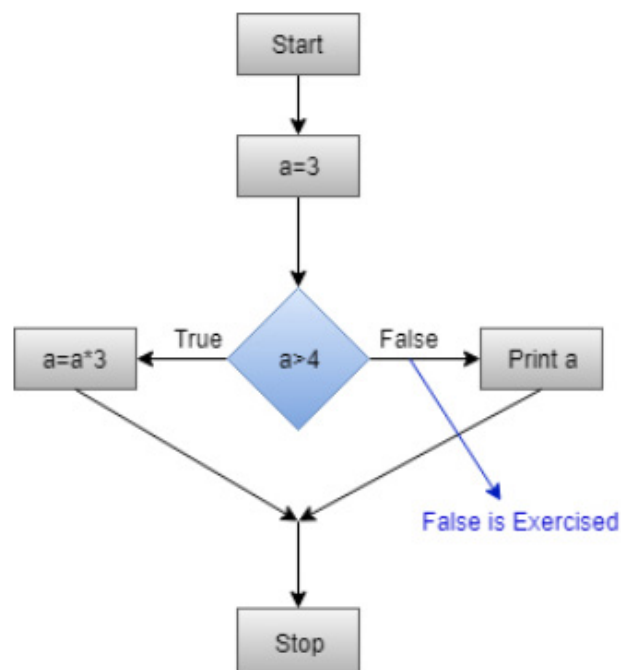
Scenario 2:

Value of a is 3 (a=3)

```
Test (int a=3)
{ if (a>4)
  a=a*3
  print (a)
}
```

The yellow-highlighted code will be run. What is the result of this code? False? if the condition (a>4) is true

When the value of an is 3, the control flow graph looks like this.



Calculation of Decision Coverage percent:

Notes

$$\text{Decision Coverage} = \frac{\text{Number of Decision Outcomes Exercised}}{\text{Total Number of Decision Outcomes}} * 100$$

$$= \frac{1}{2} * 100 \text{ (Only "False" is exercised)}$$

$$= \frac{100}{2}$$

$$= 50$$

$$\text{Decision Coverage} = 50\%$$

Result table of Decision Coverage:

Test Case	Value of A	Output	Decision Coverage
1	3	3	50%
2	7	21	50%

2.4.4 MCC Coverage

MCC stand for Multiple Condition Coverage.

All combinations of truth values of conditions must be covered, according to the Multiple Condition Coverage requirements.

There will be a total of 2^n test cases. If we have two conditions, we have $n=2$, which means $2^2=4$ test cases, and if we have three criteria, we have $2^3=8$ test cases.

Condition C1	Condition C2	Condition D1
T	T	T
T	F	F
F	T	F
F	F	F

C1 and C2 both are covered.

Now, we can see in table 1 that all combinations of conditions are covered because;

C1 T is tested with C2 T.

C1 T is tested with C2 F.

C1 F is tested with C2 T.

C1 F is tested with C2 F.

The following are some facts concerning Multiple Condition Coverage (MCC)

Notes

MCDC is subsumed under Multiple Condition Coverage (MCC). As a result, the following results:

- MCDC is the new name for the Centers for Disease Control and Prevention.
- CDC encompasses both CC and DC.
- It means that if we did MCC testing, we didn't have to do MCDC, CDC, CC, or DC.
- MCC has a higher strength than MCDC.
- The MCC is more powerful than the CDC.
- MCC has a higher strength than CC.
- MCC has a higher strength than DC.

2.5.1 MC/DC Coverage

MCDC= MCDC stands for Modified Condition Decision Coverage.

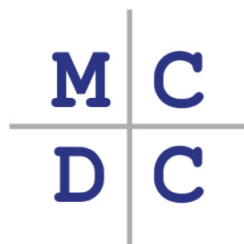
CDC= CDC stands for Condition Decision Coverage.

DC= DC stands for Decision Coverage.

During testing, MC/DC requires the following:

- Each point of entry and exit is activated.
- Each choice considers all conceivable outcomes.
- Every possible consequence is taken into account in each condition of a decision.
- Each circumstance in a decision is proven to have an independent effect on the decision's outcome.
- A condition's independence is demonstrated by demonstrating that only one condition changes at a time.

MC/DC is used in the DO-178B and DO-178C avionics software development guidelines to assure proper testing of the most critical (Level A) software, which is defined as software that could provide (or prevent failure of) continuing safe flying and landing of an aircraft. It is also strongly suggested for SIL 4 in part 3 Annex B of the fundamental safety publication[2] and ASIL D in part 6 of the ISO 26262 automotive standard.



The MC/DC criterion (Modified Condition/Decision Coverage) is a code coverage criterion that is extensively used in software testing. In the aircraft industry, for example, DO-178C software development advice mandates MC/DC for the most important software level (DAL A).

Notes

Code coverage is a metric for determining how effective your test cases are. When compared to code with a lower coverage score, the higher the proportion of code that has been covered by testing, the less likely it is to contain defects. Statement coverage, Decision coverage, and Multiple condition coverage are three further types of code coverage to examine in relation to MC/DC.

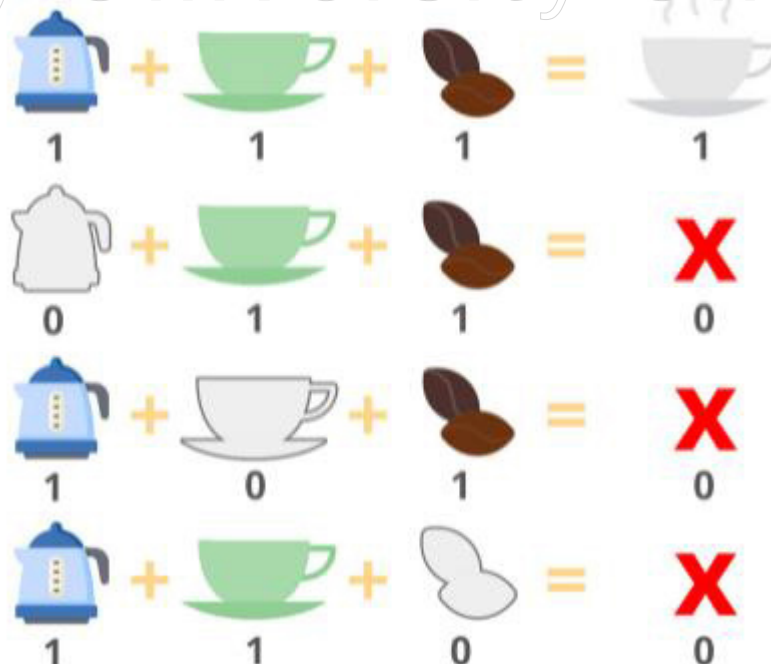
In addition to the statement and decision coverage criteria, MC/DC stipulates that “each circumstance in a decision has been proved to independently affect the decision’s outcome.”

Example of a Cup of Coffee

Consider preparing a cup of coffee. We’ll need ALL of the following to prepare a warm and good cup of coffee: a kettle, a cup, and coffee. We wouldn’t be able to create coffee if any of the components were missing. Alternatively, to put it another way:

```
if( kettle && cup && coffee ) {
    return cup_of_coffee;
}
else {
    return none;
}
```

Illustration



Tests 4 & 8 demonstrate that ‘kettle’ can independently affect the outcome.

Tests 6 & 8 demonstrate that ‘mug’ can independently affect the outcome.

Tests 7 & 8 demonstrate that ‘coffee’ can independently affect the outcome.

Notes

	Inputs			Outputs
Test	Kettle	Mug	Coffee	Result
1	0	0	0	0
2	0	0	1	0
3	0	1	0	0
4	0	1	1	0
5	1	0	0	0
6	1	0	1	0
7	1	1	0	0
8	1	1	1	1

Please note that the specified tests cover decision and statement coverage as well.

Tests 4, 6, 7, and 8 are necessary for MC/DC analysis in this situation. We don't need tests 1, 2, 3, or 5, because we can meet the MC/DC criterion without them. The coverage report does not need to include these redundant tests.

2.5.2 System integration, Deployment testing, Beta Testing

System Integration Testing (SIT)

System Integration Testing (SIT) is a black box testing technique that evaluates the system's compliance against specified requirements. System Integration Testing is usually performed on subset of system while system testing is performed on a complete system and is preceded by the user acceptance test (UAT).

The SIT can be performed with minimum usage of testing tools, verified for the interactions exchanged and the behaviour of each data field within individual layer is investigated. After the integration, there are three main states of data flow:

System Integration Testing - Main States:

- Data state within the integration layer
- Data state within the database layer
- Data state within the Application layer

Granularity in System Integration Testing:

- Intra-system testing
- Inter-system testing
- Pairwise testing

Notes

System Integration Testing Techniques:

- Top-down Integration Testing
- Bottom-up Integration Testing
- Sandwich Integration Testing
- Big-bang Integration Testing

Brief of integration testing

Integration testing is the second level of the software testing process comes after unit testing. In this testing, units or individual components of the software are tested in a group. The focus of the integration testing level is to expose defects at the time of interaction between integrated components or units.

Unit testing

uses modules for testing purpose, and these modules are combined and tested in integration testing. The Software is developed with a number of software modules that are coded by different coders or programmers. The goal of integration testing is to check the correctness of communication among all the modules.

Once all the components or modules are working independently, then we need to check the data flow between the dependent modules is known as **integration testing**.

Let us see one sample example of a banking application, as we can see in the below image of amount transfer.

The image shows a web form titled "Amount Transfer" with a purple background. It contains three input fields labeled "FAN:", "TAN:", and "AMOUNT:". Below these fields are two buttons: "Transfer" and "Cancel".

- First, we will login as a user **P** to amount transfer and send Rs200 amount, the confirmation message should be displayed on the screen as **amount transfer successfully**. Now logout as P and login as user **Q** and go to amount balance page and check for a balance in that account = Present balance + Received Balance. Therefore, the integration test is successful.
- Also, we check if the amount of balance has reduced by Rs200 in P user account.
- Click on the transaction, in P and Q, the message should be displayed regarding the data and time of the amount transfer.

Notes

Guidelines for Integration Testing

- We go for the integration testing only after the functional testing is completed on each module of the application.
- We always do integration testing by picking module by module so that a proper sequence is followed, and also we don't miss out on any integration scenarios.
- First, determine the test case strategy through which executable test cases can be prepared according to test data.
- Examine the structure and architecture of the application and identify the crucial modules to test them first and also identify all possible scenarios.
- Design test cases to verify each interface in detail.
- Choose input data for test case execution. Input data plays a significant role in testing.
- If we find any bugs then communicate the bug reports to developers and fix defects and retest.
- Perform **positive and negative integration testing**.

Here **positive** testing implies that if the total balance is Rs15, 000 and we are transferring Rs1500 and checking if the amount transfer works fine. If it does, then the test would be a pass.

And **negative testing** means, if the total balance is Rs15, 000 and we are transferring Rs20, 000 and check if amount transfer occurs or not, if it does not occur, the test is a pass. If it happens, then there is a bug in the code, and we will send it to the development team for fixing that bug.

Note: Any application in this world will do functional testing compulsory, whereas integration testing will be done only if the modules are dependent on each other. Each integration scenarios should compulsorily have source→ data→destination. Any scenarios can be called as integration scenario only if the data gets saved in the destination.

For example: In the Gmail application, the **Source** could be **Compose**, **Data** could be **Email** and the **Destination** could be the **Inbox**.

Example of integration testing

Let us assume that we have a **Gmail** application where we perform the integration testing.

First, we will do **functional testing** on the **login page**, which includes the various components such as **username, password, submit, and cancel** button. Then only we can perform integration testing.

The different integration scenarios are as follows:

Notes

Scenarios1:

- First, we login as **P** users and click on the **Compose** mail and performing the functional testing for the specific components.
- Now we click on the **Send** and also check for **Save Drafts**.
- After that, we send a **mail** to **Q** and verify in the **Sent Items** folder of P to check if the send mail is there.
- Now, we will **log out** as P and login as Q and move to the **Inbox** and verify that if the mail has reached.

Secanrios2: We also perform the integration testing on **Spam** folders. If the particular contact has been marked as spam, then any mail sent by that user should go to the spam folder and not in the inbox.

Note: We will perform functional testing for all features, such as to send items, inbox, and so on.

As we can see in the below image, we will perform the **functional testing**

for all the **text fields and every feature**. Then we will perform **integration testing** for the related functions. We first test the **add user, list of users, delete user, edit user**, and then **search user**.

Notes

Note:

- There are some features, we might be performing only the **functional testing**, and there are some features where we are performing both **functional** and **integration testing** based on the feature's requirements.
- **Prioritizing is essential**, and we should perform it at all the phases, which means we will open the application and select which feature needs to be tested first. Then go to that feature and choose which component must be tested first. Go to those components and determine what values to be entered first. And don't apply the same rule everywhere because testing logic varies from feature to feature.
- While performing testing, we should test one feature entirely and then only proceed to another function.
- Among the two features, we must be performing **only positive integrating testing** or both **positive and negative integration testing**, and this also depends on the features need.

Reason Behind Integration Testing

Although all modules of software application already tested in unit testing, errors still exist due to the following reasons:

1. Each module is designed by individual software developer whose programming logic may differ from developers of other modules so; integration testing becomes essential to determine the working of software modules.
2. To check the interaction of software modules with the database whether it is an erroneous or not.
3. Requirements can be changed or enhanced at the time of module development. These new requirements may not be tested at the level of unit testing hence integration testing becomes mandatory.

Notes

4. Incompatibility between modules of software could create errors.
5. To test hardware's compatibility with software.
6. If exception handling is inadequate between modules, it can create bugs.

Integration Testing Techniques

Any testing technique (Blackbox, Whitebox, and Greybox) can be used for Integration Testing; some are listed below:

Black Box Testing

- State Transition technique
- Decision Table Technique
- Boundary Value Analysis
- All-pairs Testing
- Cause and Effect Graph
- Equivalence Partitioning
- Error Guessing

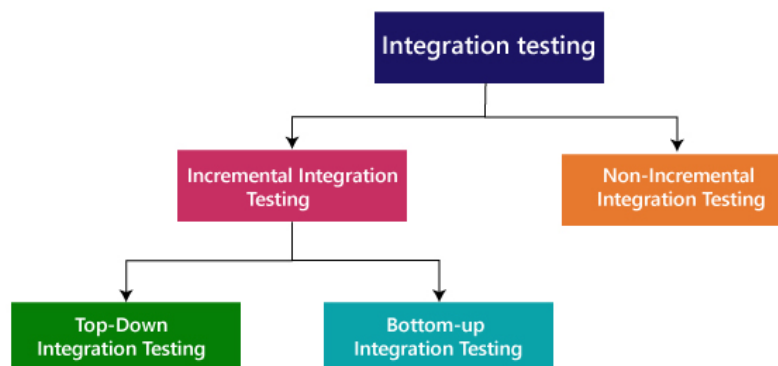
White Box Testing

- Data flow testing
- Control Flow Testing
- Branch Coverage Testing
- Decision Coverage Testing

Types of Integration Testing

Integration testing can be classified into two parts:

- **Incremental integration testing**
- **Non-incremental integration testing**



Incremental Approach

In the Incremental Approach, modules are added in ascending order one by one or according to need. The selected modules must be logically related. Generally, two or

more than two modules are added and tested to determine the correctness of functions. The process continues until the successful testing of all the modules.

OR

In this type of testing, there is a strong relationship between the dependent modules. Suppose we take two or more modules and verify that the data flow between them is working fine. If it is, then add more modules and test again.



For example: Suppose we have a Flipkart application, we will perform incremental integration testing, and the flow of the application would like this:

Flipkart → Login → Home → Search → Add cart → Payment → Logout

Incremental integration testing is carried out by further methods:

- Top-Down approach
- Bottom-Up approach

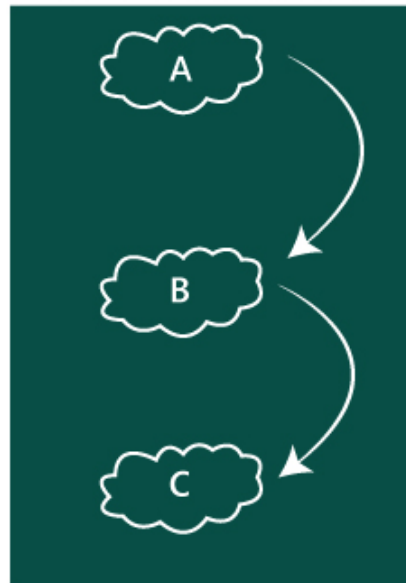
Top-Down Approach

The top-down testing strategy deals with the process in which higher level modules are tested with lower-level modules until the successful completion of testing of all the modules. Major design flaws can be detected and fixed early because critical modules tested first. In this type of method, we will add the modules incrementally or one by one and check the data flow in the same order.

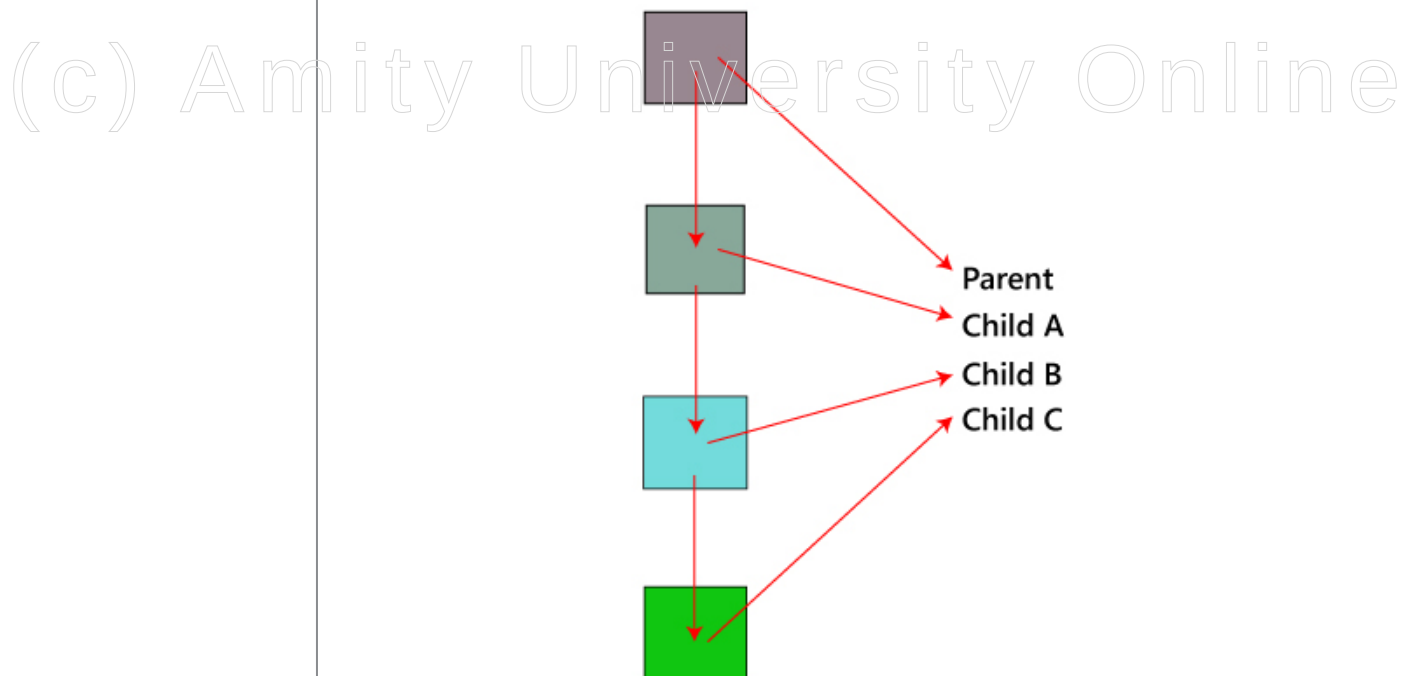
Notes

Notes

Top-Down Approach



In the top-down approach, we will be ensuring that the module we are adding is the **child of the previous one** like **Child C is a child of Child B** and so on as we can see in the below image:



Advantages:

- Identification of defect is difficult.
- An early prototype is possible.

Disadvantages:

- Due to the high number of stubs, it gets quite complicated.

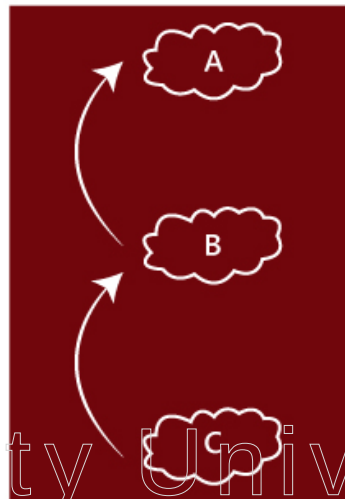
- Lower level modules are tested inadequately.
- Critical Modules are tested first so that fewer chances of defects.

Notes

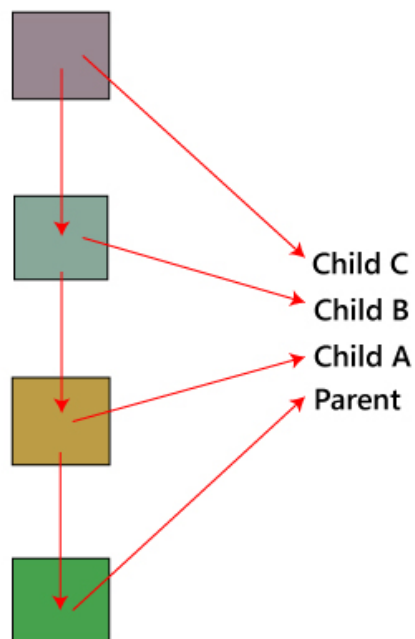
Bottom-Up Method

The bottom to up testing strategy deals with the process in which lower level modules are tested with higher level modules until the successful completion of testing of all the modules. Top level critical modules are tested at last, so it may cause a defect. Or we can say that we will be adding the modules from **bottom to the top** and check the data flow in the same order.

Bottom-up Approach



In the bottom-up method, we will ensure that the modules we are adding **are the parent of the previous one** as we can see in the below image:



Advantages

- Identification of defect is easy.

Notes

- Do not need to wait for the development of all the modules as it saves time.

Disadvantages

- Critical modules are tested last due to which the defects can occur.
- There is no possibility of an early prototype.

In this, we have one addition approach which is known as **hybrid testing**.

Hybrid Testing Method

In this approach, both **Top-Down** and **Bottom-Up** approaches are combined for testing. In this process, top-level modules are tested with lower level modules and lower level modules tested with high-level modules simultaneously. There is less possibility of occurrence of defect because each module interface is tested.



Hybrid Method

Advantages

- The hybrid method provides features of both Bottom Up and Top-Down methods.
- It is most time reducing method.
- It provides complete testing of all modules.

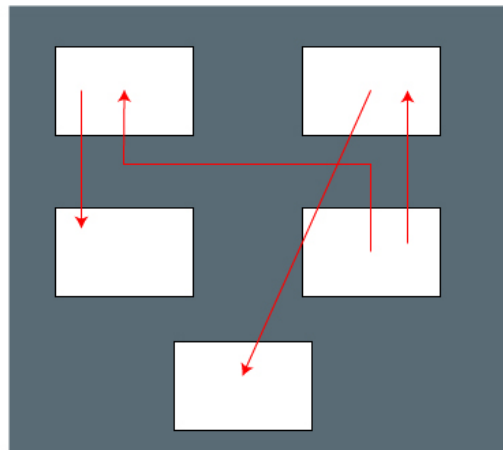
Disadvantages

- This method needs a higher level of concentration as the process carried out in both directions simultaneously.
- Complicated method.

Non- incremental integration testing

We will go for this method, when the data flow is very complex and when it is difficult to find who is a parent and who is a child. And in such case, we will create the data in any module bang on all other existing modules and check if the data is present. Hence, it is also known as the **Big bang method**.

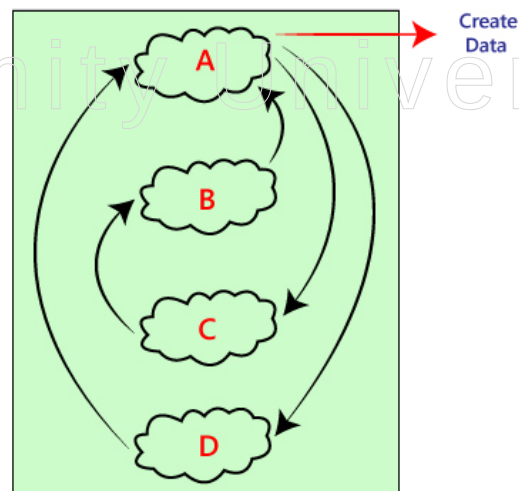
Notes



Big Bang Method

In this approach, testing is done via integration of all modules at once. It is convenient for small software systems, if used for large software systems identification of defects is difficult.

Since this testing can be done after completion of all modules due to that testing team has less time for execution of this process so that internally linked interfaces and high-risk critical modules can be missed easily.



Advantages:

- It is convenient for small size software systems.

Disadvantages:

- Identification of defects is difficult because finding the error where it came from is a problem, and we don't know the source of the bug.
- Small modules missed easily.
- Time provided for testing is very less.
- We may miss to test some of the interfaces.

Notes

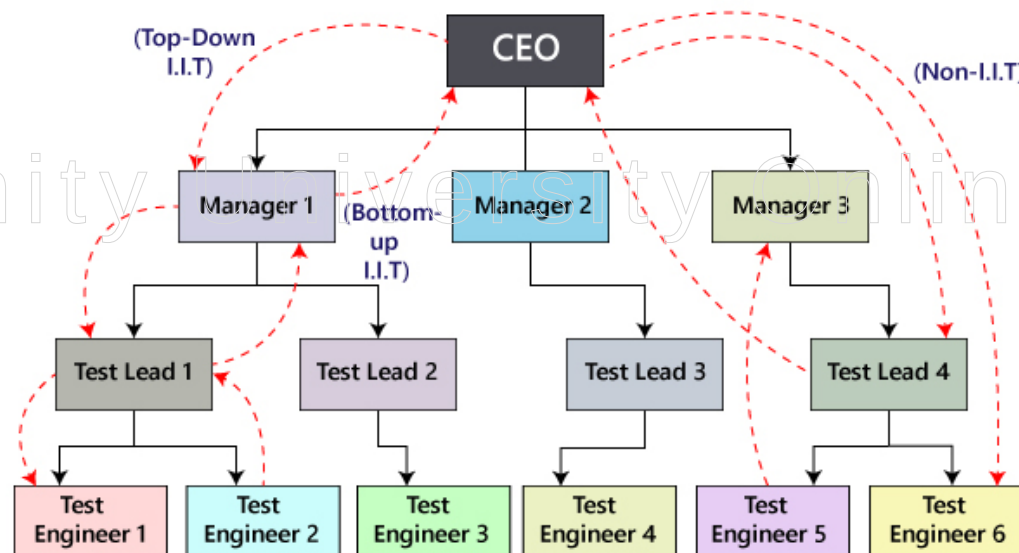
Let us see examples for our better understanding of the non-incremental integrating testing or big bang method:

Example1 In the below example, the development team develops the application and sends it to the CEO of the testing team. Then the CEO will log in to the application and generate the username and password and send a mail to the manager. After that, the CEO will tell them to start testing the application.

Then the manager manages the username and the password and produces a username and password and sends it to the **test leads**. And the **test leads** will send it to the **test engineers** for further testing purposes. This order from the CEO to the test engineer is **top-down incremental integrating testing**.

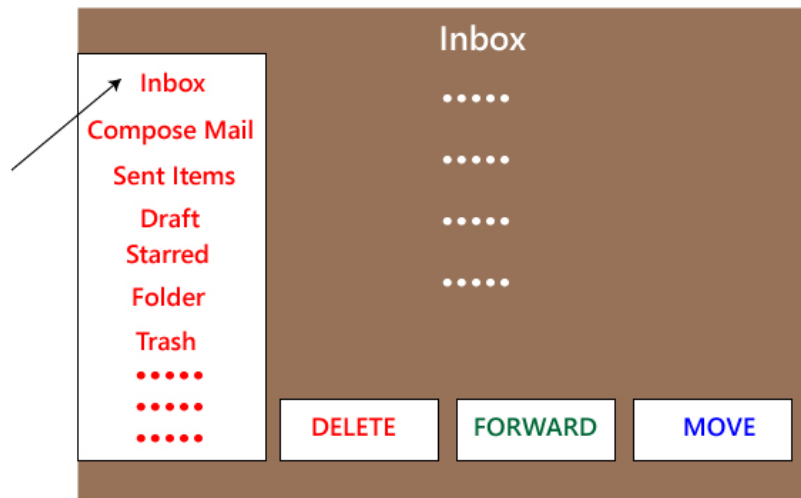
In the same way, when the test engineers are done with testing, they send a report to the **test leads**, who then submit a report to the **manager**, and the manager will send a report to the **CEO**. This process is known as **Bottom-up incremental integration testing** as we can see in the below image:

Note: The combination incremental integration testing (I.I.T) and non-incremental integration testing is known as sandwich testing.



Example2

The below example demonstrates a home page of **Gmail's Inbox**, where we click on the **Inbox** link, and we are moved to the inbox page. Here we have to do **non-incremental integration testing** because there is no parent and child concept.

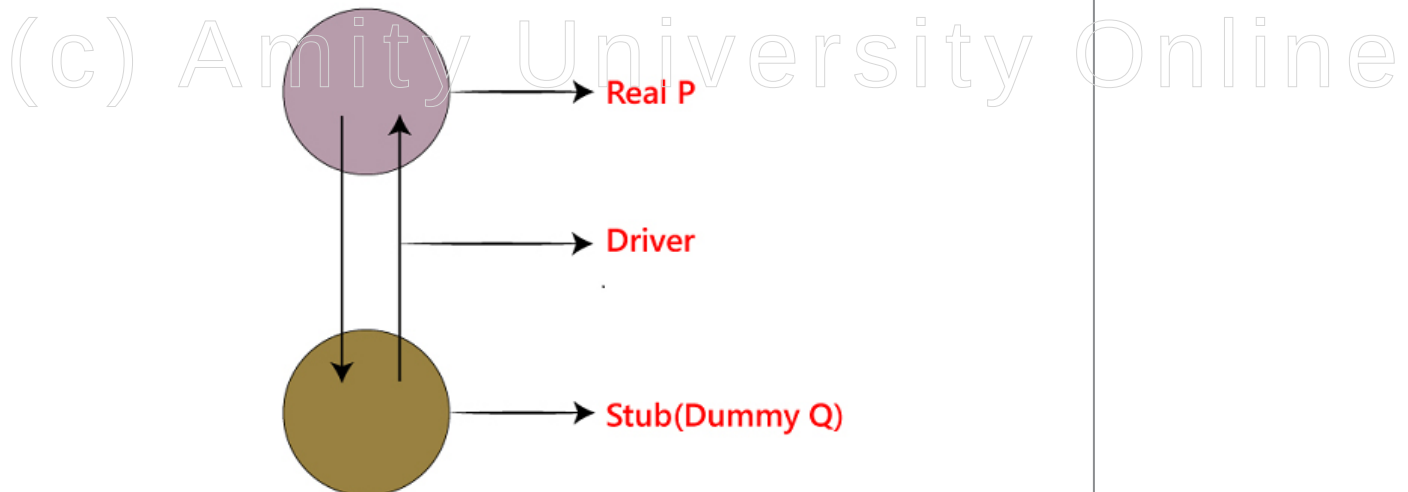


Notes

Note

Stub and driver

The **stub** is a dummy module that receives the data and creates lots of probable data, but it performs like a real module. When a data is sent from module P to Stub Q, it receives the data without confirming and validating it, and produce the estimated outcome for the given data.



The function of a driver is used to verify the data from P and sends it to stub and also checks the expected data from the stub and sends it to P.

The **driver** is one that sets up the test environments and also takes care of the communication, evaluates results, and sends the reports. We never use the stub and driver in the testing process.

In White box testing

, **bottom-up integration testing** is ideal because writing drivers is accessible. And in **black box testing**

Notes

, no preference is given to any testing as it depends on the application.

Deployment Testing

Testing a software project before and after deploying it on production is not that difficult. But too often, major bugs appear on production server after the deployment process. To avoid situations in which your production environment is threatened by these bugs, you should use a streamlined deployment and testing flow.

Generally speaking, this flow consists of three phases: pre-deploy, deploy and post-deploy.

During the **Pre-Deploy** testing phase, both the development team and the QA engineer should be tasked with the following items:

1. Ask developers to make Production and Stage environment backups.
2. Ask developers to copy database from Production to Stage.
3. Announce a Code Freeze for the entire development team.
4. Retest new features and bug fixes.
5. Perform general smoke testing using checklists.
6. Report all bugs, categorizing them with a consistent metric for urgency.
7. Test urgent bug fixes and engage in regression testing of this functionality.
8. When Stage is tested and has no major and/or minor bugs (depending on the project), — announce deployment.

When the QA engineer announces the Code Freeze, developers cannot push, commit, or deploy anything. But they can still continue working on their local environment, so no time is wasted. When the Code Freeze is cancelled, developers can push, commit, and deploy their work to Stage.

After finishing the pre-deploy testing cycle, the manager has to approve **the deploy process** before the developer who is responsible for this process puts it into motion. Having one developer be responsible for deploys is always beneficial, as it avoids misunderstandings like incorrectly merged branches.

After the deployment is done, QA begins the **Post-Deploy phase**, which consists of the following tasks:

1. Retesting new features and bug fixes.
2. Perform general smoke testing using checklists.
3. Report all bugs, categorizing them with a consistent metric for urgency.
4. Urgent bug fixes should first be deployed on the Stage environment. Here, QA can test them, and engage in regression testing before deploying them on production. Then, the testing cycle — including bug fix testing and regression testing — should be repeated.
5. When production is tested and has no issues, — deployment is complete.
6. Cancel Code Freeze, or wait for client feedback as needed before cancelling

Code Freeze.

If for any reason issues appear during the deployment process, you should quickly troubleshoot them. Here are some examples of possible problems and their solutions:

1. If production is severely broken after deploy, — use a backup to quickly restore it.
2. If developers violated Code Freeze, — they need to roll back all the changes.
3. To quickly roll back any new features, developers should use separate branches for different features.
4. To avoid timing troubles, — never make deploys late in on Friday evenings.

Testing your software is a crucial part of the deployment cycle. By running through tests before, during, and after deployment, you can improve the quality of your software and ensure a better product.

Beta testing

If we compare the various activity performed to develop ideal software, we will find the importance of software testing similar to that of the software development process. Testing is one of those activities which ensure the accuracy of the development process while validating its functionality and performance.

Beta Testing

Beta testing is a type of **User Acceptance Testing** among the most crucial testing, which performed before the release of the software. Beta Testing is a type of Field Test. This testing performs at the end of the **software** testing life cycle. This type of testing can be considered as external user acceptance testing. It is a type of salient testing. Real users perform this testing. This testing executed after the alpha testing. In this the new version, beta testing is released to a limited audience to check the accessibility, usability, and functionality, and more.

- Beta testing is the last phase of the testing, which is carried out at the client's or customer's site.

The features of beta testing

Testing of the product performs by the real users of the software application in the real environment. Beta version of the software is released to a restricted number of end-users to obtain the feedback of the product quality. Beta testing reduces the risk of failure and provides the quality of the product through customer validation. It is the final testing before shipping the product to the customers. Beta testing obtains direct feedback from the customers. It helps in testing to test the product in the customer's environment.

Features of beta testing are:

- Beta testing used in a real environment at the user's site. Beta testing helps in providing the actual position of the quality.
- Testing performed by the client, stakeholder, and end-user.

Notes

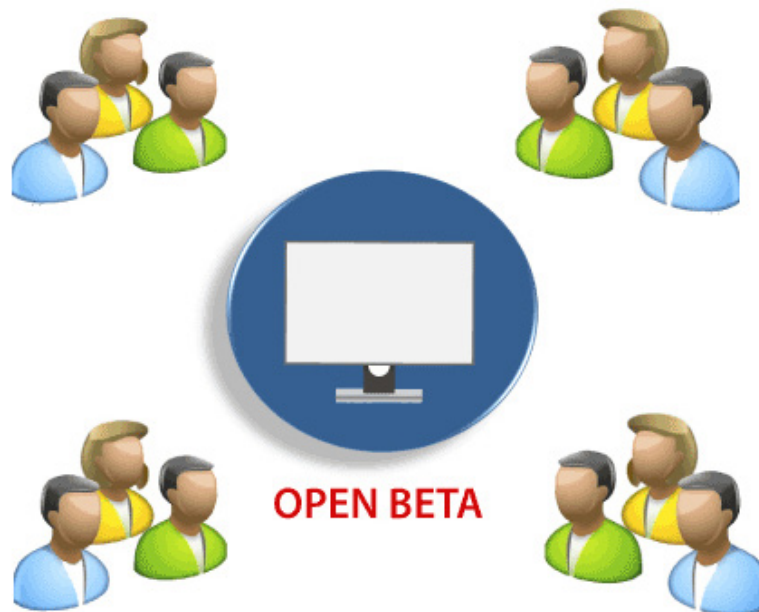
Notes

- Beta testing always is done after the alpha testing, and before releasing it into the market.
- Beta testing is black-box testing.
- Beta testing performs in the absence of tester and the presence of real users
- Beta testing is performed after alpha testing and before the release of the final product.
- Beta testing generally is done for testing software products like utilities, operating systems, and applications, etc.

What is a beta version of the software?

The beta version of the software is delivered to a restricted number of users to accept their feedback and suggestions on quality improvement. Hence, there are two types of beta version:

- 1) **Closed beta version:** Closed beta version, also known as a private beta, it is released to a group of selected and invited people. Those people will test the software and evaluate their features and specifications. This beta version represents the software which is capable of delivering value, but it is not ready to be used by everyone. Because it shows the issues like lack of documentation or missing vital features.
- 2) **Open beta version:** Open beta is also known as a public beta. The open beta opened to the public. Any user as a tester can assess the beta version to provide the relevant feedback and reviews. Open beta version improves the quality of the final release. This version helps to find the various undetected errors and issues.



The beta testing process orients this beta version.

What is the lifecycle of Beta Testing?

A group of end-users performs beta testing. This process can't execute without any strategy or test plan. Before the testers, the end-user executes this type of testing.

The process of beta testing follows the following steps:

1. **Planning:** Like another testing process, beta testing also supports proper planning. In this stage, the team prepares a testing strategy and defines the goal of testing. In this case, the team establishes the need of users for testing, duration, and necessary details related to the process.
2. **Participant Recruitment:** This is the second stage of the beta process in which the team recruits a group of selected end-users for testing. This group can change as per the requirement of the organization and the product.
3. **Product Launch:** When a team of users (testers) recruited. The beta version of the product is launched or installed at the client or user side, and users will test the product for quality assurance.
4. **Collect and Evaluate Feedback:** When the testing finished, developers will collect the feedback provided by the testers and evaluate it. In the end, based on the feedback, issues, and bugs are fixed and resolved by the responsible individual team.
5. **Closure:** When all the problems fixed and the organization meets the exit criteria, beta testing achieved, and the rewards offered to the testing team.

The types of beta testing

Beta testing has six types. Each type has different aspects of the software. All these help developers to improve the quality of the software and allow them to deliver a product that offers excellent user experience.

Here are the different types of beta testing:

1. **Open Beta Testing:** Open beta testing involves testing the software product by a large number of people before the final release. The organization decides to make a software product open to the public before releasing the product. Open Beta includes the extensive participation of the public to use and evaluate software product accordingly.

Users report the bug to the organization, along with a suggestion to improve the quality of the software.
2. **Closed Beta Testing:** Opposite to the open beta testing. Closed beta testing performed by the selective and limited number of persons. The organization recruits these. In this testing software product is not open to the public.
3. **Traditional Beta Testing:** In this testing, a software product delivered to the target market, and the feedback from the users collected. This type of testing assistance the beta testing, the quality of the software is improved, and developers can make the changes.
4. **Public Beta Testing:** This type of testing is similar to open testing. Public beta testing also allows the product is delivering to the end-users worldwide, with the

Notes

Notes

aid of various online channels available in the world. From this, the feedback and evaluated data also collected and based on the requirement changes, and the development team implements modifications.

5. **Technical Beta Testing:** Technical beta testing is also an essential type of beta testing. This testing involves delivering the software product to the internal groups of the organization. However, the data and feedback provided by the employees of the organization.
6. **Focused Beta Testing:** This type of testing focused on monitoring and evaluating a specific feature or component of the software. In focused beta testing, the software released to the market and user's experience assessed and collected to make the required changes.
7. **Post-Release Beta Testing:** In this testing, the product delivered to the market for the use of the end-users. Their feedback, reactions, and experience are collect for the future release of the software.

Perform Beta Testing

Acceptance testing is the final phase of the testing, which combines both alpha and beta testing to ensure that the product released flawlessly. Beta testing performed at the user's end. This testing always performed after the alpha testing, but before the product released to the market. In this stage, the product is expected to be 90% to 95% completed.

Any product undergoing to beta test should be reviewed for the entire checklist before launching it.

Some of them are:

- All the component of the product is ready to start this testing.
- Documentation which is going to end-user should be kept ready - Setup, installation, usage, Uninstallation should be in detail.
- The product management team should review that all the functionality is in good condition.
- Procedure to collect bugs, feedback, etc. should be identified before publishing it.

Stakeholders and participants in the Beta Testing

The product management, quality management, and user experience teams are the stakeholder in beta testing, and they closely monitor every move of the phase.

The real users who use the product are the participants.

Beta test strategy

- Business objective for the product.
- Beta test plan
- The testing approach followed by participants.

- Tools used to detect bugs, measure productivity, collect feedback.
- When and how to end this testing phase?

Beta Test plan

A beta test plan can be written in many ways,

Objective: We should have to mention the aim of the project why there is a need for beta testing even after performing the internal testing.

Scope: In this plan, we should mention the areas to be tested or not.

Test Approach: We should have to mention clearly that the testing is in the deep, what to focus on - functionality, UI, response, etc.

Schedule: We have to specify, clearly the start and ending date with time, number of cycles, and duration per cycle.

Tools: Bug logging tools and the usage of the machines should identify.

Budget: Incentive of the bugs based on the severity.

Feedback: Collecting feedback and evaluating methods.

- *Identify and review the entry and exit criteria.*

The entry criteria for Beta Testing?

- Sign off the document from alpha testing.
- Beta version of the software should ready.
- The environment should be ready to release the software application to the public.
- To capture the real-time faults environment should be ready.

The exit criteria for Beta Testing?

- All the major and minor issues resolved.
- The feedback report should prepare.
- The delivery of beta test summary report.

The advantages of Beta Testing

Beta testing performed at the end of the software testing lifecycle. Beta testing offers numerous benefits to testers, software developer, as well as the users. In the assistance of this type of testing, it enables developers, testers to test the product before its release in the market. The

1. Beta testing focuses on the customer's satisfaction.
2. It helps to reduce the risk of product failure via user validations.
3. Beta testing helps to get direct feedback from users.
4. It helps to detect the defect and issues in the system, which is overlooked and undetected by the team of software testers.

Notes

5. Beta testing helps the user to install, test, and send feedback regarding the developed software.

The disadvantages of Beta Testing

Disadvantages of beta testing is:

1. In this type of testing, a software engineer has no control over the process of the testing, as the users in the real-world environment perform it.
2. This testing can be a time-consuming process and can delay the final release of the product.
3. Beta testing does not test the functionality of the software in depth as software still in development.
4. It is a waste of time and money to work on the feedback of the users who do not use the software themselves properly.

2.5.3 Scalability testing, Reliability, Stress Testing

Scalability Testing

Scalability Testing is a type of non-functional testing in which the performance of a software application, system, network or process is tested in terms of its capability to scale up or scale down the number of user request load or other such performance attributes. It can be carried out at a hardware, software or database level. Scalability Testing is defined as the ability of a network, system, application, product or a process to perform the function correctly when changes are made in the size or volume of the system to meet a growing need. It ensures that a software product can manage the scheduled increase in user traffic, data volume, transaction counts frequency and many other things. It tests the system, processes or databases ability to meet a growing need.

Scalability Testing is to measure at what point the software product or the system stops scaling and identify the reason behind it. The parameters used for this testing differs from one application to the another. For example, scalability testing of a web page depends on the number of users, CPU usage, network usage while scalability testing of a web server depends on the number of requests processed.

Objective of Scalability Testing:

The objective of scalability testing is:

- To determine how the application scales with increasing workload.
- To determine the user limit for the software product.
- To determine client-side degradation and end user experience under load.
- To determine server-side robustness and degradation.

Scalability Testing Attributes:

- **Response Time:** Response time is the time consumed between the user's request and the application's response. Response time may increase or decrease based on different user load on the application. Basically, the response time of an application decreases as the user load increases. Application having the lesser

Notes

response time is considered as the higher performance application.

- **Throughput:** Throughput is the measurement of number of requests processed in a unit time by the application. It differs from one application to another as in web application it is measured in number of user requests processed in a unit time whereas in database application it is measured in number of queries processed in a unit time.
- **Performance measurement with number of users:** Depending on the application type, it is always tested for the number of users that it can support without its breakdown or busy standby situation.
- **Threshold load:** Threshold load is the number of requests or transactions the application can process with desired throughput.
- **CPU Usage:** CPU Usage is the measurement of the CPU utilization while executing application code instructions. It is basically measured in terms of the unit *Megahertz*.
- **Memory Usage:** Memory usage is the measurement of the memory consumed for performing a task by an application. It is basically measured in terms of the unit *bytes*.
- **Network Usage:** Network usage is the measurement of the bandwidth consumed by an application under test. It is measured in terms of bytes received per second, frames received per second, segments received and sent per second etc.

Steps involved in Scalability Testing:

Following are the steps involve in the scalability testing:

1. Define a process that is repeatable for executing scalability test.
2. Determine the criteria for scalability test.
3. Determine the software tools required to carry out the test.
4. Set the testing environment and configure the hardware required to execute scalability test.
5. Create and verify visual script.
6. Create and verify the load test scenarios.
7. Execute the test.
8. Evaluate the result.
9. Generate required report.

Reliability Testing

While developing a software or an application, there are several characteristics and aspects of the software, authenticated by the software engineers.

To deliver a quality product to the end-users is one of the essential features of software reliability that is tested and verified by the team with the help of the assistance of reliability testing.

Notes

Reliability Testing

Another essential **software testing** technique is **reliability testing**. The team implements reliability testing to guarantee the software or an application is performing and functioning reliably in each environmental condition with a particular period.

The implementation of reliability testing makes sure that the software product is defect-free and reliable for its planned objective.

In simple words, we can say that it verifies whether the software can achieve a failure-free operation for a specific period in a technological environment or not.

In general terms, Reliability signifies **yielding the same**; in other words, **reliable** means **something is dependable**, and both will provide the same outcome each time. And the same is true in the case of **Reliability testing**.

Reliability is described as the **probability of defect-free software operation** for a particular time in a specific environment. Mainly it is implemented to guarantee that the software is consistent.

The end-users think that the data shown is correct, and the software will continuously operate appropriately. In the above situations, the need for reliability testing comes into the illustration.

The function of defining that the system has been thoroughly verified and tested is the true meaning of software reliability.

The purpose of Reliability Testing

Following are some of the important objective of performing the reliability testing:

- To test the working of software under a specific condition is the significant purpose of Reliability testing.
- The purpose of executing the reliability testing is to determine the number of defects in the specific period.
- To determine the perceptual structure of monotonous breakdown.
- To find the nasty presence of the software.
- To identify the leading cause of failure.

Characterises of Reliability Testing

Some of the significance of reliability testing which can further understand with the assistance of its various features are as discussed below:

- The reliability testing guarantee that the software meets the client's along with customer's reliability necessities.
- It ensures that the software is performing without any faults and is working as per its planned determination.
- It can be described as the ability of the software to work precisely under the detailed conditions.
- It is a significant surface of reliability engineering programs.

Notes

- The data is collected from several phases **of software development** at the time of reliability testing.
- Before the product is delivered to the end-users, reliability testing helps us to identify defects and faults in the software product.
- It is executed at several software development levels to enhance the reliability and dependability of the product.

Types of Reliability Testing

In software testing, reliability testing categorizes into three types which are discussed as below:

- **Feature Testing**
- **Regression Testing**
- **Load Testing**

Feature Testing

The first type of reliability testing is **feature testing**. The feature testing concludes the suitability, which implies that it checks if it executes as expected for its intended use.

The main objective of feature testing is to test the attribute and the functionality of the software product, the correctness of the system and to verify if there are no defects identified throughout the beta testing.

In fulfilment of this testing, we will check if the application follows specific criteria such as **standards, rules**, etc.

Some of the below characteristics are checked during the execution of feature testing:

- All function needs to be done at least once by the team.
- Each function must implement completely.
- The team should also check the proper implementation of each operation.
- The communication between two or more function have to be reduced.

Note: Apart from this, the feature testing verifies some security and compliance. As we already understand, security testing is related to the inhibition of illegal access to the application, either purposely or accidentally.

Regression Testing

Another part of reliability testing is **Regression testing**. The regression testing is mainly used to check whether any new bugs or defects have been declared because of earlier bugs fixing.

In simple words, we can say that the Regression testing can be implemented on a new build when there is an essential change in the original functionality.

The implementation of regression testing guarantees that the code still works even when the changes are happening.

Notes

Regression means **Re-test** those parts of the application, which are unchanged.

In regression testing, verification of **software quality** and the new components accuracy added to the software product. And the identification of bugs or defects that are generated in the system after the execution of changes.

Load Testing

The next type of Reliability testing is **Load testing**. This test is concluded to confirm the functionality of the software under the circumstances of maximum workload.

Load testing is testing where we check an application's performance by applying some load, which is either less than or equal to the desired load.

In load testing, the term load implies that when **N-number** of users using the application simultaneously or sending the request to the server at a time.

This type of testing will help us to detect an application's maximum operating capacity and any **blockages** or bottlenecks.

The reliability testing can be evaluated in terms of *Mean time between failures* (MTBF).

- **Mean Time Between Failures (MTBF):**

The MTBF is combines **Mean time to failure (MTTF)** and **Mean time to Repair (MTTR)**.

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

Here,

- **Mean Time To Failure (MTTF)**
- The mean time to failure (MTTF) is the time between two consecutive failures.
- **Mean Time To Repair (MTTR)**
- Mean time to repair (MTTR) is the time taken to fix the failures.

How to Perform Reliability Testing

To perform any software testing, we need proper planning and management. The execution of Reliability Testing is a bit costlier as compared to other types of software testing.

The implementation of reliability testing includes **data for the test environment, test schedule, test points, etc.**

The test engineer has to retain the following aspects in order to start the implementation of reliability testing:

- The test engineer needs to use the test results to drive decisions.
- They need to determine the reliability goals.
- They need to plan and execute tests.
- The test engineer needs to cultivate the effective profile.

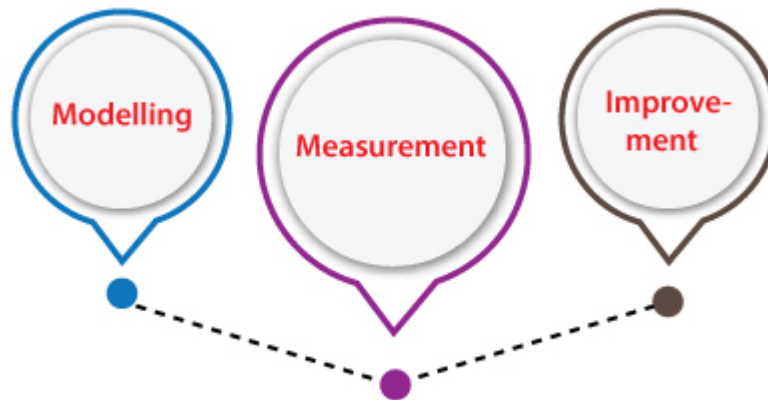
Notes

The reliability testing contains some essential constraints, which are as follows:

- Measurement of time of bug-free operation.
- Possibility of bug-free operation.
- The environment in which it is performed.

The analysis of **reliability testing** can be divided into **three parts**, which are as discussed below:

- **Modelling**
- **Measurement**
- **Improvement**



How to Perform Reliability Testing

For our better understanding, let us discuss them one by one in details:

Part1: Modelling

We can achieve a meaningful outcome by applying suitable models. In reliability testing, the **Software Modelling Technique** can be divided into two subcategories, which are as below:

- **Prediction Modelling**
- **Estimation Modelling**

The **prediction modelling and estimation modelling** have some significant differences, which we understand from the below table:

Issues	Prediction Models	Estimation Models
When used in Development Cycle	Generally, the prediction model is developed before the development or testing stages.	Generally, the estimation models are developed at the later stage of the Software Development Life Cycle (SDLC).
Data Reference	The prediction models use historical data.	The estimation models use existing data from software development.
Time Frame	Prediction models will predict consistency in the future.	The estimation models will predict the reliability for both the present time or in the future time.

Notes

Part2: Measurement

In software testing, we cannot directly measure the Software reliability. Therefore, some other connected aspects are considered in order to evaluate the software reliability.

The **Software Reliability Measurement** are divided into four types, which are as follows:

1. **Product Metrics**
2. **Fault and Failure Metrics**
3. **Process Metrics**
4. **Project Management Metrics**

Software Reliability Measurement

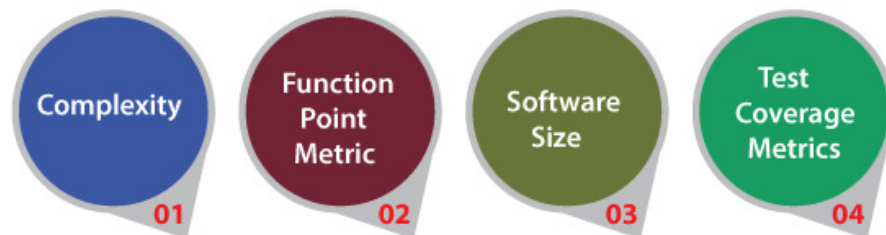


Let see a detailed information of the software reliability measurement parts:

1. Product Metrics

The **Product metrics** is the first part of software reliability measurement, which are the combination of four different types of metrics:

Product Metrics



- **Complexity**
- **Function point Metric**
- **Software size**
- **Test Coverage Metrics**

Complexity

- The representation of complexity is significant as it is directly related to software reliability.
- It is a process of defining a program's control structure's difficulty by simplifying the code into a graphical illustration.

Notes

Function point Metric

- Another metric that comes under the product metric is the **Function point Metric**, which is mainly used for determining the functionality of Software Development.
- The function point metric will take the count of inputs, outputs, master files, etc.
- It calculates those components which are delivered to the user and is independent of the programming language.

Software size

- The next part of the product metric is **software size**. In this Metric, we can use the Line of Code (LOC) for measuring the size of the software as it is an in-built initial approach.

Test Coverage Metrics

- The test coverage metrics approximate the fault and reliability by implementing the full test of software products.

2. Fault and Failure Metrics

In fault and failure metrics, we used to check whether the system is entirely bug-free or not. Both the types of faults were identified at the time of the testing process, which means before delivering the product.

And to accomplish the objective, we need to check and summarize the failure reported by users after delivery.

3. Process Metrics

The process metrics are used to evaluate, display, and enhance the software's reliability and quality. As we already knew that the quality of the product is directly related to the process.

4. Project Management Metrics

The last part of software reliability measurement is **project management metrics**. We know that good management can accomplish advanced reliability by using a better risk management process, development process, configuration management process, etc.

Part 3: Improvement

The next part of software reliability testing is **Improvement**. The Improvement is entirely depending upon the difficulties that happened in the application or system or the software features.

The method of Improvement will also vary as per the complexity of the software module. The **time and budget** are the two main constraints that will limit the attempts are put into the software reliability improvement.

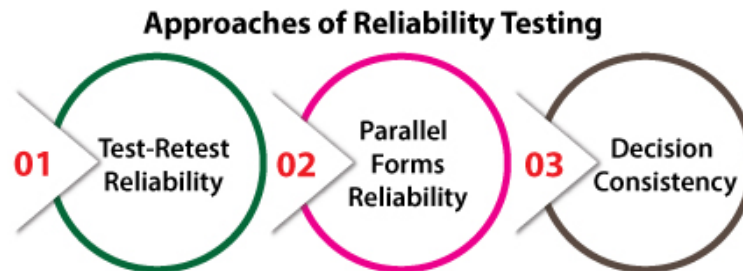
As we know that, so far, there is no good approach to overcome the complexity problem of software.

Notes

Approaches of Reliability Testing

In software testing, we will execute the reliability testing to make sure that all the bugs and faults present in the system are detected and fixed by the specified team before deployed to the user site.

Sometimes it is not easy to evaluate the precise reliability. But it can be assessed with the help of several methods. Some of the most commonly used approaches of reliability testing are as discussed below:



- **Test-Retest Reliability**
- **Parallel Forms Reliability**
- **Decision Consistency**

Test-Retest Reliability

- The first approach of reliability testing is **Test-retest reliability**. This type of reliability method explains how a test can give stable, reliable scores within time.
- In order to test the constancy and the reliability of the score, it is performed in a short period. In simple words, we can say that the team tests and **retests** the software over several testing techniques to confirm its reliability and constancy.

Parallel Forms Reliability

- The second method of software reliability testing is **parallel forms reliability**.
- In this particular approach, the software is tested by two distinct groups to test how equally the two function forms.
- By using this method, the team can validate the reliability of the outcome through several forms.

Decision Consistency

- The last approach of reliability testing is **decision consistency**.
- In this method, the output from the first two approaches (test-retest reliability and parallel forms reliability) are tested during the decision consistency. And concludes their success and failure.

Overview

In the reliability testing article, we have understood the **working of reliability testing, its types, features, different approaches, evaluation of reliability testing,**

and the process of reliability testing.

After seeing all the topics mentioned above, we can conclude that software reliability is a significant part of software quality.

Comparing Reliability Testing to other types of testing is a bit costlier process as it requires proper test planning and test management.

The execution of **reliability testing** ensures the development team and clients that the software being established is highly consistent and works in the desired manner. And it doesn't matter whether it is expansive or not.

If we are talking about the software development life cycle, reliability testing plays a significant role. As we discussed above, the usage of reliability standards will bring constancy to the software and forecast the future of the software.

Reliability testing helps to make sure that we are delivering a quality end-product. It helps us to identify and fix bugs, improve performance, and test features.

Repeatedly, if the software has a high complexity, then software reliability is hard to get. And we can guarantee that our software is working exactly how it should be with the implementation of reliability testing.

Stress Testing

In software testing, stress testing is an important part of performance testing under non-functional testing.

Stress Testing is testing used to check the accessibility and robustness of software beyond usual functional limits. It mainly considers for critical software but it can also be used for all types of software applications.

It is also known as **Endurance Testing**, **fatigue testing** or *Torture Testing*.

The stress testing includes the **testing beyond standard operational size**, repeatedly to a **breaking point**, to get the outputs.

It highlights the error handling and robustness under a heavy load instead of correct behavior under regular conditions.

In other words, we can say that **Stress testing** is used to verify the constancy and dependability of the system and also make sure that the system would not crash under disaster circumstances.

To analyses how the system works under extreme conditions, we perform **stress testing** outside the normal load.

The Objective of Stress Testing

The main objective of using stress testing is to fulfill the following aspects:

- The primary purpose of executing the stress testing is **to confirm that the software does not crash in lacking computational resources like disk space, memory, and network request.**

Notes

Notes

- The implementation of stress testing certifies that the system fails and improves effortlessly, known as **the recoverability process**.
- We can use stress testing to discover hardware issues, data corruption issues.
- Stress testing will help us to identify the security weaknesses that might sneak-in throughout constant peak load.
- It helps determine the software application's data integrity throughout the extreme load, which implies that the data should be in a dependable state after a failure.

Features of Stress Testing

Following are the basic features of stress testing:

- Stress testing also makes sure that unpredicted failures do not cause security issues.
- It is used to analyze the system works under rare circumstances and the system's behavior after a failure.
- Stress testing is used to check the system has saved the data before crashing or not.
- Stress testing guarantees to display a suitable error message when the system is under stress.

Why do we need to perform the Stress Testing?

We need to perform stress testing if we encounter the following situations:

Whenever **e-commerce or online shopping sites** announce a sale during the festival may witness a spike in traffic. Or when an article is mention in a top newspaper, its knowledges an unexpected flow in traffic.

If we fail to assist this sudden traffic can result in loss of profits and status. So, in that case, we need to execute the **Stress Testing to integrate such irregular traffic spikes**.

Stress testing is also needed to be performed for the below scenarios:

- When the system is under stress, it should display a proper error message.
- To inspect whether the system works under bizarre circumstances.
- If the system is failed under risky situations could result in huge profits loss.
- By implementing Stress Testing, we will be prepared for extreme conditions.

Example of Stress Testing

Let's see some **real-time examples** where we can discover the usage of Stress Testing.

Example 1: E-commerce website/ application

Throughout the new product releases, sales, holidays, promotions, and festival offers, the e-commerce application tends to get many users in a very short time.

Notes

Example 2: News website at the time of some major/viral event

The news website will crash or slow down when a major event happens; **for example**, when Michael Jackson passed away, a maximum number of news websites are slow down, or some of them also crashed.

To overcome this situation, we need to perform stress testing on the particular application and be prepared to recover quickly in case of any crashes or failure.

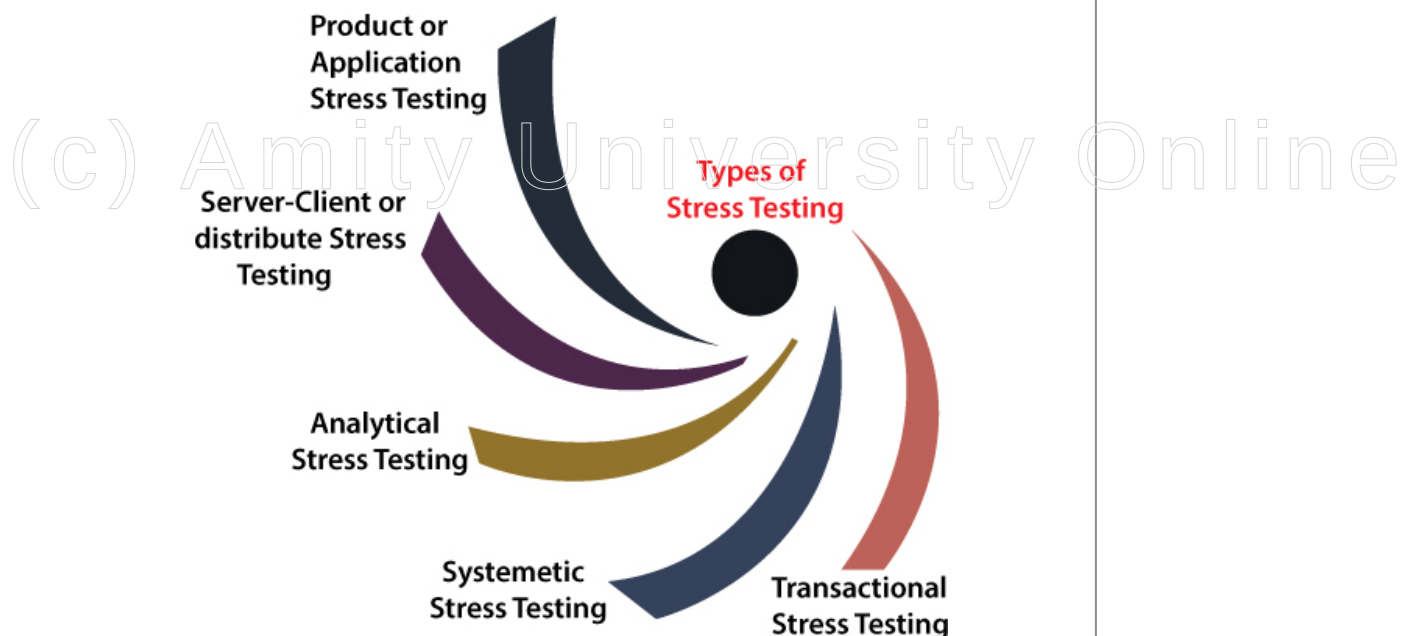
Example 3: Education Board's result website

Stress testing is important to perform on the education board's result website. On the day of some results, many students, users, and applicants will logins to the particular to check their grades.

Therefore, the execution of stress testing helps identify the failure of the application and evaluate the performance and recoverability when the ultimate load occurs in a short duration or when the result is out.

Types of Stress Testing

Stress testing can be categories into various parts, which are as follows:



- Product or Application stress testing
- Server-client or distribute Stress Testing
- Analytical Stress Testing
- Systematic Stress Testing
- Transactional Stress Testing

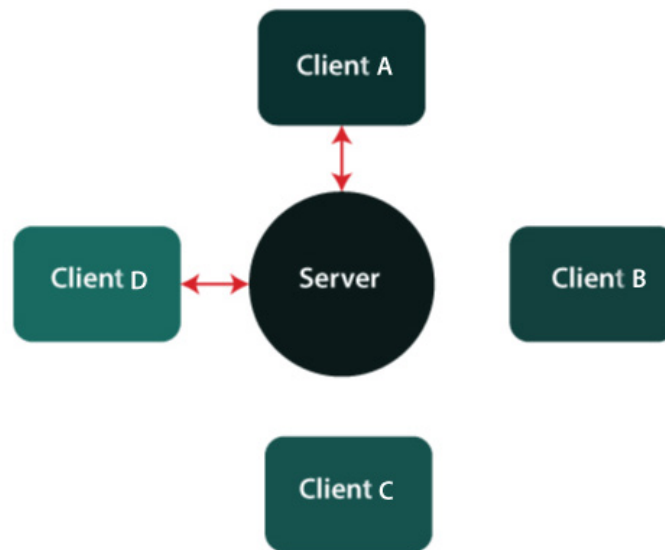
Product or Application Stress Testing

- The application or product stress testing is mainly focused on determining the faults related to network issues, data locking, blocking, and a performance bottleneck in a software product.

Notes

Server-client or Distribute Stress Testing

- In this type of stress testing, all the clients related to the server are tested.
- The distribute stress testing is used to perform across all clients from the server.
- The server can communicate with **clients A and B**, but it cannot link with **clients C and D** when there is stress on the client-server system as we can see in the below image:



Analytical/ Exploratory Stress Testing

- Analytical or exploratory testing is used to execute the system with unusual constraints unlikely to occur in a real scenario.
- It is mainly used to identify the bugs in rare situations such as **a large number of users logged simultaneously or a database went offline when retrieved from a website.**

Let see some examples of **analytical Stress Testing** where such irregular conditions are used:

- When a large number of parallel users try to log into the application.
- Data is added in enormously large quantity in the database.
- When the website tries to reach it from the front end, and database linked to the website shuts down.

Systematic Stress Testing

- It is combined testing used to execute the test across various systems running on a similar server.
- Using **systematic stress testing**, we can easily detect the bottleneck where data of one application blocks another application.

Notes

Transactional Stress Testing

- Another type of stress testing is **transactional stress testing**, which is used to implemented one or more transactions between various applications.
- The main objective of performing the transactional stress testing is **to enhance the system performance**.

Process of Stress testing / how to perform stress testing

The stress testing process will be completed into the following steps:



Step1: Detect the testing environment

In the first step of stress testing, we will identify the network, software, and hardware configurations and tools available to achieve the stress test.

Step2: Find performance acceptance criteria

After identifying the testing environment, we will find the performance acceptance criteria, which help us categorize the metrics used to test the application's performance under stress.

And also, identifying the success criteria for a stress test, for example, the maximum load can apply to the application for it to fail.

Step3: Plan and design stress tests

In the next step of the stress testing process, we will plan and design a stress test plan, identify test scenarios etc.

Notes

Step4: Configure the test environment

Once the stress test plan has been created successfully, we will move to our next step where we create the test environment, tools and resources essential to perform each approach as features and components become available for test.

Step5: Implement test design

After the test environment's configuration, we will develop the stress tests resulting the test design best performs.

Step6: Execute tests

In the next step, we will execute the particular test, observe and confirm the tests along with test data and output collection.

Step7: Analyze the results

In the last step of the stress testing process, we will analyze the outcomes, combine and share the respective teams' output data.

Stress testing tools

As we know that **stress testing** is part of **performance testing**, the tools used for performance testing can be used for stress testing. Therefore, we have various types of **Stress testing** tools available in the market, where some are commercial tools and open-source tools. Some of the most commonly **Stress testing** are listed below:

- **Apache JMeter**
- **NeoLoad**
- **Stress tester**
- **LoadRunner**

Advantages and disadvantages of Stress Testing

Advantages

Some of the vital **benefits** of performing Stress testing is as follows:

- Stress testing signifies the system's behavior after failure and makes sure that the system recovers quickly from the crashes.
- The most important advantage of executing the stress testing will make the system work in regular and irregular conditions in a suitable way.
- It determines the scalability and enhance the performance of the software.

Disadvantages

Some of the most common **drawbacks** of Stress testing are as follows:

- Even in open-source tools like JMeter, a load testing environment is required, which should be as close to the production environment setup as possible.
- If we are writing the Stress test script, the person should have enough scripting knowledge of the language supported by the particular tool.

Notes

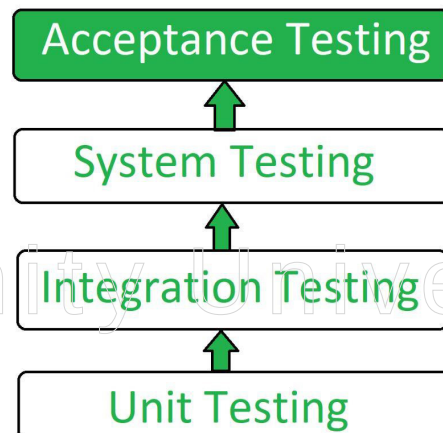
- If we are using stress testing, it will require additional resources, which makes this testing bit costlier.
- If we perform the **Stress Testing** manually, it became a tedious and complicated task to complete, and it may also not produce the expected results.

2.5.4 Acceptance testing: Acceptance criteria, test case selection and execution

Acceptance Testing

It is a formal testing according to user needs, requirements and business processes conducted to determine whether a system satisfies the acceptance criteria or not and to enable the users, customers or other authorized entities to determine whether to accept the system or not.

Acceptance Testing is the last phase of software testing performed after System Testing and before making the system available for actual use.



Types of Acceptance Testing:

1. **User Acceptance Testing (UAT):** User acceptance testing is used to determine whether the product is working for the user correctly. Specific requirements which are quite often used by the customers are primarily picked for the testing purpose. This is also termed as *End-User* Testing.
2. **Business Acceptance Testing (BAT):** BAT is used to determine whether the product meets the business goals and purposes or not. BAT mainly focuses on business profits which are quite challenging due to the changing market conditions and new technologies so that the current implementation may have to be changed which result in extra budgets.
3. **Contract Acceptance Testing (CAT):** CAT is a contract which specifies that once the product goes live, within a predetermined period, the acceptance test must be performed and it should pass all the acceptance use cases. Here is a contract termed as Service Level Agreement (SLA), which includes the terms where the payment will be made only if the Product services are in-line with all the requirements, which means the contract is fulfilled.

Notes

Sometimes, this contract happens before the product goes live. There should be a well defined contract in terms of the period of testing, areas of testing, conditions on issues encountered at later stages, payments, etc.

4. **Regulations Acceptance Testing (RAT):** RAT is used to determine whether the product violates the rules and regulations that are defined by the government of the country where it is being released. This may be unintentional but will impact negatively on the business.

Generally, the product or application that is to be released in the market, has to go under RAT, as different countries or regions have different rules and regulations defined by its governing bodies. If any rules and regulations are violated for any country then that country or the specific region then the product will not be released in that country or region. If the product is released even though there is a violation then only the vendors of the product will be directly responsible.

5. **Operational Acceptance Testing (OAT):** OAT is used to determine the operational readiness of the product and is a non-functional testing. It mainly includes testing of recovery, compatibility, maintainability, reliability etc. OAT assures the stability of the product before it is released to the production.

6. **Alpha Testing:** Alpha testing is used to determine the product in the development testing environment by a specialized testers team usually called alpha testers.

7. **Beta Testing:** Beta testing is used to assess the product by exposing it to the real end-users, usually called beta testers in their environment. Feedback is collected from the users and the defects are fixed. Also, this helps in enhancing the product to give a rich user experience.

Use of Acceptance Testing:

- To find the defects missed during the functional testing phase.
- How well the product is developed.
- A product is what actually the customers need.
- Feedbacks help in improving the product performance and user experience.
- Minimize or eliminate the issues arising from the production.

Acceptance criteria (AC)

Acceptance criteria (AC) are the conditions that a software product must meet to be accepted by a user, a customer, or other systems. They are unique for each user story and define the feature behaviour from the end-user's perspective.

Acceptance criteria are the lowest-level functional requirements

Well-written acceptance criteria help avoid unexpected results in the end of a development stage and ensure that all stakeholders and users are satisfied with what they get.

An important aspect in regard to acceptance criteria is that they have to be defined before the development team starts working on a particular user story. Otherwise, there's a decent chance the deliverables won't meet the needs and expectations of a client.

Notes

Acceptance criteria main purposes

Clarifying the stakeholder's requirements is a high-level goal. To make the purposes of AC clearer, let's break them down.

Making the feature scope more detailed. AC define the boundaries of user stories. They provide precise details on functionality that help the team understand whether the story is completed and works as expected.

Describing negative scenarios. Your AC may require the system to recognize unsafe password inputs and prevent a user from proceeding further. Invalid password format is an example of a so-called negative scenario when a user does invalid inputs or behaves unexpectedly. AC define these scenarios and explain how the system must react to them.

Setting communication. Acceptance criteria synchronize the visions of the client and the development team. They ensure that everyone has a common understanding of the requirements: Developers know exactly what kind of behavior the feature must demonstrate, while stakeholders and the client understand what's expected from the feature.

Streamlining acceptance testing. AC are the basis of the user story acceptance testing. Each acceptance criterion must be independently testable and thus have clear pass or fail scenarios. They can also be used to verify the story via automated tests.

Conducting feature evaluations. Acceptance criteria specify what exactly must be developed by the team. Once the team has precise requirements, they can split user stories into tasks that can be correctly estimated.

Since different people can have different points of view and solution ideas regarding one problem, creating a unified vision of how the functionality should be implemented is necessary. That's exactly what clearly written acceptance criteria do.

Acceptance criteria types and structures

Based on the initial task and the complexity of requirements, acceptance criteria can be written in different formats, namely:

- scenario-oriented (the Given/When/Then template);
- rule-oriented (the checklist template); and
- custom formats.

Most commonly used, the first and the second formats have very specific structures, so we'll mainly focus on them. However, you may find that other formats fit your product better so we'll briefly touch on them as well.

Scenario-oriented acceptance criteria

As the name suggests, the scenario-oriented format is the acceptance criteria type that comes in the scenario form and illustrates each criterion. It is approached through the *Given/When/Then* (GWT) sequence that looks like this:

- *Given* some precondition
- *When* I do some action

Notes

- *Then* I expect some result

This approach was inherited from behaviour-driven development (BDD) and provides a consistent structure that helps testers define when to begin and end testing a particular feature. It also reduces the time spent on writing test cases as the behaviour of the system is described upfront.

The acceptance criteria template in this format includes the following statements:

1. Scenario – the name for the behavior that will be described
2. Given – the beginning state of the scenario
3. When – specific action that the user makes
4. Then – the outcome of the action in “When”
5. And – used to continue any of three previous statements

Test Cases Selection and Execution

A test case includes not only input data but also any relevant execution conditions and procedures, and a way of determining whether the program has passed or failed the test on a particular execution.

The term “input” is used in a very broad sense, which may include all kinds of stimuli that contribute to determining program behaviour. For example, an interrupt is as much an input as is a file. The pass/fail criterion might be given in the form of expected output, but could also be some other way of determining whether a particular program execution is correct. A test case specification is a requirement to be satisfied by one or more actual test cases. The distinction between a test case specification and a test case is similar to the distinction between a program specification and a program. A test case specification might be met by several different test cases, and vice versa. Suppose, for example, we are testing a program that sorts a sequence of words. “The input is two or more words” would be a test case specification, while test cases with the input values “alpha beta” and “Milano Paris London” would be two among many test cases satisfying the test case specification. A test case with input “Milano Paris London” would satisfy both the test case specification “the input is two or more words” and the test case specification “the input contains a mix of lower- and upper-case alphabetic characters.” Characteristics of the input are not the only thing that might be mentioned in a test case specification. A complete test case specification includes pass/fail criteria for judging test execution and may include requirements, drawn from any of several sources of information, such as system, program, and module interface specifications; source code or detailed design of the program itself; and records of faults encountered in other software systems. Test specifications drawn from system, program, and module interface specifications often describe program inputs, but they can just as well specify any observable behavior that could appear in specifications. For example, the specification of a database system might require certain kinds of robust failure recovery in case of power loss, and test specifications might therefore require removing system power at certain critical points in processing. If a specification describes inputs and outputs, a test specification could prescribe aspects of the input, the output, or both. If the specification is modeled as an extended finite-state machine, it might require

executions corresponding to particular transitions or paths in the state-machine model. The general term for such as

Notes

Testing Terms

While the informal meanings of words like “test” may be adequate for everyday conversation, in this context we must try to use terms in a more precise and consistent manner. Unfortunately, the terms we will need are not always used consistently in the literature, despite the existence of an IEEE standard that defines several of them. The terms we will use are defined below.

Test case: A test case is a set of inputs, execution conditions, and a pass/fail criterion. (This usage follows the IEEE standard.)

Test case specification: A test case specification is a requirement to be satisfied by one or more actual test cases. (This usage follows the IEEE standard.) **Test obligation:** A test obligation is a partial test case specification, requiring some property deemed important to thorough testing. We use the term “obligation” to distinguish the requirements imposed by a test adequacy criterion from more complete test case specifications.

Test suite: A test suite is a set of test cases. Typically, a method for functional testing is concerned with creating a test suite. A test suite for a program, system, or individual unit may be made up of several test suites for individual modules, subsystems, or features. (This usage follows the IEEE standard.)

Test or test execution: We use the term test or test execution to refer to the activity of executing test cases and evaluating their results. When we refer to “a test,” we mean execution of a single test case, except where context makes it clear that the reference is to execution of a whole test suite. (The IEEE standard allows this and other definitions.)

Adequacy criterion: A test adequacy criterion is a predicate that is true (satisfied) or false (not satisfied) of a program, test suite pair. Usually a test adequacy criterion is expressed in the form of a rule for deriving a set of test obligations from another artifact, such as a program or specification. The adequacy criterion is then satisfied if every test obligation is satisfied by at least one test case in the suite.

Test Execution: Test execution is the process of executing the code and comparing the expected and actual results. Following factors are to be considered for a test execution process:

- Based on a risk, select a subset of test suite to be executed for this cycle.
- Assign the test cases in each test suite to testers for execution.
- Execute tests, report bugs, and capture test status continuously.
- Resolve blocking issues as they arise.
- Report status, adjust assignments, and reconsider plans and priorities daily.
- Report test cycle findings and status.

Notes

Check your Understanding

1. What is testing process' first goal?
 - a) Bug prevention
 - b) Testing
 - c) Execution
 - d) Analyses
2. Software mistakes during coding are known as
 - a) errors
 - b) failures
 - c) bugs
 - d) defects
3. Name an evaluation technique to assess the quality of test cases.
 - a) Mutation analysis
 - b) Validation
 - c) Verification
 - d) Performance analysis
4. Test should be conducted for every possible
 - a) data
 - b) case
 - c) variable
 - d) all of the mentioned
5. Which of the following is not a part of bug report?
 - a) Test case
 - b) Output
 - c) Software Version
 - d) LOC
6. Which of the following is not a part of Execution Flow during debugging?
 - a) Step Over
 - b) Step Into
 - c) Step Up
 - d) Step Out
7. Cyclomatic Complexity method comes under which testing method.
 - a) Yellow box
 - b) White box
 - c) Gray box

Notes

- d) Black box
8. Which is a black box testing technique appropriate to all levels of testing?
- a) Acceptance testing
 - b) Regression testing
 - c) Equivalence partitioning
 - d) Quality assurance
9. Which of the following is the way of ensuring that the tests are actually testing code?
- a) Control structure testing
 - b) Complex path testing
 - c) Code coverage
 - d) Quality assurance of software
10. Effective testing will reduce _____ cost.
- a) Maintenance
 - b) design
 - c) coding
 - d) documentation

Summary

- It is a type of **non-functional testing** and generally executed after the **functional testing**.
- **Stress testing** entirely concentrate on testing the system under extreme load situations to detect its **breaking point** and see if suitable messages are shown when the system is not responsive.
- The system status or behaviour after you conduct a test. An anomaly or deviation is when your actual results differ from the expected results.
- Testing carried out informally without test cases or other written test instructions.
- A development method that emphasizes working in short iterations. Automated testing is often used. Requirements and solutions evolve through close collaboration between team members that represent both the client and supplier
- Testing in which the test object is seen as a “black box” and the tester has no knowledge of its internal structure. The opposite of white box testing.
- The process in which developers identify, diagnose, and fix errors found. See also bug and defect.
- Testing performed without running the system. Document review is an example of a static test.
- Testing meant to assess how the system reacts to workloads (network, processing, data volume) that exceed the system’s specified requirements. Stress testing shows which system resource (e.g. memory or bandwidth) is first to fail.

Notes

Activity

1. State the importance of various approaches to testing.

Questions & Exercises

1. Explain Static Testing
2. What is Structured Group Examinations?
3. Explain Static Analysis Control flow & Data flow Determining Metrics Dynamic Testing
4. What is Black Box Testing with Advantages and disadvantages?
5. Discuss the Cause Effect Graphing and Decision Table Technique and Used Case Testing and Advanced black box techniques.
6. What is White Box Testing? Also explain Deployment testing, Beta testing, Scalability testing, Reliability testing and Stress testing.
7. Briefly explanation acceptance testing.

Glossary

- **Actual result:** The system status or behaviour after you conduct a test. An anomaly or deviation is when your actual results differ from the expected results.
- **Ad hoc testing:** Testing carried out informally without test cases or other written test instructions.
- **Agile development:** A development method that emphasizes working in short iterations. Automated testing is often used. Requirements and solutions evolve through close collaboration between team members that represent both the client and supplier
- **Black box testing:** Testing in which the test object is seen as a “black box” and the tester has no knowledge of its internal structure. The opposite of white box testing.
- **Debugging:** The process in which developers identify, diagnose, and fix errors found. See also bug and defect.
- **Static testing:** Testing performed without running the system. Document review is an example of a static test.
- **Stress testing:** Testing meant to assess how the system reacts to workloads (network, processing, data volume) that exceed the system’s specified requirements. Stress testing shows which system resource (e.g. memory or bandwidth) is first to fail.

Further Readings

1. Software Engineering, Sixth Edition, 2001, Ian Sommerville; Pearson Education.
2. Software Engineering – A Practitioner’s Approach, Roger S. Pressman; McGraw-Hill International Edition.
3. An Integrated approach to Software Engineering, Pankaj Jalote; Narcosis Publishing House.

Notes

4. Software Quality, 1997, Mordechai Ben-Menachem and Garry S.Marliss; Thomson Learning.
5. Emden, D. and Blakeman, K. (1990). Going Automated. London: ASLIB.
6. Gates, H. (1985). Directory of Library and Information Retrieval Software for Microcomputers. Brook-field: Gower Pub. Co.
7. Levert, V.M. (1985). Library Specific Microcomputer Software. Library Software Review. 4 (May-June), 3, 5-38.
8. Nolan, J.M. (1984). Micro Software Evaluation. Torrance CA: Nolan Information Services. Poole, B.J. (1995). Education for an Information Age. Wisconsin: WCB Brown & Benchmark.
9. Rowley, Jennifer (1998). The Electronic Library, 4th Edition of Computers for Libraries. London: Library Association Publishing.
10. Tenopir, Carol (1984), Identification and Evaluation of Software for Micro-computerbased in-house Databases. Information Technology, and Libraries. March , 21-34.

Check your Understanding – Answers

- 1) a
- 2) c
- 3) a
- 4) d
- 5) d
- 6) c
- 7) b
- 8) c
- 9) c
- 10) a

(c) Amity University Online

Module - 3: Test Management

Structure:

- 3.1.1 Test Organization
- 3.1.2 Test teams
- 3.1.3 Tasks and Qualifications
- 3.1.4 Test Planning
- 3.2.1 Quality Assurance Plan
- 3.2.2 Test Plan
- 3.2.3 Prioritization Plan
- 3.2.4 Test Exit Criteria, Cost and economy Aspects
- 3.3.1 Test Strategies
- 3.3.2 Preventive versus Reactive Approach
- 3.3.3 Analytical versus heuristic Approach
- 3.3.4 Test Activity Management
- 3.4.1 Incident Management
- 3.4.2 Configuration Management - Need and Identification
- 3.4.3 Configuration Control
- 3.4.4 Test Progress Monitoring and Control
- 3.5.1 Performance Testing
- 3.5.2 Load Testing
- 3.5.3 Stress testing
- 3.5.4 Security Testing

Objectives:

After studying this chapter, you will be able to:

- Explain the meaning of test management
- Discuss the test organization
- Explain the test teams, tasks and qualifications
- Discuss the, test progress monitoring and control
- Understand the specialized testing

Introduction

Test management, process of managing the tests. A test management is also performed using tools to manage both types of tests, automated and manual, that have been previously specified by a test procedure.

Notes

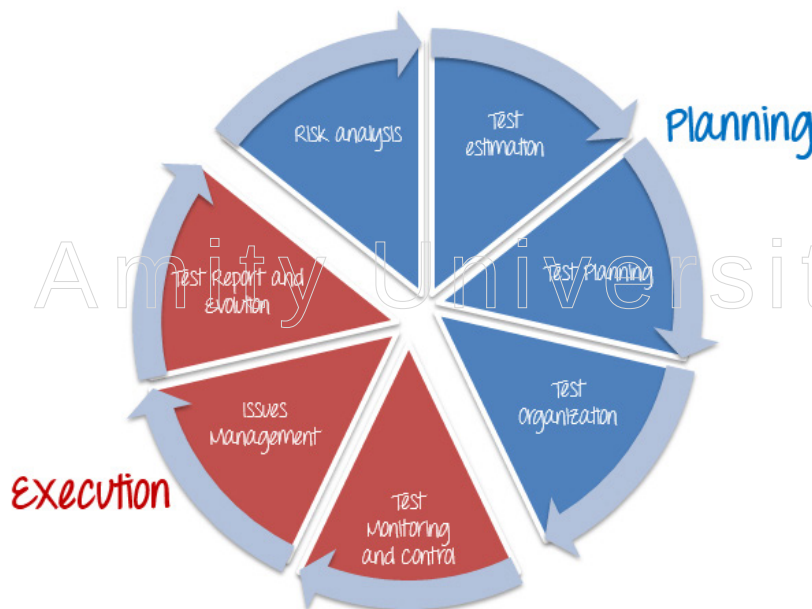
Test management tools allow automatic generation of the requirement test matrix (RTM), which is an indication of functional coverage of the application under test (SUT).

Test Management tool often has multifunctional capabilities such as test ware management, test scheduling, the logging of results, test tracking, incident management and test reporting.

Test Management is a process of managing the testing activities in order to ensure high quality and high-end testing of the software application. The method consists of organizing, controlling, ensuring traceability and visibility of the testing process in order to deliver the high quality software application. It ensures that the software testing process runs as expected.

Test Management Phases

This topic **briefly** introduces Test Management Process and shows you an **overview** of Test Management Phases. You will learn more details about each Test Management Phases in the next articles.



Test Management Process

Test Management Process is a procedure of managing the software testing activities from start to the end. The test management process provides planning, controlling, tracking and monitoring facilities throughout the whole project cycle. The process involves several activities like test planning, designing and test execution. It gives an initial plan and discipline to the software testing process.

There are two main Parts of Test Management Process: –

- Planning
- Risk Analysis
- Test Estimation
- Test Planning

Notes

- Test Organization
- Execution
- Test Monitoring and Control
- Issue Management
- Test Report and Evaluation
- Planning

Risk Analysis and Solution



Risk is the potential loss (an undesirable outcome, however not necessarily so) resulting from a given action or an activity.

Risk Analysis is the first step which Test Manager should consider before starting any project. Because all projects may contain risks, early risk detection and identification of its solution will help Test Manager to **avoid** potential loss in the future & save on project cost.

You will learn more detail about the Risk Analysis and Solution in here.

Test Estimation

An estimate is a forecast or prediction. Test Estimation is approximately determining **how long** a task would take to complete. Estimating effort for the test is one of the **major** and **important** tasks in Test Management.

Benefits of correct estimation:

Accurate test estimates lead to better planning, execution and monitoring of tasks under a test manager's attention.

Allow for more accurate scheduling and help realize results more confident

Test Planning



A Test Plan can be defined as a document describing the **scope**, **approach**, **resources**, and **schedule** of intended Testing activities.

A project may **fail** without a complete Test Plan. Test planning is particularly important in large software system development.

Test Management Responsibilities:

- Test Management has a clear set of roles and responsibilities for improving the quality of the product.
- Test management helps the development and maintenance of product metrics during the course of project.
- Test management enables developers to make sure that there are fewer design or coding faults.

3.1.1 Test Organization

A test organization defines who is responsible for what activity in the test process. The organization defines the test functions, test facilities and test activities. It defines the competencies and knowledge of the people involved.

In general, everyone is responsible for quality and therefore (almost) everyone is, in one form or the other, involved with testing. The structure of testing must resemble the associated system development approach, be it Agile, waterfall or a hybrid form. In many cases, this translates to a project or team organization with testing either as an integral part of the project/team or as a separate, possibly independent, entity. Where there is frequent (re)testing, especially in combination with scarce test knowledge and experience, testing can also be organized in a line or staff function, the so called permanent test organization.

Next to these 'in company' test approaches, testing can be conducted outside the realm of 'the company' through outsourcing or 'Crowd testing'. These 'out of company' test approaches yield their own, specific needs for test organization.

This table gives examples number of common organizational forms.

	Independent activity	integrated with other activities
Project organization	System test Acceptance test	Unit test Tests in Agile environments (e.g. Scrum, DevOps)
Permanent organization	• Test factory • Test line	• Maintenance process • end-to-end test organization
Cloud	• Crowd test (e.g. beta, compatibility, usability, game, mobile test)	

3.1.2 Test teams

Testing Project Team' is a team of experts dedicated to ensuring the best quality

Notes

of software products. In terms of software development, the testing project team is responsible for maintaining high levels of quality in software applications and making sure that it meets all user and business requirements.

It closely relates to various project roles, which are: QA project manager, Test Analyst/ Designer, Tester, Testing Developers, SQA Members. Project QA managers are the ones who initiate tests and works together with testing leads and the client to define acceptance tests. Here the job of the development team is to addresses and removes defects within the software and cooperate with project testing team after analysing them.

Effective Test Team is the Key Point of the Success

High-quality software can give businesses a significant competitive advantage and play a key role in the success of companies. However, in markets with tough competition high-quality software is not just an advantage but also a basic requirement for staying relevant.

The project testing team helps companies detect hidden errors and defects applications. Identifying and detecting these problems before the product goes live is essential for its success. Without the project testing team, these defects can seep into the final product and affect its performance in the live environment.

If a company does stop bugs before the production phases, the cost of resolving these errors later is much higher. Furthermore, since these defects and errors directly impact the end-user, it leads to bad user experience, which is a prime reason for software failure.

The project testing team not only detects these issues before they affect the user but also helps software teams prevent bugs proactively. Needless to say, investing in a project testing team is not only beneficial to your product but indispensable given the fierce competition in IT.

The project testing team needs to work together to achieve the overall objective of projects. QA Test Managers are people responsible for building these teams. These teams must work through shared environments, high levels of interdependence, and ensure QA cooperative agility.

Project testing teams build the testing infrastructure, related to automated test design and integration. Since complex software products rely on several testing procedures, QA project managers help create large, multi-dimensional project testing teams in oprder to maintain the best quality of software.

How to Start Organization in the Right Way

Here we will give a step-by-step procedure for building the best project testing team. To build and manage an effective a project testing team, it's important to follow the steps below:

Identify Resources Your Company Needs

To create the best project testing team, it's important to develop a comprehensive human resource plan. This means QA project managers must identify current and future human resource needs for their company.

Notes

Human resource planning ensures that your team doesn't have shortage of staff or redundancy of roles. Therefore, it has a key role in helping managers find the best fit for their project testing team. You can divide the process into the following three stages:

Estimate Demand

First, QA project managers have to forecast which human resources they will need based on different project plans. Your team composition and size can change based on the type and requirements of the project.

If you can estimate what kind of team you need, it will be easier to shortlist software testing team roles and responsibilities. You can also vary the number of members within your team according to the project complexity and the volume of the project activities.

Usually, project testing team is composed of following members:

Designation	Responsibilities
Test Manager	Coordinates the testing team Assigns the individual responsibilities Is responsible for the planning, implementation and reporting during testing
Test Analyst, Designer	Creates testing plans Defines testing procedures Decides on the appropriate selection of testing tools Builds up and ensures Test Environment and assets are managed and maintained Facilitates members in test environment for test execution
Tester	Prepares specified automated and manual tests Builds up the Test Cases Generate Test Suites Performs various tests Execute the tests Records the results Clearly describes the errors Report the defect
Developers in Test teams	Creates program to test Creates test automation scripts
SQA Members	Takes charge of Quality Assurance

This is why you may need at least 5 members in a project testing team. Everyone has a different role and you have to assign tasks according to each member's core competencies.

Evaluate Expertise

Evaluating the expertise of your testing resources is an essential part of human resource planning. Unless you assign your team members with the right task, they can fail to maintain the quality of software your client expects.

For instance, you have a developer within your team, but you use him as a tester in the Testing Team. You assign him the execution of test cases and request him to report defects to the QA Test Manager within a week.

However, since the developer isn't skilled in testing, he or she will fail to provide similar results as an experienced software tester. Being a skilled programmer, he may

Notes

resolve defects within the software more easily, but asking him to speed through testing will delay the project.

This is why it's essential to identify the key competencies of your team members before you assign them any task. Not only that, but it's also important to see what kind of skills a certain task demands.

You should measure the team member skills and abilities based on the project goal and mission. When you have to assign project members tasks that they lack necessary skills for, it's important to fill in skill gaps.

Planning Skill Upgrade

Identify Skill Gaps

Sometimes, your team members have gaps in their skills and the type of expertise the project demands. As a QA project manager, you have to identify what skills your testing resources lack, so you can create an appropriate training plan for them. For instance, if some of your team members have to improve their testing skills, you should help them close that skill gap.

Training and Assessment

In human resource planning, you should consider how existing members are trained and developed to gain the required skills and develop expertise. QA project managers need to create a training plan and apply it promptly to fill in the talent gap.

Evaluation

Putting your testing resources is not sufficient on its own. Instead, it's important for QA project managers to monitor and evaluate these programs frequently, so they can ensure they are effective.

For instance, you can work on your developer's training expertise by assessing his progress and assigning him different tasks to see if he has learned from the training initiatives. If the developer finds testing difficult, the manager should either replace him or consider an alternative training method.

Build the Project Testing Team

After you are finished with the human resource plan, you should start building your project team.

Team Mission

Creating a team mission helps you set your testing team in the right direction. The team mission allows all testing resources to focus on key objectives and goals, collaborate and come to an agreement

Team Responsibility

Ensuring your team members are aware of their responsibility is something you should sort out in the very beginning. Unless testing resources have clear roles and responsibilities defined, they can't do what's expected of them.

Notes**Team Rules**

Team rules set guidelines regarding how testing resources should behave, helping them work more collaboratively. Although it isn't mandatory to impose rules on your testing team, doing so can help your team reach a consensus regarding responsibilities and share responsibilities.

Team Motivation

Motivation is a critical factor in the workplace and a key driver behind the performance. It enables the management to meet the organizational goals, enhances productivity, and improves overall output.

Unmotivated employees don't focus on work and are not as much engaged.

Such unproductive behaviour not only causes unnecessary delays for the company but also wastes considerable resources, especially if the majority of the workforce is affected by this. This is why companies need to pay attention to the level of motivation in employees.

Keeping teams motivated has always been a long-standing challenge for most project managers. To be a good project manager, you should have the expertise to manage both the functional and emotional factors of your team.

Negligence in this matter can also cause high performing and strongly skilled employees to exhibit negative attitudes due to their emotional burden. If you don't manage both factors simultaneously, you'll end up delaying important projects that cost way higher than what you expected.

Manage Project Team

Making a successful project testing team goes beyond handpicking the best talent you can find. Instead, the challenge comes when you start managing testing team members. There are three ways to ensure that your team gives the best performance:

Setting Team Target

Similar to a team mission, a team target gives direction to your software testing team. However, where a team mission gives the team strategic direction, the team target provides them with a milestone to work towards. As a QA project manager, it is up to you to break down problems into specific tasks and present them to your testing team.

Observation

QA project managers also need to constantly monitor and understand what's going around their software testing team. Not only is it important to monitor the performance of each team member, but it's also important to assess what these people are doing and how is it impacting their work.

Team Collaboration and Engagement

In many project teams, silos become an obstacle in a team's performance as a unit. Every team member has a unique personality and uses different ways of engaging and collaborating with people.

Notes

If engagement is minimal between different departments, collaboration suffers. Although project teams may still work together, the internalized nature of these group's team affects impact collaboration.

Engaging team members help diminish internal silos within project teams. This is why project managers are responsible for balancing internal teams effectively.

Project managers need to lay the groundwork to engage team members and stakeholders. This will help them to sustain a good relationship with all members and stakeholders in both the short and the long term.

You must monitor the level of engagement of individual team members across the project lifecycle. You should also address their emotional and personal expectations to bring about their best performance.

If you don't address these issues yourself, it would be extremely difficult to create a good working environment among your team. Consequently, it would affect the performance of your team and make it harder for them to achieve the targets you set for them.

Performance Lab's Experience

Creating the ideal project testing team can be challenging for any QA project manager. However, by identifying key project requirements, evaluating competencies within the team, and managing available resources effectively, QA project managers can overcome any challenge and deliver testing projects with ease.

Although all these things sound straightforward and simple, in reality, it is easier said than done. This is why it's best for software teams to rely on software testing teams with considerable building software QA testing teams.

Performance Lab is a testing service dedicated to ensuring the best software quality for our clients. We have served over 500 companies across a wide variety of domains that range from finance and healthcare to retail and technology.

With years of experience at our disposal, we can help you create the best testing team for your product. By providing you with experienced testing resources, we can fill in skill gaps and help you deliver the best software quality to your clients.

3.1.3 Tasks and Qualifications

Task: Tasks of a QA engineer

Testers play no less important role than programmers. Even if developers wrote quality code, run the app on the device, and saw that everything was working, the fully functioning system is still far away.

Why do some applications (that even seem to be good and quality) get flows of negative reviews after launch? Often one of the reasons is that the app looks or works differently on different devices.

The situation is getting worse due to the fact that the market is full of various devices. Let's say someone launched the product on an old Huawei and saw an ugly layout on it.

Notes

Or it was installed by a user from Saudi Arabia, who didn't understand anything, as the text in this country is read from right to left, and the interface wasn't customized to this cultural peculiarity.

You've probably heard of the failure of the Coca-Cola advertising campaign because of it, but know – it isn't true. A few years ago, this case spread throughout the Internet, many authors included it in the lists of the most famous advertising campaign failures, but it's not confirmed by any trusted source.

So, who will check the system functioning in all conditions? Find bugs and vulnerabilities? Consider a variety of untypical cases? And, what's more, ensure that the product complies with the customer requirements? The tester is the very person who is responsible for these tasks.

The goals of a QA engineer are to achieve the correct system functioning, prevent defects, improve software quality, and throughout the project provide the end customer with the information about its quality.

The tester must also ensure that a software solution works as intended. For this purpose, he/she compares the actual system behavior with the expected one using various types of testing.

The tasks of a tester

- Identify bugs and errors, describe and send for revision
- Check the implemented improvements
- Test the product until the desired result is achieved
- Test the software solution on all necessary devices and screen resolutions
- Check the product performance in various conditions, including the use of the key system functionality and testing it for different untypical use cases
- Test the product compliance with the requirements
- Work through all uncharacteristic cases and ensure the product functions as intended

The feature testing life cycle looks as follows:

- The programmer delivers the developed feature for testing
- QA tests the implemented feature
- QA creates a bug report for each detected bug
- Priorities of bugs and features are set
- Product debugging – programmers fix bugs and prepare the build
- Feature testing. If bugs are found, the testing and debugging processes are repeated. When everything works well, the feature is marked as implemented.

Some useful tools for software testing

- **Postman + Newman + Jenkins** – this combination is used to automate API testing.

Notes

- **Cucumber + Appium** – applied for automating mobile app testing.
- **Charles** – allows testers to monitor HTTP/HTTPS traffic.
- **PICT** – a very convenient tool for testing values (and their combinations) of the checked parameters.
- **Apache JMeter, YandexTank** – enable to test the application performance.
- **JUnit, NUnit** – serve for writing unit tests.
- **Fiddler** – used to test requests and services.
- **Pixel Perfect** – a browser extension that helps test layout.
- **Burp Suite** – a great tool for testing system security.
- **Checklist** – QAs can either compile checklists specifically for the project needs or use already prepared checklists, customizing them for the tasks.
- **Fabric Crashlytics** – a popular tool for sharing builds within the team and collecting user statistics.
- **Postman** – a set of tools for testing API. It helps test the functionality before integrating API into the client app. It also enables to create API documentation, write and run tests, and replace real data from the server with test values.

Qualification

Qualification: When it comes to who can pursue a software testing course, then I would say “software testing is the game of mind, not a game of coding”. The software testing profession is open for any graduate. Any graduate student can enrol the software testing course. No matter he/she belongs from B.A, B.Com, or B.E background.

It depends on person to person, how they approach software testing and how they learn it. In software testing course a person required to learn exploration, asking questions to investigate problem area, identifying problems in the system, writing the bug report. This shows that the job profile is not completely correlated to computer science, it mostly relies on the exploration.

Here below are the skills that you need to have to be a good software tester, you can compare that which skills you have and which not. The majority of yes signals you that software testing is really for you, you can be a good software tester.

Analytic Skills:

The ability to visualize the problem is complicated or uncomplicated, if the problem is complicated then you should have an ability to divide it in small units to better understand it and create equivalent test cases. Here below I have given one example to check your analytical skills. Try to solve yourself, then check the solution.

Problem: There are three boxes. First is labelled with “Apples”, second is labelled with “Oranges” and third is labelled with “Oranges and Apples”. You know that each of is labelled with the incorrect name. You may ask me to pick one fruit from one box which you choose? And how you will label the boxes correctly?

Notes**Solution:**

Pick from the box labelled “Apples & Oranges”. This box must have either only apples or only oranges. E.g. if you find an Orange, label the box Orange, then change the Oranges box to Apples, and the Apples box to “Apples & Oranges.”

Technical skills:

You must have an expertise in MS office that is used to create reports, have expertise in software testing tools like Selenium, Jmeter, Load runner and many more. The ability to deeply understand the product/application. To acquire these skills you can join the best institute which provides you a comprehensive training to develop these skills.

TOPS Technologies is one of the best outsourcing and training company which provides comprehensive training in software testing course with low fee structure and better job opportunity. To know more about software testing course, visit your nearer branch among 22 branches across the India.

Good Verbal and Written Communication Skills:

A software tester needs to deal with developers by creating a report of bugs which demands writing skill. You should acquire written skill that you can easily convey developer about the bugs that you have found out in the system. Sometime you need to communicate with developers to convey some errors in the system which is not possible to convey by reports which demands verbal communication skills.

Productivity:

You should have the time management skills and ability to work under pressure. It is not like you are going to work all time under pressure as a software tester. But sometimes, especially at the time of release code, you may need to work under work load. You should have an ability to deal with such a condition.

Great Attitude:

You must hold the attitude to deal with software testing task like break to test, ever time ready to learn something new, the ability to suggest required process improvements and learn process improvements, if required. New applications and technology evolving these days, which demands you to keep yourself updated to test any application with any of the technology.

Passion:

“Follow your passion, success will follow you”, it’s very famous wording. This makes clear sense that if you want to gain success you must have passion about whatever you are pursuing. If you feel that you are actually passionate about software testing, you must run with it. If not, think about the other option.

If you have found yourself fit in the software testing career after go through this article. You can join software testing course. TOPS technologies is one of the reputed outsourcing and training company which provides software testing training.

Notes

3.1.4 Test Planning

A test plan outlines the strategy that will be used to test an application, the resources that will be used, the test environment in which testing will be performed, and the limitations of the testing and the schedule of the testing activities. Typically, the Quality Assurance Team Lead will be responsible for writing a Test Plan.

What does a Test Plan Include?

A Test Plan includes the following.

- Introduction to the Test Plan document.
- Assumptions while testing the application.
- List of test cases included in testing the application.
- List of features to be tested.
- The sort of approach to be used while testing the software.
- List of deliverables that need to be tested.
- The resources allocated for testing the application.
- Any risks involved during the testing process.
- A schedule of tasks and milestones to be achieved.

Important Points for Test Planning

The following points need to be considered for Test Planning in STLC.

- Ideally, the Test Analyst (Lead)/the Manager prepares the Test Strategy/Test Plan Document.
- Analysis is more focused on application related data/information.
- It is the first phase of actual testing tasks.
- This phase answers “WHAT is to be tested” and “WHAT RESOURCES are required to test”.
- The basic entry criteria of this phase is provision of Requirement Documents (updated version of unclear/missing/clarified requirements) along with Requirement Traceability Matrix.
- If automation is in scope, Automation Feasibility Report should be prepared before entering in this phase.
- The exit criteria of this phase is completion of Test Strategy/Test Plan Document and Test effort Estimation document.

Aspects of the Test Planning Phase

The main objective of this phase is to prepare a Test Plan/Test Strategy document. It includes three major aspects – Scope of Deliverables, Effort estimation and Resource Plan.

Scope of Deliverables

Following activities need to be performed to conclude over the scope of deliverables –

- Identify suitable engagement and delivery model.

Notes

- Define test objectives, scope of testing, testing phases and activities.
- Review business requirement and system requirement to identify test feasibility.
- Define testing process, type of testing and procedures.
- Define defect management and change management procedures.
- Identify testing tools, techniques and best practices.
- Define Risk Analysis.
- Define automation solution and identify suitable candidates for automation if applicable.

Effort Estimation

Estimation is the process of finding an estimate, or approximation, which is a value that can be used for some purpose even if input data may be incomplete, uncertain, or unstable.

Estimation determines how much money, effort, resources, and time it will take to build a specific system or product. Estimation is based on –

- Past Data/Past Experience
- Available Documents/Knowledge
- Assumptions
- Identified Risks

The four basic steps in Testing Estimation are –

- Estimation of the size of the AUT (Application Under Test).
- Estimation of the effort in person-months or person-hours.
- Estimation of the schedule in calendar months.
- Estimation of the project cost in agreed currency.

Resource Plan

Resource plans are the key element in testing phases. These plans are inversely proportional to the time taken by the testing team to complete a particular task. Increasing the number of resources will decrease the number of days of completion for a certain limit after that it will be saturated and increasing the resource will not have much impact and might not lead to a decrease in the completion days.

A Resource Requester, usually a project manager, creates resource plans to ask for resources, track efforts and costs. A Resource manager can modify and approve resource plans before the plans are used.

The normal workflow for a resource plan is –

- Planning by Project Manager
- Request raised by Project Manager
- Approve/Modify/Reject by Resource Manager
- Complete – Closing the request after sign off by Resource Manager

Notes

3.2.1 Quality Assurance Plan

A quality assurance plan is a document, constructed by the project team, meant to ensure the final products are of the utmost quality. A quality assurance plan contains a set of documented activities meant to ensure that customers are satisfied with the goods or services a company provides. There are four steps of the quality assurance process: Plan, Do, Check, and Act. The focus of this article will be on what items go into a quality assurance plan. The quality assurance plan should define objectives, roles and responsibilities, coordinate with other plans, and define tasks and the schedule.

Definition of Quality Objectives

No matter what system you use, you will need to be sure that the objectives have been defined for the project. **Six Sigma** uses a predefined algorithm for determining quality objectives. Part of determining the objectives for the quality plan project involves identifying the requirements of the customers. For example, if a particular store generates many complaints about the level of customer service, more employee training may be required. Second, the level of quality must be defined. Is your company going for zero defects? Or is one in one hundred okay? Make sure your **objectives are well written** and specific.

Defining Roles and Responsibilities

Once the quality objectives have been defined and stated, the next important part of creating a quality assurance plan is to define the roles and responsibilities of team members. In doing this, you will want to list each role that will be required. Once you have listed the roles, then you can itemize the responsibilities of each role. Be specific. You want the person who is assigned the roles to understand completely what is meant by each responsibility written for them.

Coordinate with Other Plans

No project plan occurs in a vacuum. This is even more true with the quality assurance plan. Make sure you are on the same level as those working on the **risk management plans**, the **resource management plans**, and the **change management plans**. It would be a terrible thing to work hard on a quality assessment plan that contradicts something in the risk management plan. Talk to the other people and project managers in your department.

Define Tasks and Schedule

Once the objectives and roles have been defined, and the other teams have been coordinated with, you can then begin to define tasks and create a schedule. Each and every task should relate directly to the quality objectives. Once each task has been defined, using the objectives as a reference, then you can begin to set the schedule. You can schedule two ways: either from the deadline in or from the start date out. Each has their own benefits. You may wish to utilize a downloadable sample of a quality assurance template.

With a focus on quality goals, criteria, assessment, and validation methods, a QAP covers the full data lifecycle, from Acquisition through Publication, and can:

Notes

- Identify data quality objectives for your data or project
- Identify requirements for
 - Staff skills and training
 - Field and lab methods and equipment that meet data-collection standards
 - Software and file types to use for data handling and analysis that support data quality goals
 - Data standards, structure, and domains consistent with community conventions for other data in the same subject area
 - Periodic data-quality assessment using defined quality metrics
- Describe a structure for data storage that can also facilitate checking for errors and help to document data quality
- Describe approved data entry tools and procedures, when applicable
- Establish data-quality criteria and data-screening processes for all of the data you will collect
- Include quality metrics that can determine current data-quality status
- Establish a plan for 'data quality assessments' as part of the data flow
- Contain a process for handling data corrections
- Contain a process for data users to dispute and correct data

3.2.2 Test Plan

A test plan is a detailed document which describes software testing areas and activities. It outlines the test strategy, objectives, test schedule, required resources (human resources, software, and hardware), test estimation and test deliverables.

The test plan is a base of every software's testing. It is the most crucial activity which ensures availability of all the lists of planned activities in an appropriate sequence.

The test plan is a template for conducting software testing activities as a defined process that is fully monitored and controlled by the testing manager. The test plan is prepared by the Test Lead (60%), Test Manager(20%), and by the test engineer(20%).

Types of Test Plan

There are three types of the test plan

- Master Test Plan
- Phase Test Plan
- Testing Type Specific Test Plans

Master Test Plan

Master Test Plan is a type of test plan that has multiple levels of testing. It includes a complete test strategy.

Notes

Phase Test Plan

A phase test plan is a type of test plan that addresses any one phase of the testing strategy. For example, a list of tools, a list of test cases, etc.

Specific Test Plans

Specific test plan designed for major types of testing like security testing, load testing, performance testing, etc. In other words, a specific test plan designed for non-functional testing.

How to write a Test Plan

Making a test plan is the most crucial task of the test management process. According to IEEE 829, follow the following seven steps to prepare a test plan.

- First, analyze product structure and architecture.
- Now design the test strategy.
- Define all the test objectives.
- Define the testing area.
- Define all the useable resources.
- Schedule all activities in an appropriate manner.
- Determine all the Test Deliverables.

Test plan components or attributes

The test plan consists of various parts, which help us to derive the entire testing activity.



Objectives: It consists of information about modules, features, test data etc., which indicate the aim of the application means the application behavior, goal, etc.

Scope: It contains information that needs to be tested with respect to an application. The Scope can be further divided into two parts:

Notes

- In scope
- Out scope

In scope: These are the modules that need to be tested rigorously (in-detail).

Out scope: These are the modules, which need not be tested rigorously.

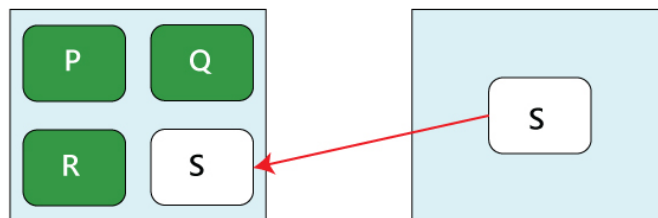
For example, suppose we have a Gmail application to test, where **features to be tested** such as **Compose mail, Sent Items, Inbox, Drafts** and the **features which not be tested** such as **Help**, and so on which means that in the planning stage, we will decide that which functionality has to be checked or not based on the time limit given in the product.

Now **how we decide which features not to be tested?**

We have the following aspects where we can decide which feature not to be tested:

- As we see above that **Help** features is not going to be tested, as it is written and developed by the technical writer and reviewed by another professional writer.
- Let us assume that we have one application that have **P, Q, R, and S** features, which need to be developed based on the requirements. But here, the S feature has already been designed and used by some other company. So the development team will purchase S from that company and integrate with additional features such as P, Q, and R.

Now, we will not perform functional testing on the S feature because it has already been used in real-time. But we will do the integration testing, and system testing between P, Q, R, and S features because the new features might not work correctly with S feature as we can see in the below image:



- Suppose in the first release of the product, the elements that have been developed, such as **P, Q, R, S, T, U, V, W.....X, Y, Z**. Now the client will provide the requirements for the new features which improve the product in the second release and the new features are **A1, B2, C3, D4, and E5**.

After that, we will write the scope during the test plan as

Scope

Features to be tested

A1, B2, C3, D4, E5 (new features)

P, Q, R, S, T

Notes

Features not to be tested

W....X, Y, Z

Therefore, we will check the new features first and then continue with the old features because that might be affected after adding the new features, which means it will also affect the impact areas, so we will do one round of regression testing for P, Q, R..., T features.

Test methodology:

It contains information about performing a different kind of testing like Functional testing, Integration testing, and System testing, etc. on the application. In this, we will decide what type of testing; we will perform on the various features based on the application requirement. And here, we should also define that what kind of testing we will use in the testing methodologies so that everyone, like the management, the development team, and the testing team can understand easily because the testing terms are not standard.

For example, for standalone application such as **Adobe Photoshop**, we will perform the following types of testing:

Smoke testing → Functional testing → Integration testing → System testing → Adhoc testing → Compatibility testing → Regression testing → Globalization testing → Accessibility testing → Usability testing → Reliability testing → Recovery testing → Installation or Uninstallation testing

And suppose we have to test the <https://www.xxx.com/> application, so we will perform following types of testing:

Smoke testing	Functional testing	Integration testing
System testing	Adhoc testing	Compatibility testing
Regression testing	Globalization testing	Accessibility testing
Usability testing	Performance testing	

Approach

This attribute is used to describe the flow of the application while performing testing and for the future reference.

We can understand the flow of the application with the help of below aspects:

- **By writing the high-level scenarios**
- **By writing the flow graph**

By writing the high-level scenarios

For example, suppose we are testing the **Gmail** application:

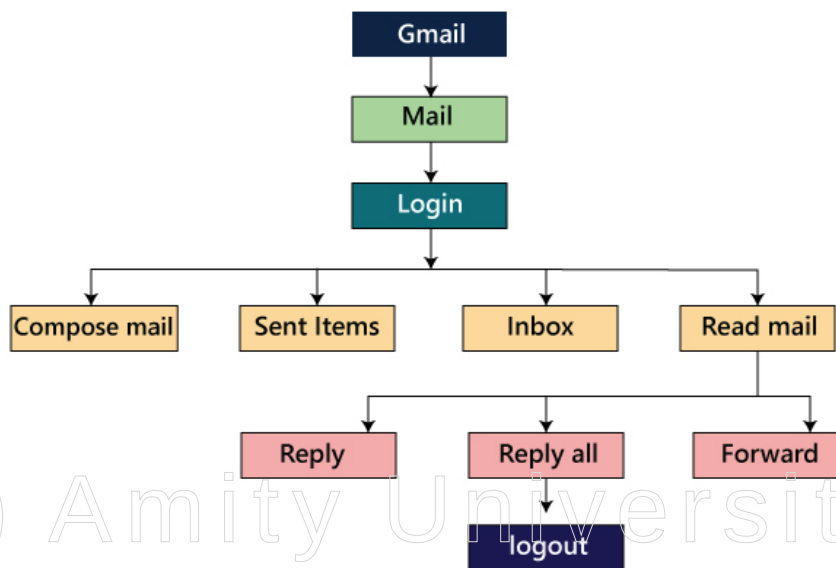
- Login to Gmail- sends an email and check whether it is in the Sent Items page
- Login to
-

-

We are writing this to describe the approaches which have to be taken for testing the product and only for the critical features where we will write the high-level scenarios. Here, we will not be focusing on covering all the scenarios because it can be decided by the particular test engineer that which features have to be tested or not.

By writing the flow graph

The flow graph is written because writing the high-level scenarios are bit time taking process, as we can see in the below image:



We are creating flow graphs to make the following benefits such as:

- Coverage is easy
- Merging is easy

The approach can be classified into two parts which are as following:

- Top to bottom approach
- Bottom to top approach

Assumption

It contains information about a problem or issue which maybe occurred during the testing process and when we are writing the test plans, the assured assumptions would be made like resources and technologies, etc.

Risk

These are the challenges which we need to face to test the application in the current release and if the assumptions will fail then the risks are involved.

For example, the effect for an application, release date becomes postponed.

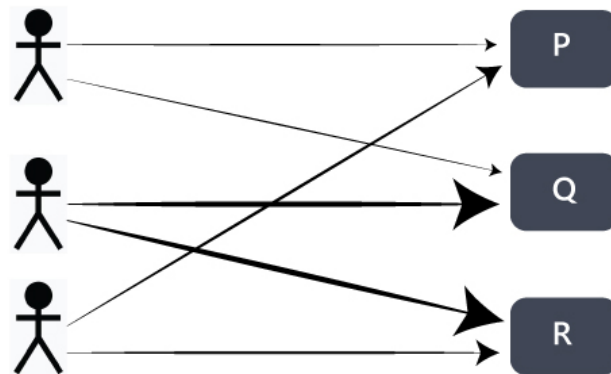
Mitigation Plan or Contingency Plan

It is a back-up plan which is prepared to overcome the risks or issues.

Notes

Notes

Let us see one example for assumption, risk, and the contingency plan together because they are co-related to each other.



In any product, the **assumption** we will make is that the all 3 test engineers will be there until the completion of the product and each of them is assigned different modules such as P, Q, and R. In this particular scenario, the **risk** could be that if the test engineer left the project in the middle of it.

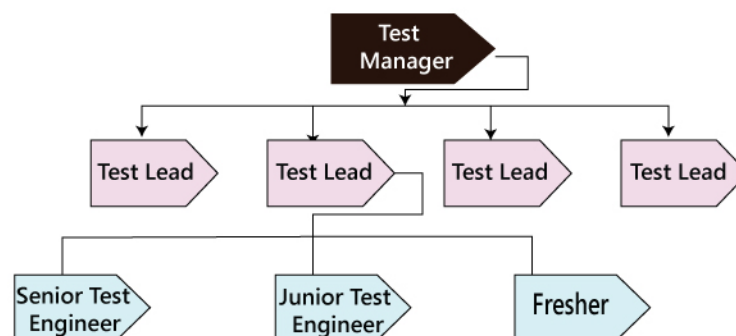
Therefore, the **contingency plan** will be assigned a primary and subordinate owner to each feature. So if the one test engineer will leave, the subordinate owner takes over that specific feature and also helps the new test engineer, so he/she can understand their assigned modules.

The assumptions, risk, and mitigation or contingency plan are always precise on the product itself. The various types of risks are as follows:

- Customer perspective
- Resource approach
- Technical opinion

Role & Responsibility

It defines the complete task which needs to be performed by the entire testing team. When a large project comes, then the **Test Manager** is a person who writes the test plan. If there are 3-4 small projects, then the test manager will assign each project to each Test Lead. And then, the test lead writes the test plan for the project, which he/she is assigned.



Let see one example where we will understand the roles and responsibility of the Test manager, test lead, and the test engineers.

Notes**Role: Test Manager****Name: Ryan****Responsibility:**

- Prepare(write and review) the test plan
- Conduct the meeting with the development team
- Conduct the meeting with the testing team
- Conduct the meeting with the customer
- Conduct one monthly stand up meeting
- Sign off release note
- Handling Escalations and issues

Role: Test Lead**Name: Harvey****Responsibility:**

- Prepare(write and review) the test plan
- Conduct daily stand up meeting
- Review and approve the test case
- Prepare the RTM and Reports
- Assign modules
- Handling schedule

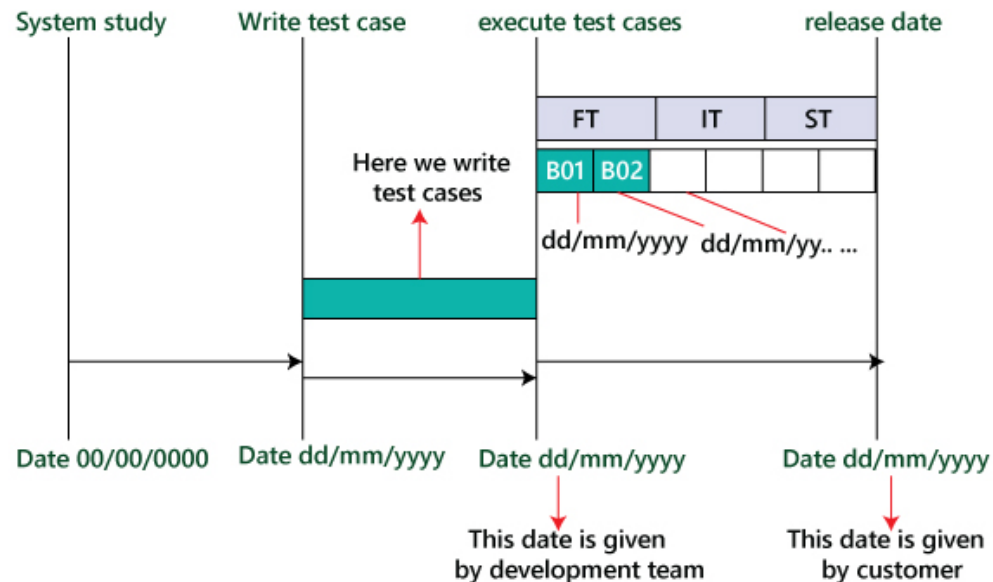
Role: Test Engineer 1, Test Engineer 2 and Test Engineer 3**Name: Louis, Jessica, Donna****Assign modules: M1, M2, and M3****Responsibility:**

- Write, Review, and Execute the test documents which consists of test case and test scenarios
- Read, review, understand and analysis the requirement
- Write the flow of the application
- Execute the test case
- RTM for respective modules
- Defect tracking
- Prepare the test execution report and communicate it to the Test Lead.

Notes

Schedule

It is used to explain the timing to work, which needs to be done or this attribute covers when exactly each testing activity should start and end? And the exact data is also mentioned for every testing activity for the particular date.



Therefore, as we can see in the below image that for the particular activity, there will be a starting date and ending date; for each testing to a specific build, there will be the specified date.

For example

- Writing test cases
- Execution process

Defect tracking

It is generally done with the help of tools because we cannot track the status of each bug manually. And we also comment about how we communicate the bugs which are identified during the testing process and send it back to the development team and how the development team will reply. Here we also mention the priority of the bugs such as high, medium, and low.

Following are various aspects of the defect tracking:

- **Techniques to track the bug**
-
-
-
-
- **Bug tracking tools**

Notes

- We can comment on the name of the tool, which we will use for tracking the bugs. Some of the most commonly used bug tracking tools are Jira, Bugzilla, Mantis, and Trac, etc.<

- **Severity**

The severity could be as following:

Blocker or showstopper

.....

..... (Explain it with an example in the test plan)

For example, there will be a defect in the module; we cannot go further to test other modules because if the bug is blocked, we can proceed to check other modules.

Critical

.....

..... (Explain it with an example in the test plan)

In this situation, the defects will affect the business.

Major

....

.... (Explain it with an example in the test plan)

Minor

.....

..... (Explain it with an example in the test plan) These defects are those, which affect the look and feel of the application.

- **Priority**

High-P1

.....

Medium-P2

.....

Low-P3

.....

.....

P4

Therefore, based on the priority of bugs like high, medium, and low, we will categorize it as P1, P2, P3, and P4.

Notes

Test Environments

These are the environments where we will test the application, and here we have two types of environments, which are of **software** and **hardware** configuration.

The **software configuration** means the details about different **Operating Systems** such as **Windows, Linux, UNIX, and Mac** and various **Browsers** like **Google Chrome, Firefox, Opera, Internet Explorer**, and so on.

And the **hardware configuration** means the information about different sizes of **RAM, ROM, and the Processors**.

For example

- The **Software** includes the following:

Server

Operating system	Linux
Webserver	Apache Tomcat
Application server	Websphere
Database server	Oracle or MS-SQL Server

Client

Operating System	Window XP, Vista 7
Browsers	Mozilla Firefox, Google Chrome, Internet Explorer, Internet Explorer 7, and Internet Explorer 8

- The **Hardware** includes the following:

Server: Sun Star Cat 1500

This particular server can be used by the testing team to test their application.

Client:

It has the following configuration, which is as follows:

Processor	Intel2GHz
RAM	2GB
....	

- Process to install the software**

.....

The development team will provide the configuration of how to install the software. If the development team will not yet provide the process, then we will write it as Task-Based Development (TBD) in the test plan.

Entry and Exit criteria

It is a necessary condition, which needs to be satisfied before starting and stopping the testing process.

Entry Criteria

The entry criteria contain the following conditions:

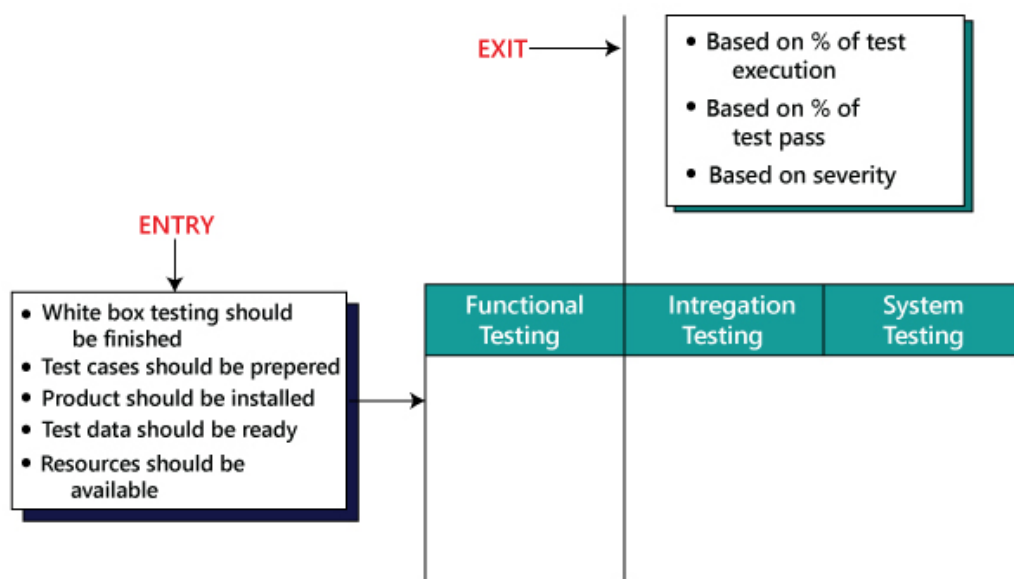
- White box testing should be finished.
- Understand and analyze the requirement and prepare the test documents or when the test documents are ready.
- Test data should be ready.
- Build or the application must be prepared
- Modules or features need to be assigned to the different test engineers.
- The necessary resource must be ready.

Exit Criteria

The exit criteria contain the following conditions:

- When all the test cases are executed.
- Most of the test cases must be **passed**.
- Depends on severity of the bugs which means that there must not be any blocker or major bug, whereas some minor bugs exist.

Before we start performing functional testing, all the above **Entry Criteria** should be followed. After we performed functional testing and before we will do integration testing, then the **Exit criteria** of the functional testing should be followed because the % of exit criteria are decided by the meeting with both development and test manager because their collaboration can achieve the percentage. But if the exit criteria of functional testing are not followed, then we cannot proceed further to integration testing.



Notes

Notes

Here **based on the severity** of the bugs means that the testing team would have decided that to proceed further for the next phases.

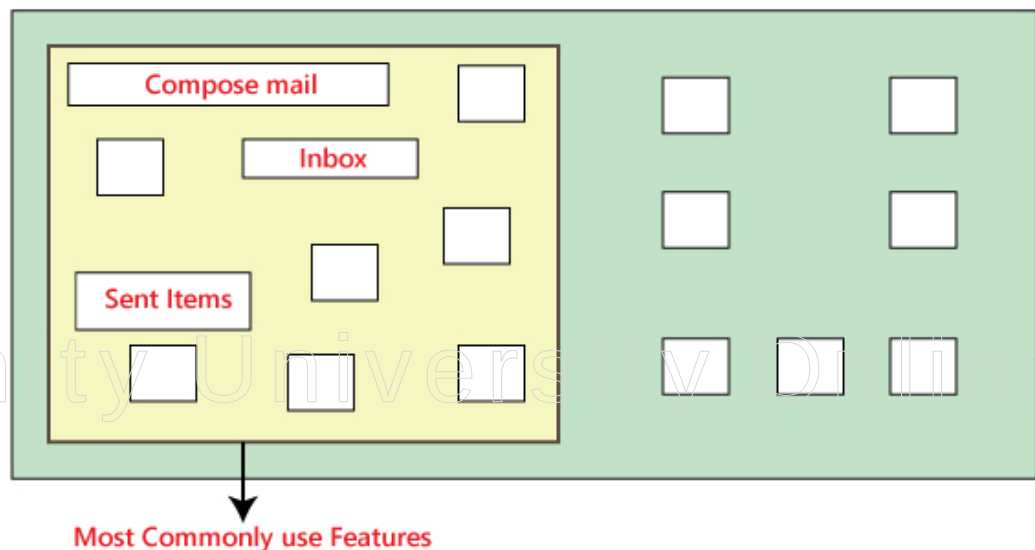
Test Automation

In this, we will decide the following:

- Which feature has to be automated and not to be automated?
- Which test automation tool we are going to use on which automation framework?

We automate the test case only after the first release.

Here the question arises that on what basis **we will decide which features have to be tested?**



In the above image, as we can see that the most commonly used features need to test again and again. Suppose we have to check the Gmail application where the essential features are **Compose mail, Sent Items, and Inbox**. So we will test these features because while performing manual testing, it takes more time, and it also becomes a monotonous job.

Now, **how we decide which features are not going to be tested?**

Suppose **the Help** feature of the Gmail application is not tested again and again because these features are not regularly used, so we do not need to check it frequently.

But **if some features are unstable and have lots of bugs**, which means that we will not test those features because it has to be tested again and again while doing manual testing.

If **there is a feature that has to be tested frequently**, but we are expecting the requirement change for that feature, so we do not check it because changing the manual test cases is more comfortable as compared to change in the automation test script.

Effort estimation

In this, we will plan the effort need to be applied by every team member.

Test Deliverable

These are the documents which are the output from the testing team, which we handed over to the customer along with the product. It includes the following:

- **Test plan**
- **Test Cases**
- **Test Scripts**
- **RTM(Requirement Traceability Matrix)**
- **Defect Report**
- **Test Execution Report**
- **Graphs and metrics**
- **Release Notes**

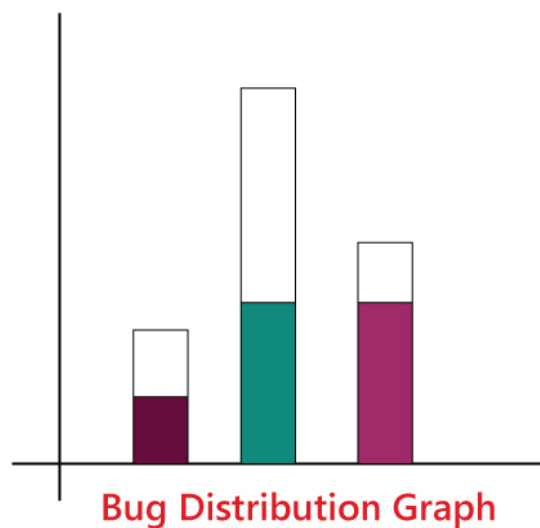
Graphs and Metrics

Graph

In this, we will discuss about the types of **graphs** we will send, and we will also provide a sample of each graph.

As we can see, we have five different graphs that show the various aspects of the testing process.

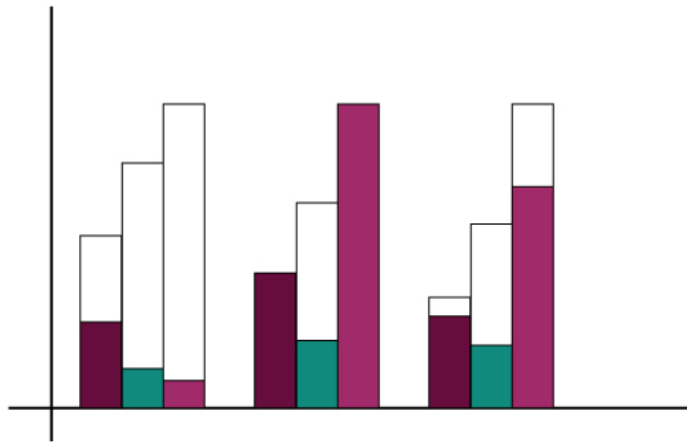
Graph1: In this, we will show how many defects have been identified and how many defects have been fixed in every module.



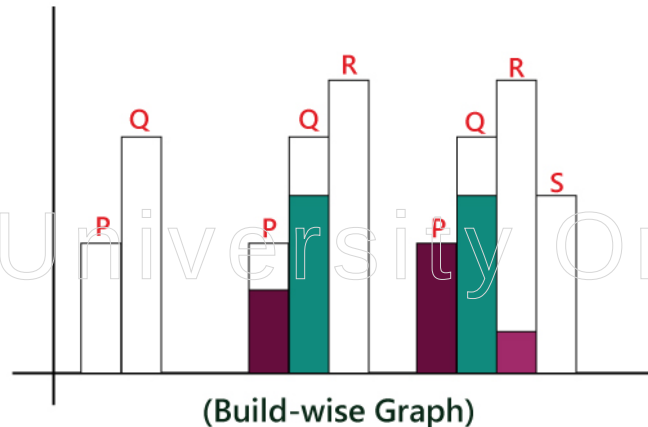
Graph 2: Figure one shows how many critical, major, and minor defects have been identified for every module and how many have been fixed for their respective modules.

Notes

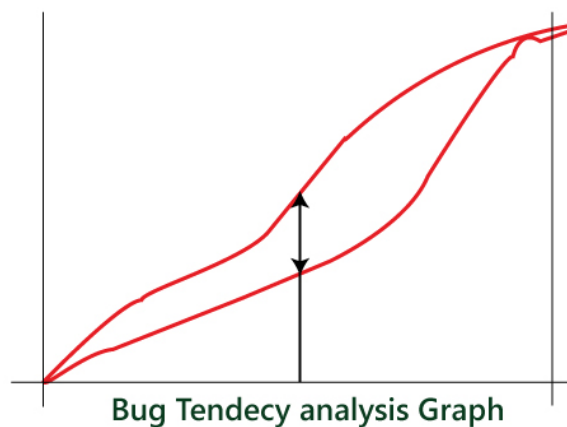
Notes



Graph3: In this particular graph, we represent the **build wise graph**, which means that in every builds how many defects have been identified and fixed for every module. Based on the module, we have determined the bugs. We will add **R** to show the number of defects in P and Q, and we also add **S** to show the defects in P, Q, and R.

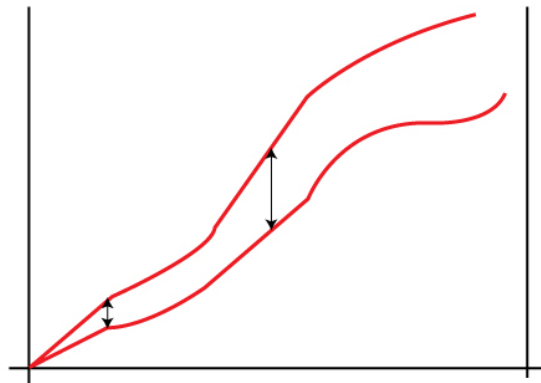


Graph4: The test lead will design the **Bug Trend analysis** graph which is created every month and send it to the Management as well. And it is just like prediction which is done at the end of the product. And here, we can also **rate the bug fixes** as we can observe that **arc** has an **upward tendency** in the below image.



Graph5: The **Test Manager** has designed this type of graph. This graph is intended to understand the gap in the assessment of bugs and the actual bugs which

have been occurred, and this graph also helps to improve the evaluation of bugs in the future.



Metrics

Module Name	Critical		Major		Minor	
	Found	Fixed	Found	Fixed	Found	Fixed
Purchase	50	46	60	20	80	10
Sales	..	...	...	...	...	...
Asset Survey	...	...	...	...	...	...

As above, we create the bug distribution graph, which is in the figure 1, and with the help of above mention data, we will design the metrics as well.

For example

Test Engineer Names	Critical		Major		Minor	
	Found	Fixed	Found	Fixed	Found	Fixed
John	50	46	60	20	80	10
James	..	...	...	...	...	...
Sophia	...	...	...	...	...	...

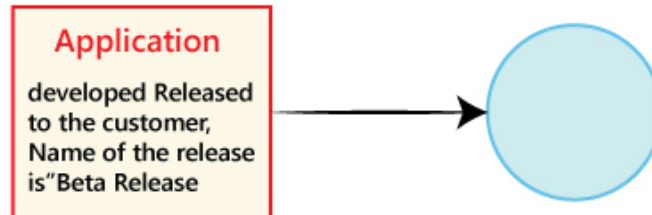
In the above figure, we retain the records of all the test engineers in a particular project and how many defects have been identified and fixed. We can also use this data for future analysis. When a new requirement comes, we can decide whom to provide the challenging feature for testing based on the number of defects they have found earlier according to the above metrics. And we will be in a better situation to know who can handle the problematic features very well and find maximum numbers of defects.

Notes

Notes

Release Note: It is a document that is prepared during the release of the product and signed by the Test Manager.

In the below image, we can see that the final product is developed and deployed to the customer, and the latest release name is **Beta**.



The **Release note** consists of the following:

- User manual.
- List of pending and open defects.
- List of added, modified, and deleted features.
- List of the platform (Operating System, Hardware, Browsers) on which the product is tested.
- Platform in which the product is not tested.
- List of bugs fixed in the current release, and the list of fixed bugs in the previous release.
- Installation process
- Versions of the software

For Example

Suppose that **Beta** is the second release of the application after the first release **Alpha** is released. Some of the defect identified in the first released and that has been fixed in the later released. And here, we will also point out the list of newly added, modified, and deleted features from alpha release to the beta release.



Template

This part contains all the templates for the documents that will be used in the product, and all the test engineers will use only these templates in the project to maintain the consistency of the product. Here, we have different types of the template which are used during the entire testing process such as:

- Test case template
- Test case review template
- RTM Template

- Bug Report Template
- Test execution Report

Let us see one sample of test plan document

Page-1

<u>Testplan</u>					
Version	Author	Reviewed By	Approved By	Comments	Approval date
1	...	...	Name of manager	Version 1.0 is developed	dd/mm/yyyy
1.1	...	..	..	Version 1.1 is developed. PQR feature is added	dd/mm/yyyy
.....					
.....					

Page3-page18

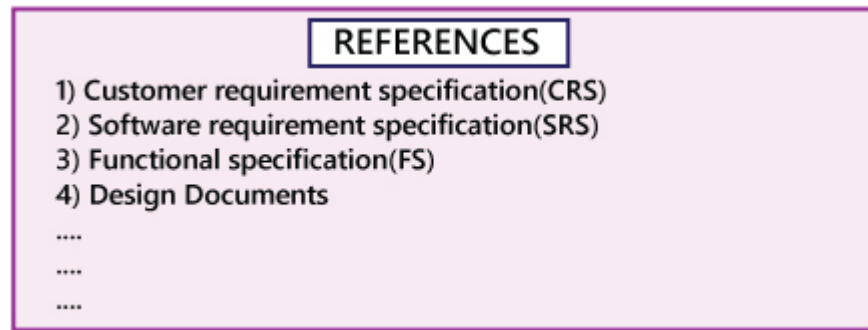
TABLE OF CONTENTS		
• Objective	pg 1	
• Scope	pg 2	
• Approach	pg 3	
..		
...		
...		
...		

.....	} Entire Test Plan document
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	

Page-20

Notes

Notes



In-Page 1, we primarily fill only the **Versions, Author, Comments, and Reviewed By** fields, and after the manager approves it, we will mention the details in the **Approved By and Approval Date** fields.

Mostly the test plan is approved by the Test Manager, and the test engineers only reviews it. And when the new features come, we will modify the test plan and do the necessary modification in **Version** field, and then it will be sent again for further review, update, and approval of the manager. The test plan must be updated whenever any changes have occurred. On page 20, the **References** specify the details about all the documents which we are going to use to write the test plan document.

Note:

Who writes the test plan?

- Test Lead→60%
- Test Manager→20%
- Test Engineer→20%

Therefore, as we can see from above that in 60% of the product, the test plan is written by the Test Lead.

Who reviews the Test Plan?

- Test Lead
- Test Manager
- Test engineer
- Customer
- Development team

The Test Engineer review the Test plan for their module perspective and the test manager review the Test plan based on the customer opinion.

Who approve the test Plan?

- Customer
- Test Manager

Notes**Who writes the test case?**

- Test Lead
- Test Engineer

Who review the test case?

- Test Engineer
- Test Lead
- Customer
- Development Team

Who approves the Test cases?

- Test Manager
- Test Lead
- Customer

Test Plan Guidelines

- Collapse your test plan.
- Avoid overlapping and redundancy.
- If you think that you do not need a section that is already mentioned above, then delete that section and proceed ahead.
- Be specific. For example, when you specify a software system as the part of the test environment, then mention the software version instead of only name.
- Avoid lengthy paragraphs.
- Use lists and tables wherever possible.
- Update plan when needed.
- Do not use an outdated and unused document.

Importance of Test Plan

- The test plan gives direction to our thinking. This is like a rule book, which must be followed.
- The test plan helps in determining the necessary efforts to validate the quality of the software application under the test.
- The test plan helps those people to understand the test details that are related to the outside like developers, business managers, customers, etc.
- Important aspects like test schedule, test strategy, test scope etc are documented in the test plan so that the management team can review them and reuse them for other similar projects.

3.2.3 Prioritization Plan

Prioritization and planning are two sides of the same coin. Planning is **thinking**

Notes

about the tasks required to achieve the desired goal on some scale. Prioritization is ensuring you are **doing the right tasks**. Planning and prioritization are two of the best skills a manager can have. They ensure good use of your own efforts and those of your team.

Prioritization is making the best use of your limited time and resources when demands are seemingly limitless. Every single day a manager is bombarded with demands with “ASAP” written all over it. Unending meeting requests, continuous daily reports, pressing operative issues and urgent project tasks — you name it—the list goes on and on and on! If you get into that vicious cycle of trying to do everything, you’ll end up burned out, frustrated and unhappy.

Prioritization in principle means doing “first things first;” as a process it means evaluating a group of items and ranking them in order of importance and urgency. – *Business Dictionary*

Your day only has a limited number of hours, this is the same for your week, your month, your year etc. There is a maximum number of things that you can possibly do (with good quality) in a period of time— therefore you need to prioritize.

If everything is important then nothing is important. If you qualify the “not-so-important-tasks” as very important it devalues any other “more-important-tasks”.

Start your day by devoting a fair amount of your creative energy to planing your day. This will jump start your day on the right track. You will know your **action items** (things that matter) and **backburners** (tasks that can wait).

Perhaps a year is a much longer period but even though, there is a maximum number of things you can do in a year—therefore, you still need to prioritize. Annual planning sessions are important endeavors for companies wanting to set priorities right for the year and align objectives with strategic goals. Nothing beats starting the year in the right direction; you have a game plan and you understand what needs to be done to accomplish your goals.

By planning ahead you are in the best position to adjust priorities. Proper planning is building enough room into your plans for additional demands. New demands pop-up and they may also be important. Adjusting priorities is commonplace; you should always assume that there will be unexpected requests. Set aside time for them. As long as they are important tasks that bring you closer to you goal, they must be done!

3.2.4 Test Exit Criteria, Cost and Economy Aspects

Test Exit Criteria

Exit criteria in testing are often viewed as a single document commemorating the end of a life cycle phase. It can be defined as “The specific conditions or on-going activities that should be fulfilled before completing the software testing life cycle. STLC specifies which exit criteria is required at each testing phase”. The exit criteria can identify the intermediate deliverables and enable you to track them as independent events.

Notes

The following exit criteria should be considered for completion of a testing phase:

- Ensuring all critical Test Cases are passed
- Achieving complete Functional Coverage
- Identifying and fixing all the high-priority defects
- Fixing all the 'Show Stopper defects' or 'Blockers' and ensuring that none of the identified Critical/Severity 1 defects are in Open Status
- Re-testing and closing all the high-priority defects to execute corresponding Regression scenarios successfully

The following output is achieved through the exit criteria:

- Test Logs
- Test Incident Report Log
- Test Summary Report/Findings Report

Listed below are some of the commonly considered exit criteria for terminating or concluding the testing process:

- Budgeting challenges
- Deadlines not meet
- Complete execution of all the test cases
- Meets the desired and sufficient coverage requirements and functionalities
- Closure of all the identified defects
- No critical bug has been left out during the testing process

Leverage Our Testing Expertise

Our testing team can help you fix the gaps and provide recommendations to meet your testing requirements

Here are some of the industry-leading best practices for Entry and Exit Criteria Considerations:

3. Before commencing any process, it is pertinent to define the entry and exit criteria for each test type explicitly
4. Quantitative over qualitative representation of measures or preconditions
5. Assigned corrective action or restart of the whole process with necessary changes must be allocated for the failure in meeting entry or exit criteria
6. Continuous vigilance with follow-up from the moderator during the process creation and review is much required.

To sum up, defining the entry and exit criteria for a software testing process is essential. The requirements discussed above will help the testing teams to plan and drive testing tasks within stipulated deadlines. It is without compromising the quality, functionality, effectiveness, and efficiency of the developed software. Rishabh Software provides end-to-end software testing services to ensure a bug-free, robust software solution for your business. Check out our Testing capabilities to leverage our matured and proven test approach.

Notes

Cost and economy Aspects

Cost of Quality:

It is the most established, effective measure of quantifying and calculating the business value of testing. There are four categories to measure cost of quality: Prevention costs, Detection costs, Internal failure costs, and External failure costs.

These are explained as follows below.

1. **Prevention costs** include cost of training developers on writing secure and easily maintainable code
2. **Detection costs** include the cost of creating test cases, setting up testing environments, revisiting testing requirements.
3. **Internal failure costs** include costs incurred in fixing defects just before delivery.
4. **External failure costs** include product support costs incurred by delivering poor quality software.

Major parts of total cost are detecting defects and internal failure cost. But, these costs less than external failure costs. That's why testing provides good business value.

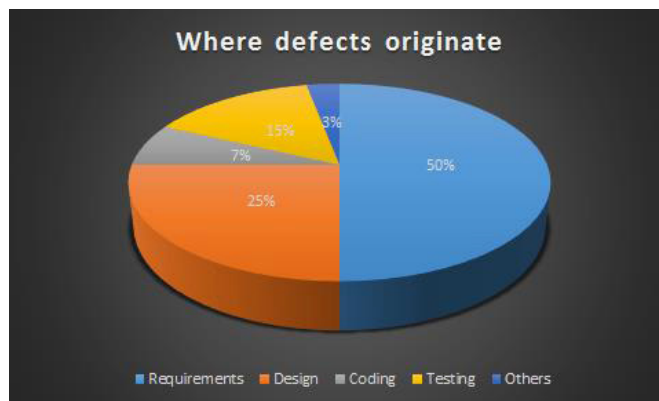
Business value of software testing:

Test manager has responsibility to identify the business value and provide communications between teams and senior management. It concerns the cost of quality.

To deliver business value, these are some of the measurable ways, like

1. Detecting defects
2. Documenting defects
3. Status reports and test metrics on project progress
4. Status reports on process implementation and product development
5. Increasing confidence in product quality
6. Legal liabilities
7. Decreasing risks
8. Ensuring predictable product
9. Enhancing reputation for product
10. Enhancing reputation for process quality
11. Testing can prevent mission failure

There is a definite economic impact of software testing. One economic impact is from the cost of defects and the another is the way we perform testing. Cost of defect is a very real and very tangible cost. The second point the way we perform testing will not consider it now.

Where defects originate?

One of the most commonly understood facts about defects is that most defects originate in the requirements definition phase of a project. The next runner-up is the design phase.

Some problems in getting accurate, clear, and testable requirements are:

- Many people do not have a solid requirements gathering process
- Few people have been trained in or understand the dynamics of requirements
- Projects, people, and the world around us change very quickly
- The English language is ambiguous and even what we consider clear language can be interpreted differently by different people.

Where testing resources are used?

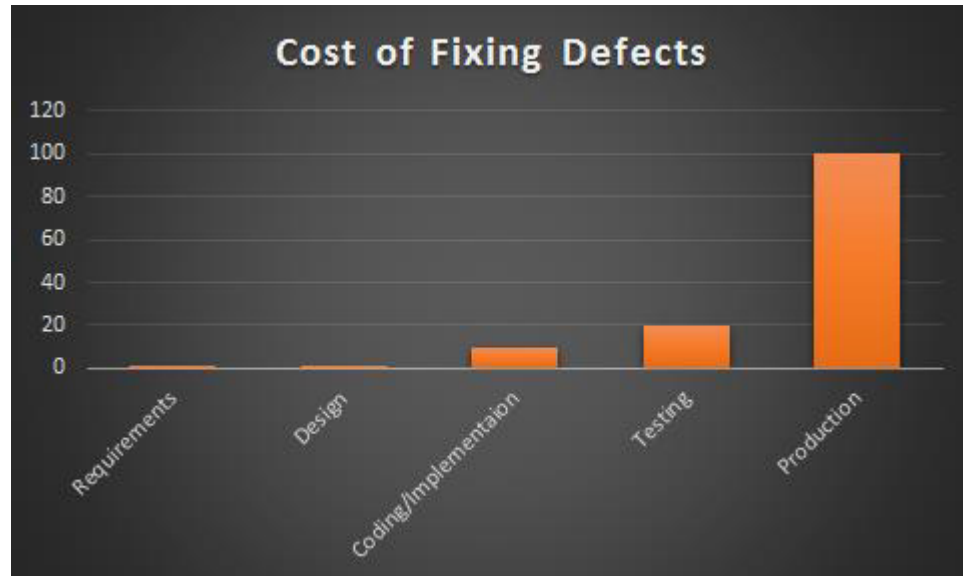
In the previous section we saw that most defects originate in requirements and design, but most of the testing effort occurs in a traditional “testing” phase toward the end of the project. The problem with the big bang approach to testing is that defects are not found until toward the end of the project. This is the most costly and risky time to fix defects. Some complex defects may even be impossible to fix.

Relative cost of fixing defects

One of the known facts about software defects is that the longer they go undetected, the more expensive they are to fix. Although research differs on the exact ratios, the general rule is 1:10:100. That is, if a defect costs one unit (hour, dollar, etc.) to fix in requirements and design, it costs 10 units to fix in testing (system/acceptance) and over 100 times to fix in production. This cost of defects doesn't even take into account the impact cost of defects. These costs of fixing in production may be even higher than 100 times. Costs could be attributed to lost revenue, reimbursements, fraud, lost customers, bad public relations, and litigation.

Notes

Notes



3.3.1 Test Strategies

A high-level document is used to validate the test types or levels to be executed for the product and specify the **Software Development Life Cycle's** testing approach is known as Test strategy document.

Once the test strategy has been written, we cannot modify it, and it is approved by the **Project Manager, development team.**

The test strategy also specifies the following details, which are necessary while we write the test document:

- **What is the other procedure having to be used?**
- **Which module is going to be tested?**
- **Which entry and exit criteria apply?**
- **Which type of testing needs to be implemented?**

In other words, we can say that it is a document, which expresses how we go about testing the product. And the approaches can be created with the help of following aspects:

- **Automation or not**
- **Resource point of view**

We can write the test strategy based on **development design documents.**

The development design document includes the following documents:

- **System design documents:** Primarily, we will use these documents to write the test strategy.
- **Design documents:** These documents are used to specify the software's functionality to be enabled in the upcoming release.
- **Conceptual design documents:** These are the document which we used In frequently.

Notes

The Objective of Test Strategy

The primary objective of writing the test strategy is to make sure that all purposes are covered entirely and understood by all stakeholders, we should systematically create a test strategy.

Furthermore, a test strategy objective is to support various quality assurance stockholders in respect of **planning of resources, language, test and integration levels, traceability, roles and responsibilities**, etc.

Features of Test Strategy Document

In SDLC (Software Development Life Cycle), the test strategy document plays an important role. It includes various significant aspects, such as who will implement the testing, what will be tested, how it will be succeeded, and what risks and incidents will be related to it.

Some of the additional characteristics of the Test Strategy document are as follows:

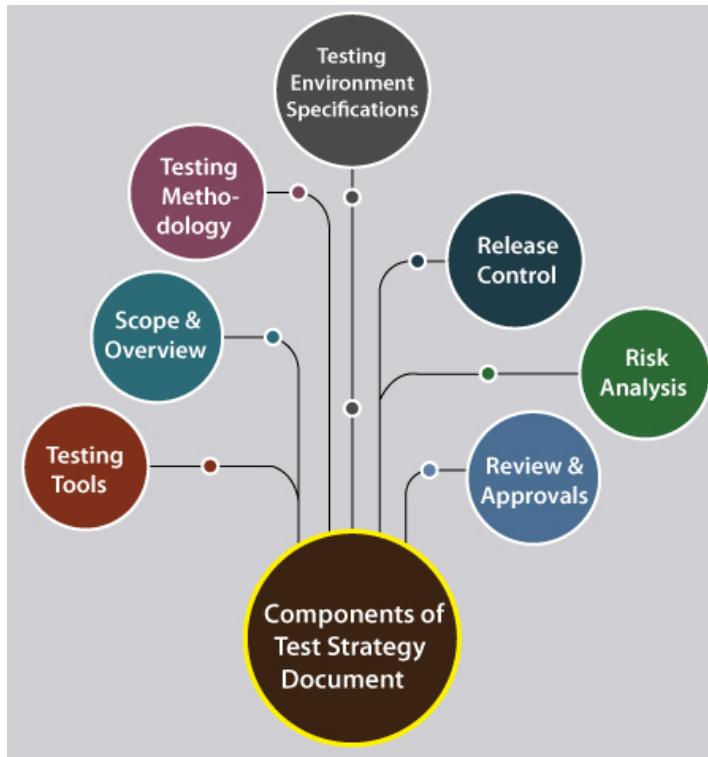
- The test strategy document is approved and reviewed by the following's peoples:
 - **Test Team Lead**
 - **Development Manager**
 - **Quality Analyst Manager**
 - **Product Manager**
- For different testing activities, the test strategy document specifies the resources, scope, plan, methodology, etc.
- In order to direct how testing will be achieved, it is used by the project test team once it is ready or completed.
- Primarily, it is obtained from the **BRS (Business Requirements Specifications)** documents.
- The test strategy document is a high-level document, which generally remains constant, implying no frequent and pointless modification is made in the document.
- The respective team easily accomplishes the objectives of testing with the help of a test strategy document.
- The respective team easily accomplishes the objectives of testing with the help of test strategy document.

Components of Test Strategy Document

We understand that the test strategy document is made during the requirements phase and after the requirements have been listed.

Like other testing documents, the test strategy document also includes various components, such as:

Notes



- **Scope and Overview**
- **Testing Methodology**
- **Testing Environment Specifications**
- **Testing Tools**
- **Release Control**
- **Risk Analysis**
- **Review and Approvals**

Let's see them one by one for our better understanding:

1. Scope and Overview

- The first component of the test strategy document is **Scope and Overview**.
- The overview of any product contains the information on who should approve, review and use the document.
- The test strategy document also specified the testing activities and phases that are needed to be approved.

2. Testing Methodology

- The next module in the test strategy document is **Testing methodology**, which is mainly used to specify the **levels of testing, testing procedure, roles, and responsibilities** of all the team members.
- The testing approach also contains the change management process involving the modification request submission, pattern to be used, and activity to manage the request.

- Above all, if the test strategy document is not established appropriately, then it might lead to **errors or mistakes** in the future.

3. Testing Environment Specifications

- Another component of the test strategy document is **Testing Environment Specification**.
- As we already aware of the specification of the **test data** requirements is exceptionally significant. Hence, clear guidelines on how to prepare test data are involved in the testing environment specification of the test strategy document.
- This module specifies the information related to **the number of environments and the setup demanded**.
- The backup and restore strategies are also offered to ensure that there is no data loss because of the coding or programming issues.

4. Testing Tools

- **Testing tools** are another vital component of the test strategy document, as it stipulates the complete information about the **test management** and **automation tools** necessary for test execution activity.;
- For **security, performance, load testing**, the necessary methodologies, and tools are defined by the details of **the open-source or commercial tool** and the number of users that can be kept by it.

5. Release Control

- Another important module of the test strategy document is **Release Control**.
- It is used to ensure that the correct and effective **test execution** and release management strategies should be systematically developed.

6. Risk Analysis

- The next component of the test strategy document is **Risk Analysis**.
- In the test strategy document, all the possible risks are described linked to the project, which can become a problem in test execution.
- Furthermore, for inclining these risks, a clear strategy is also formed in order to make sure that they are undertaking properly.
- We also create a contingency plan if the development team faces these risks in real-time.

7. Review and Approvals

- The last component of the Testing strategy document is **Review and Approval**.
- When all the related testing activities are specified in the test strategy document, it is reviewed by the concerned people like:
 - ◆ **System Administration Team**
 - ◆ **Project Management Team**

Notes

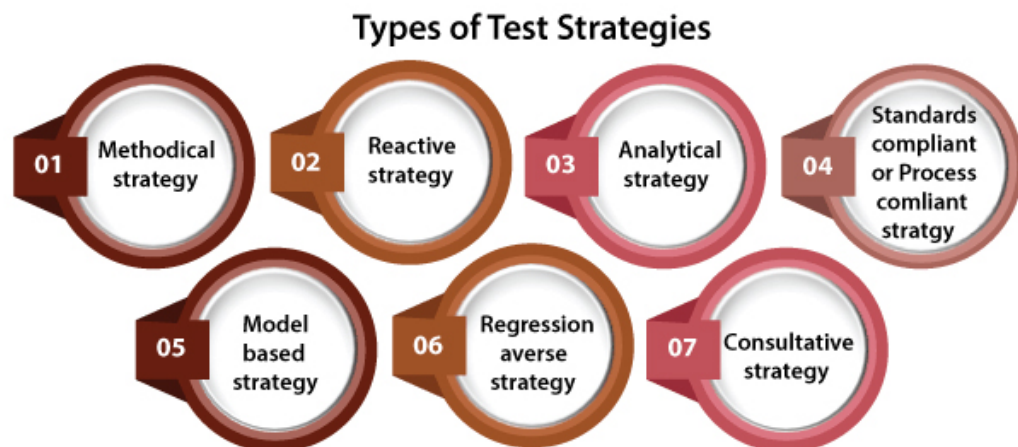
◆ Development Team

◆ Business Team

- Together with the **correct date, approver name, comment, and** summary of the reviewed variations should be followed while starting the document.
- Likewise, it should be constantly reviewed and updated with the testing process improvements.

Types of Test Strategies

Here, we are discussing some of the significant types of test strategies document:



- Methodical strategy
- Reactive strategy
- Analytical strategy
- Standards compliant or Process compliant strategy
- Model-based strategy
- Regression averse strategy
- Consultative strategy

Let's understand them one by one in detail:

1. Methodical Strategy

- The first part of test strategy document is **Methodical strategy**.
- In this, the test teams follow a **set of test conditions, pre-defined quality standard**(like ISO25000), **checklists**.
- The Standard checklists is occurred for precise types of testing, such as **security testing**.

2. Reactive Strategy

- The next type of test strategy is known as **Reactive strategy**.
- In this, we can design the test and execute them only after the real software is delivered, There fore, the **testing is based upon the identified defects** in the existing system.

- Suppose, we have used the **exploratory testing**, and the test approvals are established derived from the existing aspects and performances.
- These test approvals are restructured based on the outcome of the testing which is implemented by the test engineer.

3. Analytical strategy

- Another type of test strategy is **Analytical strategy**, which is used to perform testing based on requirements, and requirements are analyzed to derive the test conditions. And then **tests are designed, implemented, and performed** to encounter those requirements. **For example, risk-based testing or requirements-based testing.**
- Even the outcomes are recorded in terms of **requirements**, such as **requirements tested and passed.**

4. Standards compliant or Process compliant strategy

- In this type of test strategy, the test engineer will follow the **procedures or guidelines created by a panel of industry specialists or committee standards** to find test conditions, describe test cases, and put the testing team in place.
- Suppose any project follows the **Scrum** Agile technique. In that case, the test engineer will generate its complete test strategy, beginning from classifying test criteria, essential test cases, performing tests, report status, etc., around each **user story**.
- Some **good examples** of the standards-compliant process are **Medical systems following US FDA (Food and Drugs Administration) standards.**

5. Model-based strategy

- The next type of test strategy is a **model-based strategy**. The testing team selects the **current or expected situation** and produces a model for it with the following aspects: inputs, **outputs, processes, and possible behavior**.
- And the models are also established based on the current data speeds, software, hardware, infrastructure, etc.

6. Regression averse strategy

- In the regression averse strategy, the test engineer mainly **emphasizes decreasing regression risks** for **functional or non-functional** product shares.
- **For example**, suppose we have one web application to test the regression **issues** for the particular application. The testing team can develop the test automation for both **typical and exceptional use cases** for this scenario.
- And to facilitate the tests can be run whenever the application is reformed, the testing team can use **GUI-based automation tools.**

7. Consultative strategy

- The consultative strategy is used to consult **key investors as input** to **choose the scope** of test conditions as in user-directed testing.

Notes

- In order of priority, the client will provide a list of **browsers and their versions, operating systems, a list of connection types, anti-malware software**, and also the contradictory list, which they want to test the application.
- As per the need of the items given in provided lists, the test engineer may use the various testing techniques, such as **equivalence partitioning**

We can combine the two or more strategies as per the needs of the product and organization's requirements. And it is not necessary to use any one of the above listed test strategies for any testing project.

Test strategy selection

The selection of the **test strategy may depend on the below aspects:**

- The selection of test strategy depends on the **Organization type and size**.
- We can select the test strategy based on the **Project requirements**, such as **safety and security** related applications require rigorous strategy.
- We can select the test strategy based on the **Product development model**.

What kind of details may include in the test strategy document?

The final document of the test strategy contains important details about the following factors:

- Scope and Overview
- Re-usability of both software and testing work products.
- Details of different Test levels, relationships between the test levels, and procedure to integrate different test levels.
- Testing environment
- Testing techniques
- Level of automation for testing
- Different testing tools
- Risk Analysis
- For each test level Entry as well exit conditions
- Test results reports
- Degree of independence of each test
- Metrics and measurements to be evaluated during testing
- Confirmation and regression testing
- Managing defects detected
- Managing test tools and infrastructure configuration
- Roles and responsibilities of Test team members

3.3.2 Preventive versus Reactive Approach

Implementation of test strategy for a particular project is known as "test approach". The test approach is usually defined in all test plans and test designs. Test approach

refers to the commencement of various project activities such as planning the testing process, selecting the designs, defining the entry and exit criteria etc.

There are two approaches in testing:

Preventative approach.

Reactive approach.

Preventative approach

In preventative approach, tests are designed at an early stage, i.e. before the commencement of software development.

Reactive approach

In Reactive approach, tests are designed after software development.

Before selecting a 'test approach', we should consider the following factors:

Analyse the various risks associated with failure of the project - to the people, the environment, and the company.

Analyse the expertise and efficiency of the people in the proposed tools and techniques.

The nature of product and the business.

3.3.3 Analytical versus heuristic Approach

An **analytical approach** is the use of an appropriate process to break a problem down into the elements necessary to solve it. Each element becomes a smaller and easier problem to solve. It follows that a non-analytical approach is just the opposite: the use of an inappropriate process, which is unable to break a problem down into the elements necessary to solve it. Because this is not done, the problem remains too big and complex to solve. That is one reason an analytical approach is the only reliable way that will work on solving the global environmental sustainability problem, because that problem is too big and complex to solve any other way.

Let's take a longer approach to proving an analytical approach is not only a better way, it is the only way.

We will try to prove two things:

1. The analytical approach is the only known approach that works consistently on difficult problems.
2. The global environmental sustainability problem is a difficult problem.

If both propositions are true, then it follows that an analytical approach is the best way to solve the global environmental sustainability problem. Let's prove proposition (1), then (2), and finally conclude the argument.

Proposition 1 - The analytical approach is the only known approach that works consistently on difficult problems.

Notes

First, we need to prove that the analytical approach is the only known approach that works consistently on difficult problems.

The analytical approach is the formal use of reason to solve problems. The first rules to formal reasoning were invented by Aristotle (384 to 322 BC). Reasoning correctly involves representing the constituent elements of an argument with premises, intermediate conclusions, and final conclusions.

Here's a simple example showing the difference between an intuitive and an analytical approach. Suppose you need to multiply two three-digit numbers. Only a few geniuses can do it in their head, seemingly intuitively. The rest of us cannot no matter how hard we try. We are forced to take an analytical approach, which is structuring one's analysis.

A multiplication problem is structured by writing the two numbers down as shown. Solving the problem then becomes as easy as falling off a log because it requires only multiplying or adding two one-digit numbers at a time.

We use hundreds of similar analytical approaches every day, like planning a driving route, composing a meal, and planning how to best approach an important conversation or project. These are done so often and so fast they seem intuitive. But in fact, they are analytical.

An analytical approach takes a problem, breaks it down into its constituent elements so as to understand the problem, and then adds elements that represent a solution. These elements form the formal argument that this is the problem and this is the solution.

The reason an analytical approach is required for difficult problems is that all this becomes too complicated to do intuitively. Each element must be represented formally, such as with exact phrases in writing or with equations in a simulation model, so that the problem solver(s) can go over and over an evolving analysis to be certain it is correct. Complex problems have dozens or hundreds of elements, and hundreds or thousands of relationships between those elements. However, the mind has only seven (plus or minus two) short term memory banks. This causes the mind to overload quickly on any but the simplest of problems, or problems it has encountered before and memorized the solution.

Before the invention of the Scientific Method in the 17th century, science was based on tradition and guesswork. Afterward it was based on an analytical approach. This momentous change caused science to shift into a whole new mode of thinking, one so productive it quickly led to the Industrial Revolution and all that science and technology have brought us today. *Science knows of no other method that will work to produce reliable knowledge.* This should be proof enough that an analytical approach is required to solve difficult problems.

To summarize, difficult problems require analysis because finding the correct solution rationally instead of by guessing requires a rigorous structured approach. A correct analysis requires reliable understanding, i.e. reliable knowledge. And the only known way to produce reliable knowledge, knowledge that you know is true, is the Scientific Method. Therefore, because the Scientific Method is an analytical approach, an analytical approach is the only known way to solve difficult problems.

Notes

Heuristic A heuristic, or a heuristic technique, is any approach to problem-solving that uses a practical method or various shortcuts in order to produce solutions that may not be optimal but are sufficient given a limited timeframe or deadline. Heuristics methods are intended to be flexible and are used for quick decisions, especially when finding an optimal solution is either impossible or impractical and when working with complex data.

Advantages and Disadvantages of Using Heuristics

Heuristics facilitate timely decisions. Analysts in every industry use rules of thumb such as intelligent guesswork, trial and error, the process of elimination, past formulas, and the analysis of historical data to solve a problem. Heuristic methods make decision-making simpler and faster through shortcuts and good-enough calculations.

There are trade-offs with the use of heuristics that render the approach prone to bias and errors in judgment. The user's final decision may not be the optimal or best solution, the decision made may be inaccurate, and the data selected might be insufficient (thus, leading to an imprecise solution to a problem). For example, copycat investors often imitate the investment pattern of successful investment managers to avoid researching securities and the associated quantitative and qualitative information on their own.

Copycat investors hope that the formulas used by these managers will continually earn them profits, but this is not always the case. For example, the crash of Valeant Pharmaceutical International was a shock to investors; the company saw its stock plunge 90% from 2015 to 2016. Valeant was a stock held in the portfolios of many hedge fund managers and the investors copying them.

Example of Heuristics

A popular shortcut method in problem-solving is called representativeness heuristics. Representativeness uses mental shortcuts to make decisions based on past events or traits that are representative of or similar to the current situation. Say, for example, Fast Food ABC expanded its operations to India and its stock price soared. An analyst noted that India is a profitable venture for all fast-food chains. Therefore, when Fast Food XYZ announced its plan to explore the Indian market the following year, the analyst wasted no time in giving XYZ a "buy" recommendation.

Although his shortcut approach saved reviewing data for both companies, it may not have been the best decision. Fast Food XYZ may have food that is not appealing to Indian consumers, which research would have revealed.

Anchoring and adjustment is another prevalent heuristic approach. With anchoring and adjustment, a person begins with a specific target number or value—called the anchor—and subsequently adjusts that number until an acceptable value is reached over time. The major problem with this method is that if the value of the initial anchor is not the true value, then all subsequent adjustments will be systematically biased toward the anchor and away from the true value.

Notes

An example of anchoring and adjustment is a salesman begins negotiations with a very high price (that is arguably well above the fair value). Because the high price is an anchor, the final price will tend to be higher than if the car salesman had offered a fair or low price to start.

3.3.4 Test Activity Management

Test Management Process is a set of activities from the start of the testing to the end of the testing. It gives a discipline to testing. When follow a test process it gives us the plan at the initial. Test process provides the facility to plan and control the testing throughout the project cycle. It helps to track and monitor the testing throughout the project. Provides transparent of testing among stakeholders and maintains the conducted test for future reference. Affords deep level of detail of the testing that's being carrying out. Gives clear understanding of testing activities of prior project and post project to all the stakeholders. There are many tools (Tools such as qTest, JIRA, Team Service, TestLink.) available to manage the test process. Test process can be defined and practiced differently according to the necessity in test. Explained below are the typical activities in test process.

1) Test Plan:

Test plan served as an initial sketch to carry out the testing. Testing is being tracked and monitored as per the test plan. It gives a prior picture of test challenge and aspect that will be carried out for the software. By maintaining a test plan we can manage the changes in the plan. When starting new projects, based on the lesson learned in the previous tests, test plan needs to be improved to get betterment. Test plan explains the over view of particular requirement which needs to be tested, scope, functional and non-functional requirement, risk and mitigation, testing approaches, test schedule and deliverables and schedule, out of scope and assumption, test team and allocation, test environment, test activities mechanism and any other special note for testing.



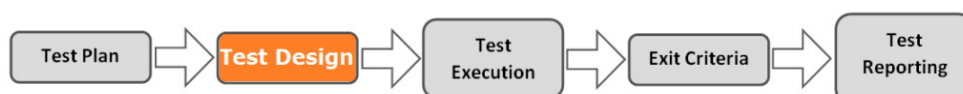
Test plan elements	Description
Over view	Over view of the test plan and purpose of this test plan. What's the project that needs to be tested? Brief of the software that needs to be tested. Purpose of providing this software to user.
Scope and out of scope	What's the purpose of the testing? What type of testing is going to be carried out? If there is any out of scope of testing. Brief explanation on the software project and what are covered in the test plan. Defining a frame to the testing based on resources, effort, budget and time line. What features or section that will be covered and what features or section will not be covered during the testing.

Notes

Functional and non-functional requirement	Explain each functional and non-fictional (performance testing, usability testing) testing that needs to be carried out. Explain each features that will be tested. Each functional and non-functional items should be placed without ambiguity.
Risk and mitigation	Explain the identified project, software and resources related risk. Explain the mitigation plan and possibility. Identify risk that we could face during the testing. Resource unavailability, delay in developer release, slip in schedule, less understanding in functions and gap between business and system requirement.
Testing approaches	What kind of testing approaches will be used? What type of testing will be carried out? Test types like installation testing, functional testing, UAT testing. Specify what tools we are going to use in testing. Specify the tools and license information that need for the testing.
Test schedule and deliverables	Describe entire start and complete date of testing. Need to find out the date of developer releases and number of releases. Mention each of the developer release date, test start date and completion date. Analyze the requirement and testing we are going to carry out and then come up with the effort. Based on the resource, plan the schedule with mile stone. We also need to consider the time frame like any specific deadline.
Assumption	There can be any assumption related to software, project, resource or any concepts. And these have to be written in this.
Test team and allocations	Who are the testers that will be involved and what their responsibilities in the project are. To whom the training is required, if any. When responsibilities are set it's easy to conduct the testing in project.
Test environment	Provide all the information related to test environment. What is the test environment? In which browsers the testing is carried out? Mentioning the UAT environment. External system that will be accessed during the testing. State the capacity of RAM and processor.

2) Test Design:

Test design affords how to implement the testing. Typically creating test cases is with inputs and expected output of the system and choosing which test cases are necessary for the execution of the test. Tester should have the clear understanding and appropriate knowledge to set the expected result. By this, coverage of the testing is defined and tester will not miss any scenario. There are two types of test design techniques one is static testing and the other one is dynamic testing. Static testing is used to test without execution mostly to artifacts like document and dynamic testing is testing by executing the system.



Notes

Test case (Element in test case document):

- Project / Test title, Test executed by, Test executed date, Version of the software and Test environment
- Test case number
- Test summary
- Steps
- Pre-condition
- Post condition
- Test data
- Actual result
- Expected result
- Test result
- Note

3) Test Execution:

Manner of executing and test the actual system result against the expected result is test execution. Test execution can be done manually and by using automation suit. During the execution tester needs to make sure, that the user's need of the software is occupied in the software. Test execution is conducted by referring the document created during test design as step by step process. Tester needs to keep the track while executing the test cases.



Example for static testing:

- Test the requirement specification document.
- Test the design document
- Test the user guide

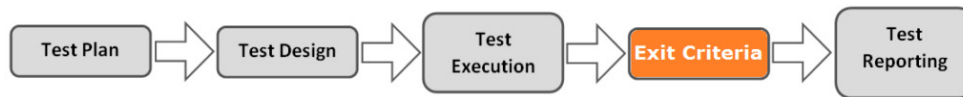
Example for dynamic testing:

- Unit testing
- Functional testing
- Integration testing

4) Exit Criteria:

Exit criteria determines when to stop the test execution. Exit criteria is defined during the test plan phase and used in the test execution phase as a mile stone. Tester needs to set the exit criteria at the beginning, exit criteria may change during the project run as well. There are factors like client need, system stability and filled function that

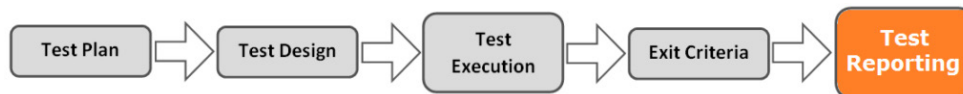
decide the exit criteria. Once the tester reached the exit criteria testing will be stopped. Below are some common exit criteria.



- All critical defects are closed.
- All the reported defects are closed and verified.
- Executed and covered the areas which used by user mostly.
- System catered all the requirements.
- All the important functions are tested and working as expected.

5) Test Reporting:

Test reporting gives the picture of test process and result for the particular testing cycle. To define the element in the test reporting the first thing that needs to be considered is whom the audiences of the test report are. For an example a project manager will like to see the high level picture of the testing, intermediate people will wish to view more detail and the client will expect the test reporting in the criteria such as requirement basis, feature basis. Test report is prepared and communicated periodically like daily, weekly, month etc. This needs to be sent in different stages and time. In the future project result of test reports needs to be analysed and apply the lesson learns. Test report contain elements such as test execution status, completed percentage, plan vs. executed test cases, test environment, test execution by resources, risk and mitigation if any, defect summary, test scenario and conditions, any assumption, any note etc.



Test coverage report: (Elements of test coverage report)

- Percentage completed
- Test scenario
- Software area
- Tested resource
- Tested date
- Test result

Defect summary report: (Elements of defect summary report)

- Defect by severity
- Defects by priority
- Defects by assigned developer
- Defects by function

Notes

Notes

- Defects by software area
- Open and closed defects

Risk and mitigation report: (Elements of risk and mitigation report)

- Identified risk
- Likelihood
- Risk level
- Risk type
- Mitigation plan

3.4.1 Incident Management

The occurrence of an activity or event during testing is referred to as an 'incident'. An incident demands further investigation in order to find the root cause and take corrective action. The reason for occurrence of an incident is deviation in actual and expected results. Incidents may arise at any point of time throughout the software development life cycle.

Incident management is basically an art of identifying, investigating and taking necessary actions to prevent such an event. The idea behind incident management is to ensure that incidents are tracked from its identification stage until its correction stage, so that the final result is bug free. Roles and responsibilities should be assigned to monitor the whole process of incident management.

Objectives of an incident summary/report:

- It is important for the developers and other team members to understand the details of a problem to be able to arrive at a solution.
- A detailed view of an incident report gives a deep understanding to the testers so that they are able to track the quality of the system undergoing test activities efficiently.
- Provide ideas to improve test process. For all incidents the point of its occurrence should be noted whether it is a defect in requirements, code, or in any subsequent phase. Hence test improvement process can focus on the right area.

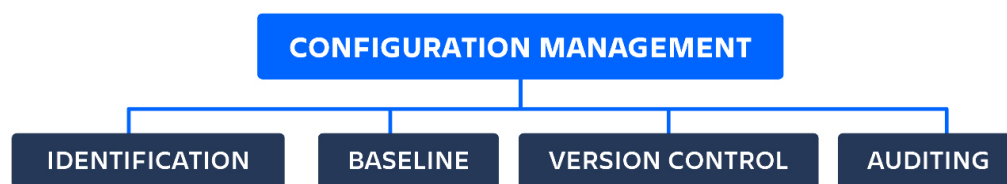
An incident report includes facts that portray the areas where it has occurred along with few other details like -

- Severity, scope and priority of an incident.
- Date of issuing the report, issuing authority, approvals and status.
- An outline of expected and actual results.
- Describing the test environment and configuration details.
- The phase of the software development lifecycle in which the incident was caught.
- Description of an incident with the help of logs, screenshots etc.
- The extent of impact an incident may pose on a stakeholder.

Notes

- Degree of severity of an incident and set the priority as per its impact on the system.
- Status of an incident whether it is a repeated one, deferred, open, waiting to be fixed, awaiting confirmation from the authority and so on.
- Recommendations and approvals.
- Due to changes made in the system as a result of an incident, note whether it causes trouble in other areas as well.

Note the sequence of changes made in the order of actions taken by the project team members.

3.4.2 Configuration Management - Need and Identification

Configuration management is a system engineering process for establishing consistency of a product's attributes throughout its life. In the technology world, configuration management is an IT management process that tracks individual configuration items of an IT system. IT systems are composed of IT assets that vary in granularity. An IT asset may represent a piece of software, or a server, or a cluster of servers. The following focuses on configuration management as it directly applies to IT software assets and software asset CI/CD.

Software configuration management is a systems engineering process that tracks and monitors changes to a software systems configuration metadata. In software development, configuration management is commonly used alongside version control and CI/CD infrastructure. This post focuses on its modern application and use in agile CI/CD software environments.

Why is configuration management important?

Configuration management helps engineering teams build robust and stable systems through the use of tools that automatically manage and monitor updates to configuration data. Complex software systems are composed of components that differ in granularity of size and complexity. For a more concrete example consider a microservice architecture. Each service in a microservice architecture uses configuration metadata to register itself and initialize. Some examples of software configuration metadata are:

- Specifications of computational hardware resource allocations for CPU, RAM, etc.
- Endpoints that specify external connections to other services, databases, or domains
- Secrets like passwords and encryption keys

Notes

It's easy for these configuration values to become an afterthought, leading to the configuration to become disorganized and scattered. Imagine numerous post-it notes with passwords and URLs blowing around an office. Configuration management solves this challenge by creating a "source of truth" with a central location for configuration.

Git is a fantastic platform for managing configuration data. Moving configuration data into a Git repository enables version control and the repository to act as a source of truth. Version control also solves another configuration problem: unexpected breaking changes. Managing unexpected changes through the use of code review and version control helps to minimize downtime.

3.4.3 Configuration Control

Configuration values will often be added, removed, or modified. Without version control this can cause problems. One team member may tweak a hardware allocation value so that the software runs more efficiently on their personal laptop. When the software is later deployed to a production environment, this new configuration may have a suboptimal effect or may break.

Version control and configuration management solve this problem by adding visibility to configuration modifications. When a change is made to configuration data, the version control system tracks it, which allows team members to review an audit trail of modifications.

Configuration version control enables rollback or "undo" functionality to configuration, which helps avoid unexpected breakage. Version control applied to the configuration can be rapidly reverted to a last known stable state.

How configuration management fits with DevOps, CI/CD and agile

Configuration data has historically been hard to wrangle and can easily become an afterthought. It's not really code so it's not immediately put in version control and it's not first-class data so it isn't stored in a primary database. Traditional and small-scale system administration is usually done with a collection of scripts and ad-hoc processes. Configuration data can be overlooked at times, but it is critical to system operation.

The rise of cloud infrastructures has led to the development and adoption of new patterns of infrastructure management. Complex, cloud-based system architectures are managed and deployed through the use of configuration data files. These new cloud platforms allow teams to specify the hardware resources and network connections they need provisioned through human and machine-readable data files like YAML. The data files are then read and the infrastructure is provisioned in the cloud. This pattern is called infrastructure as code (IaC).

DevOps configuration management

In the early years of internet application development, hardware resources and systems administration were primarily performed manually. System administrators wrangled configuration data while manually provisioning and managing hardware resources based on configuration data.

Notes

Configuration management is a key part of a DevOps lifecycle. DevOps configuration is the evolution and automation of the systems administration role, bringing automation to infrastructure management and deployment.

DevOps configuration also brings system administration responsibility under the umbrella of software engineering. Enterprises today utilize it to empower software engineers to request and provision needed resources on demand. This removes a potential organizational dependency bottleneck of a software development team waiting for resources from a separate system administration team.

CI/CD configuration management

CI/CD configuration management utilizes pull request-based code review workflows to automate deployment of code changes to a live software system. This same flow can be applied to configuration changes. CI/CD can be set up so that approved configuration change requests can immediately be deployed to a running system. A perfect example of this process is a GitOps workflow.

Agile configuration management

Configuration management enables agile teams to clearly triage and prioritize configuration work. Examples of configuration work are chores and tasks like:

- Update the production SSL certificates
- Add a new database endpoint
- Change the password for dev, staging, and production email services.
- Add API keys for a new third-party integration

Once a configuration management platform is in place, teams have visibility into the work required for configuration tasks. Configuration management work can be identified as dependencies for other work and properly addressed as part of agile sprints.

Configuration management tools

Git

Git is the industry-leading version control system to track code changes. Adding configuration management data alongside code in a Git repository provides a holistic version control view of an entire project. Git is a foundational tool in higher-level configuration management. The following list of other configuration management tools is designed to be stored in a Git repository and leverage Git version control tracking.

Docker

Docker introduced containerization that is an advanced form of configuration management -- like a configuration lockdown. Docker is based on configuration files called Dockerfiles, which contain a list of commands that are evaluated to reconstruct the expected snapshot of operating system state. Docker creates containers from these Dockerfiles that are snapshots of a preconfigured application. Dockerfiles are committed to a Git repository for version tracking and need additional configuration management to deploy them to infrastructure.

Notes

Terraform

Terraform is an open source configuration management platform by HasiCorp. Terraform uses IaC to provision and manage clusters, cloud infrastructure, or services. Terraform supports Amazon Web Services (AWS), Microsoft Azure, and other cloud platforms. Each cloud platform has its own representation and interface for common infrastructure components like servers, databases, and queues. Terraform built an abstraction layer of configuration tools for cloud platforms that enable teams to write files that are reproducible definitions of their infrastructure.

Ansible, Salt Stack, Chef, Puppet

Ansible, Salt Stack, and Chef are IT automation frameworks. These frameworks automate many traditional system administrators' processes. Each framework uses a series of configuration data files -- usually YAML or XML -- that are evaluated by an executable.

The configuration data files specify a sequence of actions to take to configure a system. The actions are then run by the executable. The executable differs in language between the systems -- Ansible and Salt Stack are Python based and Chef is Ruby. This workflow is similar to running ad-hoc shell scripts but offers a more structured and refined experience through the respective platforms ecosystems. These tools are what will bring enable the automation needed to achieve CI/CD.

How to implement configuration management

Identification

The first action towards configuration management is information gathering. Configuration data should be aggregated and compiled from different application environments, development, staging, and production for all the components and services in use. Any secret data like passwords and keys should be identified and securely encrypted and stored. At this point configuration data should be organized into data files that can be pointed to as a central source of truth.

Baseline

After configuration data has been aggregated and organized a baseline can be established. A baseline configuration is a known state of configuration that will successfully operate the dependent software without error. This baseline is usually created by reviewing the configuration of a functioning production environment and committing those configuration settings.

Version Control

Your development project should use a version control system. If not, install Git, initialize a repository for the project, and add the configuration data files to the repository. A word of caution before adding configuration data to a repository: make sure that any secret data like passwords or keys are encrypted with an external key. Secret data accidentally committed to a repository is a huge risk. It needs to be scrubbed from the repositories history or it will be at risk of being exploited.

Auditing

Having configuration data organized and added to a repository enables collaboration and visibility into the system's configuration. The popular pull request workflow that software teams use to review and edit code can then be applied to configuration data files. This helps build out an audit and accounting system. Any changes applied to the configuration must be reviewed and accepted by the team. This adds accountability and visibility into configuration changes.

3.4.4 Test Progress Monitoring and Control

Test Monitoring in test execution is a process in which the testing activities and testing efforts are evaluated in order to track current progress of testing activity, finding and tracking test metrics, estimating the future actions based on the test metrics and providing feedback to the concerned team as well as stakeholders about current testing process.

Test Control in test execution is a process of taking actions based on results of the test monitoring process. In the test control phase, test activities are prioritized, test schedule is revised, test environment is reorganized and other changes related to testing activities are made in order to improve the quality and efficiency of future testing process.

Congratulation! We now start with **Test Execution** phase. While your team works on the assigned tasks, you need to monitor and control their work activity.

In the Test Management Phases tutorial, we briefly introduced Test Monitoring and Control. In this tutorial, you will learn it in detail.

Cost

Costs are an important aspect of project monitoring and control. You have to estimate and track basic cost information for your project. Having accurate project estimates and a robust project budget is necessary to deliver project within the decided budget. Suppose, your boss has agreed to fund the project with \$100,000. You must keep an eye on the actual costs while the project is being implemented. As mentioned in Test estimation article, there's a ton of project activities which need money. You have to monitor and manage the project budget in order to control all that activities. Without monitoring the project cost, the project will most likely never be delivered on-budget.

Schedules

How can you work without a schedule? It can be compared to driving your car but without any idea of how long it takes you get to the destination. No matter how big or small is the size & scope of your project, you must prepare a project schedule. The schedule tells you

- When should each activity be done?
- What has already been completed?
- The sequence in which things need to be finished.

Notes

Notes

This task should be finished in one week. You can create a schedule as given below

Task Name	Duration	Start	Finish	Feb 16						
				S	M	T	W	T	F	S
Setup Test Environment	1	02/17/14	02/17/14							
Perform Integration Test	3	02/18/14	02/20/14							
Test Report	1	02/21/14	02/21/14							

Resources

As mentioned in previous articles, **resources** are all things required to carry out the project tasks. They can be people or equipment required to complete the project activity. Lack of resources can affect the project progress.

The truth is, everything may not happen as planned, employees will leave, the project budget may be cut, or the schedule will get pushed. Monitoring resources will help you to early detect any resource crunch and find a solution to deal with it.

Quality

Quality monitoring involves monitoring the results of specific **work products** (like test case suite, test execution log), to evaluate whether its meets the defined quality standards. In case results do not meet quality standards, you need to identify potential resolution.

Example: Suppose that you monitored and controlled the project progress very well. Finally, you delivered the product at the deadline. The project seems to be successful.

But after delivering 2 weeks, you got this feedback from customer

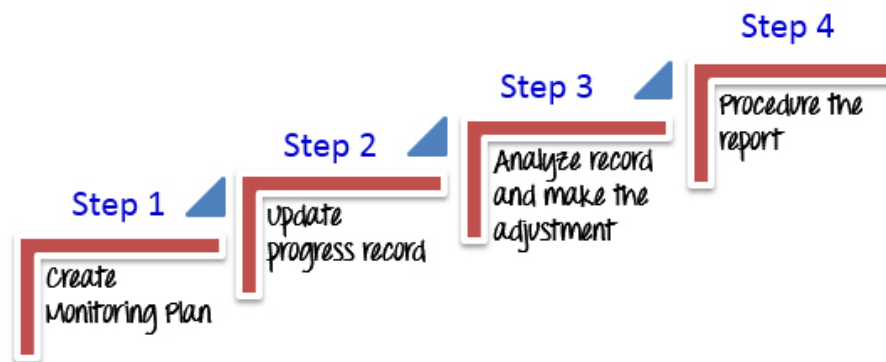
How to monitor?

As your project comes to life, keep these questions in mind:

- Are you on **schedule**? If not, how far behind are you, and how can you catch up?
- Are you over **budget**?
- Are you still working toward the same project goal?
- Are you running low on **resources**?
- Are there warning signs of impending **problems**?
- Is there **pressure** from management to complete the project sooner?

These are just a few of the questions you should ask yourself as you monitor the progress of your project.

It is important to monitor the progress of the project so you will know if adjustments need to be made to get it moving back in the right direction. To monitor project progress effectively, you should follow the following steps



Notes

Step 1) Create Monitoring Plan

You cannot monitor progress unless you have a plan to monitor progress with DEFINED metrics. Similar to Test Plan, Monitoring Plan is the first and one of the most important steps in progress monitoring.

In the Monitoring Plan, you must plan carefully about

What metrics need to collect and measure?

In the monitoring plan, you should clearly define what metrics you need to collect and measure.

- The **cost** (time, money) spent for the project so far
- How much **resource** (employees, equipment) are used for the project
- The status of the task (on **schedule**, behind or before the schedule)
- The **quality** of the work product (Run rate/pass rate, defect metrics)

When to collect the data?

Now decide **when** or **how often** you are going to collect the data for monitoring in the monitoring plan –Weekly or monthly? Or just at the start and end of the project?

As the Plan, the Guru99 Bank project will be completed in one month. In that case, we recommend you monitor the project progress **weekly** or **daily** basis.

How to evaluate the project's progress via metrics?

In the monitoring plan, you should define the **methods** to evaluate the project's progress via collected metrics. Some methods you can refer are

- Compare the progress in plan with the actual progress that the team has made
- Define the **criteria** which are used to evaluate the project's progress. For example, if the effort to complete a task took more than **30%** effort than planned a project delay.

Step 2) Update progress record

With time, your team member will be making progress on their project task. You must track their activity as per schedule and ask them frequently update the progress information such as time spent, task status...etc. By checking these records, you can

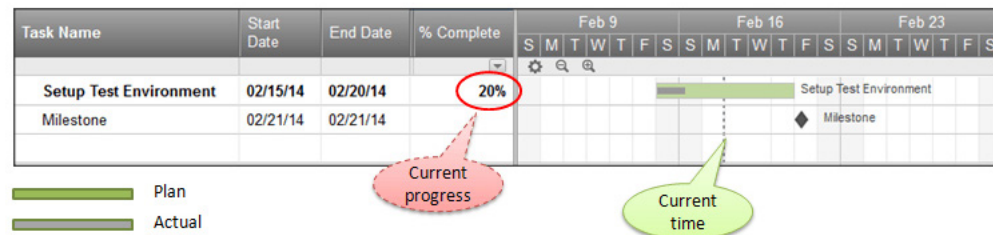
Notes

immediately see the impact on the project plan. One of the best methods to track the member progress is holding **regular meetings**.

In the meeting, all members report their current status and issues if any. If a team member or members have fallen behind or have run into obstacles, formulate a plan for identifying and solving the problem.

Let's practice with following scenario

As defined in monitoring plan, you assigned a task "Setting up Test environment" for testing website Guru99 bank to a member in your Team. His role is a Test Administrator. He has to set up the Test Environment in 6 days. You required him to report the current status in every team meeting. Here is an example of his record of current progress



Step 3) Analyse record and make the adjustment

There're 2 sub-steps in the steps



Step 3.1) Analyze

In this step, you compare the progress you defined in plan with the actual progress that the team has made. By analysing the record, you can also see how much time has been spent on individual task and the total time spent on the project overall.

By tracking and analysing the project progress, you can early detect any issue which may happen to the project, and you can find out the solution to solve that issue.

Step 3.2) Adjustment

Make the necessary adjustments keep your project on track. Reassign tasks, modify schedules, or reassess your goals. This will help you keep moving toward the finish line.

Step 4) Produce the report

If your boss asks you about the current project progress, whether the progress is behind or ahead the schedule, what will you answer? You need to prepare progress report of the project. Using the report is a good option to share the overall project progress with team members or the Management Board. It is also a useful way to show your boss whether the project is on track. You can use some template reports to ensure

the progress data is presented consistently and clearly. This article includes the report template which you can refer. Also, check a sample report for Guru99 Banking project as reference

Notes

Best Practices in Test Monitoring and Control

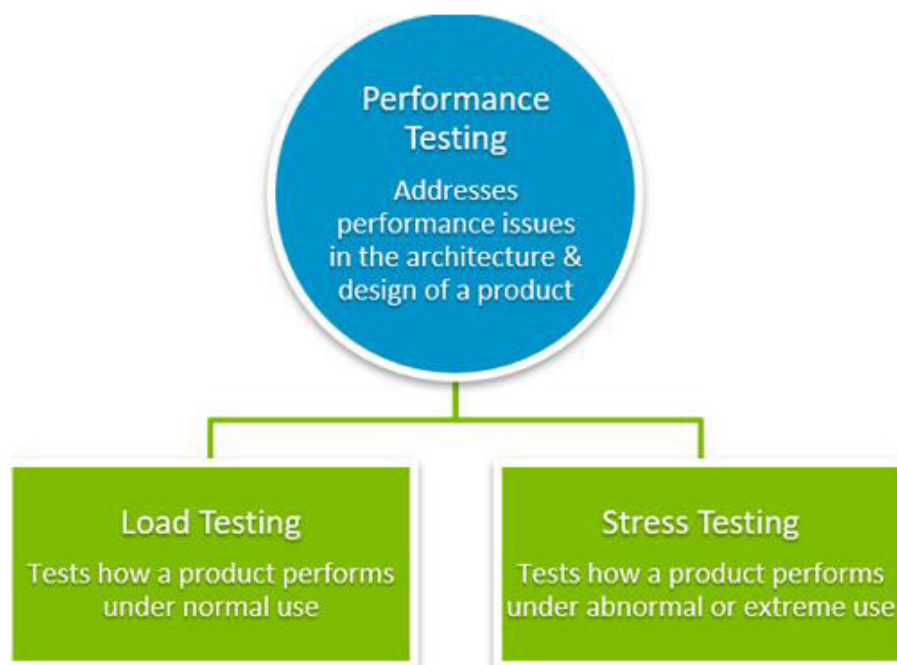
- **Follow the standards:** One important consideration of project planning is to ensure standardization. It means that all the project activities must follow the standard process guideline. Standardized processes, tools, templates, and measurement values make analysis easy, facilitate easy communications, and help the project team members understand the situation better.
- **Documentation:** What will happen if you do not write down any discussion or decision in a document? You may forget them and lose many things. You should write down discussions and decisions at the appropriate place, and establishing a formal documentation procedure for meetings. Such documentation helps you to resolve issues of miscommunication or misunderstandings among the project team.
- **Proactivity:** Issues occur in all projects. The important thing is that you have to adopt a proactive approach to solve issues and problems that arise during project execution. Such issues could be budget, scope, time, quality, and human resources

3.5.1 Performance Testing

Performance testing is an umbrella term for load testing and stress testing.

When developing an application, software, or website, you likely set a benchmark (standard) for performance. This covers what happens under:

- **Regular parameters:** If everything goes as planned, how does it work?
- **Irregular parameters:** Can my website application survive a DDoS attack?



Notes

What is performance testing?

When you want to find out how the components of a system are performing under a given situation, performance testing is the way to go. You'll want to find out three key factors about your product:

- The scalability
- The resource usage
- The reliability

As a subset of performance engineering, this type of testing addresses performance issues in the architecture and design of the system. Using performance testing tools, we can begin practicing risk management.

Load testing and stress testing are merely subsets of performance testing. As such, performance testing is a wide term that encompasses:

- Spike testing
- Endurance testing
- Volume testing
- Scalability testing

How frequently you test performance depends on your development methodology:

- A team using a waterfall approach will want to test with each individual release.
- If they are using the agile methodology, performance testing should be a constant practice alongside development.

Why use performance testing?

Want to establish the benchmark behavior of your system? The first step to testing your functionality is performance testing. During performance testing, you're looking to meet or exceed a number of industry-defined benchmarks.

Importantly, performance testing is not in any way aimed at finding defects with the application.

Rather, it's meant to set the benchmark and standard for the application. The key is to ensure that utmost accuracy is observed during performance testing. Close monitoring is necessary for ensuring the performance of a system or application.

When setting a benchmark, examine attributes such as:

- Throughput
- Response time
- Speed
- Stability
- Resource usage

These are examples of KPIs—Key Performance Indicators. Essentially, performance testing analyses KPIs like these.

Notes

Performance testing best practices

When or how often to performance test depends on what kind of testing you are doing. Here are some tips for getting the most out of your performance testing measures.

Your performance testing should not include any actual users. Performance testing should use software to measure how well it performs or does not perform. User Acceptance Testing (UAT) is when we start to use real software or system users to see how it performs in the real world.

Test as often as possible. There are specific times to test, usually near the last step. When it is time to test, do it repeatedly. Repeat tests over and over to get the most accurate results. If your application, software, or website can successfully pass the tests over and over again, you can tick it off as ready for the next stage.

Do not change what you are testing. When we make lots of changes to an experiment, it becomes more difficult to identify what exactly we're testing. Between each test, your environment should stay the same. This includes the load or stress that you place upon the system.

Use automated tools to make the process as simple as possible. Automating your testing process will boost efficiency. You can:

- Save time and money
- Minimize navigating complex architecture
- Provide more accurate results faster

Common tools for performance testing include:

- Apache JMeter
- Ready API
- BlazeMeter
- Visual Studio Test Professional
- Micro Focus LoadRunner

3.5.2 Load Testing

Load testing is testing how an application, software, or website performs when in use under an expected load. We intentionally increase the load, searching for a threshold for good performance. This tests how a system functions when it faces normal traffic.

Load testing is the process of putting pressure on a software, system, or device to measure its response under an expected load.

By simulating production (pressure), load testing shows the behaviour under normal and *expected* peak conditions. The goal is to ensure a given function, system, or program can handle what it's designed for. This is important because when you're building your product, you're only accounting for a few individual users. Load testing helps you prepare for what happens when you deploy your product to hundreds or thousands of users.

Notes

More specifically, the goals of load testing include:

- **Determining the upper limit of all the components of an application** (the hardware, database, network, etc.). This test ensures that the application has the ability to manage any load it will be exposed to in the future.
- **Detecting defects in applications** that relate to buffer overflow, memory mismanagement, and memory leaks. Load testing is likely to expose a number of issues including bandwidth issues, load balancing problems, and the carrying capacity of the system.

Usually, you load test a system that is as close as possible to becoming a finished product, ready for deployment to the masses. It shows your team whether the product is working as expected and/or intended, under the given conditions.

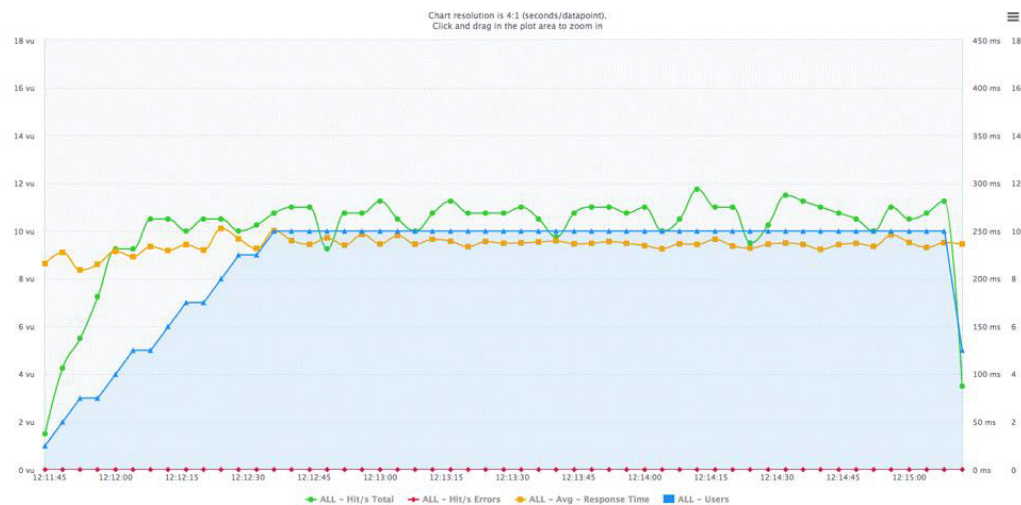
How load testing works

During load testing, the system is taken through a steadily and constantly increasing load until it reaches the threshold. You want to find the weight limit—the weight (load) of activity that an application, software, or website can handle under normal conditions.

The weight limit should be our peak load. Load testing should help you identify:

- Page load issues
- System lag
- Additional issues arising when multiple users accessing the system

Good load testing probably looks something like this:



We have four bars to look at:

- Green means hits (visitors)
- Red means errors
- Yellow means response time
- Blue means users

Notes

This is a good experiment for a number of reasons. We want to see the blue and green lines climbing while the yellow one stays relatively flat. If the yellow line drops, we know that the software can't stand up to high demands (users and visitors).

As we add hits to the software, the system should be able to withstand traffic until we reach our expected weight limit. Seeing a drop in the yellow line on the graph would indicate that there is a problem and that the weight limit is a lot lower than desired. If it can't perform well with the expected load, there may be some serious problems in the design.

Of course, you want to know what will happen to the system when the traffic increases suddenly. You may not be able to detect some of these issues during development and testing. This could include testing for:

- Traffic from different or varying geo-locations
- Sudden spikes in traffic

When to load test

Load testing as a process doesn't start till a project is close to its end. This is when system performance and actual user engagement can undergo accurate simulation and testing.

(Technically, you don't have to wait till software development is completed before testing. You can test a specific component in a focused way for the load at any stage of the development process. Still, to test the demand of hundreds of thousands of users, it's best to wait till development is nearly complete.)

Load testing tools usually mimic the actions of multiple concurrent users of the program, website, app, etc. It is a repeated procedure that involves collecting and monitoring both software and hardware statistics. Of particular interest are the:

- CPU
- Disk IO
- Memory of the physical servers

The test will also show the response time and throughput of the system along with other KPIs. When checking backend performance issues, the endurance of input over prolonged periods of time or simultaneous user input is important to understand. However, checking for things that could cause lag, broken functionality, or memory leaks before completing development is load testing in a limited form.

After analysis, the load testing software will generate a load testing report. Compare these results to your expected goals (benchmarks).

3.5.3 Stress testing

Stress testing is testing how an application, software, or website performs when under extreme pressure—an unexpected load. We increase the load to its upper limit to find out how it recovers from possible failure. This tests how a system functions when it faces abnormal traffic.

Notes

What is stress testing?

Where load testing tests expected loads, stress testing applies unrealistic load scenarios—to the point of overloading the system till it breaks. The aim is to find out how stable your product is by stretching it beyond its bandwidth capability. Stress testing evaluates how an application will behave beyond normal conditions and normal peak load.

Why do you need to know this ahead of time? The biggest reason is to see whether your sensitive data remains protected when the system fails. Is there a particular situation where you're more vulnerable to data leaks?



When you stress test, you deliberately induce failures to analyze the risks at the breaking points. You can then tweak the programs to make inevitable failures more graceful. If it's going to break, we want our functionality testing to show that it can recover! Exploring the outer limits of a system is the best way to do this.

Some of the issues that can be exposed during stress testing include:

- Memory leaks
- Race conditions
- Synchronization issues

When you've completed a round of stress testing, analyze the post-crash report to understand whether your application failed gracefully, as expected—or not.

How to stress test

Stress testing works on anything you're developing: software, an app, a website, or a backend system, like a host server. The key to successful stress testing is ensuring a controlled environment before and during the test.

Stress testing can include various activities such as:

- Overloading the system with lots of jobs
- Removing system components
- Checking the behavior when the number of users suddenly increases (known as spike tests)
- Checking the sustainability of a system over a period of time through a gradual increase in the number of users (known as soak tests)

Performance testing is critical

Performance testing of any kind is not merely a development need—it's a business

requirement.

Stakeholders and customers demand fast and reliable digital experiences. This means that your applications must be able to support thousands or even hundreds of thousands of users seamlessly. Knowing what test to do at what time and for what purpose will help you achieve this goal.

3.5.4 Security Testing

Security testing is a sort of software testing that identifies system vulnerabilities and ensures that the system's data and resources are safe from potential invaders. It assures that the software system and application are free of dangers or risks that could result in a data loss. The goal of security testing for any system is to uncover any possible flaws and weaknesses in the system that could lead to the loss of data or the organization's reputation.

Security Testing's Objectives

Security testing's goal is to:

- To identify the system's dangers.
- To assess the system's potential weaknesses.
- To assist in the detection of all potential security risks in the system.
- To assist developers in creating solutions to security issues.

Security Testing Concepts

The six basic principles of security testing are as follows:

- Confidentiality
- Integrity
- Authentication
- Authorization
- Availability
- Non-repudiation

Major Focus Areas in Security Testing

- Network Security
- System Software Security
- Client-side Application Security
- Server-side Application Security

Check your Understanding

1. _____ is a series of planning, execution, monitoring and control activities that help achieve project goals.

Notes

(c) Amity University Online

Notes

- A. Verification
 - B. Test Management
 - C. Validation
 - D. None of the mentioned above
2. Test Management is a process of managing the testing activities in order to ensure high quality and ____ of the software application.
- A. High-end testing
 - B. Low-end testing
 - C. Average testing
 - D. None of these
3. Test Management Process is a procedure of ____ the software testing activities from start to the end.
- A. Verification
 - B. Validation
 - C. Managing
 - D. None of these
4. How many parts of test management process have?
- A. One
 - B. Two
 - C. Three
 - D. None of these
5. ____ in Software Testing is a procedure of defining roles in the testing process.
- A. Test Organization
 - B. Test Management
 - C. Test Process
 - D. None of these
6. The structure of testing must resemble the associated system development approach.
- A. True
 - B. False
7. The test process gives the ability to plan and control testing throughout the project cycle, which is invaluable.
- A. True
 - B. False
8. An analysis of functional and non-functional requirements is carried out in,
- A. The Test Analysis Phase

Notes

- B. Testing Phase
 - C. Testing Methods
 - D. None of the mentioned above
9. Test execution executes and tests the actual system result against the expected result.
- A. True
 - B. False
10. Test closure includes creating all the test reports,
- A. True
 - B. False

Summary

- After understanding the **test strategy document**, at last, we can say that the test strategy document provides a vibrant vision of what the test team will do for the whole project.
- The test strategy document could prepare only those who have good experience in the **product domain** because the test strategy document will drive the entire team.
- And it cannot be modified or changed in the complete project life cycle as it is a static document.
- Before any testing activities begin, the Test strategy document can distribute to the entire testing team.
- If the test strategy document is written correctly, it will develop a high-quality system and expand the complete testing process.
- A set of activities intended to evaluate software and other deliverables to determine if that they meet requirements, to demonstrate that they are fit for purpose and to find defects.
- Testing performed without running the system. Document review is an example of a static test.
- Testing meant to assess how the system reacts to workloads (network, processing, data volume) that exceed the system's specified requirements. Stress testing shows which system resource (e.g. memory or bandwidth) is first to fail.
- A document describing what should be tested by whom, when, how, and why. The test plan is bounded in time, describing system testing for a particular version of a system, for example. The test plan is to the test leader what the project plan is to the project manager.
- A group of test cases e.g. all the test cases for system testing.
- The person responsible for planning the test activities at a specific test level. Usually responsible for writing the test plan and test report. Often involved in writing test cases

Notes

- A structured test script that describes how a function or feature should be tested, including test steps, expected results preconditions and post conditions.
- Information that completes the test steps in a test case with e.g. what values to input. In a test case where you add a customer to the system the test data might be customer name and address. Test data might exist in a separate test data file or in a database.

Activity

1. Briefly explain in an article the topic 'Test implementation and dimension'.

Questions & Exercises

1. What is Test Management and Test Organization?
2. Explain Test teams, tasks and Qualifications also discuss the Test Planning
3. Explain Quality Assurance Plan, Test Plan and Prioritization Plan.
4. Describe the Test Exit Criteria and Cost and economy Aspects.
5. Explain Test Strategies also describe the Preventive versus Reactive Approach.
6. What do you understand by Analytical versus heuristic Approach and Test Activity Management?
7. What is Test Progress Monitoring and Control?
8. Explain Specialized Testing with Performance, Load, stress & Security Testing.

Glossary

- **Testing:** A set of activities intended to evaluate software and other deliverables to determine if that they meet requirements, to demonstrate that they are fit for purpose and to find defects.
- **Static testing:** Testing performed without running the system. Document review is an example of a static test.
- **Stress testing:** Testing meant to assess how the system reacts to workloads (network, processing, data volume) that exceed the system's specified requirements. Stress testing shows which system resource (e.g. memory or bandwidth) is first to fail.
- **Test plan:** A document describing what should be tested by whom, when, how, and why. The test plan is bounded in time, describing system testing for a particular version of a system, for example. The test plan is to the test leader what the project plan is to the project manager.
- **Test suite:** A group of test cases e.g. all the test cases for system testing.
- **Test manager:** The person responsible for planning the test activities at a specific test level. Usually responsible for writing the test plan and test report. Often involved in writing test cases
- **Test case:** A structured test script that describes how a function or feature should be tested, including test steps, expected results preconditions and post conditions.

Notes

- **Test data:** Information that completes the test steps in a test case with e.g. what values to input. In a test case where you add a customer to the system the test data might be customer name and address. Test data might exist in a separate test data file or in a database.

Further Readings

1. Software Engineering, Sixth Edition, 2001, Ian Sommerville; Pearson Education.
2. Software Engineering – A Practitioner's Approach, Roger S. Pressman; McGraw-Hill International Edition.
3. An Integrated approach to Software Engineering, Pankaj Jalote; Narcosis Publishing House.
4. Software Quality, 1997, Mordechai Ben-Menachem and Garry S. Marliss; Thomson Learning.
5. Emden, D. and Blakeman, K. (1990). Going Automated. London: ASLIB.
6. Gates, H. (1985). Directory of Library and Information Retrieval Software for Microcomputers. Brook-field: Gower Pub. Co.
7. Levert, V.M. (1985). Library Specific Microcomputer Software. Library Software Review. 4 (May-June), 3, 5-38.
8. Nolan, J.M. (1984). Micro Software Evaluation. Torrance CA: Nolan Information Services. Poole, B.J. (1995). Education for an Information Age. Wisconsin: WCB Brown & Benchmark.
9. Rowley, Jennifer (1998). The Electronic Library, 4th Edition of Computers for Libraries. London: Library Association Publishing.
10. Tenopir, Carol (1984), Identification and Evaluation of Software for Micro-computerbased in-house Databases. Information Technology, and Libraries. March, 21-34.

Check your Understanding – Answers

- 1) b
- 2) a
- 3) c
- 4) b
- 5) a
- 6) a
- 7) a
- 8) a
- 9) a
- 10) a

Module - 4: Software Reliability, Reliability Growth Models

Structure:

- 4.1.1 Defining Software Reliability
- 4.1.2 Software Reliability Attributes and Specification
- 4.1.3 Concept of Introduction to Measurement and Inspection Process
- 4.1.4 Basics of Reliability Theory
- 4.2.1 Software Reliability Problem
- 4.2.2 Modeling Process
- 4.2.3 Software Reliability Models (SRGM)
- 4.2.4 preliminary Concepts of Reliability Engineering, Parameter Estimation
- 4.3.1 Model Validation.
- 4.3.2 Execution Time Models
- 4.3.3 Calendar Time Models
- 4.3.4 Erlang Model
- 4.4.1 Modeling Fault Dependency and Debugging Time Lag
- 4.4.2 Testing Effort Dependent Modeling
- 4.4.3 Distributed Environment, Imperfect Debugging
- 4.4.4 Testing-Domain Models
- 4.5.1 Change-Point
- 4.5.2 Unified Approach Concepts
- 4.5.3 Artificial Neural Networks based SRGM
- 4.5.4 Introduction to Discrete SRGM

Objectives:

After studying this chapter, you will be able to:

- Explain the meaning of Software Reliability
- Discuss the Software Reliability Attributes and Specification
- Explain the Concept of Introduction to Measurement and Inspection Process
- Discuss the , Documents and Metrics, Basics of Reliability
- Understand the Model Validation
- Explain the meaning of Software Reliability Growth Model
- Discuss the Execution Time Models
- Explain the Testing-Domain Models, Change-Point and Unified Approach Concepts

- Discuss the Artificial Neural Networks based SRGM
- Understand the Introduction to Discrete SRGM

Introduction

Reliability is defined as the probability of failure-free software operation for a specified period of time in a particular environment.

Reliability testing is performed to ensure that the software is reliable, it satisfies the purpose for which it is made, for a specified amount of time in a given environment and is capable of rendering a fault-free operation.

In this mechanized world, people nowadays blindly believe in any software. Whatever result the software system shows, people follow it believing that the software will always be correct. Indeed that is a common mistake that we all do.

Users think that the data shown is correct, and the software will always operate correctly. This is where the need for reliability testing comes into the picture.

According to ANSI, Software Reliability is defined as the probability of failure-free software operation for a specified period of time in a particular environment.

If a software product is operating in a failure-free manner for a particular period of time in a specified environment then it is known as reliable software.

Software reliability will reduce failures during software development. In electronic devices or mechanical instruments, the software cannot have a 'wear and tear', here 'wear and tear' only happens due to the 'defects' or 'bugs' in the software system.

Factors Influencing Software Reliability

1. The number of faults presents in the software
2. The way users operate the system
 - Reliability Testing is one of the key to better software quality. This testing helps discover many problems in the software design and functionality.
 - The main purpose of reliability testing is to check whether the software meets the requirement of customer's reliability.

Reliability testing will be performed at several levels. Complex systems will be tested at unit, assembly, subsystem and system levels.

4.1.1 Defining Software Reliability

Software Reliability means **Operational reliability**. It is described as the ability of a system or component to perform its required functions under static conditions for a specific period.

Software reliability is also defined as the probability that a software system fulfils its assigned task in a given environment for a predefined number of input cases, assuming that the hardware and the input are free of error.

Notes

Software Reliability is an essential connect of software quality, composed with functionality, usability, performance, serviceability, capability, installability, maintainability, and documentation. Software Reliability is hard to achieve because the complexity of software turn to be high. While any system with a high degree of complexity, containing software, will be hard to reach a certain level of reliability, system developers tend to push complexity into the software layer, with the speedy growth of system size and ease of doing so by upgrading the software.

For example, large next-generation aircraft will have over 1 million source lines of software on-board; next-generation air traffic control systems will contain between one and two million lines; the upcoming International Space Station will have over two million lines on-board and over 10 million lines of ground support software; several significant life-critical defence systems will have over 5 million source lines of software. While the complexity of software is inversely associated with software reliability, it is directly related to other vital factors in software quality, especially functionality, capability, etc.

In today's world, Software Applications are being used in each and every aspect of our life including healthcare, government sectors, telecommunication, etc.

Hence, we need to have accurate data in which the users can rely on. Reliability testing is concerned with the quality of the software and the standardization of products. If we are able to repeat the test cases and if we get the same output consistently, then the product is said to be 'reliable'.

Reliability testing is performed to ensure that the software is reliable, it satisfies the purpose for which it is made, for a specified amount of time in a given environment and is capable of rendering a fault-free operation.

When do we use Reliability Testing?

Given below are the scenarios where we use this testing:

- To find the faults present in the system and the reason behind it.
- To ensure the quality of the system.

Test cases should be designed in such a way in which it ensures the total coverage of the software. The test cases should be executed at regular intervals so that we can cross check the current result and the previous result and verify if there is any difference between them. If it shows the same or similar result then the software can be considered as a reliable one.

Also, we can test the Reliability by executing the test cases for a particular amount of time and check if it is showing the result correctly without any failures after that particular period of time. While doing Reliability Testing, we must check the environment constraints like memory leakage, low battery, low network, database errors, etc.

Fundamental Types to Gauge the Reliability of Software

Enlisted below are few fundamental types to gauge the Software Reliability.

Notes

1) Test-retest Reliability

Consider the following situation in which we are testing a functionality, Say at 9:30 am and testing the same functionality at 1 pm again. Later, we compare both the results. We are getting a high correlation in the results. Then we can say that the test is 'Reliable'. Usually, a Reliability of 0.8 or more means that the system can be considered as a highly reliable product.

Here, it is very important to note that the length of the test remains the same if we have 10 steps in a test case, then the number of steps will remain the same for performing the test next time.

Logic used in Test-Retest: A reliable test must show similar result over time.

Consider the particular **Example** of a person attending an 'IQ Test' and scoring 144 points. After 6 months he takes the same 'IQ test' and scores 68 points. In such a case he cannot be considered as a 'reliable' source.

2) Parallel or Alternate form of Reliability

It is called so as the testers are conducting the test in two forms at the same time.

Logic used in Parallel-Form Reliability: Different Version of same test should produce similar result for a given individual.

3) Inter-Rater Reliability

Inter-Rater Reliability is otherwise known as Inter-Observer or Inter-Coder Reliability. It is a special type of reliability which consists of multiple raters or judges. It deals with the consistency of the rating put forward by different raters/observers.

Logic used in Inter-Rater Reliability: To get a reliable result the independent rating by multiple judges should be highly similar.

For Example, consider a contestant participating in a singing competition and earning 9,8,9 (out of 10) points from multiple judges. This score can be considered as 'reliable' as they are fairly consistent. But if he had scored 9,3,7 (out of 10) then it cannot be considered as 'reliable'.

Note: These ratings will highly depend on the general agreement among the different judges/raters. Once you have a series of observation done, then you can decide that there is a kind of stability across the scores and after that period of time, we can say that they are consistent.

Thus, the scoring stability is a measurement across multiple observers. It's very important to note that the skill of the observer also plays an important role when it comes to discussing the inter-rater reliability. For improving the inter-rater reliability, the raters need training or proper guidance.

Notes

ITEMS	RATER1	RATER2	ISMATCHING
Item0	9	8	0
Item1	8	8	1
Item2	7	7	1
Item3	8	9	0
Item4	7	6	0
Item5	6	6	1
Item6	7	7	1
Item7	9	9	1
Item8	8	8	1
Item9	9	9	1
Item10	7	6	0
Item11	9	9	1
	Number of Matching item =8		

Consider the Excel sheet above and view the ratings given by two different raters Rater1 and Rater2 for 12 different items. Rater1 has independently rated on the scoring board. Here, using the scoreboard, we are now going to calculate the percentage of the agreement between the two raters. This is called inter-rater reliability or inter-rater agreement between the two raters.

In the third column, we will put '1' if the scores put by the raters are matching. We will give '0' if the scores are not matching. After that, we will find the number '1's and '0's in the column. Here it is 8.

Number of '1' = 8

Total Number of items = 12

Percentage of agreement = $(8/12) * 100 = 67\%$. 67% is not so much. Raters need to have more agreement so that they can discuss and improve the result accordingly.

Different Types of Reliability Test

The various types of reliability testing are discussed below for your reference:

1) Feature Testing:

This testing determines suitability, i.e. it tests if the application performs as expected for its intended use. Here, it will check the Interoperability of an application for testing it with the other components and the system that interacts with the application.

It ensures the accuracy of the system to check if there are no bugs found during Beta testing.

Apart from this, it tests some sort of security and compliance. Security testing is related to prevention of unauthorized access to the application either intentionally or unintentionally. In compliance, we will check if the application follows certain criteria like standard, rules, etc.

Notes**2) Load Testing**

Load testing will check how well the system performs when compared to the competition system or performance. It is also based on the number of concurrent users who are using the system and the behaviour of the system to the users.

The system must respond to the user commands with less response time (say 5 seconds) and meet the user expectations.

3) Regression Testing

In Regression testing, we will check if the system is performing well and no bugs have been introduced as a result of the addition of new functionality in the software. It is also done when a bug has been fixed and the tester needs to test it again.

Reliability Test Plan

During the different Phases of SDLC (Software Development Life Cycle) many questions about the future of the product may rise by its the users such as 'if they are reliable or not'. We need to have a clear solution for such questions. With a proper model, we can predict the product.

The two types of models include:

- Prediction Model
- Estimation Model

In Predictive testing, we predict the result with the historical data, statistics, and machine & learning. All we need is to write a report. In a predictive model, we get only some historical information. Using this info, we can construct a scatterplot and draw an extrapolate line to the existing historical data and we can predict the upcoming data.

This type of model is performed before the development or testing stage itself. In Estimation Testing, apart from using the historical data, we will use the current data. Here we can predict the reliability of a product in the present time or future time. This type of testing is performed during the last stages of the Software Development Life Cycle.

Reliability Testing Tools

Testers need to determine the Estimation of Reliability of a Software. This will lead to the use of various tools in Software Reliability.

By using a standardized tool, we can:

- Detect the failure information.
- Choose the correct model to make a prediction about the software.
- Generate Reports about the Failures.

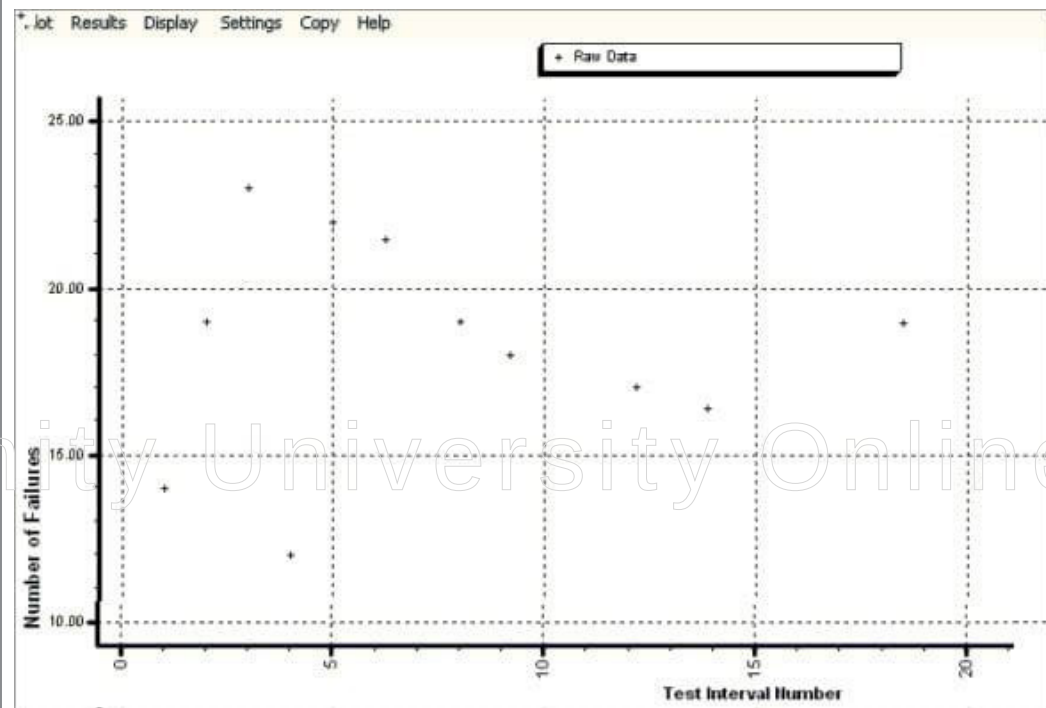
There are various tools that are available in the market for measuring software reliability, and some of them are mentioned below:

Notes

CASRE (Computer Aided Software Reliability Estimation Tool): This is not a freeware, we need to purchase it.

CASRE reliability measurement tool is built based on the existing reliability models which help in better estimations of the reliability of a software product. The GUI of the tool provides a better understanding of the software reliability and it is very easy to use as well.

During a test, it helps the users to find out if the reliability of the system is increasing or decreasing while using a set of failure data. Carse provides a 2D view by plotting the number of failure against the test Interval time and thereby a user can obtain a graph representing the system as shown in the below Figure.



Using CASRE

- The user can select failure data.
- Specifying how far in the future, we want to predict the reliability of the product.
- Select the reliability models.
- Select an appropriate model for the result.
- Print the failure result.
- Save the result to disk.

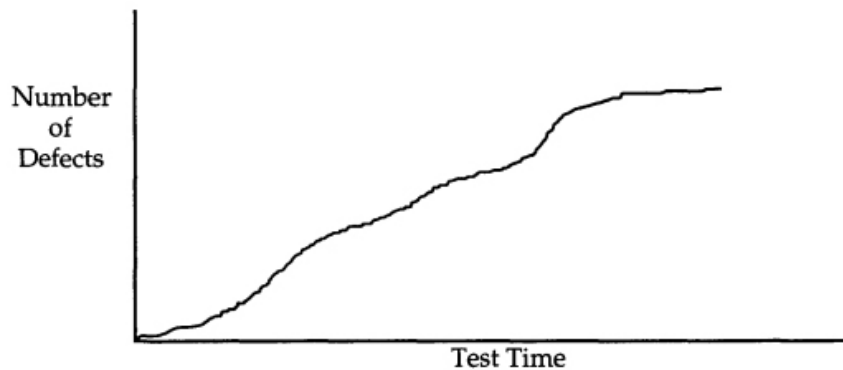
Other tools used for Testing Reliability include SOFTREL, SoRel (Software Reliability Analysis and Prediction), WEIBULL++, etc.

Software Reliability Growth Models

Software Reliability Growth Models Reliability is usually defined as the probability that a system will operate without failure for a specified time period under specified operating conditions. Reliability is concerned with the time between failures or

Notes

its reciprocal, the failure rate. In this report we are considering data from a test environment, so we report defect detection rate rather than failure rate. A defect detection is usually a failure during a test, but test software may also detect a defect even though the test continues to operate. Defects can also be detected during design reviews or code inspections, but we do not consider those sorts of activities in this report. Time in a test environment is a synonym for amount of testing, which can be measured in several ways. Defect detection data consists of a time for each defect or group of defects and can be plotted as shown in Figure. We can derive defect detection rates from this data.



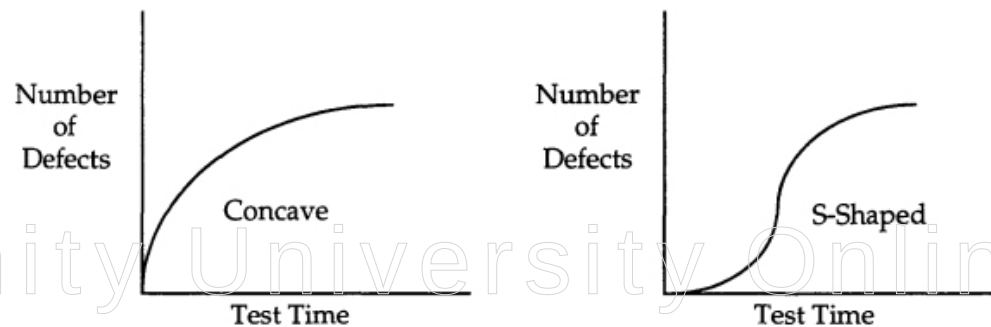
A cumulative plot of defects vs amount offsetting such as Figure should show that the defect discovery rate decreases as the amount of testing increases. The theory is that each defect is fixed as it is discovered. This decreases the number of defects in the code, so the defect discovery rate should decrease (the length of times between defect discoveries should increase). When the defect discovery rate reaches an acceptably low value, the software is deemed suitable to ship. However, it is difficult to extrapolate from defect discovery rate in a test environment to failure rate during system operation, primarily because it is hard to extrapolate from test time to system operation time. Instead, we look at the expected quantity of remaining defects in the code. These residual defects provide an upper limit on the number of unique failures our customers could encounter in field use. Software reliability growth models are a statistical interpolation of defect detection data by mathematical functions. The functions are used to predict future failure rates or the number of residual defects in the code. Current software reliability literature is inconclusive as to which data representation, software reliability growth model, and statistical correlation technique works best. The advice in the literature seems to be to try a number of the different techniques and see which works best in your environment.

Software Reliability Growth Model Types

Software reliability growth models have been grouped into two classes of models - concave and S-shaped. These two model types are shown in Figure. The most important thing about both models is that they have the same asymptotic behavior, i.e., the defect detection rate decreases as the number of defects detected (and repaired) increases, and the total number of defects detected asymptotically approaches a finite value. The theory for this asymptotic behavior is that: (1) A finite amount of code should have a finite number of defects. Repair and new functionality may introduce new defects, which increases the original finite number of defects. Some models explicitly

Notes

account for new defect introduction during test while others assume they are negligible or handled by the statistical fit of the software reliability growth model to the data. (2) It is assumed that the defect detection rate is proportional to the number of defects in the code. Each time a defect is repaired, there are fewer total defects in the code, so the defect detection rate decreases as the number of defects detected (and repaired) increases. The concave model strictly follows this pattern. In the S-shaped model, it is assumed that the early testing is not as efficient as later testing, so there is a ramp-up period during which the defect detection rate increases. This could be a good assumption if the first QA tests are simply repeating tests that developers have already run or if early QA tests uncover defects in other products that prevent QA from finding defects in the product being tested. For example, an application test may uncover as defects that need to be corrected before the application can be run. Application test hours are accumulated, but defect data is minimal because as defects don't count as part of the application test data. After the as defects are corrected, the remainder of the application test data (after the inflection point in the S-shaped curve) looks like the concave model.



There are many different representations of software reliability models. In this paper we use the model representation shown in Figure This representation shows the expected number of defects at time t and is denoted $J(t)$, where t can be calendar time, execution time, or number of tests executed

$J(t) = a(1 - e^{-bt})$, where a = expected total number of defects in the code b = shape factor = the rate at which the failure rate decreases, i.e., the rate at which we approach the total number of defects. The Goel-Okumoto model is a concave model, and the parameter “ a ” would be plotted as the total number of defects in Figure 2-2. The Goel-Okumoto model has 2 parameters; other models can have 3 or more parameters. For most models, $J(t) = aF(t)$, where a is the expected total number of defects in the code and $F(t)$ is a cumulative distribution function. Note that $F(0) = 0$, so no defects are discovered before the test starts, and $F(\infty) = 1$, so $J(\infty) = a$ and a is the total number of defects discovered after an infinite amount of testing. Table 2-1 provides a list of the models that were evaluated as part of this effort. A derivation of the properties of most of these models can be found in.

Notes

Model Name	Model Type	$\mu(t)$	Reference	Comments
Goel-Okumoto (G-O)	Concave	$a(1-e^{-bt})$ $a \geq 0, b > 0$	Goel, 79	Also called Musa model or exponential model
G-O S-Shaped	S-Shaped	$a(1-(1+bt)e^{-bt})$ $a \geq 0, b > 0$	Yamada, 83	Modification of G-O model to make it S-shaped (Gamma function instead of exponential)
Hossain-Dahiya/G-O	Concave	$a(1-e^{-bt})/(1+ce^{-bt})$ $a \geq 0, b > 0, c > 0$	Hossain, 93	Solves a technical condition with the G-O model. Becomes same as G-O as c approaches 0.
Gompertz	S-Shaped	$a(b^t)$ $a \geq 0, 0 \leq b \leq 1, 0 < c < 1$	Kececioglu, 91	Used by Fujitsu, Numazu Works
Pareto	Concave	$a(1-(1+t/\beta)^{1-\alpha})$ $a \geq 0, \beta > 0, 0 \leq \alpha \leq 1$	Littlewood, 81	Assumes failures have different failure rates and failures with highest rates removed first
Weibull	Concave	$a(1-e^{-bt^c})$ $a \geq 0, b > 0, c > 0$	Musa, 87	Same as G-O for $c=1$
Yamada Exponential	Concave	$a(1-e^{-r\alpha(1-e^{-\beta t})})$ $a \geq 0, r\alpha > 0, \beta > 0$	Yamada, 86	Attempts to account for testing effort
Yamada Raleigh	S-Shaped	$a(1-e^{-r\alpha(1-e^{-\beta t^2/2})})$ $a \geq 0, r\alpha > 0, \beta > 0$	Yamada, 86	Attempts to account for testing effort
Log Poisson	Infinite Failure	$(1/c)\ln(c\alpha t + 1)$ $c > 0, \alpha > 0$	Musa, 87	Failure rate decreases but does not approach 0

Table -1. Software Reliability Growth Model Examples

The Log Poisson model is a different type of model. This model assumes that the code has an infinite number of failures. Although this is not theoretically true, it may be essentially true in practice since all the defects are never found before the code is rewritten, and the model may provide a good fit for the useful life of the product.

The models all make assumptions about testing and defect repair. Some of these assumptions seem very reasonable, but some are questionable. Table contains a list and discussion of these assumptions.

Notes

Assumption	Reality
Defects are repaired immediately when they are discovered	Defects are not repaired immediately, but this can be partially accommodated by not counting duplicates. Test time may be artificially accumulated if a non-repaired defect prevents other defects from being found.
Defect repair is perfect	Defect repair introduces new defects. The new defects are less likely to be discovered by test since the retest for the repaired code is not usually as comprehensive as the original testing.
No new code is introduced during QA test	New code is frequently introduced throughout the entire test period, both defect repair and new features. This is accounted for in parameter estimation since actual defect discoveries are used, but may change the shape of the curve, i.e., make it less concave. The multi-stage model, discussed in Section 2.4, is an attempt to account for new code introduction.
Defects are only reported by the product testing group	Defects are reported by lots of groups because of parallel testing activity. If we add the test time for those groups, we have the problem of equivalency between an hour of QA test time and an hour of test time from a group that is testing a different product. This can be accommodated by restricting defects to those discovered by QA, but that eliminates important data. This problem means that defects do not correlate perfectly with test time.
Each unit of time (calendar, execution, number of test cases) is equivalent	This is certainly not true for calendar time or test cases as discussed earlier. For execution time, "corner" tests sometimes are more likely to find defects, so those tests create more stress on a per hour basis. When there is a section of code that has not been as thoroughly tested as other code, e.g., a product that is under schedule pressure, tests of that code will usually find more defects. Many tests are rerun to ensure defect repair has been done properly, and these reruns should be less likely to find new defects. However, as long as test sequences are reasonably consistent from release to release, this can be accounted for if necessary from lessons learned on previous releases.
Tests represent operational profile	Customers run so many different configurations and applications that it is difficult to define an appropriate operational profile. In some cases, the sheer size and transaction volume of the production system makes the operational environment impractical to replicate. The tests contained in the QA test library test basic functionality and operation, error recovery, and specific areas with which we have had problems in the past. Additional tests are continually being added, but the code also learns the old tests, i.e., the defects that the old tests would have uncovered have been repaired.

It is difficult to determine how the violation of the model assumptions will affect the models. For instance, introducing new functionality may make the curve less concave, but test reruns could make it more concave. Removing defects discovered by other groups comes closer to satisfying the model assumptions but makes the model less useful because we are not including all the data (which may also make the results less statistically valid). In general, small violations probably get lost in the noise while significant violations may force us to revise the models, e.g., see the discussion of Release 4 test hours at the end. Given the uncertainties about the effects of violating model assumption, the best strategy is to try the models to see what works best for a particular style of software development and test.

4.1.2 Software Reliability Attributes and Specification

Reliability attributes are the measure of software reliability. There may be many reliability requirements for different software but all the software have some common reliability measures. Reliability attributes for different categories of software products may be different. Therefore, it is necessary to specify the that the level of reliability required for a software product in the software requirements specification document.

Notes

There are some attributes that are required to significantly express the reliability of a software product. A good reliability measure should be dependent of observer so that different people can agree on the degree of reliability that a system has.

For example, there are various techniques for measuring performance which possibly would result in obtaining the same performance value irrespective of who is carrying out the performance measurement. But practically it is very difficult to formulate a precise reliability measurement technique. The next base case is to have measures that relate with reliability.

There are six reliability attributes which can be used to express the reliability of a software product.

Rate of occurrence of failure (ROCOF) :

ROCOF measures the frequency of occurrence of unexpected behaviour of the software. It basically measures how many times the software product fails. ROCOF measure of a software product can be obtained by having the record of the behaviour of a software product. ROCOF is basically the total number of failures occurring during the specified time interval.

Mean Time To Failure (MTTF) :

The average time between two successive failures is known as MTTF. It is obtained by observing a large number of failures in the software product. To measure MTTF, failure data for n failures is recorded. If the failures occur at the time instants $T_1, T_2, \dots, T_n$ then MTTF can be calculated as:

$$MTTF = ((T_2 - T_1) + (T_3 - T_2) + \dots + (T_{n+1} - T_n)) / (n-1)$$

It is notable that only run time is considered in the time measurements. The time for which the system is down to fix the error, the boot time and any other negligible time are neglected and are not included in the time measurements.

Mean Time To Repair (MTTR):

When the software fails, it requires sometimes to fix the error. MTTR is the measure of the average time system takes to find the errors that cause the failure and to fix them.

Mean Time Between Failure (MTBR) :

MTBR attribute can be obtained by combining the MTTF and MTTR attributes.

$$MTBF = MTTF + MTTR$$

Thus, MTBF indicates that once a failure occurs, the next failure is expected after same time. In this case, time measurements are real time and not the execution time like MTTF.

Probability of Failure on Demand (POFOD):

This attribute does not involve time measurements like the other attributes. POFOD measures the behavior of the system failing when a service request is made.

Notes

For example, if POFOD measures 0.001 then it means that out of every 1000 one time the service requests would result in a failure.

Availability:

Availability of a system is a measure of possibility of availability of the system for use over a given period of time. It considers the number of failures occurring during a time interval. It also takes into measure the down time of a system when a failure occurs. This attribute is important for systems which are supposed to be never down. This plays a vital role in telecommunication systems and operating systems. These systems need to run even during the repair time.

Reliability Specification: -The communication between suppliers or vendors and their customers is often using a mix of specifications and requirements.

Customers set requirements and suppliers offer specifications. When they match, or when supplier component specifications meet the customer's requirements, we have the potential for a transaction. Most products on the market include materials, components, or subassemblies, from other organizations. The product development team conceives of a product concept then organizes the necessary elements to create a product. Many of those elements are purchased. Elements may be standard items, like bolts or capacitors, essentially bought off the shelf. Some elements are custom. Custom elements may include drawings and extensively documented requirements. Examples of custom elements of a product include the printed circuit boards and molded enclosure parts.

4.1.3 Concept of Introduction to Measurement and Inspection Process

Measurement is an manifestation of the size, quantity, amount or dimension of a particular attributes of a product or process. Software measurement is a titrate impute of a characteristic of a software product or the software process. It is an authority within software engineering. Software measurement process is defined and governed by ISO Standard.

Need of Software Measurement:

1. Create the quality of the current product or process.
2. Anticipate future qualities of the product or process.
3. Enhance the quality of a product or process.
4. Regulate the state of the project in relation to budget and schedule.

Classification of Software Measurement:

There are 2 types of software measurement:

1. Direct Measurement:

In direct measurement the product, process or thing is measured directly using standard scale.

2. Indirect Measurement:

In indirect measurement the quantity or quality to be measured is measured using related parameter i.e. by use of reference.

Metrics:

A metrics is a measurement of the level that any impute belongs to a system product or process. There are 4 functions related to software metrics:

1. Planning
2. Organizing
3. Controlling
4. Improving

Characteristics of software Metrics:**1. Quantitative:**

Metrics must possess quantitative nature. It means metrics can be expressed in values.

2. Understandable:

Metric computation should be easily understood, the method of computing metric should be clearly defined.

3. Applicability:

Metrics should be applicable in the initial phases of development of the software.

4. Repeatable:

The metric values should be same when measured repeatedly and consistent in nature.

5. Economical:

Computation of metric should be economical.

6. Language Independent:

7. Metrics should not depend on any programming language.

Classification of Software Metrics:

There are 2 types of software metrics:

1. Product Metrics:

Product metrics are used to evaluate the state of the product, tracing risks and under covering prospective problem areas. The ability of team to control quality is evaluated.

2. Process Metrics:

Process metrics pay particular attention on enhancing the long term process of the team or organization.

Notes

3. Project Metrics:

Project matrix is describes the project characteristic and execution process.

- Number of software developer
- Staffing pattern over the life cycle of software
- Cost and schedule
- Productivity

Inspection process: Inspections are a formal type of review that involves checking the documents thoroughly before a meeting and is carried out mostly by moderators. A meeting is then held to review the code and the design.

Inspection meetings can be held both physically and virtually. The purpose of these meetings is to review the code and the design with everyone and to report any bugs found.

Benefits

- It is easier to find defects for the people who have not done the implementation themselves and are unaware of its correctness beforehand.
- Knowledge sharing about specific software artifacts and designs.
- Knowledge sharing regarding defect detection practices.
- Flaws are identified at early stages.
- It reduces the rework and testing effort.

Who is involved?

- **Moderator:** Inspector who is responsible for organizing and reporting on inspection.
- **Author:** Owner of the report.
- **Reader:** A person who guides the examination of the product.
- **Recorder:** An inspector who notes down all the defects on the defect list.
- **Inspector:** Member of the inspection team.

Steps of inspection

Planning

The planning phase starts when the entry criteria for the inspection state are met. A moderator verifies that the product entry criteria are met.

Overview

In the overview phase, a presentation is given to the inspector with some background information needed to review the software product properly.

Preparation

This is considered an individual activity. In this part of the process, the inspector

collects all the materials needed for inspection, reviews that material, and notes any defects.

Meeting

The moderator conducts the meeting. In the meeting, the defects are collected and reviewed.

Rework

The author performs this part of the process in response to defect disposition determined at the meeting.

Follow-up

In follow-up, the moderator makes the corrections and then compiles the inspection management and defects summary report.

4.1.4 Basics of Reliability Theory**Basic Concepts of Reliability**

Reliability is a broad concept. It is applied whenever we expect something to behave in a certain way. Reliability is one of the metrics that are used to measure quality. It is a user-oriented quality factor relating to system operation. Intuitively, if the users of a system rarely experience failure, the system is considered to be more reliable than one that fails more often. A system without faults is considered to be highly reliable. Constructing a correct system is a difficult task. Even an incorrect system may be considered to be reliable if the frequency of failure is “acceptable.” The key concepts in discussing reliability are:

- Fault
- Failure
- Error
- Time

Failure

A failure is said to occur if the observable outcome of a program execution is different from the expected outcome. It is something dynamic. The program has to be executing for a failure to occur. The term failure relates to the behavior of the program. It includes such things as deficiency in performance attributes and excessive response time. Example: A failure may be caused by a defective block of code. Fault: The adjudged cause of failure is called a fault. A fault is the defect in the program that, when executed under particular conditions, causes a failure. There can be different sets of conditions that cause failures, or the conditions can be repeated. Hence a fault can be source of more than one failure. A fault is a property of the program rather a property of its execution or behavior. It is what we are really referring to in general when we use the term “bug”. A fault is created when a programmer makes an error. One fault can

Notes

cause more than one failure depending upon how the system executes the faulty code. Depending whether fault will manifest itself as a failure; we have three types of faults:

- Faults that are never executed so they don't trigger failures.
- Faults that are executed but does not result in failures.
- Faults that are executed and that result in failures. Software reliability focuses solely on faults that have the potential to cause failures by detecting and removing faults that result in failures and implementing fault tolerance techniques to prevent faults from producing failures or mitigating the effects of the resulting failures.

Error

It can be defined as incorrect or missing human action that result in system/ component containing a fault (i.e. incorrect system). Examples include omission of misinterpretation of user requirements in a product specification, incorrect translation, or omission of a requirement in the design specification.

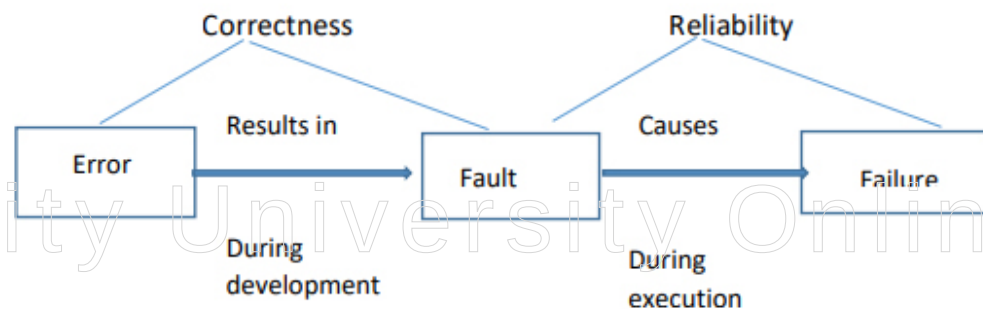


Figure: Relation of error, fault and failure

Time

Time is a key concept in the formulation of reliability. If the time gap between two successive failures is short, we say that the system is less reliable. Two forms of time are considered.

- Execution time (τ):** The execution time for a software system is the actual time spent by a processor in executing the instructions of the software system. Sometimes a processor may execute code from different programs, and therefore, their individual execution times must be considered independently.
- Calendar time (t):** The calendar time is the time people normally experience in terms of years, months, weeks, days, etc. Calendar time is useful in order to express reliability suitable with the calendar time, because it offers to the managers and to persons that develop software the chance to see the date when the system attains the objectives of reliability. In other words, if you start a stopwatch when you start a program and check it when the program is done, you are measuring calendar time. This is equivalent to the "real" time. If there are many things running on the machine at the same time, the Execution time

could obviously be much less than calendar time. Also, if the program is sitting suspended while it waits for IO, the execution time might be low while calendar time might be high

Notes

4.2.1 Software Reliability Problem

Software Reliability is the probability of failure-free software operation for a specified period of time in a specified environment. Software Reliability is also an important factor affecting system reliability. It differs from hardware reliability in that it reflects the design perfection, rather than manufacturing perfection. The high complexity of software is the major contributing factor of Software Reliability problems. Software Reliability is not a function of time - although researchers have come up with models relating the two. The modeling technique for Software Reliability is reaching its prosperity, but before using the technique, we must carefully select the appropriate model that can best suit our case. Measurement in software is still in its infancy. No good quantitative methods have been developed to represent Software Reliability without excessive limitations. Various approaches can be used to improve the reliability of software, however, it is hard to balance development time and budget with software reliability.

Software failure mechanisms

Software failures may be due to errors, ambiguities, oversights or misinterpretation of the specification that the software is supposed to satisfy, carelessness or incompetence in writing code, inadequate testing, incorrect or unexpected usage of the software or other unforeseen problems. [Keiller91] While it is tempting to draw an analogy between Software Reliability and Hardware Reliability, software and hardware have basic differences that make them different in failure mechanisms. Hardware faults are mostly physical faults, while software faults are design faults, which are harder to visualize, classify, detect, and correct. [Lyu95] Design faults are closely related to fuzzy human factors and the design process, which we don't have a solid understanding. In hardware, design faults may also exist, but physical faults usually dominate. In software, we can hardly find a strict corresponding counterpart for «manufacturing» as hardware manufacturing process, if the simple action of uploading software modules into place does not count. Therefore, the quality of software will not change once it is uploaded into the storage and start running. Trying to achieve higher reliability by simply duplicating the same software modules will not work, because design faults can not be masked off by voting.

A partial list of the distinct characteristics of software compared to hardware is listed below [Keene94]:

- **Failure cause:** Software defects are mainly design defects.
- **Wear-out:** Software does not have energy related wear-out phase. Errors can occur without warning.
- **Repairable system concept:** Periodic restarts can help fix software problems.
- **Time dependency and life cycle:** Software reliability is not a function of operational time.

Notes

- **Environmental factors:** Do not affect Software reliability, except it might affect program inputs.
- **Reliability prediction:** Software reliability can not be predicted from any physical basis, since it depends completely on human factors in design.
- **Redundancy:** Can not improve Software reliability if identical software components are used.
- **Interfaces:** Software interfaces are purely conceptual other than visual.
- **Failure rate motivators:** Usually not predictable from analyses of separate statements.
- **Built with standard components:** Well-understood and extensively-tested standard parts will help improve maintainability and reliability. But in software industry, we have not observed this trend. Code reuse has been around for some time, but to a very limited extent. Strictly speaking there are no standard parts for software, except some standardized logic structures.

4.2.2 Modeling Process

Software Processes is a coherent set of activities for specifying, designing, implementing and testing software systems. A software process model is an abstract representation of a process that presents a description of a process from some particular perspective. There are many different software processes but all involve:

- Specification – defining what the system should do;
- Design and implementation – defining the organization of the system and implementing the system;
- Validation – checking that it does what the customer wants;
- Evolution – changing the system in response to changing customer needs.

Types of Software Process Model

Software processes, methodologies and frameworks range from specific prescriptive steps that can be used directly by an organization in day-to-day work, to flexible frameworks that an organization uses to generate a custom set of steps tailored to the needs of a specific project or group. In some cases a “sponsor” or “maintenance” organization distributes an official set of documents that describe the process.

Software Process and Software Development Lifecycle Model

One of the basic notions of the software development process is SDLC models which stands for Software Development Life Cycle models. There are many development life cycle models that have been developed in order to achieve different required objectives. The models specify the various stages of the process and the order in which they are carried out. The most used, popular and important SDLC models are given below:

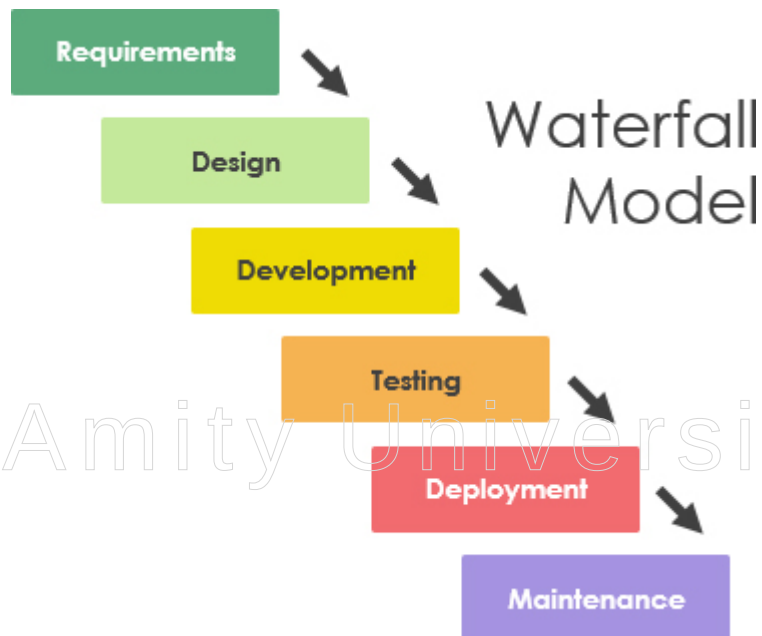
- Waterfall model
- V model
- Incremental model

- RAD model
- Agile model
- Iterative model
- Spiral model
- Prototype model

Notes

Waterfall Model

The waterfall model is a breakdown of project activities into linear sequential phases, where each phase depends on the deliverables of the previous one and corresponds to a specialisation of tasks. The approach is typical for certain areas of engineering design.

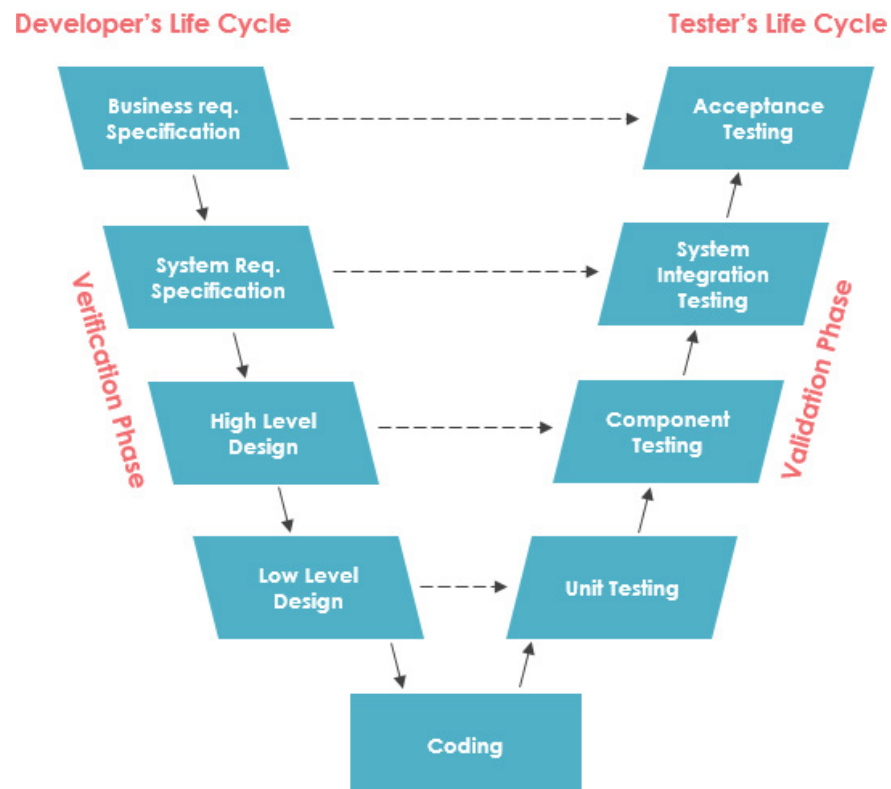


V Model

The V-model represents a development process that may be considered an extension of the waterfall model and is an example of the more general V-model. Instead of moving down in a linear way, the process steps are bent upwards after the coding phase, to form the typical V shape. The V-Model demonstrates the relationships between each phase of the development life cycle and its associated phase of testing. The horizontal and vertical axes represent time or project completeness (left-to-right) and level of abstraction (coarsest-grain abstraction uppermost), respectively.

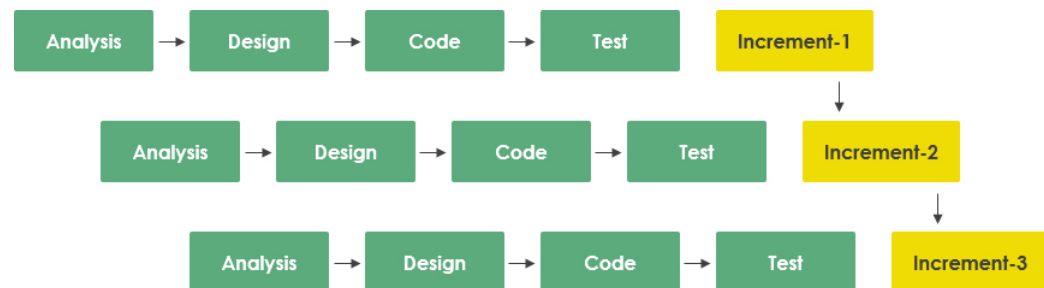
Notes

V-Model



Incremental model

The incremental build model is a method of software development where the model is designed, implemented and tested incrementally (a little more is added each time) until the product is finished. It involves both development and maintenance. The product is defined as finished when it satisfies all of its requirements. Each iteration passes through the requirements, design, coding and testing phases. And each subsequent release of the system adds function to the previous release until all designed functionality has been implemented. This model combines the elements of the waterfall model with the iterative philosophy of prototyping.



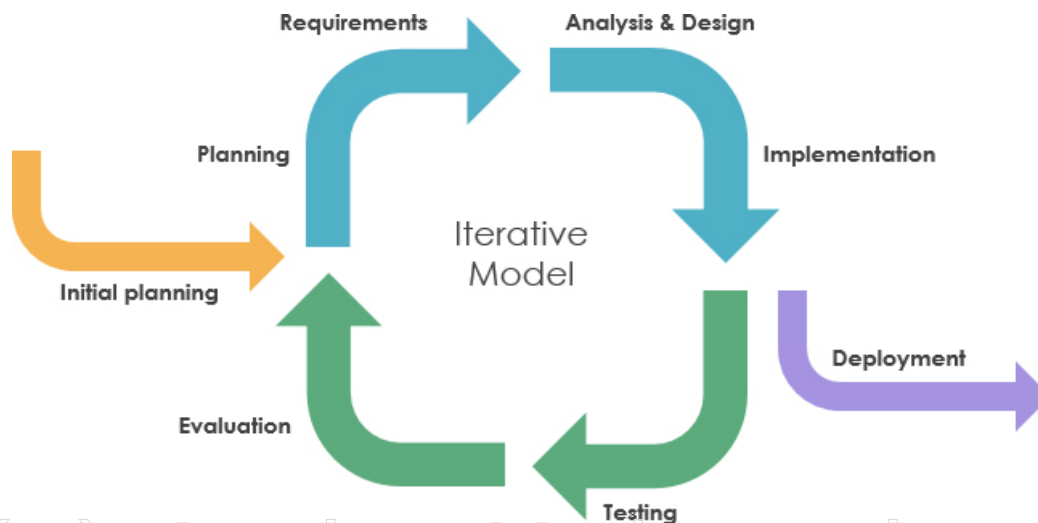
Incremental Model

Iterative Model

An iterative life cycle model does not attempt to start with a full specification of requirements by first focusing on an initial, simplified set user features, which then progressively gains more complexity and a broader set of features until the

targeted system is complete. When adopting the iterative approach, the philosophy of incremental development will also often be used liberally and interchangeably.

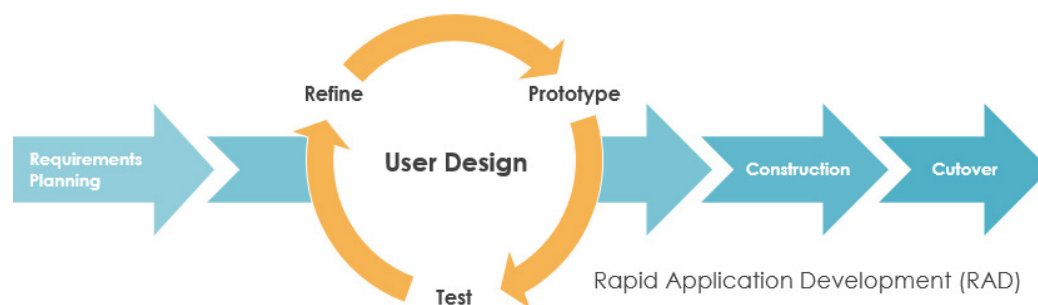
In other words, the iterative approach begins by specifying and implementing just part of the software, which can then be reviewed and prioritized in order to identify further requirements. This iterative process is then repeated by delivering a new version of the software for each iteration. In a light-weight iterative project the code may represent the major source of documentation of the system; however, in a critical iterative project a formal software specification may also be required.



RAD model

Rapid application development was a response to plan-driven waterfall processes, developed in the 1970s and 1980s, such as the Structured Systems Analysis and Design Method (SSADM). Rapid application development (RAD) is often referred as the adaptive software development. RAD is an incremental prototyping approach to software development that end users can produce better feedback when examining a live system, as opposed to working strictly with documentation. It puts less emphasis on planning and more emphasis on an adaptive process.

RAD may result in a lower level of rejection when the application is placed into production, but this success most often comes at the expense of a dramatic overrun in project costs and schedule. RAD approach is especially well suited for developing software that is driven by user interface requirements. Thus, some GUI builders are often called rapid application development tools.



Notes

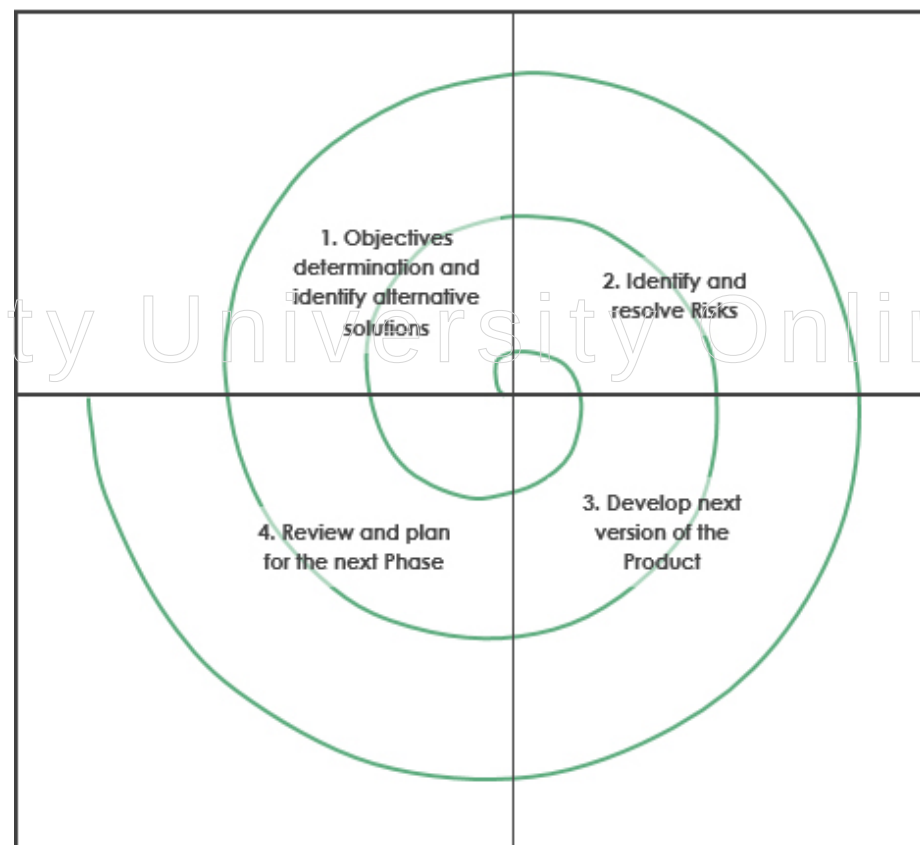
Notes

Spiral model

The spiral model, first described by Barry Boehm in 1986, is a risk-driven software development process model which was introduced for dealing with the shortcomings in the traditional waterfall model. A spiral model looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. This model supports risk handling, and the project is delivered in loops. Each loop of the spiral is called a Phase of the software development process.

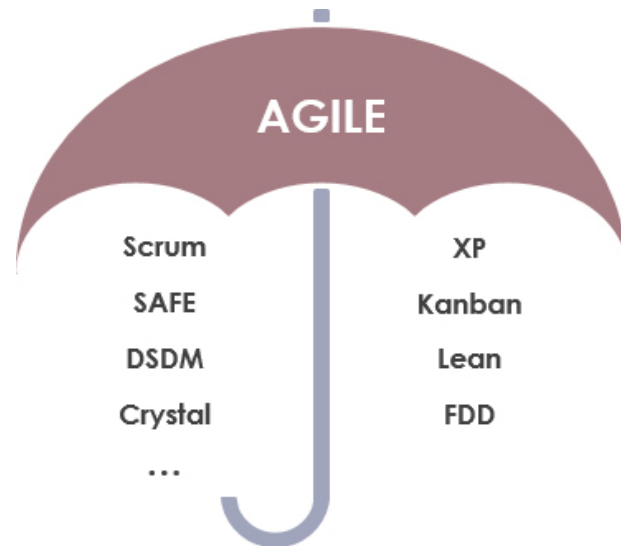
The initial phase of the spiral model in the early stages of Waterfall Life Cycle that is needed to develop a software product. The exact number of phases needed to develop the product can be varied by the project manager depending upon the project risks. As the project manager dynamically determines the number of phases, so the project manager has an important role to develop a product using a spiral model.

Spiral Model



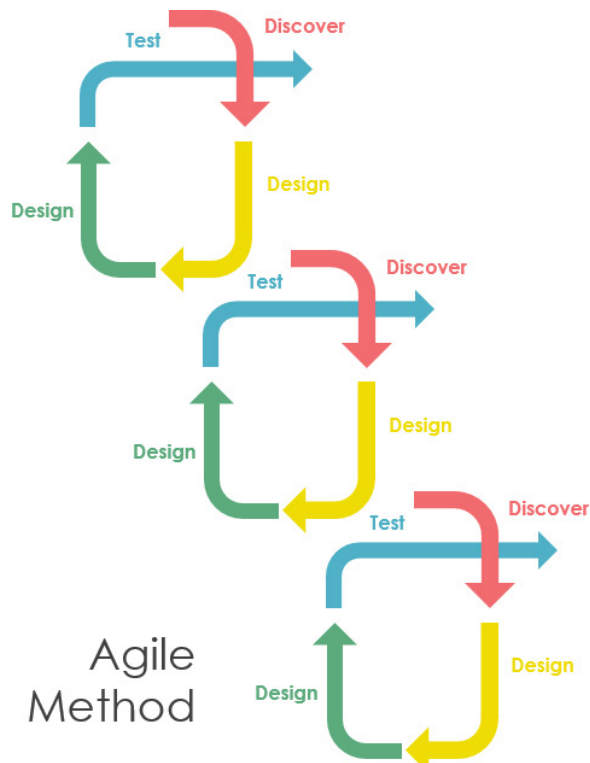
Agile model

Agile is an umbrella term for a set of methods and practices based on the values and principles expressed in the Agile Manifesto that is a way of thinking that enables teams and businesses to innovate, quickly respond to changing demand, while mitigating risk. Organizations can be agile using many of the available frameworks available such as Scrum, Kanban, Lean, Extreme Programming (XP) and etc.

Notes

The Agile movement proposes alternatives to traditional project management. Agile approaches are typically used in software development to help businesses respond to unpredictability which refer to a group of software development methodologies based on iterative development, where requirements and solutions evolve through collaboration between self-organizing cross-functional teams.

The primary goal of being Agile is empowered the development team the ability to create and respond to change in order to succeed in an uncertain and turbulent environment. Agile software development approach is typically operated in rapid and small cycles. This results in more frequent incremental releases with each release building on previous functionality. Thorough testing is done to ensure that software quality is maintained.



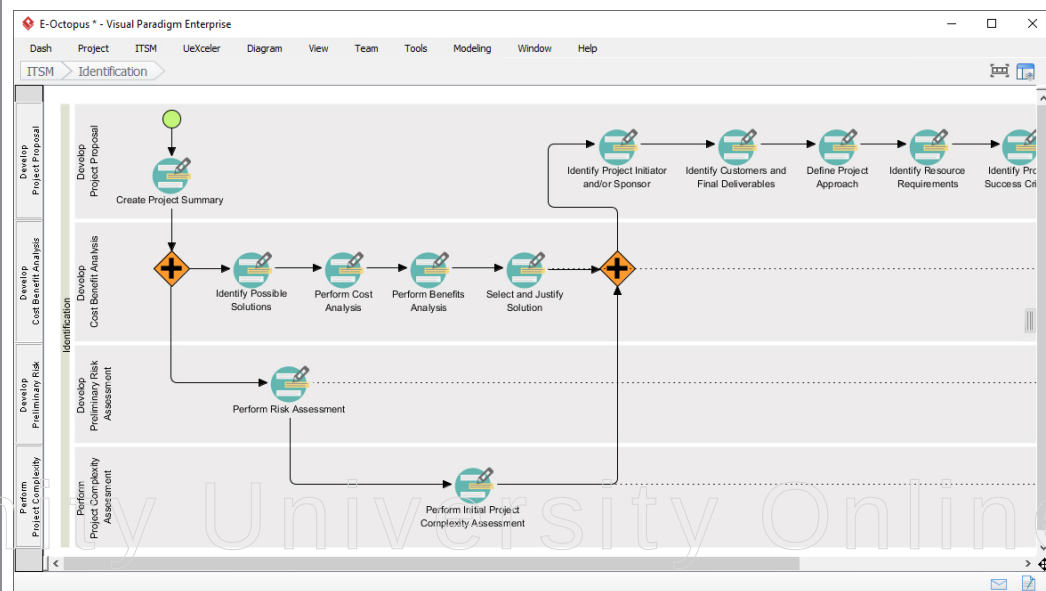
Notes

Managing Software Process with Visual Paradigm

Visual Paradigm provides a rich set of project management tools that help software team perform major development activities and manage the artifact created throughout the process.

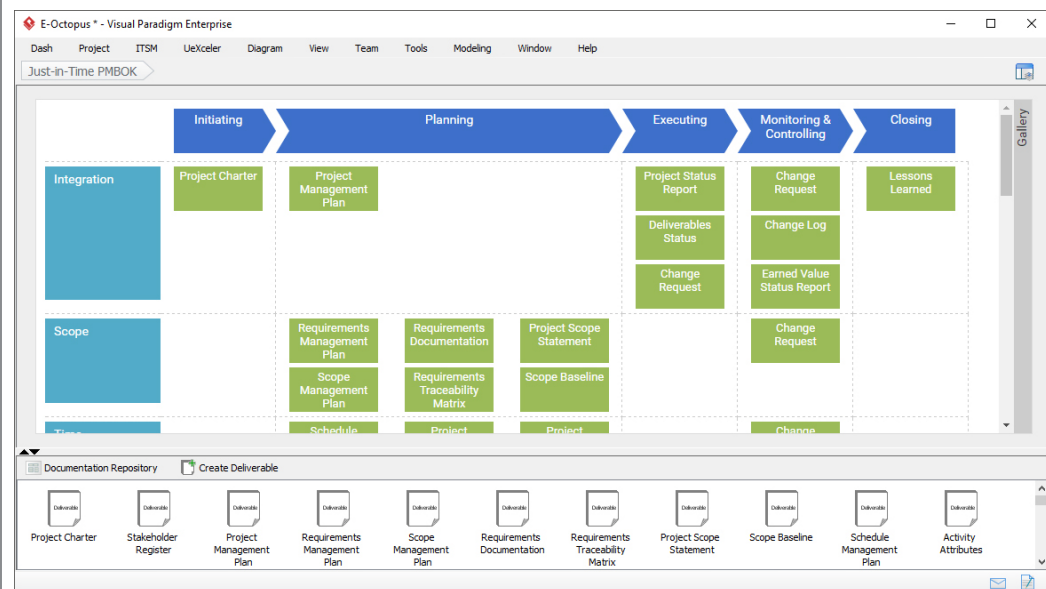
Project Manage Guide-Through

Kick-start any size of IT projects with automated guide-through process, with step-by-step instruction, input references and samples. Develop deliverables incrementally and collaboratively with your team members.



Just-in-Time PMBOK / Project Management Process Map

People are facing great deal of difficulties on Project Management such as steep learning curve and high costs hiring certified professionals. Visual Paradigm's unique automated PMBOK tool provides all the help and guidance for jumpstarting IT project management with minimal costs.



4.2.3 Software Reliability Models (SRGM)

Reliability of software is possibility of no failure during a given operating time in a specified environment. Software reliability can be defined as the probability of failure-free software operation for a specified period of time in a specified environment. Software reliability growth models are helping the software industries to develop software which is error free and reliable. They try to predict software reliability from test data. These models try to show a relationship between fault detection data (i.e. test data) and known mathematical functions such as logarithmic or exponential functions. Software reliability growth models (SRGM) captures failure behaviour of software during testing and extrapolates it to determine its behaviour during operation. Hence this category of models uses failure data information and trends observed in the failure data to derive reliability prediction. The SRGM techniques are specifically useful for developers and testers during testing and debugging phase. This study aims to apply and compare the predictive capability of SRGM.

A proliferation of software reliability models have emerged as people try to understand the characteristics of how and why software fails, and try to quantify software reliability. Over 200 models have been developed since the early 1970s, but how to quantify software reliability still remains largely unsolved. Interested readers may refer to As many models as there are and many more emerging, none of the models can capture a satisfying amount of the complexity of software; constraints and assumptions have to be made for the quantifying process. Therefore, there is no single model that can be used in all situations. No model is complete or even representative. One model may work well for a set of certain software, but may be completely off track for other kinds of problems.

Most software models contain the following parts: assumptions, factors, and a mathematical function that relates the reliability with the factors. The mathematical function is usually higher order exponential or logarithmic.

Software modeling techniques can be divided into two subcategories: prediction modeling and estimation modeling. Both kinds of modeling techniques are based on observing and accumulating failure data and analyzing with statistical inference. The major difference of the two models are shown in Table 1.

ISSUES	PREDICTION MODELS	ESTIMATION MODELS
DATA REFERENCE	Uses historical data	Uses data from the current software development effort
WHEN USED IN DEVELOPMENT CYCLE	Usually made prior to development or test phases; can be used as early as concept phase	Usually made later in life cycle(after some data have been collected); not typically used in concept or development phases
TIME FRAME	Predict reliability at some future time	Estimate reliability at either present or some future time

Table 1. Difference between software reliability prediction models and software

Notes

Notes

reliability estimation models

Representative prediction models include Musa's Execution Time Model, Putnam's Model, and Rome Laboratory models TR-92-51 and TR-92-15, etc. Using prediction models, software reliability can be predicted early in the development phase and enhancements can be initiated to improve the reliability.

Representative estimation models include exponential distribution models, Weibull distribution model, Thompson and Chelson's model, etc. Exponential models and Weibull distribution model are usually named as classical fault count/fault rate estimation models, while Thompson and Chelson's model belong to Bayesian fault rate estimation models.

The field has matured to the point that software models can be applied in practical situations and give meaningful results and, second, that there is no one model that is best in all situations. Because of the complexity of software, any model has to have extra assumptions. Only limited factors can be put into consideration. Most software reliability models ignore the software development process and focus on the results -- the observed faults and/or failures. By doing so, complexity is reduced and abstraction is achieved, however, the models tend to specialize to be applied to only a portion of the situations and a certain class of the problems. We have to carefully choose the right model that suits our specific case. Furthermore, the modelling results can not be blindly believed and applied.

Software Reliability Growth Models

The software reliability growth is one of the fundamental techniques to assess software reliability quantitatively. SRGM takes failure specification as the input and provides the reliability of the software as output. The specifications used must be the number of failures within an interval and the time between two successive failures. The factors on which the failures depend on are the failure identification, removal and operational usage. The models applicable to the assessment of software reliability are called SRGM. SRGM are useful for estimating how software reliability improves as faults are detected and repaired. Software Reliability Models can be classified in two ways, one is based on Failure History and the other one is Data Requirements as depicted in the Fig 1:

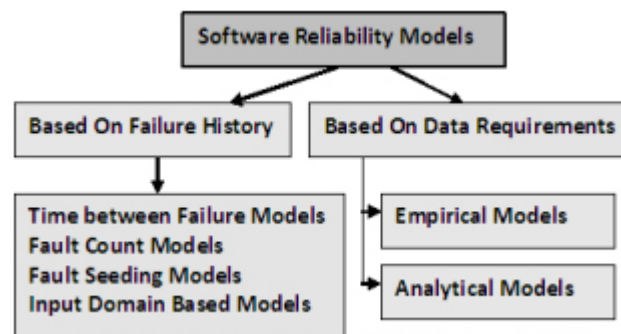


Fig. 1 Classification of Software Reliability Growth Models

The focus of this review study is on the software reliability model classification based on the failure history. The basic data requirement under each model category is summarized in Fig 2:

Model Category	Data Requirements
Time Between Failure	Time at each failure and failures are recorded
Failure Count	Operational Profile and corresponding tests
Fault Seeding	Failure count in the interval and length of interval
Category	Number of seeded faults
	Types and number of fault discovered

Fig. 2 Data requirements of the SRGM Models 1) Time between Failure Models Under these models the study is based on the time between failures. It works on the assumption that the time between (i-1) th and ith failures is a random variable, which follows a distribution whose parameters depend on the number of faults remaining in the program during this interval. Estimates of the parameters are obtained from the observed values of time between failures, mean time to next failure, etc., are then obtained from the fitted model. Jelinski Moranda Model Jelinski Moranda (JM) model is an exponential model but is differs from geometric model in that the parameter used is proportional to the remaining number of faults rather than constant . In JM model, we have N software faults at the start of testing, each is independent of others and is equally likely to cause a failure during testing. Fault removal technique is applied to remove defects and no new defects are introduced during debugging.

4.2.4 Preliminary Concepts of Reliability Engineering, Parameter Estimation

Basics of Reliability Engineering

One of best features about working in reliability engineering is everything fails, eventually. This fact provides a bit of career stability.

Another aspect I enjoy is the concepts and approaches that create the foundation for reliability engineering knowledge do not change very much over time. The basics of reliability engineering are the same as when the earliest engineers began design structures and products.

What is Reliability Engineering?

I define reliability engineering as the discipline to optimize the system or product dependability in a cost effective manner.

We use tools, techniques, and knowledge to accomplish answers to two fundamental engineering questions.

1. What will fail?
2. When will it fail?

Reliability engineering includes design, manufacture, transport, installation, operation, maintenance, and retirement of systems and products. We work with design and manufacturing teams primarily, yet also work closely with procurement, suppliers, marketing, finance, and customers.

The focus for a reliability professional is on creating a product that meets or exceeds customer expectations with respect to reliability.

Notes

Notes

Building Block for Reliability Engineering

As with any profession, reliability engineers have a body of knowledge that in part defines the profession. We have a common language with other engineering disciplines since work so closely with them. Yet, we bring unique specific concepts to the table.

One set of tools (building blocks) we rely on is in the determination of what will fail.

Risk assessment includes applying our knowledge of failure mechanisms to identify weaknesses in a particular design when exposed to the use environment. Specific tools include failure mode and effect analysis, discovery evaluations, environmental and stress testing.

Know failure

We work to understand failure mechanisms well enough to find engineering solutions (root cause analysis) plus to model and predict future failures (physics of failure modeling).

Our knowledge of failures permits us to assist the entire team to recognize likely failures and minimize their existence or mitigate their effect.

Another set of tools includes the resolution of when something will fail. Obvious this relies to a large extent on know what will fail.

The modeling tools of reliability block diagrams, physics of failure, field data analysis, and others provide means to estimate the future failure rate for a specific product.

The experimental and laboratory skills of environmental, stress, and life testing permit focus on specific failure mechanisms and evidence to support predictive modeling. I like to say “we break things” as a description of this part of our profession.

It is the mix of analytical and empirical approaches that permit us to learn and create suitably reliable products.

People skills are necessary

One more building block for successful reliability engineers is the ability to influence others.

We rarely work alone and rarely have the position to impose our recommendations. We support design and manufacturing teams and have to enable those engineers to make good decisions.

Having a solid grasp of the basic tools of reliability engineering is essential, and not sufficient. Knowing how to work with other engineers, what drives them to make decisions, what they need to make a fully informed decision and what motivates them all to play a part in gaining influence.

Summary

The reliability engineering body of knowledge has basic concepts around understanding failure mechanisms and interpersonal influence. The specific knowledge required to be successful involves many fields of science and engineering with

Notes

emphasis on those topics related to your system or product. The basics of reliability engineering involve some concepts that are easy to master. It involved some tools that allow you to determine what and when something will fail. And, a basic element of reliability engineering involves working well with others.

Parameter Estimation

Test Estimation is a management activity which approximates **how long** a Task would take to complete. Estimating effort for the test is one of the **major** and **important** tasks in Test Management.

Why Test Estimation?

Two questions you can expect from your clients when discussing potential test engagements are

Estimate?



- **Resources:** Resources are required to **carry out** any project tasks. They can be people, equipment, facilities, funding, or anything else capable of definition required for the completion of a project activity.
- **Times:** Time is the most valuable resource in a project. Every project has a deadline to delivery.
- **Human Skills:** Human skills mean the **knowledge** and the **experience** of the Team members. They affect to your estimation. For example, a team, whose members have low testing skills, will take more time to finish the project than the one which has high testing skills.
- **Cost:** Cost is the project **budget**. Generally speaking, it means **how much money** it takes to finish the project.

How to estimate?

List of Software Test Estimation Techniques

- Work Breakdown Structure
- 3-Point Software Testing Estimation Technique
- Wideband Delphi technique
- Function Point/Testing Point Analysis
- Use – Case Point Method
- Percentage distribution
- Ad-hoc method

Notes

Work Breakdown Structure (WBS)

- Breaking down the test project into small pieces

Three Point Estimation

- Estimation method is based on statistical data

Functional Point Method

- Measure the size and give weightage to each function point

Following is the 4 Step process to arrive at an estimate

Step 1) Divide the whole project into the smallest tasks

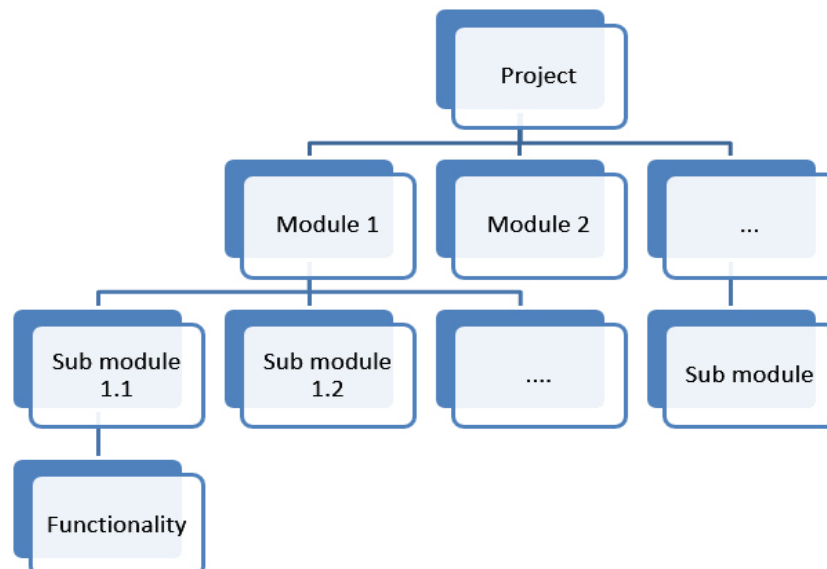
Step 2) Allocate each task to team members

Step 3) Estimate the effort required to complete each task

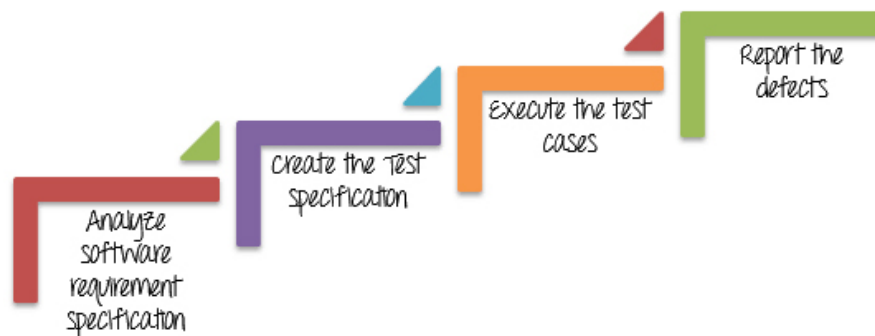
Step 4) Validate the estimation

Step 1) Divide the whole project task into subtasks

Task is a piece of work that has been given to someone. To do this, you can use the **Work Breakdown Structure** technique. In this technique, a complex project is divided into modules. The modules are divided into sub-modules. Each sub-module is further divided into functionality. It means divide the whole project task into the **smallest** tasks.



Use the Work Break Down structure to break out the Guru99 Bank project into 5 smaller tasks-



After that, you can break out each task to the **subtask**. The purpose of this activity is create task as **detailed** as **possible**.

Task	Sub task
Analyze software requirement specification	Investigate the soft requirement specs
	Interview with the developer & other stakeholders to know more about the website
Create the Test Specification	Design test scenarios
	Create test cases
	Review and revise test cases
Execute the test cases	Build up the test environment
	Execute the test cases
	Review test execution results
Report the defects	
	Create the Defect reports
	Report the defects

Step 2) Allocate each task to team member

In this step, each task is assigned to the **appropriate** member in the project team. You can assigned task as follows

Task	Members
Analyze software requirement specification	All the members
Create the test specification	Tester/Test Analyst
Build up the test environment	Test Administrator
Execute the test cases	Tester, Test Administrator
Report defects	Tester

Step 3) Effort Estimation For Tasks

There are 2 techniques which you can apply to estimate the effort for tasks

1. **Functional Point Method**
2. **Three Point Estimation**

Notes

Notes

Method 1) Function Point Method

In this method, the Test Manager estimates Size, Duration, and Cost for the tasks

Step A) Size



Step B) Duration



Step C) Cost

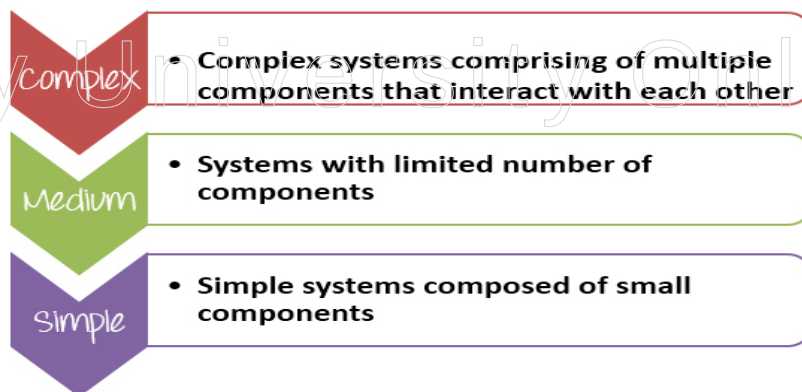


Step A) Estimate size for the task

In Step 1, you already have broken the whole project task into small task by using WBS method. Now you estimate the size of those tasks. Let's practice with a particular task "**Create the test specification**"

The size of this task depends on the functional size of the system under test. The functional size reflects the **amount** of functionality that is relevant to the user. The more **number** of functionality, the more **complex** system is.

Prior to start actual estimating tasks effort, functional points are divided into three groups like **Complex, Medium Simple** as following:



Based on the complex of software functions, the Test Manager has to give enough **weightage** to each functional point. For example

Group	Weightage
Complex	5
Medium	3
Simple	1

Let's take a simple example exercise to get clearer:

the software engineer have already described the software modules in detail, can you determine the **complexity** of website's features by giving the weightage for each modules?

More complex the function point, more is the effort to test it is. The website is divided into **12 function** points, you can determine the **complexity** of each function

points as follows-

Notes

No.	Module Name	Applicable Roles	Description	Weightage
1.	Balance Enquiry	Manager Customer	Customer: A customer can have multiple bank accounts. He can view balance of his accounts only Manager: A manager can view balance of all the customers who come under his supervision	3
2.	Fund Transfer	Manager Customer	Customer: A customer can have transfer funds from his "own" account to any destination account. Manager: A manager can transfer funds from any source bank account to destination account	5
3.	Mini Statement	Manager Customer	A Mini statement will show last 5 transactions of an account Customer: A customer can see mini-statement of only his "own" accounts Manager: A manager can see mini-statement of any account	3
4.	Customized Statement	Manager Customer	A customized statement allows you to filter and display transactions in an account based on date, transaction value Customer: A customer can see Customized- statement of only his "own" accounts Manager: A manager can see Customized -statement of any account	5
5.	Change Password	Manager Customer	Customer: A customer can change password of only his account. Manager: A manager can change password of only his account. He cannot change passwords of his customers	1
6.	New Customer	Manager	Manager: A manager can add a new customer. Manager: A manager can edit details like address, email, telephone of a customer.	3

Notes

No.	Module Name	Applicable Roles	Description	Weightage
7.	New Account	Manager	Currently system provides 2 types of accounts • Saving • Current A customer can have multiple saving accounts (one in his name, other in a joint name etc). He can have multiple current accounts for different companies he owns. Or he can have a multiple current and saving accounts. Manager: A manager can add a new account for an existing customer.	5
8.	Edit Account	Manager	Manager: A manager can add an edit account details for an existing account	1
9.	Delete Account	Manager	Manager: A manager can add a delete an account for a customer.	1
10.	Delete Customer	Manager	A customer can be deleted only if he/she has no active current or saving accounts Manager: A manager can delete a customer.	1
11.	Deposit	Manager	Manager: A manager can deposit money into any account. Usually done when cash is deposited at a bank branch.	3
12.	Withdrawal	Manager	Manager: A manager can withdraw money from any account. Usually done when cash is withdrawn at a bank branch.	3

STEP B) Estimate duration for the task

After classifying the **complexity** of the function points, you have to estimate the **duration** to test them. Duration means **how much** time needs to finish the task.

$$\boxed{\text{Total Effort}} = \boxed{\text{Total Function Points}} * \boxed{\text{Estimate defined per Function Points}}$$

Notes

- **Total Effort:** The effort to completely test all the functions of the website
- **Total Function Points:** Total modules of the website
- **Estimate defined per Function Points:** The average effort to complete one function points. This value depends on the **productivity** of the member who will take in charge this task.

	Weightage	# of Function Points	Total
Complex	5	3	15
Medium	3	5	15
Simple	1	4	4
Function Total Points			34
Estimate define per point			5
Total Estimated Effort (Person Hours)			170

So the total effort to complete the task “Create the test specification” of Guru99 Bank is around 170 man-hours

Once you understand the effort that is required, you can assign resources to determine how long the task will take (duration), and then you can estimate labor and non-labor costs. Above example also shows the importance of the member in your team. If you have **talented** and **experienced** members, you can finish the assigned task in the **small** time, and your project will finish at the deadline or sooner.

STEP C) Estimate the cost for the tasks

This step helps you to answer the last question of customer “**How much does it cost?**”

Suppose, on average your team salary is \$5 per hour. The time required for “Create Test Specs” task is 170 hours. Accordingly, the cost for the task is $5 \times 170 = \$850$. Now you can calculate budget for other activities in WBS and arrive at overall budget for the project. As a project manager, you have to decide how to get the **most return** for your company’s investment. The more **accurate** your estimate of project cost is, the **better** able you will be to manage your project’s budget.

METHOD 2) Three Point Estimation

Three-Point estimation is one of the techniques that could be used to estimate a task. The simplicity of the Three-point estimation makes it a very useful tool for a Project Manager that who wants to estimate. In three-point estimation, **three** values are produced initially for every task based on **prior experience** or **best-guesses** as follows



Notes

When estimating a task, the Test Manager needs to provide three values, as specified above. The three values identified, estimate what happens in an **optimal state**, what is the **most likely**, or what we think it would be the **worst case** scenario.

You can estimate as following

- The **best case** to complete this task is **120** man-hours (around 15 days). In this case, you have a talented team, they can finish the task in smallest time.
- The **most likely** case to complete this task is **170** man-hours (around 21 days). This is a normal case, you have enough resource and ability to complete the task
- The **worst case** to complete this task is **200** man-hours (around 25 days). You need to perform much more work because your team members are not experienced.

Now, assign the value to each parameter as below

$$a = 120 \quad m = 170 \quad b = 200$$

The effort to complete the task can be calculated using **double-triangular distribution** formula as follows-

$$E = (a + 4m + b)/6$$

$$E = (120 + 4 * 170 + 200)/6$$

$$E = 166.6 \text{ (man - hours)}$$

In the above formula, parameter E is known as **Weighted Average**. It is the **estimation of the task "Create the test specification"**.

But your boss may ask you



In the above estimation, you just determine a **possible** and not a **certain** value, we must know about the **probability** that the estimation is correct. You can use the other formula:

$$SD = (b - a)/6$$

$$SD = (200 - 120)/6$$

$$SD = 13.33 \text{ (man - hours)}$$

Notes

In above formula, the SD mean Standard Deviation, this value could give you the information about the **probability** that the estimation is correct.

Now you can conclude the estimation for the task “Create the test specification” To complete the task “Create the test specification” of Guru99 Bank website, you need **166.6 ± 13.33** Man-hour (153.33 to 179.99 man-hour)

Step 4) Validate the estimation

Once you create an aggregate estimate for all the tasks mentioned in the WBS, you need to forward it to the **management board**, who will **review** and **approve** it. The member of management board could comprise of the CEO, Project Manager & other stakeholders. The management board will review and discuss your estimation plan with you. You may explain them your estimation **logically** and **reasonably** so that they can approve your estimation plan.

Test estimation best practices

This topic introduces general tips on how to estimate Testing accuracy.

- **Add some buffer time:** Many unpredictable things may happen to your project, such as a talented team member quits his job suddenly, the testing takes more time than estimated to complete... etc. That why you need include some buffer in your estimation. Having a buffer in the estimation enables to cope for any delays that may occur.
- **Account Resource planning in estimation:** What should you do if some members in your team take long leaves? It may delay the project. Resource planning in estimation plays a key role. The availability of resources will help to make sure that the estimations are realistic. Here you have to consider the leaves for your team member, generally long leaves.
- **Use the past experience as reference:** Experiences from past projects play a vital role while preparing the time estimates. Because some project may be some similarity, you can reuse the past estimation. For example, if you use to do a project like testing a website, you can learn from that experience, try to avoid all the difficulties or issues that were faced in past projects.
- **Stick to your estimation:** Estimation is just estimate because it may go **wrong**. In early stages of the project, you should frequently **re-check the test estimations and make modification** if needed. We should not extend the estimation after we fix it, unless there are major changes in requirement, or you have to negotiate with customer about the re-estimation

Software Test Estimation Template

Other Techniques

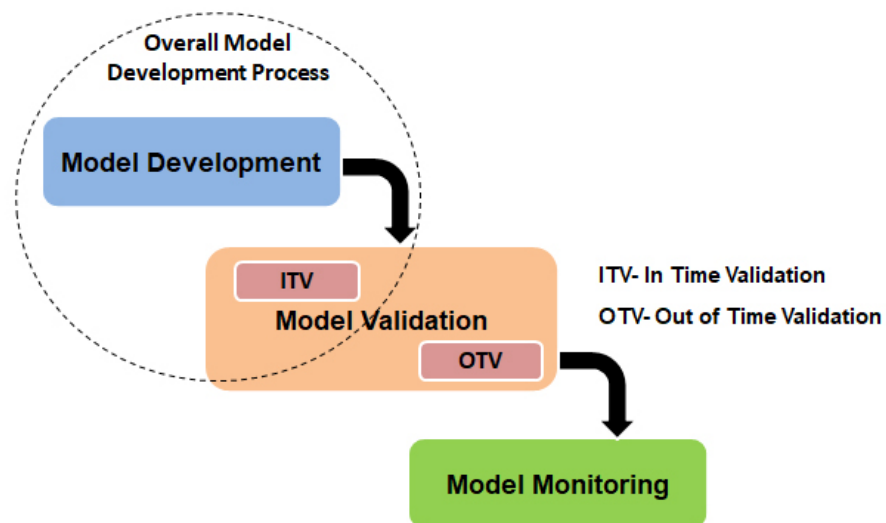
Wideband Delphi Technique, Use – Case Point Method, Percentage distribution, Ad-hoc method are other estimation techniques in Software Engineering.

- Let's do an exercise -for the Flight Reservation Application prepare a Work Breakdown Structure of the
- various testing tasks like – Check Login Functionality, Check New Order Functionality, Check Fax Functionality, and other similar functionality and Estimate the effort required to test these functionalities

Notes

- For example login functionality can be tested in 2 hours. Likewise prepare a list of all the tasks and corresponding effort. Pause the training tutorial and complete the exercise. I hope you made an educated guess of the effort required
- This is Bottom-Up Strategy for Test Estimation. The technique is called bottom-up since based on the tasks which is at the lowest level of the work breakdown hierarchy you estimate the duration , dependencies and resources.
- In bottom-Up strategy , estimates are not taken by a single person but all stakeholders, individual contributors , experts and experienced staff members collectively. The idea is to draw on the collaborative wisdom of the team members to arrive at accurate test estimates
- Now since you have considerable experience on the flight reservation system. Use this experience to estimate the effort required for full Functional Testing of the website. – <http://newtours.demoaut.com/>
- This site's functionality is identical to the Flight Reservation Application , just that it is web based. Pause the tutorial and do the exercise now
- I hope based on your experience you made a good estimate on the effort required to test the website
 - This is the Top – Down Approach to estimation which is based on experience.
 - Another technique is to classify project based on their size and complexity and then seeing how long a project of a particular size and complexity have taken in past.
 - Another approach is determining Average Effort Per Test Case in past for similar projects and then using estimated test cases of the current project and arriving at total effort
 - More sophisticated estimation models involve complex mathematical models. In practice, majority of the projects use top-down approach for estimation.
 - Test estimates can be affected by many factors like timing pressures , people factors , geographic distribution of the test team and so on

4.3.1 Model Validation



Notes

Once the model development steps are complete, model validation comes into the picture. In fact model validation is an important part of the overall model development process. If a developer is spending X amount of time in developing the model, most of the time they spend X or even more than the X amount of time in validating the model and making sure of its robustness and accuracy.

In this post, I'll emphasize the importance of the Model Validation process and how it is different from the Model monitoring processes.

Consequences of improper Model Validation

1. Poor model performance on unseen data

If the model has not been validated properly then it would not be able to perform well on the unseen data which is the ultimate purpose of a predictive model. There are various model validation techniques, the most important categories would be In time validation and Out of time validation.

A part of the development dataset is kept aside and the model is then tested on it to see how it is performing on the unseen data from the similar time segment using which it was built in the first place (*i.e. In Time validation*).

A dataset is pulled from a different time segment and the model is tested on this unseen chunk of data too to judge its response to unseen data (*i.e. Out of time validation*).

These validations ensure that the developers can be confident of the model performance.

2. Questionable robustness

As explained in the above point, if a model has been validated properly, the developers have confidence in its performance. A model which has completed the validation process is deemed fit to act robustly in the future scenarios. There are sensitivity analysis tests incorporated in the validation processes which ensure that by varying the independent model variables to a certain degree to account for economic ups and downs, the dependent variable is not affected to an extreme degree which might render the model unusable.

3. Failure to adapt to stress scenarios

In extreme scenarios like recession or this current pandemic situation, the predictive models have difficulty in adapting and predicting still-good-enough predictions despite the volatility. But if the model validation process included the stress testing measures as well then it helps in putting a version of a model into production that's already been well tested for stress scenarios and will not suddenly fail when any calamity strikes.

4. Unreliable model outputs

If the model has not gone through basic out of time validation tests, then it might be possible that the model is an overfitted model which performed well on the development sample but fails on unseen data. To avoid such a scenario, always validate!

Notes

Model Validation VS Model Monitoring

To begin understanding this distinction, we'll cover three-pointers: The purpose, the metrics, and the time period.

The Time period:

Model validation is carried out in tandem with the model development process. Many times, it's an iterative process. If a model fails to perform in the validation stage, it goes back to the development stage. This is immediately after the model development. It's a one-time process.

Model Monitoring comes into effect after a model has gone into the PROD (production) stage. It's an ongoing process. A specific monitoring frequency is decided for every model and it's evaluated then to make sure that the model is performing up to the mark and its results are reliable. Also, we check that the population distribution should not be significantly different as compared to the development time period to ensure that the model is still relevant and okay to be used.

The Purpose:

The purpose of model validation is to check the accuracy and performance of the model basis on the past data for which we already have actuals.

Once the model is deployed, the model monitoring processes ensure the relevance of the model by judging the population distribution and also recording back-dated error % comparisons between the model predictions and actuals data as soon as that starts coming in to make sure that the model performance is in the acceptable range.

The Metrics:

This is the most important distinction between the two stages. In the model validation stage, we focus majorly on the statistical metrics that can decode the model performance and response for us. Whereas in the model monitoring stage, we focus both on the statistical as well the business metrics to derive our conclusion of being confident in the relevance and the reliability of a particular model.

4.3.2 Execution Time Models

Basic Execution Time Model

This model was established by J.D. Musa in 1979, and it is based on execution time. The basic execution model is the most popular and generally used reliability growth model, mainly because:

- It is practical, simple, and easy to understand.
- Its parameters clearly relate to the physical world.
- It can be used for accurate reliability prediction.

The basic execution model determines failure behaviour initially using execution time. Execution time may later be converted in calendar time.

The failure behaviour is a **nonhomogeneous Poisson process**, which means the

Notes

associated probability distribution is a Poisson process whose characteristics vary in time.

It is equivalent to the **M-O logarithmic Poisson execution time model**, with different mean value function.

The mean value function, in this case, is based on an exponential distribution.

Variables involved in the Basic Execution Model:

- **Failure intensity (λ):** number of failures per time unit.
- **Execution time (τ):** time since the program is running.
- **Mean failures experienced (μ):** mean failures experienced in a time interval.

In the basic execution model, the mean failures experienced μ is expressed in terms of the execution time (τ) as

$$\mu(\tau) = v_0 \times \left(1 - e^{-\frac{\lambda_0}{v_0} \tau} \right)$$

Where

- λ_0 : stands for the initial failure intensity at the start of the execution.

- v_0 : stands for the total number of failures occurring over an infinite time period; it corresponds to the expected number of failures to be observed eventually.

The failure intensity expressed as a function of the execution time is given by

$$\lambda(\tau) = \lambda_0 \times e^{-\frac{\lambda_0}{v_0} \tau}$$

It is based on the above formula. The failure intensity λ is expressed in terms of μ as:

$$\lambda(\tau) = \lambda(\mu) = \lambda_0 \left(1 - \frac{\mu}{v_0} \right)$$

Where

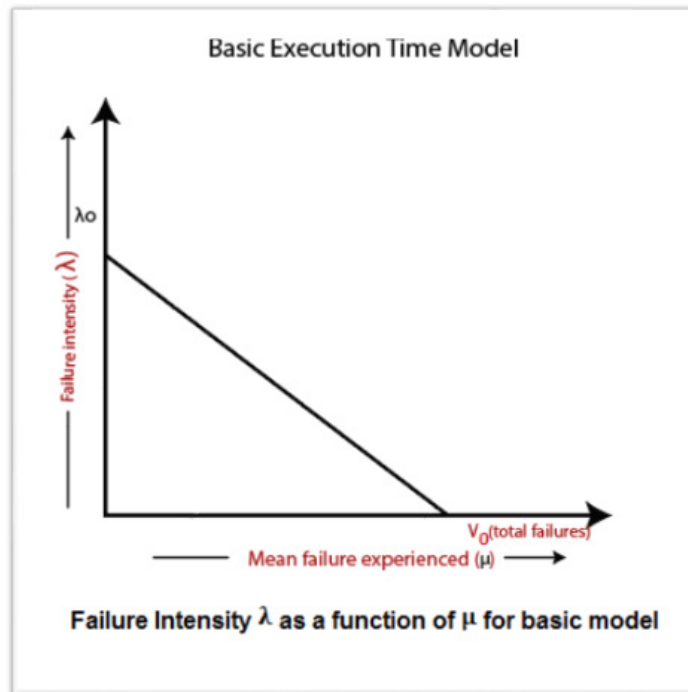
λ_0 : Initial

v_0 : Number of failures experienced, if a program is executed for an infinite time period.

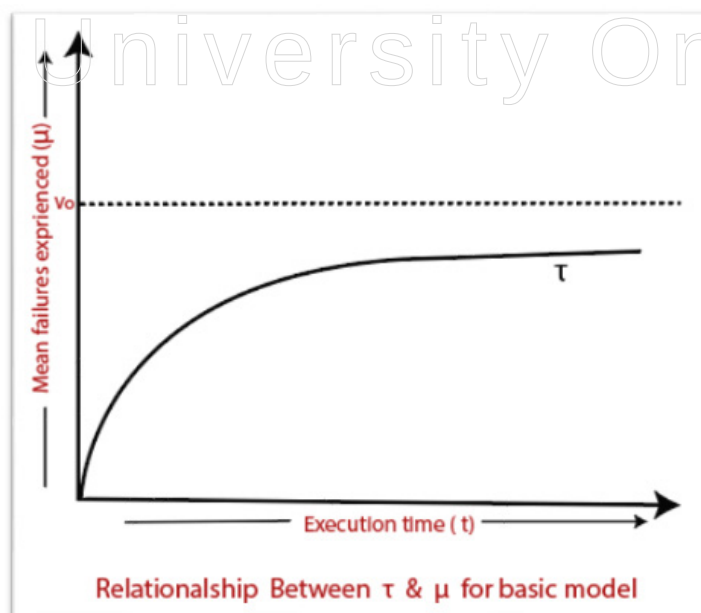
μ : Average or expected number of failures experienced at a given period of time.

τ : Execution time.

Notes



$$\frac{d\lambda}{d\mu} = \frac{-\lambda_0}{v_0} \dots \dots \text{equation 2}$$



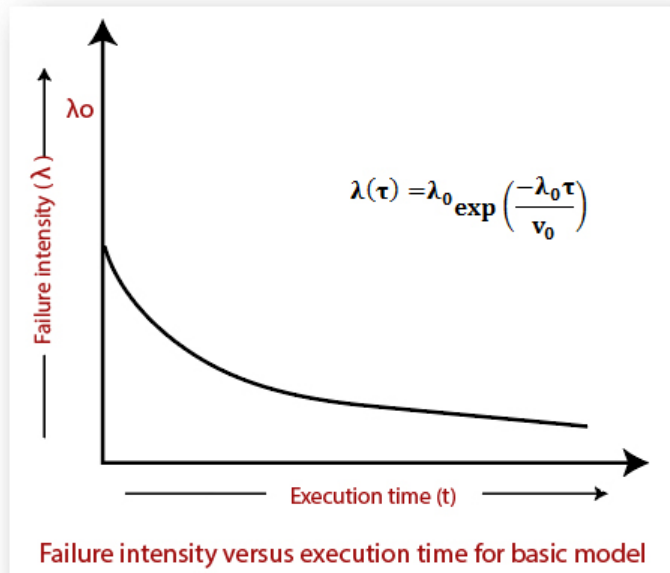
For a derivation of this relationship, equation 1 can be written as:

$$\frac{d\mu(\tau)}{d\tau} = \lambda_0 \left(1 - \frac{\mu(\tau)}{v_0} \right)$$

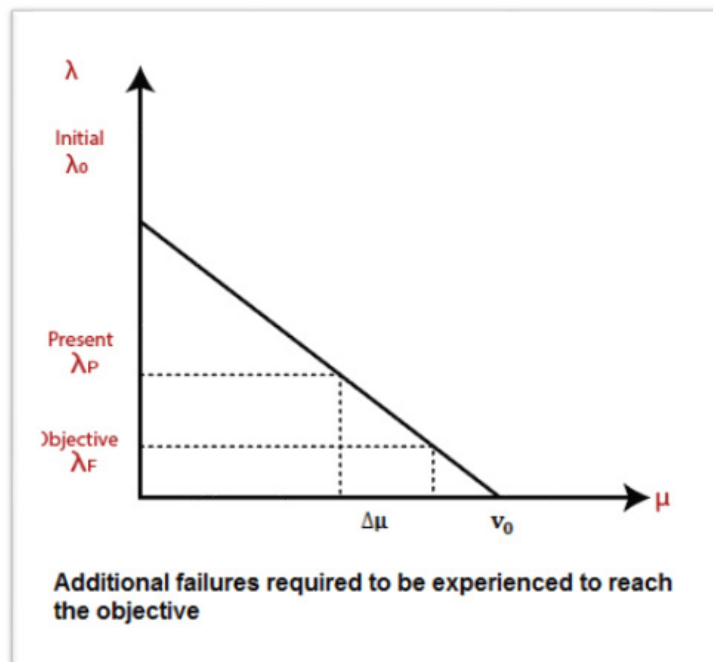
The above equation can be solved for $\lambda(\tau)$ and result in:

$$\mu(\tau) = v_0 \left(1 - \exp\left(\frac{-\lambda_0 \tau}{v_0}\right) \right)$$

The failure intensity as a function of execution time is shown in fig:



Based on the above expressions, given some failure intensity objective, one can compute the expected number of failures $\Delta\lambda$ and the additional execution time $\Delta\tau$ required to reach that objective.



Notes

Notes

$$\Delta\mu = \frac{v_0}{\lambda_0} x (\lambda_P - \lambda_F)$$

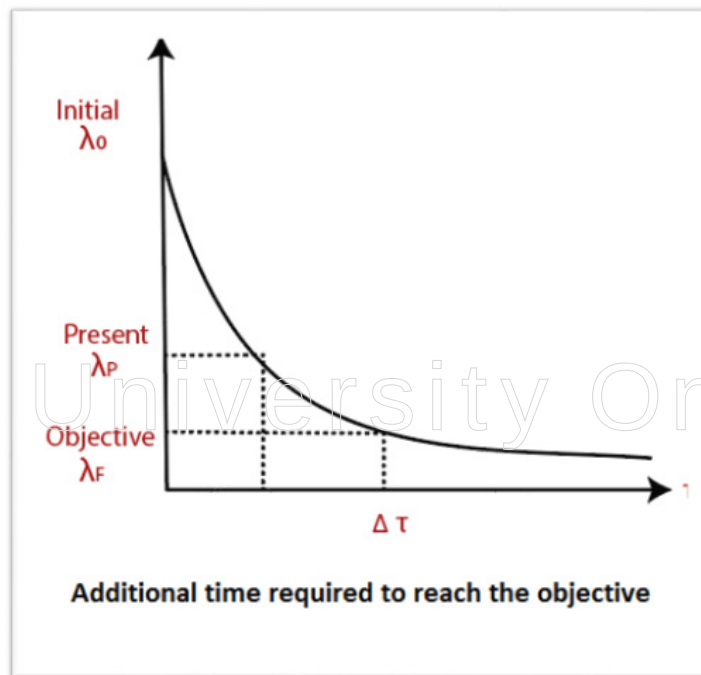
Where

λ_0 : Initial failure Intensity

λ_P : Present failure Intensity

λ_F : Failure of Intensity objective

$\Delta\mu$: Expected number of additional failures to be experienced to reach failure intensity objectives.



This can be derived in mathematical form:

$$\Delta\tau = \frac{v_0}{\lambda_0} \ln \left(\frac{\lambda_P}{\lambda_F} \right)$$

Example: Assume that a program will experience 200 failures in infinite time. It has now experienced 100. The initial failure intensity was $20 \frac{\text{Failures}}{\text{CPU hr}}$. Determine the current failure intensity.

1. Find the decrement of failure intensity per failure.
2. Calculate the failures experienced and failure intensity after 20 and 100 CPU hrs. of execution.
3. Compute addition failures and additional execution time required to reach the failure intensity objective of 5 failures/CPU hr.

Use the basic execution time model for the above-mentioned calculations.

Solution:

Here $v_0 = 200$ Failures

$\mu = 100$ Failures

$\lambda_0 = 20 \frac{\text{Failures}}{\text{CPU hr.}}$

(1) Current Failure Intensity:

$$\begin{aligned}\lambda(\mu) &= \lambda_0 \left(1 - \frac{\mu}{v_0}\right) \\ &= 20 \left(1 - \frac{100}{200}\right) \\ &= 20(1 - 0.5) \\ &= 10 \frac{\text{Failures}}{\text{CPU hr.}}\end{aligned}$$

(2) Decrement of failure Intensity per failure can be calculated as:

$$\frac{d\lambda}{d\mu} = \frac{-\lambda_0}{v_0} = \frac{-20}{200} = -\frac{0.1}{\text{CPU hr.}}$$

(3) (a) Failures experienced & Failure Intensity after 20 CPU hr.

$$\begin{aligned}\mu(\tau) &= v_0 \left(1 - \exp\left(\frac{-\lambda_0 \tau}{v_0}\right)\right) \\ &= 200 \left(1 - \exp\left(\frac{-20 \times 20}{200}\right)\right) \\ &= 200(1 - \exp(1 - 2)) \\ &= 200(1 - 0.1353) \\ &= 173 \text{ Failures.}\end{aligned}$$

$$\begin{aligned}\lambda(\tau) &= \lambda_0 \exp\left(\frac{-\lambda_0 \tau}{v_0}\right) \\ &= 20 \exp\left(\frac{-20 \times 20}{200}\right) \\ &= 20 \exp(-2) \\ &= 2.71 \frac{\text{Failures}}{\text{CPU hr.}}\end{aligned}$$

(b) Failures experienced & Failure Intensity after 100 CPU hr.

Notes

Notes

$$\begin{aligned}
 \mu(\tau) &= v_0 \left(1 - \exp\left(\frac{-\lambda_0 \tau}{v_0}\right) \right) \\
 &= 200 \left(1 - \exp\left(\frac{-20 \times 100}{200}\right) \right) \\
 &= 200 \text{ Failures (almost)} \\
 \lambda(\tau) &= \lambda_0 \exp\left(\frac{-\lambda_0 \tau}{v_0}\right) \\
 &= 20 \exp\left(\frac{-20 \times 100}{200}\right) \\
 &= 0.000908 \frac{\text{Failures}}{\text{CPU}} \text{ hr}
 \end{aligned}$$

4. Additional failures ($\Delta\mu$) required to reach the failure intensity objectives of $5 \frac{\text{Failures}}{\text{CPU}} \text{ hr.}$

$$\begin{aligned}
 \Delta\mu &= \frac{v_0}{\lambda_0} \times (\lambda_P - \lambda_F) \\
 &= \left(\frac{200}{20}\right) (10 - 5) = 50 \text{ Failures}
 \end{aligned}$$

The additional execution time required to reach the failure intensity objectives of $5 \frac{\text{Failures}}{\text{CPU}} \text{ hr.}$

$$\begin{aligned}
 \Delta\tau &= \frac{v_0}{\lambda_0} \text{Ln} \left(\frac{\lambda_P}{\lambda_F} \right) \\
 &= \frac{200}{20} \text{Ln} \left(\frac{10}{5} \right) = 6.93 \text{ CPU hr.}
 \end{aligned}$$

4.3.3 Calendar Time Models

Calendar time is derived from execution time by identifying and considering resource usage constraints and requirements in the project. For instance, during test phase the amount of testing is constrained by the rates of failure identification (I) and correction (F) by the test personnel, and the computer time (C) available. In general, these quantities are established and known at the beginning of the testing stage and remains fairly stable throughout the process. For given execution time, some of these resources become limiting, affecting as a result the testing process, and as such determining the pace of the entire process. Limiting resources determine the rate at which execution times are used across the calendar time. Specifically they allow computing the ratio between execution time and calendar time. Hence, knowledge of the ratio and the execution time can be used to compute the calendar time.

4.3.4 Erlang Model

Erlang is a programming language used to build massively scalable soft real-time systems with requirements on high availability. Some of its uses are in telecoms, banking, e-commerce, computer telephony and instant messaging. Erlang's runtime system has built-in support for concurrency, distribution and fault tolerance.

Processes

Erlang applications are built of very lightweight Erlang processes in the Erlang runtime system. Erlang processes can be seen as “living” objects (Object-oriented programming), with data encapsulation and message passing, but capable of changing behavior during runtime. The Erlang runtime system provides strict process isolation between Erlang processes (this includes data and garbage collection, separated individually by each Erlang process) and transparent communication between processes on different Erlang nodes (on different hosts).

Joe Armstrong, co-inventor of Erlang, summarized the principles of processes in his PhD thesis:

- Everything is a process.
- Processes are strongly isolated.
- Process creation and destruction is a lightweight operation.
- Message passing is the only way for processes to interact.
- Processes have unique names.
- If you know the name of a process you can send it a message.
- Processes share no resources.
- Error handling is non-local.
- Processes do what they are supposed to do or fail.

Joe Armstrong remarked in an interview with Rackspace in 2013: “If Java is ‘write once, run anywhere’, then Erlang is ‘write once, run forever’.

4.4.1 Modeling Fault Dependency and Debugging Time Lag

Software reliability growth modelling research has witnessed considerable growth since the early 1970s. Software reliability growth models (SRGMs) are generally applicable to the late stages of software development testing and can provide important information on how to improve software product reliability. Most researchers think that defects are instantly discovered and repaired in most existing SRGMs. In practise, however, this assumption may not be realistic or satisfying. We begin with an overview of fault detection and rectification mechanisms in SRGMs in this work. We demonstrate how the time-dependent delay function may be used to derive numerous existing SRGMs based on NHPP models. We also illustrate how to model software reliability growth using both failure dependency and a time-dependent delay function. Based on NHPPs, we develop stochastic reliability models for software failure phenomena. There are some numerical examples based on genuine software failure data sets. The findings suggest that the proposed methodology for including both failure dependency and a time-dependent delay function into software reliability modelling has a practical application in software testing and correction.

Notes

4.4.2 Testing Effort Dependent Modeling

As software becomes more extensively used, both its functionality and integrity are becoming increasingly important. Testing is frequently done to ensure high reliability, during which software flaws reveal themselves as failures that may be found and removed using debuggers. On the other hand, even when rigorous and disciplined development techniques are followed, making bug-free software is nearly impossible. Many software reliability growth models (SRGMs) have been presented during the last 30 years as a tool to track the software testing process's reliability growth trend. SRGMs are quite valuable in that they may assist management in making critical decisions such as allocating testing resources and determining software release dates.

Testing necessitates a significant number of resources, such as people and CPU hours, which are not always available throughout the testing process. Testing effort function (TEF) is a function that represents how testing resources are distributed and has been used into software reliability studies by some researchers. The TEF, according to Yamada et al., can be defined by a Weibull-type distribution, which contains the Exponential, Rayleigh, and Weibull curves. The Weibull-type curve fits most data well and is commonly used in software reliability modelling. Some researchers employ logistic TEF instead of Weibull-type TEF, and it appears to be fairly accurate in quantifying the consumption of testing effort.

In most cases, a discovered issue cannot be fixed instantly, and the time it takes to fix it is referred to as debugging lag/delay. The fault correction process (FCP) was first proposed by Schneidewind, who characterised it as a separate process that followed the fault detection process (FDP) with a constant time lag. Xie et al. and Wu et al. presented other paired FDP and FCP models based on this architecture, integrating other versions of debugging delay. Hwang and Pham went on to build a generalised NHPP model that took into account quasi-renewal time-delay fault elimination. Jia et al. suggested a Markovian software reliability model that took the fault rectification process into consideration. These articles, however, do not address the impact of testing effort on debugging lag. Intuitively, the time required to correct a discovered error, or the debugging lag, appears to be shorter when more testing effort is spent between the time of detection and the time of rectification. As a result, incorporating the testing effort function into the modelling framework for both FDP and FCP is more realistic.

Furthermore, the debugging process is rarely ideal, and many of the defects reported by consumers are the result of debugging. Imperfect debugging must be incorporated into FDP and FCP models.

4.4.3 Distributed Environment, Imperfect Debugging

Distributed Environment

A distributed environment is chosen for various applications where:

- The data is generated at various geographical locations and needs to be available locally most of the time.
- The data and software processing is distributed to reduce the impact of any particular site or hardware failure.

Notes

Accessing Remote Databases in a Distributed Environment

When a relational database management system (RDBMS) allows data to be distributed while providing the user with a single logical view of data, it supports “location transparency.” Location transparency eliminates the need to know the actual physical location of the data. Location transparency thus helps make the development of the application easier. Depending on the needs of the application, the database administrator (DBA) can hide the location of the relevant data.

To access a remote object, the local server must establish a connection with the remote server. Each server requires unique names for the remote objects. The methods used to establish the connection with the remote server, and the naming conventions for the remote objects, differ from database to database.

Oracle and Remote Objects

Oracle allows remote objects (such as tables, views, and procedures) throughout a distributed database to be referenced in SQL statements using global object names. In Oracle, the global name of a schema object comprises the name of the schema that contains the object, the object name, followed by an “at” sign (@), and a database name. For example, the following query selects information from the table named scott.emp in the SALES database that resides on a remote server:

```
SELECT * FROM  
scott.emp@sales.division3.acme.com
```

A distributed database system can be configured so that each database within the system has a unique database name, thereby providing “effective” global object names.

Furthermore, by defining synonyms for remote object names, you can eliminate references to the name of the remote database. The synonym is an object in the local database that refers to a remote database object. Synonyms shift the responsibility of distributing data from the application developer to the DBA. Synonyms allow the DBA to move the objects as desired without impacting the application.

The synonym can be defined as follows:

```
CREATE PUBLIC SYNONYM emp FOR  
scott.emp@sales.division3.acme.com;
```

Using this synonym, the preceding SQL statement can be changed to the following:

```
SELECT * FROM emp;
```

Microsoft SQL Server or Sybase Adaptive Server and Remote Objects

Microsoft SQL Server or Sybase Adaptive Server requires schema objects throughout a distributed database to be referenced in SQL statements by fully qualifying the object names. The complete name of a schema object has the following format:

```
server_name.database_name.object_owner_name.object_name
```

Notes

The **server_name** is the name of a remote server. The **database_name** is the name of a remote database on the remote server.

Microsoft SQL Server or Sybase Adaptive Server does not support the concept of synonyms or location transparency. In a distributed environment, objects cannot be moved around without impacting the application, as the developers must include the location of the object in the application code.

Most of the static queries tend to include the references to the remote server and remote database. Some applications maintain a user table to map the complete object names (including the remote server name and the database name) to dummy object names. The queries refer to these dummy object names. The translations are performed in real-time with the help of the map in the user table. This limitation precludes any common scheme of referring to remote objects that can work for Oracle and Microsoft SQL Server or Sybase Adaptive Server.

The Microsoft SQL Server Omni SQL Gateway server allows location transparency, but this requires that the schema definitions of all the databases participating in the distribution must be available with the Omni SQL Gateway server.

Replication

Replication functionality in Microsoft SQL Server or Sybase Adaptive Server has the following characteristics:

- Unidirectional
- Table-based, not transaction-based
- No automatic conflict resolution (must be manual)
- Heterogeneous replication through Open Database Connectivity (ODBC)

In addition to the preceding characteristics, Microsoft SQL Server 7.0 replication provides heterogeneous replication through ODBC.

Oracle replication has richer replication functionality, which includes the following:

- Bi-directional
- Any database object can be replicated
- Automatic resynchronization
- Automatic conflict resolution
- Heterogeneous replication provided through gateways

Since Oracle distributed environment and replication support is a superset of Microsoft SQL Server or Sybase Adaptive Server, conversion of distributed applications from Microsoft SQL Server or Sybase Adaptive Server to Oracle is feasible.

Application Development Tools

Several application development tools that are currently available use specific features of one of the various database servers; you may have to invest significant effort to port these products to other database servers. With critical applications, it is sometimes best to develop and maintain a different set of application development tools that work best with the underlying database, as ODBC support is not adequate in such cases.

Notes

The majority of Microsoft SQL Server or Sybase Adaptive Server applications are written using ODBC application programming interfaces (APIs) or Visual Basic. DB-Library is widely used to develop 3GL applications with Microsoft SQL Server or Sybase Adaptive Server as the back end.

Since Oracle provides ODBC connectivity, it is possible to convert ODBC-based Microsoft SQL Server or Sybase Adaptive Server applications to work with an Oracle back end. If a Visual Basic application is written with ODBC as the connection protocol to access Microsoft SQL Server or Sybase Adaptive Server, it is possible to modify and fix the Visual Basic application to work with an Oracle back end.

Many Visual Basic applications use VB-SQL which is DB-Library for Visual Basic. VB-SQL allows Visual Basic programs to access Microsoft SQL Server or Sybase Adaptive Server natively (as opposed to using ODBC). Such applications can also be converted to work with an Oracle back end, if you replace the VB-SQL database access routines with Oracle Objects for OLE. Oracle provides a call interface known as Oracle Call Interface (OCI), which is functionally equivalent to the DB-Library API. Conversion of DB-Library applications to OCI applications is feasible.

Imperfect Debugging

Due to the human aspect involved in debugging software, perfect debugging is an unreasonable expectation. Furthermore, the complexity of some softwares may allow for the introduction of new flaws as well as the chance that the bug(s) that caused the failure have not been totally eradicated. It was one of the key practical drawbacks of previous models, according to Goel, who improved the J-M model by incorporating the idea of imperfect debugging. The phenomenon of imperfect debugging occurs when new faults are introduced during the correction process, and the number of faults rectified is lower than the number of defects that cause failures. Imperfect debugging occurs when some discovered flaws are not removed with confidence or additional errors are introduced throughout the debugging process. It indicates that if a defect is found during the debugging stage but not permanently eliminated (for example, by adding new faults), then an imperfect debugging has occurred (software reliability stays the same or worsens).

4.4.4 Testing-Domain Models

Before delving into testing domain models, you first need to understand what exactly I mean by testability. After all, a group of team members manually harnessing your domain model with reams of test data is also doing testing. But that's not the definition of testing that we're using here. The main problem with manual testing is that it's not repeatable. After you make changes to your domain-model implementation, how do you ensure that it still works as per the specification? Would you employ the same set of testing team members and repeat the entire ceremony? How do you find out which parts of the model have been impacted by the changes that you made last night? Manual testing is no testing.

- You need automated testing that is
- Repeatable
- Scalable with an additional dataset

Notes

- Maintainable
- Noninvasively extensible with addition of features
- Capable of being integrated with the build system

Tests can be at various levels of granularity. You can have unit tests, module tests, and integration tests, for example, and we won't spend too much time talking about how they differ. The type of testing that we focus on in this chapter is white-box testing: You write tests at the function level and verify that the function honors all the promises that it makes. In previous chapters, we talked a lot about the algebra of a function. In this chapter, you'll see how to use that algebra to verify the properties that a function satisfies. A function embodies a set of domain behaviors that you can express as a collection of algebraic properties. As an example from our banking domain, a withdrawal (or debit) from a bank account followed by a deposit (or credit) of an equal amount restores the original balance of the account. This is an algebraic property that domain behaviors such as *debit* and *credit* have to honor. Our approach to testing focuses on verifying those algebraic properties. Remember that when we're talking about functions, we're talking about *pure* functions. So verifying algebraic properties of pure functions is repeatable, deterministic, and scalable with additional datasets.

But first you need to understand what it means to make a model *testable*.

Designing testable domain models

An abstraction is testable if it's *testable in isolation*. You should be able to test the abstraction without testing the other abstractions that it collaborates with. Let's start with an example of an initial implementation of a function that opens a customer account with a bank. The following listing starts with a simple implementation for Account and focuses on the behavioral aspects of the open function.

Listing . Toward testable domain models

```
case class Account(no: String, name: String, idNo: String,
  dateOpened: DateTime, dateClosed: Option[DateTime])

trait AccountService {
  type Error = String
  type ErrorOr[A] = Error \/ A

  def open(no: String, name: String, idNo: String,
    dateOpened: DateTime): ErrorOr[Account] = {
    val isValid: Boolean = verifyId(idNo, name)
    if (isValid) {
      //.. other validations
      Account(no, name, idNo, dateOpened, None).right
    } else s"Id ($idNo) validation failed".left
  }

  def verifyId(idNo: String, name: String): Boolean = {
    // possibly an expensive call that needs to communicate
    // with external systems, maybe over a Web service
    // stubbed here
    true
  }
}
```

The Account algebraic data type

open returns either an error or the opened account.

External service call—a side effect

How easily testable is the open function? At the beginning, it calls the function verifyId that does the actual verification of the customer's ID and name. Though you've stubbed out the implementation of verifyId, in reality it may need to

Notes

reach out to a third-party implementation, possibly through a web service, before returning the result of the verification. It's a call that induces a side effect in open and is considered impure in our vocabulary of pure functional programming. Admittedly, it's difficult to test open with such an external service call in between. But more than that, testing verifyId for correctness isn't the responsibility of the suite that tests the correctness of the open function.

4.5.1 Change-Point

We describe our process for automatic detection of performance changes for a software product in the presence of noise. A large collection of tests run periodically as changes to our software product are committed to our source repository, and we would like to identify the commits responsible for performance regressions. Previously, we relied on manual inspection of time series graphs to identify significant changes. That was later replaced with a threshold-based detection system, but neither system was sufficient for finding changes in performance in a timely manner. This work describes our recent implementation of a change point detection system built upon the E-Divisive means algorithm. The algorithm produces a list of change points representing significant changes from a given history of performance results. A human review the list of change points for actionable changes, which are then triaged for further inspection. Using change point detection has had a dramatic impact on our ability to detect performance changes. Quantitatively, it has dramatically dropped our false positive rate for performance changes, while qualitatively it has made the entire performance evaluation process easier, more productive (ex. catching smaller regressions), and more timely.

4.5.2 Unified Approach Concepts

Towards a Unified Approach to C&A

As previously mentioned, the JTFTI, led by NIST, with participating members from the Civilian, Defense, and Intelligence Communities, is a joint partnership focused on transforming the federal government's C&A processes. One of the primary goals is to establish a "unified information security framework that harmonizes security standards and guidelines for NSSs and non-NSSs". This harmonization effort not only eliminates the duplication among the various federal C&A processes, but also aims at reducing the cost associated with managing and operating multiple overlapping C&A processes.

In addition, the unified process enables the government to more effectively share information when responding to the growing number of advanced cyber threats.

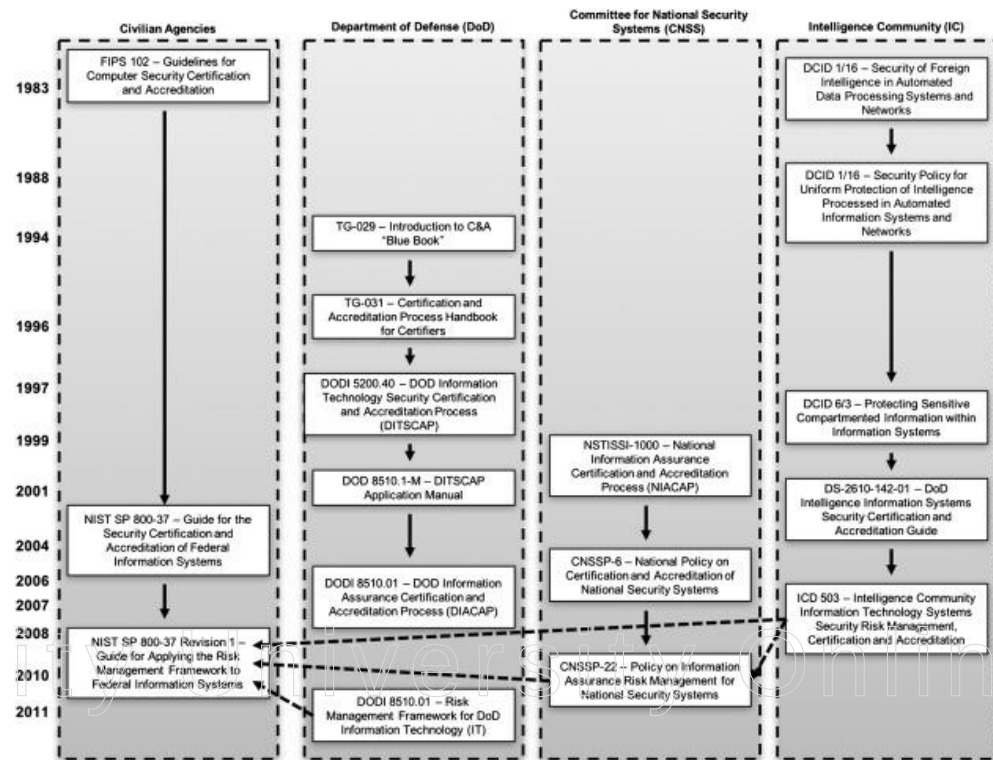
Note

The DoD Cloud Computing Strategy, published by the DoD CIO, directed leveraging efforts such as Fed RAMP, which prescribes the use of the NIST standards and guidelines as a standardized and streamlined C&A process for commercial and federal cloud providers.

These cyber threats have led to challenges within the federal government's ability to seamlessly share information and authorizations through reciprocity, effectively limiting the reuse of evidence when verifying the implementation of security controls

Notes

between interconnected systems. As depicted in Fig. 7.1, the unification of C&A processes establishes a bridge across the various federal communities by harmonizing policies and guidance. This harmonization process focuses on using the NIST standards and guidelines currently applied to non-NSSs, leaving the DoD and IC to shift their focus to addressing the unique security requirements through community-specific²⁶ policies and guidance.



4.5.3 Artificial Neural Networks based SRGM

Software reliability growth models (SRGM) are statistical interpolation of software failure data by mathematical functions. The functions are used to estimate future failure rates and reliability or the number of residual defects in the software. The SRGM facilitates reliability engineers to decide when to stop testing. Although more than 200 traditional SRGMs have been proposed to estimate failure occurrence times, the research is still continuing to develop more robust models. Inherently the SRGMs are based on assumptions. In order to increase the estimation accuracy of the models we propose the SRGM based on Feed-Forward Neural Network (FFNN) approach. It seems to have significant advantages over the traditional SRGMs. Traditional parameter estimation of SRGMs need estimation ranges of parameter beforehand. The proposed artificial neural network (ANN) model does not have this requirement and hence the parameter estimation gives consistent results without any assumptions. In this paper a new neural network combination model based on the dynamically evaluated weights is proposed in order to improve the goodness of fit of already proposed traditional SRGMs and ANN based combination models. The performance comparison from practical software failure data sets seems to confirm that, the goodness of fit of proposed model is better than that of traditional SRGMs, both independent and ANN based models.

4.5.4 Introduction to Discrete SRGM

Computer based systems typically consist of hardware and software. Quality hardware can now be produced at a reasonable cost but the same cannot be said about software. Software development consists of a sequence of activities where perfection is yet to be achieved. Hence there is every possibility that fault can be introduced and can remain in a software. A fault occurs, when a human makes a mistake, called an error, in performing some software activity. These faults can lead to failures with catastrophic results. Hence lot of emphasis is put on avoiding introduction of faults during software development and to remove dormant faults before the product is released for use. The testing phase is given a lot of importance and nearly half the developmental resources are spent during this phase. This phase aims at detecting and removing the faults, which may have been introduced during software development, thereby increasing the reliability of the software. It is the testing phase, which is amenable to mathematical modeling. Models that describe the failure phenomenon and consequent enhancement in reliability due to fault removal are termed as Software Reliability Growth Models (SRGMs). Several SRGMs have been developed in the literature to estimate the fault content and fault removal rate per fault in software. Apart from the ability to fit the failure curves, the parameters of SRGM should be interpretable in terms of software testing phenomenon. Models have been developed under various set of assumptions representing factors affecting the testing phase.

Check your Understanding

1. Which of the following is not a phase of "bathtub curve" of hardware reliability?
 - a) Useful Life
 - b) Burn-in
 - c) Wear-out
 - d) Time
2. How is reliability and failure intensity related to each other?
 - a) direct relation
 - b) inverse relation
 - c) no relation
 - d) none of the mentioned
3. How many product quality factors are proposed in McCall quality model?
 - a) 2
 - b) 3
 - c) 11
 - d) 8
4. Which one of the following is not a software quality model?
 - a) ISO 9000
 - b) McCall model

Notes

Notes

- c) Boehm model
- d) ISO 9126
- 5. What is MTTF ?
 - a) Maximum time to failure
 - b) Mean time to failure
 - c) Minimum time to failure
 - d) None of the mentioned
- 6. How is software reliability defined?
 - a) time
 - b) efficiency
 - c) quality
 - d) speed
- 7. Suitability, Accuracy, Interoperability, and security are what type quality attribute of ISO 9126 ?
 - a) Reliability
 - b) Efficiency
 - c) Functionality
 - d) Usability
- 8. Time Behavior and Resource Behavior fall under which quality attribute of ISO 9126 ?
 - a) Reliability
 - b) Efficiency
 - c) Functionality
 - d) Usability
- 9. NHPP stands for
 - a) Non Homogeneous Poisson Product
 - b) Non-Hetrogenous Poisson Product
 - c) Non-Hetrogenous Poisson Process
 - d) Non Homogeneous Poisson Process
- 10. The CMM model is a technique to
 - a) automatically maintain the software reliability
 - b) improve the software process.
 - c) test the software
 - d) all of the mentioned
- 11) What is the first step in the software development lifecycle?

Notes

- a) System Design
 - b) Coding
 - c) System Testing
 - d) Preliminary Investigation and Analysis
- 12) What does the study of an existing system refer to?
- a) Details of DFD
 - b) Feasibility Study
 - c) System Analysis
 - d) System Planning
- 13) Which of the following is involved in the system planning and designing phase of the Software Development Life Cycle (SDLC)?
- a) Sizing
 - b) Parallel run
 - c) Specification freeze
 - d) All of the above
- 14) What does RAD stand for?
- a) Rapid Application Document
 - b) Rapid Application Development
 - c) Relative Application Development
 - d) None of the above
- 15) Which of the following prototypes does not associated with Prototyping Model?
- a) Domain Prototype
 - b) Vertical Prototype
 - c) Horizontal Prototype
 - d) Diagonal Prototype
- 16) The major drawback of RAD model is _____.
- a) It requires highly skilled developers/designers.
 - b) It necessitates customer feedbacks.
 - c) It increases the component reusability.
 - d) Both (a) & (c)

Summary

- Reliability Testing is costly when compared to other forms of Testing. Hence, in order to do it cost-effectively, we need to have a proper Test Plan and Test Management.
- In SDLC, Reliability Test plays an important role. As explained above, using the reliability metrics will bring reliability to the software and predict the future of the

Notes

software. Many times software reliability is hard to obtain if the software has high complexity.

- Software reliability is a key part in software quality. The study of software reliability can be categorized into three parts: modelling, measurement and improvement.
- Software reliability modelling has matured to the point that meaningful results can be obtained by applying suitable models to the problem. There are many models exist, but no single model can capture a necessary amount of the software characteristics. Assumptions and abstractions must be made to simplify the problem. There is no single model that is universal to all the situations.
- Software reliability measurement is naive. Measurement is far from commonplace in software, as in other engineering field. “How good is the software, quantitatively?” As simple as the question is, there is still no good answer. Software reliability can not be directly measured, so other related factors are measured to estimate software reliability and compare it among products. Development process, faults and failures found are all factors related to software reliability.
- Software reliability improvement is hard. The difficulty of the problem stems from insufficient understanding of software reliability and in general, the characteristics of software. Until now there is no good way to conquer the complexity problem of software. Complete testing of a moderately complex software module is infeasible. Defect-free software product can not be assured. Realistic constraints of time and budget severely limits the effort put into software reliability improvement.
- As more and more software is creeping into embedded systems, we must make sure they don't embed disasters. If not considered carefully, software reliability can be the reliability bottleneck of the whole system. Ensuring software reliability is no easy task. As hard as the problem is, promising progresses are still being made toward more reliable software. More standard components, and better process are introduced in software engineering field.
- A slang term for fault, defect, or error. Originally used to describe actual insects causing malfunctions in mechanical devices that predate computers. The International Software Testing Qualifications Board (ISTQB) glossary explains that “a human being can make an error (mistake), which produces a defect (fault, bug) in the program code, or in a document. If a defect in code is executed, the system may fail to do what it should do (or do something it shouldn't), causing a failure. Defects in software, systems or documents may result in failures, but not all defects do so.” See also debugging.
- SQA: Software quality assurance (SQA) is **a means and practice of monitoring the software engineering processes and methods used in a project to ensure proper quality of the software**. It may include ensuring conformance to standards or models, such as ISO/IEC 9126 (now superseded by ISO 25010), SPICE or CMMI.
- Quality control plans are often viewed as a set of instructions that should be followed. They **document the planning, implementation, and assessment procedures for a project**, as well as any QA or QC activities. Some areas may be more detailed than others, based on the project, process, or organization's needs.

Notes

- **Quality planning** is the task of determining what factors are important to a project and figuring out how to meet those factors. Such factors often include the resources that will be used, the steps needed to complete the project and any other specifications. So for Molly, this means she needs to plan what resources the project will need, determine the cost of those resources, plan a timeline for completing the project, outline the steps she and her team will take, and she will need to assign the tasks and responsibilities to each person.

Activity

1. Write about Software Reliability and Reliability Growth Models in relation to software testing.

Questions & Exercises

1. Defining Software Reliability
2. Explain the Software Reliability Attributes and Specification.
3. Describe the Concept of Introduction to Measurement and Inspection Process.
4. What is Documents and Metrics also describe Basics of Reliability Theory,
5. Explanation of Software Reliability Problem?
6. What is Software Reliability Models (SRGM).
7. Describe the preliminary Concepts of Reliability Engineering.
8. What is Execution Time Model?
9. Explain Calendar Time Models and Erlang Model.
10. What is Distributed Environment, Imperfect Debugging?
11. Describe Testing-Domain Models and Change-Point.
12. Explain Unified Approach Concepts Also describe Artificial Neural Networks based SRGM.

Glossary

- **V-model:** A software development lifecycle model that describes requirements management, development, and testing on a number of different levels.
- **Validation:** Tests designed to demonstrate that the developers have built the correct system. Contrast with verification, which means testing that the system has been built correctly. A large number of validation activities take place during acceptance testing.
- **Verification:** Tests designed to demonstrate that the developers have built the system correctly. Contrast with validation, which means testing that the correct system has been built. A large number of verification activities take place during component testing.
- **Quality:** The degree to which a component, system or process meets specified requirements and/or user/customer needs and expectations.
- **Inspection:** An example of a formal review technique

Notes

- **Failure:** Deviation of the component or system under test from its expected result.
- **Entry criteria:** Criteria that must be met before you can initiate testing, such as that the test cases and test plans are complete
- **Exit criteria:** Criteria that must be fulfilled for testing to be considered complete, such as that all high-priority test cases are executed, and that no open high-priority defect remains. Also known as completion criteria
- A slang term for fault, defect, or error. Originally used to describe actual insects causing malfunctions in mechanical devices that predate computers. The International Software Testing Qualifications Board (ISTQB) glossary explains that “a human being can make an error (mistake), which produces a defect (fault, bug) in the program code, or in a document. If a defect in code is executed, the system may fail to do what it should do (or do something it shouldn’t), causing a failure. Defects in software, systems or documents may result in failures, but not all defects do so.” See also debugging.
- **SQA:** Software quality assurance (SQA) is **a means and practice of monitoring the software engineering processes and methods used in a project to ensure proper quality of the software.** It may include ensuring conformance to standards or models, such as ISO/IEC 9126 (now superseded by ISO 25010), SPICE or CMMI.
- Quality control plans are often viewed as a set of instructions that should be followed. They **document the planning, implementation, and assessment procedures for a project**, as well as any QA or QC activities. Some areas may be more detailed than others, based on the project, process, or organization’s needs.
- **Quality planning** is the task of determining what factors are important to a project and figuring out how to meet those factors. Such factors often include the resources that will be used, the steps needed to complete the project and any other specifications. So for Molly, this means she needs to plan what resources the project will need, determine the cost of those resources, plan a timeline for completing the project, outline the steps she and her team will take, and she will need to assign the tasks and responsibilities to each person.

Further Readings

1. Software Engineering, Sixth Edition, 2001, Ian Sommerville; Pearson Education.
2. Software Engineering – A Practitioner’s Approach, Roger S. Pressman; McGraw-Hill International Edition.
3. An Integrated approach to Software Engineering, Pankaj Jalote; Narcosis Publishing House.
4. Software Quality, 1997, Mordechai Ben-Menachem and Garry S.Marliss; Thomson Learning.
5. Emden, D. and Blakeman, K. (1990). Going Automated. London: ASLIB.
6. Gates, H. (1985). Directory of Library and Information Retrieval Software for

Notes

Microcomputers. Brook-field: Gower Pub. Co.

7. Levert, V.M. (1985). Library Specific Microcomputer Software. Library Software Review. 4 (May-June), 3, 5-38.
8. Nolan, J.M. (1984). Micro Software Evaluation. Torrance CA: Nolan Information Services. Poole, B.J. (1995). Education for an Information Age. Wisconsin: WCB Brown & Benchmark.
9. Rowley, Jennifer (1998). The Electronic Library, 4th Edition of Computers for Libraries. London: Library Association Publishing.
10. Tenopir, Carol (1984), Identification and Evaluation of Software for Micro-computerbased in-house Databases. Information Technology, and Libraries. March, 21-34.

Check your Understanding – Answers

- 1) d
- 2) b
- 3) b
- 4) a
- 5) b
- 6) a
- 7) c
- 8) b
- 9) d
- 10) b
- 11) d
- 12) c
- 13) d
- 14) b
- 15) d
- 16) d

(c) Amity University Online

Module - 5: Software Reliability Allocation Models, Software Verification, Validation & Testing

Structure:

- 5.1.1 Software Reliability Allocation Models
- 5.1.2 Criteria for Model Evaluation
- 5.1.3 Optimal Reliability Allocation
- 5.1.4 Quality Planning
- 5.2.1 Quality Control
- 5.2.2 Quality Improvement Process
- 5.2.3 Evolution of Software Quality Assurance
- 5.2.4 Major SQA Activities
- 5.3.1 Major SQA Issues
- 5.3.2 Zero Defect Software
- 5.3.3 Verification
- 5.3.4 Validation
- 5.4.1 Evolutionary Nature of Verification and Validation
- 5.4.2 Impracticality of Testing all Data and Paths
- 5.4.3 Proof of Correctness
- 5.4.4 Testing
- 5.5.1 Functional, Structural and Error-Oriented Analysis and Testing
- 5.5.2 Static Testing Tools
- 5.5.3 Dynamic Testing Tools
- 5.5.4 Characteristics of Modern Testing Tools

Objectives:

After studying this chapter, you will be able to:

- Understand software reliability allocation models
- Evaluate the criteria for model evaluation
- Learn about optimal reliability allocation
- Discuss quality planning, control and quality improvement process
- Analyse evolution of software quality assurance
- Understand major SQA activities and issues
- Learn about zero defect software
- Explain the meaning of Cloud Computing

Notes

- Discuss the Characteristics of Cloud computing as per NIST
- Explain the Private, public and hybrid cloud
- Discuss the Cloud types; IaaS, SaaS, PaaS
- Understand the Benefits and challenges of cloud computing

Introduction

A software reliability allocation model is developed. This model determines how reliable software modules and programs must be in order to maximize the user's utility, while taking into account the financial and technical constraints of the system. The model is shown to provide a unified approach in which the user's requirements and preferences are formally integrated with the technical structure of the software and its module and program reliabilities. The model determines reliability goals at the planning and design stages of the software project, making reliability a singular measure for performance evaluation and project control. An example for the application of the model is provided.

The software reliability model indicates the form of a random process that defines the behavior of software failures to time.

Software reliability models have appeared as people try to understand the features of how and why software fails, and attempt to quantify software reliability.

Over 200 models have been established since the early 1970s, but how to quantify software reliability remains mostly unsolved.

There is no individual model that can be used in all situations. No model is complete or even representative.

Most software models contain the following parts:

- Assumptions
- Factors

A mathematical function that includes the reliability with the elements. The mathematical function is generally higher-order exponential or logarithmic.

Software Reliability Modelling Techniques

Both kinds of modelling methods are based on observing and accumulating failure data and analyzing with statistical inference.

Differentiate between software reliability prediction models and software reliability estimation models

Basics	Prediction Models	Estimation Models
Data Reference	Uses historical information	Uses data from the current software development effort.

Notes

When used in development cycle	Usually made before development or test phases; can be used as early as concept phase.	Usually made later in the life cycle (after some data have been collected); not typically used in concept or development phases.
Time Frame	Predict reliability at some future time.	Estimate reliability at either present or some next time.

Reliability Models

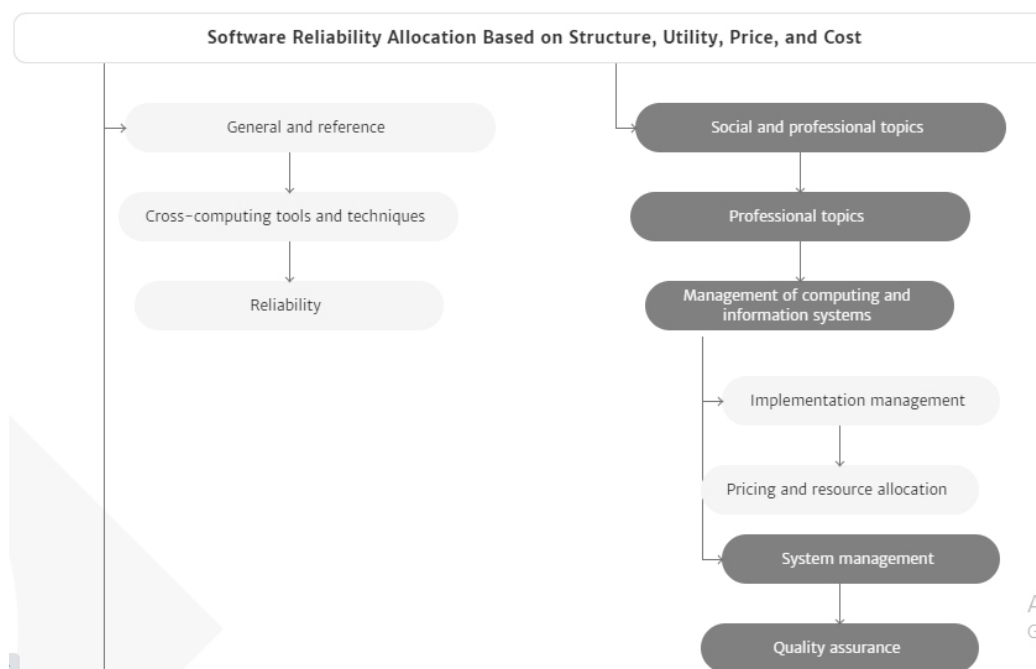
A reliability growth model is a numerical model of software reliability, which predicts how software reliability should improve over time as errors are discovered and repaired. These models help the manager in deciding how much efforts should be devoted to testing. The objective of the project manager is to test and debug the system until the required level of reliability is reached.

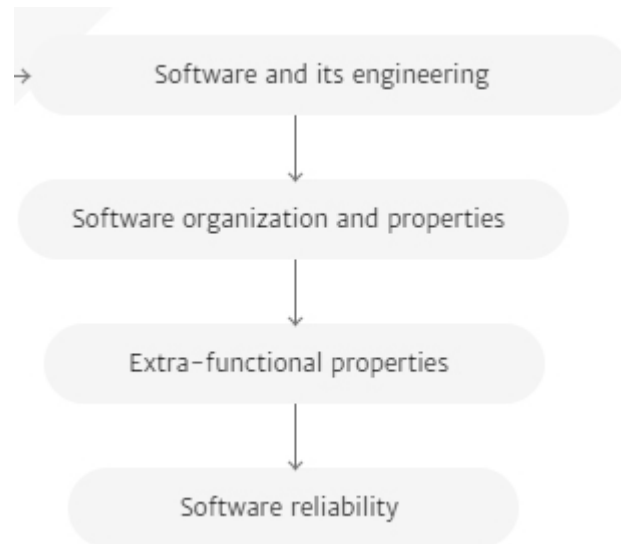
Following are some of the Software Reliability Models.

Software verification and validation is an important model that allows the team of software developers and testers to ensure the accurate development of the product throughout the development life cycle. Both **software verification and validation** help create a software product that efficiently meets the specified requirements of the customer and offers them optimum services. Therefore, let us discuss software verification in detail to better understand its relevance in the **software development life cycle (SDLC)**.

5.1.1 Software Reliability Allocation Models

1. Software Reliability Allocation Based on Structure, Utility, Price, and Cost



Notes**5.1.2 Criteria for Model Evaluation**

For reliability models, in 1984 a group of experts devised a set of criteria for model assessment and comparison. The criteria are listed as follows, by order of importance as determined by the group :

- Predictive validity: The capability of the model to predict failure behaviour or the number of defects for a specified time period based on the current data in the model.
- Capability: The ability of the model to estimate with satisfactory accuracy quantities needed by software managers, engineers, and users in planning and managing software development projects or controlling change in operational software systems.
- Quality of assumptions: The likelihood that the model assumptions can be met, and the assumptions' plausibility from the viewpoint of logical consistency and software engineering experience.
- Applicability: The model's degree of applicability across different software products (size, structure, functions, etc.).
- Simplicity: A model should be simple in three aspects: (1) simple and inexpensive to collect data, (2) simple in concept and does not require extensive mathematical background for software development practitioners to comprehend, and (3) readily implemented by computer programs.

From the practitioner's point of view and with recent observations of software reliability models, we contend that the most important criteria are predictive validity, simplicity, and quality of assumptions, in that order of importance. Capability and applicability are less significant. As the state of the art is still maturing and striving to improve its most important objective (predictive accuracy), the extra criteria of demanding more functions (capability) for multiple environments (applicability) seems burdensome. Perhaps the accuracy of software reliability models can best be summarized as follows: Some models sometimes give good results, some are almost

Notes

universally awful, and none can be trusted to be accurate at all times. A model with good predictive validity but poor capability and narrow applicability is certainly superior to one with good capability and wide applicability but with very poor ability to predict.

In contrast to the order of importance determined by the 1984 group, we think that simplicity is much more important, second only to predictive validity. Experts in software reliability models are usually academicians who are well versed in mathematics and statistics. Many modelling concepts and terminologies are outside the discipline of computer science, let alone easy to comprehend and implement by software developers in the industry. As mentioned earlier, some reliability models have not been tested and used in real-life development projects simply because they are not understandable. Simplicity, therefore, is a key element in bridging the gap between the state of the art and the state of practice in software reliability modelling.

The quality of the assumptions is also very important. Early models tend to have restrictive and unrealistic assumptions. More recent models tend to have more realistic assumptions. Better assumptions make the model more convincing and more acceptable by software practitioners; they also lead to better predictive validity.

Given a fixed set of smoothing parameters λ in which each λ controls the smoothness of each spline term, you can fit a generalized additive model by the penalized likelihood estimation. There are infinitely many sets of smoothing parameters. In order to search optimum models, some model evaluation criteria need to be defined to quantify the model goodness-of-fit. The GAMPL procedure uses the following model evaluation criteria:

- generalized cross validation (GCV), V_g
- unbiased risk estimator (UBRE), V_u
- generalized approximate cross validation (GACV), V_a

Consider the optimization problem

$$\min(\mathbf{y} - \mathbf{X}\beta)'(\mathbf{y} - \mathbf{X}\beta) + \beta' \mathbf{S}_\lambda \beta \quad \text{with respect to } \beta$$

The parameter estimate for β can be represented as

$$\hat{\beta} = (\mathbf{X}'\mathbf{X} + \mathbf{S}_\lambda)^{-1} \mathbf{X}'\mathbf{y}$$

And the smoothing matrix (also called the influence matrix or hat matrix) is thus represented as

$$\mathbf{H}_\lambda = \mathbf{X}(\mathbf{X}'\mathbf{X} + \mathbf{S}_\lambda)^{-1} \mathbf{X}'$$

With the defined smoothing matrix, you can form the model evaluation criteria as follows:

$$\begin{aligned} V_g(\lambda) &= \frac{n \|\mathbf{y} - \mathbf{H}_\lambda \mathbf{y}\|^2}{(\text{tr}(\mathbf{I} - \gamma \mathbf{H}_\lambda))^2} \\ V_u(\lambda) &= \frac{1}{n} \|\mathbf{y} - \mathbf{H}_\lambda \mathbf{y}\|^2 - \frac{2}{n} \sigma^2 \text{tr}(\mathbf{I} - \gamma \mathbf{H}_\lambda) + \sigma^2 \\ V_a(\lambda) &= \frac{1}{n} \|\mathbf{y} - \mathbf{H}_\lambda \mathbf{y}\|^2 \left(1 + 2\gamma \frac{\text{tr}(\mathbf{H}_\lambda)}{\text{tr}(\mathbf{I} - \mathbf{H}_\lambda)} \right) \end{aligned}$$

In the equations, $\gamma \geq 1$ (which corresponds to the GAMMA= suboption of the CRITERION= option) is the tuning parameter that is sometimes used to enforce smoother models.

The GAMPL procedure uses fitting algorithms that involve minimizing the model evaluation criterion with respect to unknown smoothing parameters λ .

5.1.3 Optimal Reliability Allocation

In the process of developing a new product, the engineer is often faced with the task of designing a system that conforms to a set of reliability specifications. The engineer is given the goal for the system and must then develop a design that will achieve the desired reliability of the system, while performing all of the system's intended functions at a minimum cost. This involves a balancing act of determining how to allocate reliability to the components in the system so the system will meet its reliability goal while at the same time ensuring that the system meets all of the other associated performance specifications.

BlockSim provide three allocation methods: equal allocation, weighted reliability allocation and cost optimization allocation. In these three methods, the simplest method is equal reliability allocation, which distributes the reliabilities uniformly among all components. For example, suppose a system with five components in series has a reliability objective of 90% for a given operating time. The uniform allocation of the objective to all components would require each component to have a reliability of 98% for the specified operating time, since $0.985 \approx 0.90$. While this manner of allocation is easy to calculate, it is generally not the best way to allocate reliability for a system. The optimum method of allocating reliability would take into account the cost or relative difficulty of improving the reliability of different subsystems or components.

The reliability optimization process begins with the development of a model that represents the entire system. This is accomplished with the construction of a system reliability block diagram that represents the reliability relationships of the components in the system. From this model, the system reliability impact of different component modifications can be estimated and considered alongside the costs that would be incurred in the process of making those modifications. It is then possible to perform an optimization analysis for this problem, finding the best combination of component reliability improvements that meet or exceed the performance goals at the lowest cost.

Reliability engineers are very often called upon to make decisions as to whether to improve a certain component or components in order to achieve a minimum required system reliability. There are two approaches to improving the reliability of a system: fault avoidance and fault tolerance. Fault avoidance is achieved by using high-quality and high-reliability components and is usually less expensive than fault tolerance. Fault tolerance, on the other hand, is achieved by redundancy. Redundancy can result in increased design complexity and increased costs through additional weight, space, etc.

Before deciding whether to improve the reliability of a system by fault tolerance or fault avoidance, a reliability assessment for each component in the system should be made. Once the reliability values for the components have been quantified, an analysis can be performed in order to determine if that system's reliability goal will be met. If it

Notes

becomes apparent that the system's reliability will not be adequate to meet the desired goal at the specified mission duration, steps can be taken to determine the best way to improve the system's reliability so that it will reach the desired target.

Consider a system with three components connected reliability-wise in series. The reliabilities for each component for a given time are: $R_1=70$ $R_2=80$, and $R_3=90$. A reliability goal, $R_G=85$ is required for this system.

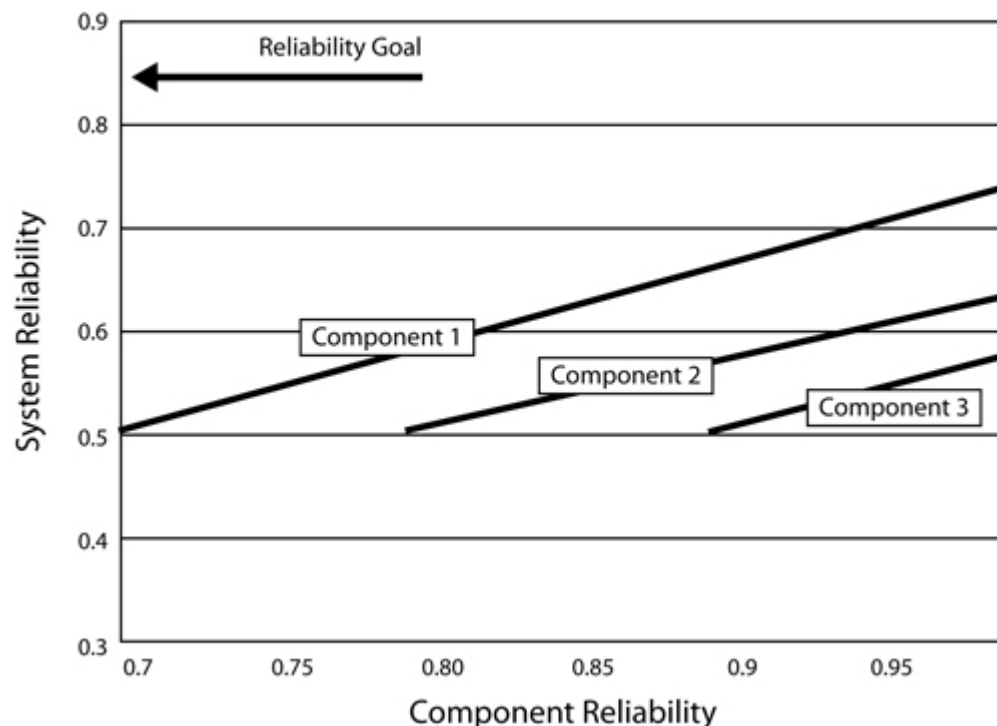
The current reliability of the system is:

$$R_s = R_1 \cdot R_2 \cdot R_3 = 50.4$$

Obviously, this is far short of the system's required reliability performance. It is apparent that the reliability of the system's constituent components will need to be increased in order for the system to meet its goal. First, we will try increasing the reliability of one component at a time to see whether the reliability goal can be achieved.

The following figure shows that even by raising the individual component reliability to a hypothetical value of 1 (100% reliability, which implies that the component will never fail), the overall system reliability goal will not be met by improving the reliability of just one component. The next logical step would be to try to increase the reliability of two components. The question now becomes: which two? One might also suggest increasing the reliability of all three components. A basis for making such decisions needs to be found in order to avoid the *trial and error* aspect of altering the system's components randomly in an attempt to achieve the system reliability goal.

System Reliability vs. Component Reliability
(by changing the reliability of one component at a time)

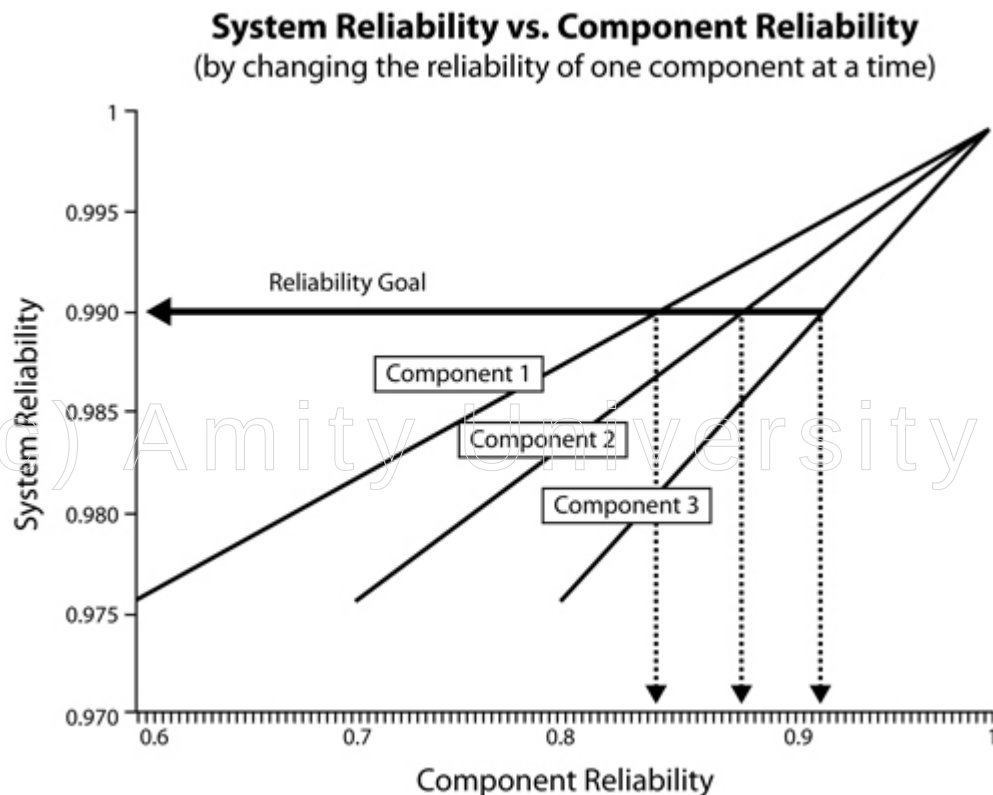


Notes

As we have seen, the reliability goal for the preceding example could not be achieved by increasing the reliability of just one component. There are cases, however, where increasing the reliability of one component results in achieving the system reliability goal. Consider, for example, a system with three components connected reliability-wise in parallel. The reliabilities for each component for a given time are: $R_1=60$ $R_2=70$ and $R_3=80$. A reliability goal, $R_G=99$ is required for this system. The initial system reliability is:

$$RS=1-(1-0.6) \cdot (1-0.7) \cdot (1-0.8)=0.976$$

The current system reliability is inadequate to meet the goal. Once again, we can try to meet the system reliability goal by raising the reliability of just one of the three components in the system.



From the above figure, it can be seen that the reliability goal can be reached by improving Component 1, Component 2 or Component 3. The reliability engineer is now faced with another dilemma: which component's reliability should be improved? This presents a new aspect to the problem of allocating the reliability of the system. Since we know that the system reliability goal can be achieved by increasing at least one unit, the question becomes one of how to do this most efficiently and cost effectively. We will need more information to make an informed decision as to how to go about improving the system's reliability. How much does each component need to be improved for the system to meet its goal? How feasible is it to improve the reliability of each component? Would it actually be more efficient to slightly raise the reliability of two or three components rather than radically improving only one?

In order to answer these questions, we must introduce another variable into the problem: cost. Cost does not necessarily have to be in dollars. It could be described

Notes

in terms of non-monetary resources, such as time. By associating cost values to the reliabilities of the system's components, we can find an optimum design that will provide the required reliability at a minimum cost.

5.1.4 Quality Planning

We defined quality and touched on how to define quality standards. This section will look at the steps in quality planning and control.

Organizations perform quality planning and control to ensure products or services satisfy expectations consumers have about the design specifications of the product or service. Meeting consumer needs depends on the design of these specifications.

One of the most important assisting procedures throughout project planning is quality assurance. It is a continuous action that is carried out in conjunction with other planning procedures. The goal of quality planning is to identify the appropriate quality standards, incorporate them into products and processes, and determine how these requirements will be met.

A project manager must first understand the standard expectations before he can prepare for quality. In other words, he must be informed of the organization's top management's quality policy. It expresses the organization's overall goal with regard to quality as formally articulated by the senior management. It needs to be documented. To guide project implementation, the project team should follow the organization's quality policy. In the event of a joint venture, if there is no quality policy or project that involves many performing organisations, the project management team should design one. The quality policy should be well understood by all project stakeholders.

Another crucial component of successful planning is the project scope statement. It specifies crucial information for quality planning. It specifies what will and will not be provided as part of the project, as well as cost, timeline, and cost targets. A description of the product to be designed, standards, and regulations that may affect the project, as well as outputs from other processes, are all inputs to quality planning.

The project manager can measure the quality after gathering the relevant inputs and evaluating the product description and quality assurance. Estimating, tracking project progress, identifying problems and attaining continuous improvement, and reviewing tools all involve quality metrics.

Quality plans should define:

- Defining the goals that must be met (for example, characteristics or specifications, uniformity, effectiveness, aesthetics, cycle time, cost, natural resources, utilization, yield, dependability, and so on)
- Steps in a process that make up an organization's operational practise or procedures.
- Specific defined standards, practises, procedures, and instructions to be applied during the various phases of the process or project Allocation of roles, authority, and resources during the various phases of the process or project

Notes

- At the relevant phases, proper testing, inspection, examination, and audit programmes
- As a process is enhanced, a defined mechanism for changes and revisions to a quality plan is created.
- A method for determining whether or not the quality objectives have been met.
- Other steps required to achieve the goals

Quality goals and plans should be linked into the organization's overall strategic plans at the highest level. Each function develops its own best means of contributing to the top-level goals and objectives as organisational objectives and plans are disseminated throughout the organisation.

The quality plan takes on the role of an executable plan at lower levels. Depending on the desired objective, such plans might take many various shapes. To achieve a certain result, quality plans may be expressed by more than one type of document.

Example

A manufacturing company that machines metal parts is an example of a quality strategy. Its quality plan includes relevant procedures (descriptions of the manufacturing process and duties), applicable workmanship standards, allowable measurement tolerances, material standards description, and so on. These might be different docs.

Individual work orders may include more varied information specific to a specific customer (sometimes called travelers). Work orders detail machine settings and tolerances, as well as operations, tests, inspections, handling, storing, packaging, and delivery procedures.

An operating-level quality plan translates customer requirements (the what) into actions required to achieve the desired outcome (the how), and couples this with applicable procedures, standards, practises, and protocols to specify exactly what is required, who will perform it, and how it will be accomplished. Product tolerances, testing parameters, and acceptance criteria may all be specified in a quality control plan. While the terminology for service and other types of organisations may differ, the underlying concept remains the same.

5.2.1 Quality Control**Steps in quality planning and control****Step 1**

Defining quality characteristics of the product or service takes place in this step. These quality characteristics are directly related to the product or service's design specifications. The most common quality characteristics include:

1. **Functionality:** whether the product performs its desired function.
2. **Appearance:** whether the physical aspects are attractive.
3. **Reliability:** the ability of the product or service to perform continuously.
4. **Durability:** the total life expectancy.
5. **Serviceability:** the ability for the organization (or third party) to repair the product

Notes

or service.

6. Contact: the interaction available between the client/consumer and an organization.

Step 2

After determining quality characteristics, try express them in measurable terms. Characteristics that are measurable are controllable.

Step 3

Each defined quality characteristic of the product or service should have set standards. Quality standards for quality characteristics enable the operations manager to measure the conformance and achievement of the defined quality characteristics. Having fewer than 0.5 bugs per 1000 lines of code is an example of a quality standard. If 3 bugs per 1000 lines of code were observed then quality standards weren't met to ensure the satisfaction of 'reliability' quality characteristics.

Step 4

Determine whether the product or service satisfies realistic quality standards by analysing them. This is the control step. The operations manager is concerned with three areas in this control process:

1. Whether to Implement preventative, in-time or reactive control.
2. Deciding if they will check each product or service individually or if they will take a general sample to determine if all products and services meet quality standards.
3. Statistical process control or ac
4. ceptance sampling is conducted. Statistical process control is when inspection is conducted during the manufacturing or rendering process. Acceptance sampling is when inspection occurs after the manufacturing or rendering process.

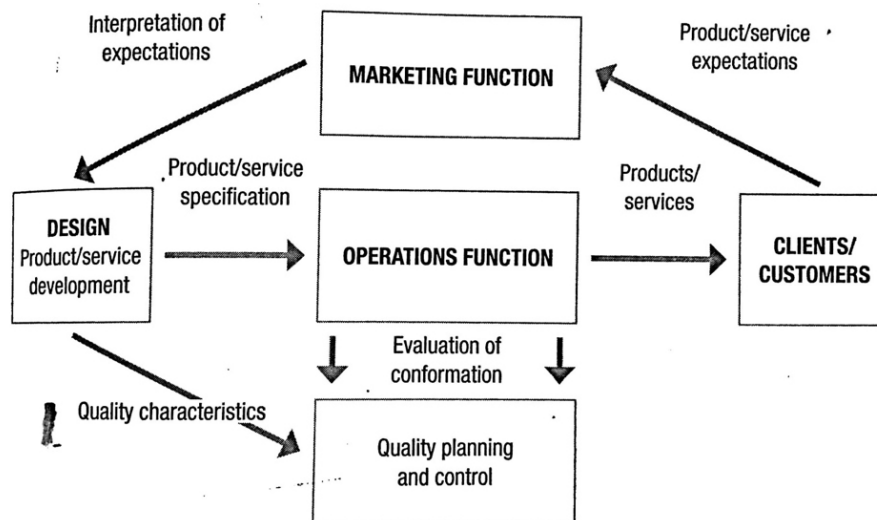
Step 5:

Never perceive quality deviations negatively. Although they can be costly, they give the organization a chance to identify and correct causes of poor quality. Conduct identifying and rectifying poor quality at this stage.

Step 6:

The last step is to continuously look for ways to improve the quality of products and services. Improvement of products and services is based on performance measurement. Improvement can either be dramatic and large scale – such as breakthrough improvement or it can be incremental where regular and small scale changes take place – continuous improvement.

Notes



The customer/client-marketing-design-operations cycle

5.2.2 Quality Improvement Process

Quality improvement (QI) is a systematic, formal approach to the analysis of practice performance and efforts to improve performance.

A variety of approaches—or QI models—exist to help you collect and analyse data and test change. While it's important to choose a reputable QI model to guide your efforts, it's more important that you fully commit to using the QI process and good QI practices.

Benefits of QI

- Understanding and properly implementing QI is essential to a well-functioning practice, and is necessary for any practice interested in improving efficiency, patient safety, or clinical outcomes.
- In addition, good QI practices and improved patient outcomes position your practice for success by:
- Helping you prepare for the transition to value-based payment models.
- Allowing you to participate in the public reporting of physician-quality data.
- Giving you the opportunity to participate in the federal Quality Payment Program (QPP) following one of two tracks: the Merit-based Incentive Payment System (MIPS) or the Alternative Payment Model (APM).
- Equipping you with the skills necessary to apply for and complete national recognition programs, such as National Committee for Quality Assurance's (NCQA) Diabetes, Heart/Stroke, and patient-centered medical home (PCMH)-recognition programs.
- Helping you earn Family Medicine Certification-Performance Improvement activity credit (formerly Maintenance of Certification) requirements.

Quality Improvement Basics

Notes

The QI process is grounded in the following basic concepts:

- **Establish a culture of quality in your practice.** Your practice's organization, processes, and procedures should support and be integrated with your QI efforts. The culture of a practice—attitudes, behaviors, and actions—reflect how passionately the practice team embraces quality. The QI culture looks different for every practice, but may include establishing dedicated QI teams, holding regular QI meetings, or creating policies around your QI goals.
- **Determine and prioritize potential areas for improvement.** You will need to identify and understand the ways in which your practice could improve. Examine your patient population (e.g., to identify barriers to care, frequently diagnosed chronic conditions, or groups of high-risk patients) and your practice operations (e.g., to identify management issues such as low morale, long patient wait times, or poor communication). Use established quality measures, such as those from the National Quality Forum Agency for Healthcare Research and Quality and the Quality Payment Program to guide your efforts.
- **Collect and analyze data.** Data collection and analysis lie at the heart of quality improvement. Your data will help you understand how well your systems work, identify potential areas for improvement, set measurable goals, and monitor the effectiveness of change. It's important to collect baseline data before you begin a QI project, commit to regular data collection, carefully analyze your results throughout the project, and make decisions based on your analysis.
- **Communicate your results.** Quality improvement efforts should be transparent to your staff, physicians, and patients. Include the entire practice team and patients when planning and implementation QI projects, and communicate your project needs, priorities, actions, and results to everyone (patients included). When a project is successful, celebrate and acknowledge that success.
- **Commit to ongoing evaluation.** Quality improvement is an ongoing process. A high-functioning practice will strive to continually improve performance, revisit the effectiveness of interventions, and regularly solicit patient and staff feedback.
- **Spread your successes.** Share lessons learned with others to support wide-scale, rapid improvement that benefits all patients and the health care industry as a whole.

A quality improvement process can offer organizations the following benefits:

- Solutions that focus on failures in processes, not flaws in people
- A reliance on objective, data-driven solutions, rather than subjective opinions, to identify inefficiencies, preventable errors, and inadequate processes
- Improvements that provide better customer service, increased efficiency, greater safety, and higher revenues
- A localized focus on testing small, incremental improvements that is less risky than a focus on making changes at one time
- Data collection to monitor improvement efforts, which can provide the basis for reimbursement and certification programs, particularly in healthcare organizations

Difficulties in Pursuing a Quality Improvement Process Plan

Here are some of the common difficulties in following through with a QI plan:

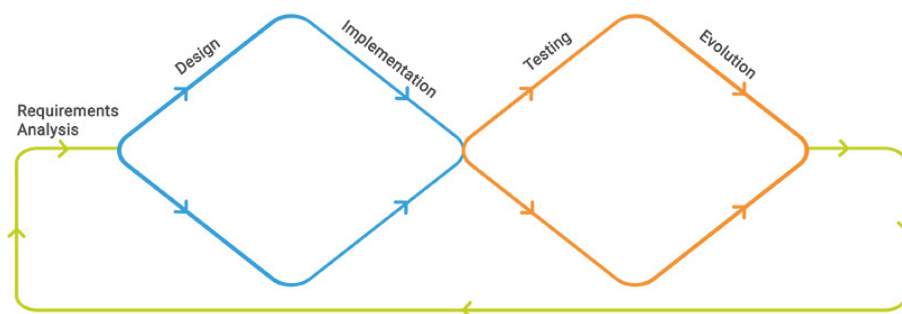
- Expectations are not clear.
- Leadership is not adequately engaged, making bottom-up initiatives difficult.
- There is insufficient time and resources to properly implement the initiative.
- In healthcare settings, some physicians don't implement new systems until they have confidence in new processes.
- There is an inadequate emphasis on the importance and use of new measures.
- There is a poor level of collaboration between teams.
- People underestimate the time required to implement a program.
- The extent to which a result depends on a change can't be measured because the extent of the original problem has not been measured.
- Specific improvement cycles can't be evaluated.
- Costs can't be evaluated.
- Changes result from the Hawthorne effect rather than the QI program. In this interpretation of the Hawthorne effect, stakeholder behaviour changes because their activities and results are monitored.
- A small sample size makes generalizations impossible.
- Solving some problems creates additional problems.
- Targets are overly ambitious and therefore difficult to achieve.
- There are too many diverse stakeholder conditions.

Notes

5.2.3 Evolution of Software Quality Assurance

Software quality assurance is a validation process to check whether the developed software meets predefined quality specifications or not. It is an ongoing process which goes along with the software development life cycle to confirm that the developed software is exactly in line with the client requirements.

Test Engineering



The Evolution of Software testing

Earlier on organizations used the waterfall development cycle in which no

Notes

working software was available during the development cycle. The testing started post development and there was no room for making any changes to the requirements.

This led to the evolution of the entire software testing industry as it shifted from quality control to quality assurance. Companies adopted agile development practices to improve team engagement and quality of software.

Today, the development process is complicated as there are frequent changes in the requirements, test plans, and resource allocation. Testers and developers need to align with changes as they can significantly impact software quality and delivery. The quality assurance process needs to evolve to accommodate all the changes.

The ongoing transformation from quality assurance to test engineering

A new way of testing is required to deliver digital products quickly with high-quality. The customers are looking for an Omni-channel experience which means an integrated experience across different devices and browsers. The software development companies need to ensure that the software is compatible with all the platforms. The change is only possible when the development and testing teams have a higher level of collaboration.

Changing customers' needs

The testing process is also undergoing transformation with the evolution of the software being built. Quality assurance now involves the entire team to evaluate the user requirements from the end-user point and a testability perspective.

More secure applications required

It is vital for organizations to prevent and protect their software from all kinds of threats, breaches, and data leaks by performing test engineering. Test engineering is the continuous testing process which focuses on fixing bugs quickly at the time of development.

What is Test Engineering or Continuous Testing?

Test Engineering is the process of continuous testing in which testing is an integral part of the development process from end to end. This type of testing is most suitable for DevOps and Agile development initiatives.

Test engineering or continuous testing is a process where the focus is on feedback to reduce business associated risk with the software release; It takes place by executing automated tests which are an integral part of the software development lifecycle.

Key features of continuous testing:

- Continuous testing evaluates complete coverage in terms of business risk.
- It also provides insights on the risk involved in releasing the application through the delivery pipeline.
- This form of testing ensures testing to be in line with the development process instead of the released software.

- Continuous testing avoids software failures by creating a safe environment which protects user experience while the application is in the development process.
- It ensures the availability of valid test data for each and every test run.
- It easily implements “shift left” (unit, component, coverage...) to “shift right” (monitoring/ APM, Testing in Production) testing.
- This testing prevents bottleneck by testing at the right stage of the delivery pipeline.
- It also offers actionable feedback for each stage of the development process.
- Continuous testing also checks the application change and its impact on functionality.
- It also reduces rework, prioritize robust, flexible and modern framework.
- Continuous testing reduces redundancy by consistently optimizing and reviewing the test suites.

5.2.4 Major SQA Activities

Software quality assurance (SQA) is a process which assures that all software engineering processes, methods, activities and work items are monitored and comply against the defined standards. These defined standards could be one or a combination of any like ISO 9000, CMMI model, ISO15504, etc.

SQA incorporates all software development processes starting from defining requirements to coding until release. Its prime goal is to ensure quality.

SQA Activities

Given below is the list of SQA activities:

#1) Creating an SQA Management Plan:

The foremost activity includes laying down a proper plan regarding how the SQA will be carried out in your project.

Along with what SQA approach you are going to follow, what engineering activities will be carried out, and it also includes ensuring that you have a right talent mix in your team.

#2) Setting the Checkpoints:

The SQA team sets up different checkpoints according to which it evaluates the quality of the project activities at each checkpoint/project stage. This ensures regular quality inspection and working as per the schedule.

#3) Apply software Engineering Techniques:

Applying some software engineering techniques aids a software designer in achieving high-quality specification. For gathering information, a designer may use techniques such as interviews and FAST (Functional Analysis System Technique).

Later, based on the information gathered, the software designer can prepare the project estimation using techniques like WBS (work breakdown structure), SLOC (source line of codes), and FP(functional point) estimation.

Notes

#4) Executing Formal Technical Reviews:

An FTR is done to evaluate the quality and design of the prototype.

In this process, a meeting is conducted with the technical staff to discuss regarding the actual quality requirements of the software and the design quality of the prototype. This activity helps in detecting errors in the early phase of SDLC and reduces rework effort in the later phases.

#5) Having a Multi- Testing Strategy:

By multi-testing strategy, we mean that one should not rely on any single testing approach, instead, multiple types of testing should be performed so that the software product can be tested well from all angles to ensure better quality.

#6) Enforcing Process Adherence:

This activity insists the need for process adherence during the software development process. The development process should also stick to the defined procedures.

This activity is a blend of two sub-activities which are explained below in detail:

(i) Product Evaluation:

This activity confirms that the software product is meeting the requirements that were discovered in the project management plan. It ensures that the set standards for the project are followed correctly.

(ii) Process Monitoring:

This activity verifies if the correct steps were taken during software development. This is done by matching the actually taken steps against the documented steps.

#7) Controlling Change:

In this activity, we use a mix of manual procedures and automated tools to have a mechanism for change control.

By validating the change requests, evaluating the nature of change and controlling the change effect, it is ensured that the software quality is maintained during the development and maintenance phases.

#8) Measure Change Impact:

If any defect is reported by the QA team, then the concerned team fixes the defect.

After this, the QA team should determine the impact of the change which is brought by this defect fix. They need to test not only if the change has fixed the defect, but also if the change is compatible with the whole project.

For this purpose, we use software quality metrics which allows managers and developers to observe the activities and proposed changes from the beginning till the end of SDLC and initiate corrective action wherever required.

#9) Performing SQA Audits:

The SQA audit inspects the entire actual SDLC process followed by comparing it against the established process.

It also checks whatever reported by the team in the status reports were actually performed or not. This activity also exposes any non-compliance issues.

#10) Maintaining Records and Reports:

It is crucial to keep the necessary documentation related to SQA and share the required SQA information with the stakeholders. The test results, audit results, review reports, change requests documentation, etc. should be kept for future reference.

#11) Manage Good Relations:

In fact, it is very important to maintain harmony between the QA and the development team.

We often hear that testers and developers often feel superior to each other. This should be avoided as it can affect the overall project quality.

5.3.1 Major SQA Issues

Identifying SQA Issues

Software Quality Assurance is a good practice that every large scale business should employ. IT related businesses have never hesitated to use SQA to ensure that the application they will release for their users or sell to their customers will live up to their expectations.

Identifying SQA Issues

Software Quality Assurance is a good practice that every large scale business should employ. IT related businesses have never hesitated to use SQA to ensure that the application they will release for their users or sell to their customers will live up to their expectations.

On the other hand, there are companies that has opted to forgo with the usage of SQA and relied on application testers just to make sure the application will never have internal errors. The SDLC on the other hand, will ensure the development of the application will work as planned.

Like other development plans, there are issues why developers and companies do not use SQA.

SQA Cost

This is the main reason why small scale companies hesitated to use SQA as part of their development plan. SQA is a different entity that works separately from the development team. Hiring another set of developers will mean another set of people to pay for. Developing software takes months to take and if you are a small-scaled company you will definitely have to cut costs.

This problem cannot just be answered with more money and resources. Unfortunately, the developers have to face the fact that SQA cannot just be implemented when everyone has to cut costs. Only the SDLC and the testing team could compensate what the SQA is lacking.

Notes

SQA Complexity

Years ago, an application could be easily built by a single developer and let the whole world enjoy them. Today, a highly efficient application would take years for a single developer to develop. By the time it could be integrated and implemented, it is already outdated.

The point is, software today is very complicated that team after team will be working on developing a single application and will take them months to develop. SQA has already been here for years and there are models that are not as good as it was.

To answer this concern the SQA team should carefully use CASE tools. These tools could easily summarize the functions. They ensure consistency in the evaluation of the application. That includes documentation of every stage of development.

SQA Integration

This is one of the many concerns of the SQA models today. Because application development has been very rapid, the standardization of the application might not be the same as it was five years ago. The result for this is that although it has been deemed as an application developed with SQA, the type of SQA might not live up according to what is expected.

The standardization models might ensure that the application developed ten years ago is good but it might work today. Fortunately, this issue was answered with the development of new standardizations. This ensured that the application was developed according to the plan and today's need. Today's standards usually come with set of tools with an understanding that the application is more complex than it was.

Demand for Faster Development

More than ever, the market for better software is increasing and everyone wants to sell and developing something new. The software has to be developed faster more than every to be ahead of everyone. Complimenting that speed is also the increasing complexity of the application.

It is not only the developers that have this problem of rapid software development but also the SQA as well. They have to act really fast and decide whether or not the application is worthy to be released for operation. If that was the case 10 years ago, everyone will have a very hard time. Checking the application used to be manual and if that will be the case today, everyone will definitely have a very hard time approving the application.

Today, there are tools that have great testing capabilities that could work faster like any other manual testers. That does not include the stability the application could offer. Formal verification could be attained using this application since it has been sanctioned by standards to work. The development of the application for formal verification has ease up the demand

Defective Software

After months and moths of development and manual coding, it is very frustrating when your application is disregarded and has to start all over again. SQA has that function to be a little bit perfectionists and anything bad is disregarded. Considering the

Notes

need to develop an application rapidly, everyone has to complete the application really fast. Perfection and speed is always a requirement for SQA and have been one of the biggest reasons why developers and small scale companies have opted not to use SQA too much. They have limited themselves to code testing and SDLC.

Today, that problem is addressed with the aid of libraries. There are many libraries today that can easily be used to develop the application. The good thing about it is that they do not have to code. The well known libraries can easily be used in an application and in no time the application could be used and tested.

SQA has been tested on these applications and have passed so many times. Although there are no frameworks that have been approved to bypass SQA, they would almost pass the SQA as soon as they are tested. What's left is just the documentation and proper commenting in each function.

These are the issues that have been holding back the small scale companies. However, there are now solutions to these concerns that small scale companies does not need to have second doubts in using SQA for developing a good looking and well functioning application. Besides the monetary consideration, every issue that surrounds the SQA has been addressed.

One of the biggest reasons why SQA could now be used by any company is automation. CASE tools can easily gauge if the application has enough software or user requirements. Although there are things that the SQA team should work on themselves, automation has helped application to be the best it could be. Testing tools has also developed and it has not checked the codes only but also on stresses. Anything that concerns around SQA has already been address so anyone could easily use this service.

5.3.2 Zero Defect Software

Zero Defects is a Lean philosophy, a mentality, or a movement introduced by Philip Crosby* that aims to reduce and minimize the number of defects and errors in a process in hopes of getting things done right the first time. The ultimate aim of the theory is to reduce the level of defects to zero. Its purpose is to design every process so that it is not possible to produce something of low quality. This is done assuming that the cost of preventing problems is less than fixing them later. Zero Defects certainly seems good in theory, but how does it actually apply to software development? The following are few things that you should be aware before starting such initiative in your organization:

Is Zero Defects possible?

OK, before we go any further we need to address this question; the answer is NO. In reality, especially when we are talking about software development, it is just not possible to have zero defects. Those who promote the philosophy of zero defects say that the quest for perfection, even though everyone knows it is not possible, will at least increase quality. But then why are they motivating people to work towards something that is not possible? Wouldn't it be better to just use something as the "Defect Reduction Initiative" instead of a Zero Defect approach?

Notes

When did all defects become the same?

In the real world, we prioritize defects and take care of them in order from greatest to least importance. Also, if we consider Return Of Investment (ROI), some defects at the bottom of this scale may never be taken care of at all. The Zero Defects movement assumes that all defects are equal and requires teams to spend time analyzing and resolving them without ever considering ROI. To make things worse, organizations that are into Zero Defects will add quality (check) points that will slow down overall delivery but “ensure” these defects will never be repeated despite the fact that the process is not actually adding value.

So how the Zero Defects is actually reducing cost?

While one of the main ideas behind Zero Defects is to reduce cost, it will more than often do the opposite. Teams that need to fix all defects add tremendous labor-related effort and delays product releases to customer; both of these issues are highly correlated with cost. One more thing to consider is that as teams resolve defects, there will be fewer defects left to find. This often means that finding defects that are left will take more time and effort, and with that, increases in cost occur.

Zero Defects Mentality

Often I hear the purpose of promoting Zero Defects is to shift mindsets in your organization, but is that an environment you really want to build? A few years ago Mary Poppendieck posted a quote from Defense Secretary William J. Perry: “This zero defects mindset creates conditions that will lead inevitably to failure. Demanding such a rigid standard produces timid leaders afraid to make tough decisions in crisis, unwilling to take the risks necessary for success in military operations.” In addition to significantly damaging your environment, there is a point of diminishing return in product quality that Zero Defects does not take into consideration. In the end, we all certainly want our products to be as perfect as possible, but the Zero Defect approach is not the best way to achieve that. Companies should understand that any efforts to reach perfection will come alongside with significant cost increases and changes to your current environment.

5.3.3 Verification

What is Software Verification?

Verification is the process of checking or verifying the credentials, data or information to confirm their credibility and accuracy. In the field of software engineering, software verification is defined as the process of evaluating software product, to ensure that the development phase is being carried out accurately, to build the desired software product. It is performed during the ongoing phase of software development, to ensure the detection of defects and faults in the early stage of the development life cycle and to determine whether it satisfies the requirements of the customer.

Software verification offers answers to our query of **“Are we building the software product in a right manner?”** However, if the software passes during the verification process, it does not guarantee its validity. It is highly possible that a software product goes well through the verification process, but might fail to achieve the desired requirements.

Notes

Features of Software Verification:

In the realm of the software industry, software verification plays an integral role in building the product as per the requirements and needs of the customer. Other features of software verification that signify its importance are:

- Performed during the early stages of the software development process to determine whether the software meets the specified requirements.
- Verification denotes precision of the end or final product.
- It conducts **software review**, walk through, inspection, and evaluates documents, **plans**, requirements, and specifications.
- It demonstrates the consistency, completeness, and correctness of the software during each stage of the software development life cycle.
- Software verification can be termed as the first stage of the **software testing life cycle (STLC)**.

Types of Verification:

Software verifications are of two types, each of which, is focused on verifying various aspects of the software and ensuring its high quality. Together, these two verification types ensure that the software conforms to and satisfies specified requirements. The two types of software verification are:

1. **Static Verification:** Static verification involves inspection of the code before its execution, which ensures that the software meets its specified requirements and specifications.
 - It is an analysis based approach, usually carried out by just making an analysis of static aspects of the software system, such as the code conventions, software metrics calculation, anti-pattern detection, etc.
 - It does not involve the operation of the system or **component**.
 - Static verification includes both **manual** as well as **automated testing** techniques, such as consistency techniques and measurement techniques.
2. **Dynamic Verification:** It concerns with the working behavior of the software and is being carried out along with the execution of software. Also, known by 'Test phase' dynamic verification executes **test data** on the software, to assess the behavior of the software.
 - Unlike static verification, dynamic verification involves **execution** of the system or its components.
 - The team selects a group of test cases consisting of test data, which are then used to find out the output test results.
 - There are three subtypes of dynamic verification: **functional testing**, structural testing, and **random testing**.

How to Perform Software Verification?

The process of software verification involves the assessment of the development phase and intermediary software product, based on the pre-defined specifications and guidelines.

Notes

It ensures that the methodology used for software development adheres to the specified specifications, set-up before the development phase, to build the correct software product.

Thus, this process requires cross-checking of these pre-defined specifications with the partially developed software product, to satisfy the process of carrying out the development of the product, in a right way.

Approaches:

As stated above, the process of software verification involves reviews, walkthrough, and inspection, which together helps to evaluate the accuracy of the product and ensure its quality. Therefore, defined below are these approaches of software verification:

The process of software verification can be performed through following approaches:

1. **Reviews:** It is an organized way of examining the documents such as design specifications, requirements specifications, code, etc. by one or more than one person, to explore defects in the software.
2. **Walkthroughs:** It is, usually an informal way of evaluating the software product, where different teams, whether associated or non-associated with the software development, goes through the software product and have discussion on possible errors and other defects, present in the software.
3. **Inspection:** It is one of the preferred and most common methods of the **static testing**. Unlike, walkthroughs, it's a formal way of testing the software, through examination of documents, carried out by the skilled moderator. This method usually, checklist, rules, entry and exist criteria along with preparation and sharing of reports, in order to take corrective & necessary actions.

Advantages of Software Verification:

The advantages offered by software verification are numerous. It is among those processes, which makes the process of software development easy and allows the team to create an end product that conforms to the rules and regulations, as well as customer's requirements. Other advantages of this process are:

- It helps reduce the number of defects found during the later stages of development.
- Verifying the software in its initial development phase allows the team to get a better understanding of the product.
- It helps reduce the chances of system failures and crashes.
- Assists the team in developing a software product that conforms to the specified requirements and design specifications.

5.3.4 Validation

Software validation

Software validation checks that the software product satisfies or fits the intended use (high-level checking), i.e., the software meets the user requirements, not as

Notes

specification artifacts or as needs of those who will operate the software only; but, as the needs of all the stakeholders (such as users, operators, administrators, managers, investors, etc.). There are two ways to perform software validation: internal and external. During internal software validation, it is assumed that the goals of the stakeholders were correctly understood and that they were expressed in the requirement artefacts precisely and comprehensively. If the software meets the requirement specification, it has been internally validated. External validation happens when it is performed by asking the stakeholders if the software meets their needs. Different software development methodologies call for different levels of user and stakeholder involvement and feedback; so, external validation can be a discrete or a continuous event.

Successful final external validation occurs when all the stakeholders accept the software product and express that it satisfies their needs. Such final external validation requires the use of an acceptance test which is a dynamic test. However, it is also possible to perform internal static tests to find out if the software meets the requirements specification but that falls into the scope of static verification because the software is not running.

Artifact or specification validation

Requirements should be validated before the software product as a whole is ready (the waterfall development process requires them to be perfectly defined before design starts; but iterative development processes do not require this to be so and allow their continual improvement).

Examples of artifact validation:

- User Requirements Specification validation: User requirements as stated in a document called User Requirements Specification are validated by checking if they indeed represent the will and goals of the stakeholders. This can be done by interviewing the stakeholders and asking them directly (static testing) or even by releasing prototypes and having the users and stakeholders to assess them (dynamic testing).
- User input validation: User input (gathered by any peripheral such as keyboard, bio-metric sensor, etc.) is validated by checking if the input provided by the software operators or users meets the domain rules and constraints (such as data type, range, and format).

Difference between Software Verification and Validation:

Software verification and validation are two of the most important processes used for ensuring the quality and accuracy of the product. However, these are usually confused by people and used interchangeably. Hence, to help you here is a comparison between these two software testing techniques.

Software Verification	Software Validation
1. Software verification is the process of evaluating the development phase to ensure the accuracy of the software.	1. Software validation evaluates the software during the development phase to ensure its compliance with business requirements.

Notes

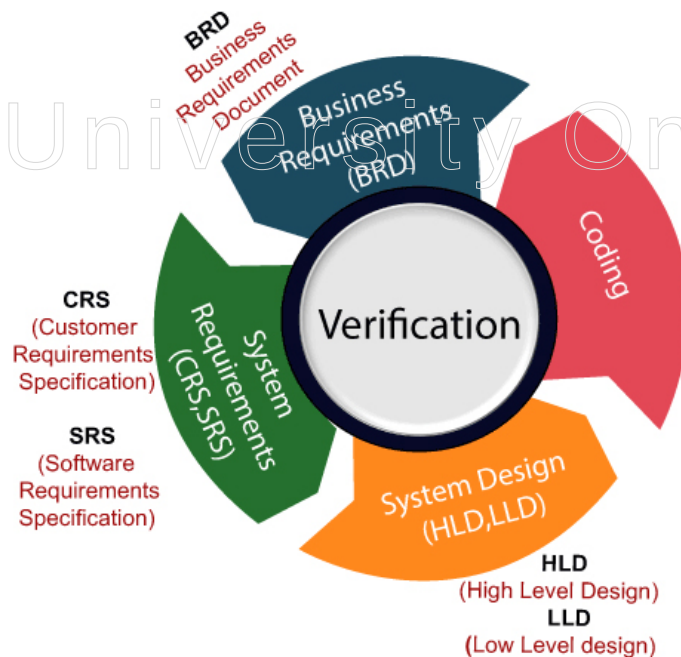
2. It is performed before the execution of software validation.	2. It is performed once the process of software verification is completed.
3. Verification ensures the product is being developed as per the requirements and design specification.	3. Validation ensures that the software meets the user's needs and is fit to be used by them.
4. Evaluates plans, requirement specification, documents, test cases , etc.	4. Evaluates the software product to verify its readiness to be released.

5.4.1 Evolutionary Nature of Verification and Validation

Verification testing

Verification testing includes different activities such as business requirements, system requirements, design review, and code walkthrough while developing a product.

It is also known as static testing, where we are ensuring that “**we are developing the right product or not**”. And it also checks that the developed application fulfilling all the requirements given by the client.



Validation testing

Validation testing is testing where tester performed functional and non-functional testing. Here **functional testing** includes Unit Testing

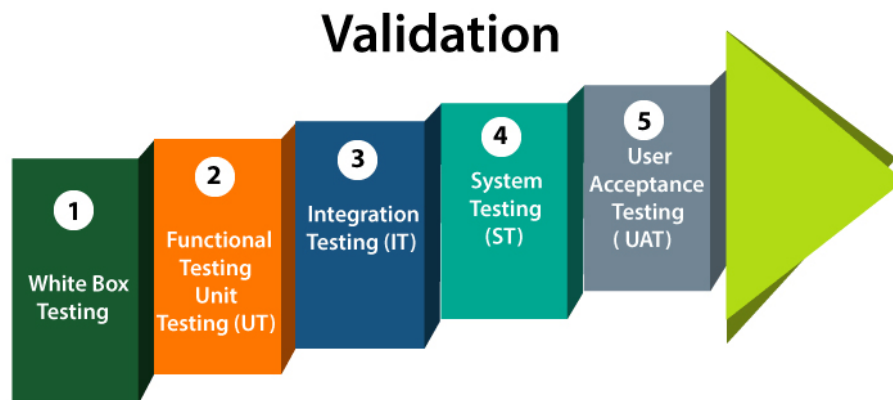
(UT), Integration Testing

(IT) and System Testing (ST), and **non-functional** testing includes User acceptance testing (UAT).

Validation testing is also known as dynamic testing, where we are ensuring

that “we have developed the product right.” And it also checks that the software meets the business needs of the client.

Notes



Note: Verification and Validation process are done under the V model of the software development life cycle.

Difference between verification and validation testing

Verification	Validation
We check whether we are developing the right product or not.	We check whether the developed product is right.
Verification is also known as static testing .	Validation is also known as dynamic testing .
Verification includes different methods like Inspections, Reviews, and Walkthroughs.	Validation includes testing like functional testing, system testing, integration, and User acceptance testing.
It is a process of checking the work-products (not the final product) of a development cycle to decide whether the product meets the specified requirements.	It is a process of checking the software during or at the end of the development cycle to decide whether the software follow the specified business requirements.
Quality assurance comes under verification testing.	Quality control comes under validation testing.
The execution of code does not happen in the verification testing.	In validation testing, the execution of code happens.
In verification testing, we can find the bugs early in the development phase of the product.	In the validation testing, we can find those bugs, which are not caught in the verification process.
Verification testing is executed by the Quality assurance team to make sure that the product is developed according to customers' requirements.	Validation testing is executed by the testing team to test the application.
Verification is done before the validation testing.	After verification testing, validation testing takes place.

Notes

In this type of testing, we can verify that the inputs follow the outputs or not.

In this type of testing, we can validate that the user accepts the product or not.

5.4.2 Impracticality of Testing all Data and Paths

Impracticality of Testing All Data

- For almost every program, it is impossible to make an attempt to test the program with all sets of inputs.
- The correctness of the output for a particular test input is determined using a testing oracle.
- A testing oracle is a mechanism that can be used for determining whether a test has pass or failed.

Impracticality of Testing All Paths

- For almost every program, it is impossible to attempt to test all executable paths through the project because of the number of combination i.e. combination explosion.
- Developing an algorithm for this purpose is also not possible.

5.4.3 Proof of Correctness

A proof of correctness is a mathematical proof that a computer program or a part thereof will, when executed, yield correct results i.e., results fulfilling specific requirements. Before proving a program correct, the theorem to be proved must, of course, be formulated.

Hypothesis: The hypothesis of such a correctness theorem is typically a condition that the relevant program variables must satisfy immediately “before” the program is executed. This condition is called the “precondition”.

Thesis: The thesis of the correctness theorem is typically a condition that the relevant program variables must satisfy immediately “after” execution of the program. This latter condition is called the ‘post-condition’.

Thus the correctness theorem can be stated as:

“If the condition, “V”, is true before execution of the program, “S”, then the condition, “P”, will be true after execution of “S” “.

Where “V” is pre-condition and “P” is post-condition.

Notation: Such a correctness theorem is usually written as $\{V\} S \{P\}$, where V, S and P have been explained above.

By “program variable” we broadly include input and output data, e.g., data entered via a keyboard, displayed on a screen or printed on paper. Any externally observable aspect of the program’s execution may be covered by the precondition and post-condition.

5.4.4 Testing

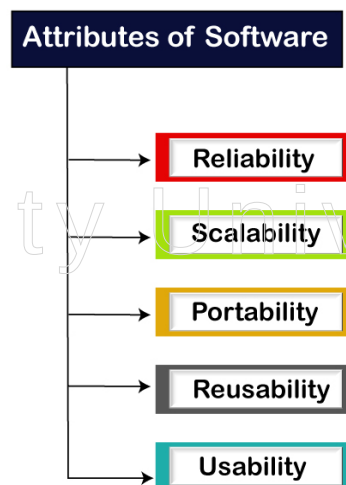
Software testing tutorial provides basic and advanced concepts of software testing. Our software testing tutorial is designed for beginners and professionals.

Software testing is widely used technology because it is compulsory to test each and every software before deployment.

Our Software testing tutorial includes all topics of Software testing such as Methods such as Black Box Testing, White Box Testing, Visual Box Testing and Gray Box Testing. Levels such as Unit Testing, Integration Testing, Regression Testing, Functional Testing. System Testing, Acceptance Testing, Alpha Testing, Beta Testing, Non-Functional testing, Security Testing, Portability Testing.

What is Software Testing

Software testing is a process of identifying the correctness of software by considering its all attributes (Reliability, Scalability, Portability, Re-usability, Usability) and evaluating the execution of software components to find the software bugs or errors or defects.



Software testing provides an independent view and objective of the software and gives surety of fitness of the software. It involves testing of all components under the required services to confirm that whether it is satisfying the specified requirements or not. The process is also providing the client with information about the quality of the software.

Testing is mandatory because it will be a dangerous situation if the software fails any of time due to lack of testing. So, without testing software cannot be deployed to the end user.

What is Testing

Testing is a group of techniques to determine the correctness of the application under the predefined script but, testing cannot find all the defect of application. The main intent of testing is to detect failures of the application so that failures can be discovered and corrected. It does not demonstrate that a product functions properly under all conditions but only that it is not working in some specific conditions.

Notes

Notes

Testing furnishes comparison that compares the behavior and state of software against mechanisms because the problem can be recognized by the mechanism. The mechanism may include past versions of the same specified product, comparable products, and interfaces of expected purpose, relevant standards, or other criteria but not limited up to these.

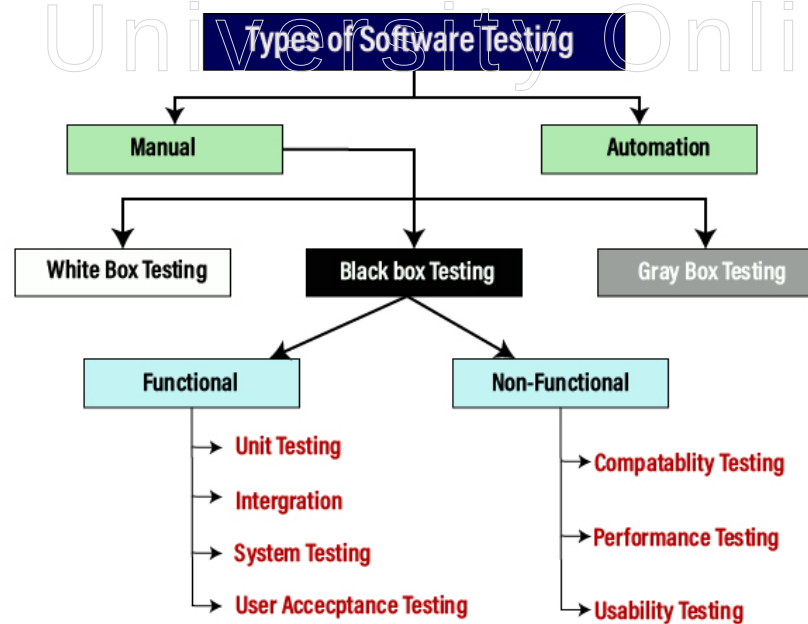
Testing includes an examination of code and also the execution of code in various environments, conditions as well as all the examining aspects of the code. In the current scenario of software development, a testing team may be separate from the development team so that Information derived from testing can be used to correct the process of software development.

The success of software depends upon acceptance of its targeted audience, easy graphical user interface, strong functionality load test, etc. For example, the audience of banking is totally different from the audience of a video game. Therefore, when an organization develops a software product, it can assess whether the software product will be beneficial to its purchasers and other audience.

Type of Software testing

We have various types of testing available in the market, which are used to test the application or the software.

With the help of below image, we can easily understand the type of software testing:



Manual testing

The process of checking the functionality of an application as per the customer needs without taking any help of automation tools is known as manual testing. While performing the manual testing on any application, we do not need any specific knowledge of any testing tool, rather than have a proper understanding of the product so we can easily prepare the test document.

Manual testing can be further divided into three types of testing, which are as follows:

- **White box testing**
- **Black box testing**
- **Gray box testing**

Automation testing

Automation testing is a process of converting any manual test cases into the test scripts with the help of automation tools, or any programming language is known as automation testing. With the help of automation testing, we can enhance the speed of our test execution because here, we do not require any human efforts. We need to write a test script and execute those scripts.

5.5.1 Functional, Structural and Error-Oriented Analysis and Testing

Functional

It is a type of software testing which is used to verify the functionality of the software application, whether the function is working according to the requirement specification. In functional testing, each function tested by giving the value, determining the output, and verifying the actual output with the expected value. Functional testing performed as black-box testing which is presented to confirm that the functionality of an application or system behaves as we are expecting. It is done to verify the functionality of the application.

Functional testing also called as black-box testing, because it focuses on application specification rather than actual code. Tester has to test only the program rather than the system.

Goal of functional testing

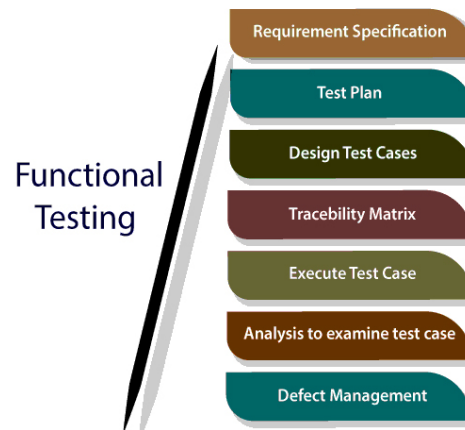
The purpose of the functional testing is to check the primary entry function, necessarily usable function, the flow of screen GUI. Functional testing displays the error message so that the user can easily navigate throughout the application.

What is the process of functional testing?

- Testers follow the following steps in the functional testing:
- Tester does verification of the requirement specification in the software application.
- After analysis, the requirement specification tester will make a plan.
- After planning the tests, the tester will design the test case.
- After designing the test, case tester will make a document of the traceability matrix.
- The tester will execute the test case design.
- Analysis of the coverage to examine the covered testing area of the application.

Notes

- Defect management should do to manage defect resolving.



What to test in functional testing? Explain

The main objective of functional testing is checking the functionality of the software system. It concentrates on:

- **Basic Usability:** Functional Testing involves the usability testing of the system. It checks whether a user can navigate freely without any difficulty through screens.
- **Accessibility:** Functional testing test the accessibility of the function.
- **Mainline function:** It focuses on testing the main feature.
- **Error Condition:** Functional testing is used to check the error condition. It checks whether the error message displayed.

Explain the complete process to perform functional testing.

There are the following steps to perform functional testing:

- There is a need to understand the software requirement.
- Identify test input data
- Compute the expected outcome with the selected input values.
- Execute test cases
- Comparison between the actual and the computed result

Identify test input(input data)

Compute the expected outcome with the selected input values

Execute test cases

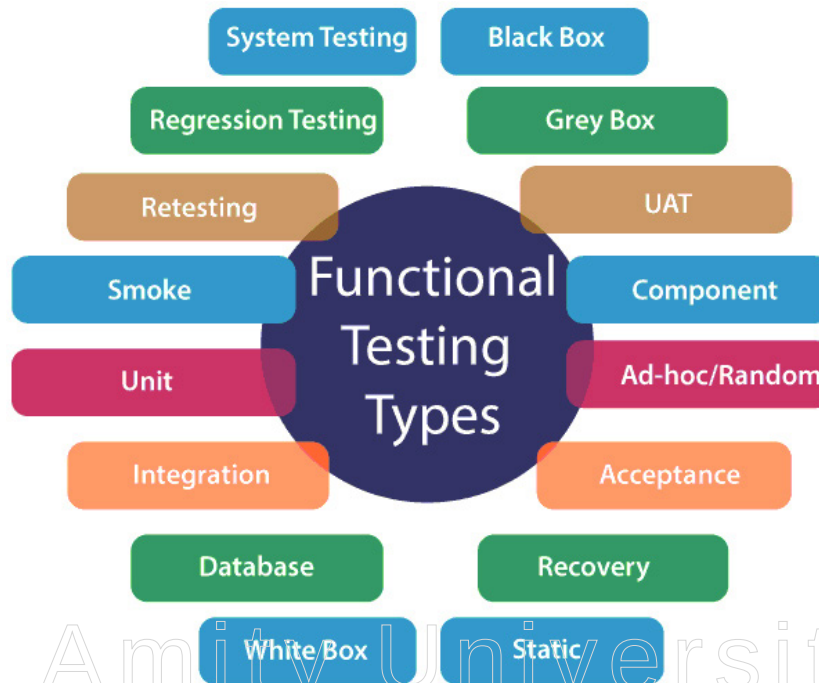
Comparison of actual and computed expected result

Explain the types of functional testing.

The main objective of functional testing is to test the functionality of the component.

Functional testing is divided into multiple parts.

Here are the following types of functional testing.



Unit Testing: Unit testing is a type of software testing, where the individual unit or component of the software tested. Unit testing, examine the different part of the application, by unit testing functional testing also done, because unit testing ensures each module is working correctly.

The developer does unit testing. Unit testing is done in the development phase of the application.

Smoke Testing: Functional testing by smoke testing. Smoke testing includes only the basic (feature) functionality of the system. Smoke testing is known as «**Build Verification Testing**» Smoke testing aims to ensure that the most important function work.

For example, Smoke testing verifies that the application launches successfully will check that GUI is responsive.

Sanity Testing: Sanity testing involves the entire high-level business scenario is working correctly. Sanity testing is done to check the functionality/bugs fixed. Sanity testing is little advance than smoke testing.

For example, login is working fine; all the buttons are working correctly; after clicking on the button navigation of the page is done or not.

Regression Testing: This type of testing concentrate to make sure that the code changes should not side effect the existing functionality of the system. Regression

Notes

(c) Amity University Online

Notes

testing specifies when bug arises in the system after fixing the bug, regression testing concentrate on that all parts are working or not. Regression testing focuses on is there any impact on the system.

Integration Testing: **Integration testing** combined individual units and tested as a group. The purpose of this testing is to expose the faults in the interaction between the integrated units.

Developers and testers perform integration testing.

White box testing: **White box testing** is known as Clear Box testing, code-based testing, structural testing, extensive testing, and glass box testing, transparent box testing. It is a software testing method in which the internal structure/design/implementation tested known to the tester.

The white box testing needs the analysis of the internal structure of the component or system.

Black box testing: It is also known as behavioral testing. In this testing, the internal structure/ design/ implementation not known to the tester. This type of testing is functional testing. Why we called this type of testing is black-box testing, in this testing tester, can't see the internal code.

For example, A tester without the knowledge of the internal structures of a website tests the web pages by using the web browser providing input and verifying the output against the expected outcome.

User acceptance testing: It is a type of testing performed by the client to certify the system according to requirement. The final phase of testing is user acceptance testing before releasing the software to the market or production environment. UAT is a kind of black-box testing where two or more end-users will involve.

Retesting: **Retesting** is a type of testing performed to check the test cases that were unsuccessful in the final execution are successfully pass after the defects fixed. Usually, tester assigns the bug when they find it while testing the product or its component. The bug allocated to a developer, and he fixes it. After fixing, the bug is assigned to a tester for its verification. This testing is known as retesting.

Database Testing: Database testing is a type of testing which checks the schema, tables, triggers, etc. of the database under test. Database testing may involve creating complex queries to load/stress test the database and check its responsiveness. It checks the data integrity and consistency.

Example: let us consider a banking application whereby a user makes a transaction. Now from database testing following, things are important. They are:

- Application store the transaction information in the application database and displays them correctly to the user.
- No information lost in this process
- The application does not keep partially performed or aborted operation information.
- The user information is not allowed individuals to access by the

Ad-hoc testing: Ad-hoc testing is an informal testing type whose aim is to break the system. This type of software testing is unplanned activity. It does not follow any

test design to create the test cases. Ad-hoc testing is done randomly on any part of the application; it does not support any structured way of testing.

Recovery Testing: Recovery testing is used to define how well an application can recover from crashes, hardware failure, and other problems. The purpose of recovery testing is to verify the system's ability to recover from testing points of failure.

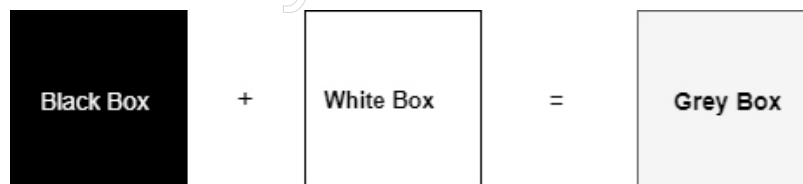
Static Testing: Static testing is a software testing technique by which we can check the defects in software without actually executing it. Static testing is done to avoid errors in the early stage of the development as it is easier to find failure in the early stages. Static testing used to detect the mistakes that may not found in dynamic testing.

Why we use static testing?

Static testing helps to find the error in the early stages. With the help of static testing, this will reduce the development timescales. It reduces the testing cost and time. Static testing also used for development productivity.

Component Testing: Component Testing is also a type of software testing in which testing is performed on each component separately without integrating with other parts. Component testing is also a type of black-box testing. Component testing also referred to as Unit testing, program testing, or module testing.

Grey Box Testing: Grey Box Testing defined as a combination of both white box and black-box testing. Grey Box testing is a testing technique which performed with limited information about the internal functionality of the system.



What are the functional testing tools?

The functional testing can also be executed by various apart from manual testing. These tools simplify the process of testing and help to get accurate and useful results.

It is one of the significant and top-priority based techniques which were decided and specified before the development process.

The tools used for functional testing are:

Tools	Features/ Characteristics
Sahi	- It is an open-source and automation testing tool, released under Apache License open source license, used for testing of the web application. - Sahi is written in Java and JavaScript and considered for most of the testing techniques. - It runs as a proxy server; it is browser-independent.

Notes

Notes

SoapUI	- It is an open-source functional testing tool, used for web application testing. - It is simple and easy to design. - It supports multiple environments, i.e., at any instance, the target environment may be set up.
Watir	- Watir, is an abbreviated form of web application testing in ruby, is an open-source tool for automating web browser. - It uses a ruby scripting language, which is concise and easy to use. - Watir supports multiple browsers on various platform.
Selenium	The open-source tool, used for functional testing on both web application and applications of the desktop.
	- It automates browsers and web application for testing purpose. - It gives the flexibility to customize the automated test case - Provides the advantage of writing test scripts, as per the requirements, using web driver.
Canoo WebTest	- An open-source tool for performing functional testing of the web application. - Platform independent - Easy and fast - Easy to extend to meet growing and incoming requirements.
Cucumber	Cucumber is an open-source testing tool written in Ruby language. This tool works best for test-driven development. It is used to test many other languages like java, c#, and python. Cucumber for testing using some programming.

What are the advantages of Functional Testing?

Advantages of functional testing are:

- It produces a defect-free product.
- It ensures that the customer is satisfied.
- It ensures that all requirements met.
- It ensures the proper working of all the functionality of an application/software/ product.
- It ensures that the software/ product work as expected.
- It ensures security and safety.
- It improves the quality of the product.

Example: Here, we are giving an example of banking software. In a bank when money transferred from bank A to bank B. And the bank B does not receive the correct amount, the fee is applied, or the money not converted into the correct currency, or incorrect transfer or bank A does not receive statement advice from bank B that the payment has received. These issues are critical and can be avoided by proper functional testing.

What are the disadvantages of functional testing?

Disadvantages of functional testing are:

- Functional testing can miss a critical and logical error in the system.
- This testing is not a guarantee of the software to go live.
- The possibility of conducting redundant testing is high in functional testing.

Structural and Error-Oriented Analysis and Testing

Introduction of Structural Testing

Another type of **software testing**

Technique is **Structural testing**, which is used to test the internal design of the software or structure of the coding for the particular software.

In this testing, the development team members are included in the testing team to execute the software's internal design. The working of structural testing is opposite to **Behavioural testing**.

In other words, we can say that structural testing tests the different features of an application based on its types.

Structural testing is also known as white-box testing, **glass box testing**, and **clear-box testing**. Developers mostly implement it to identify the issue and fix them quickly.

The structural testing process requires an in-depth knowledge of the programming language and is opposite to Functional Testing. The knowledge of the code's internal executions and how the software is implemented is a necessity for the test engineer to implement the structural testing.

Throughout the structural testing, the test engineer intends on how the software performs, and it can be used at all levels of testing.

For example, the coverage of menu options or significant business transactions could be the system's structural element or acceptance testing.

Types of Structural Testing

Structural testing is divided into four different categories, which are as follows:

- **Mutation testing**
- **Data flow testing**
- **Control flow testing**
- **Slice-based testing**

Mutation testing

- It is used to check the quality of the test case that should fail the mutant code.
- Mutation testing involves the development of new tests to be implemented on the software for its testing process.
- When we identify various errors, it implies that either the program is correct or the test case is inefficient in locating the fault.

Notes

- In the mutation testing, the developers make small modifications to the previously accessible software tests and generate a mutant of the old software test.
- It used to cause an error in the program, which implies that the mutation testing is performed to evaluate the test case's productivity.

Data flow testing

- It is a group of testing approaches used to observe the control flow of programs to discover the sequence of variables as per the series of events.
- It implements a control flow graph and analysis the points where the codes can change the data.
- If we execute the data flow testing technique, the information is kept safe and unchanged during the code's implementation.

Control flow testing

- The **control flow testing** is the basic model of **Structural testing**.
- It is to check the implementation order of commands or statements of the code over a control structure.
- In the control flow testing, a specific part of an extensive program is selected by the test engineer to set the testing path.
- Generally, the control flow testing technique is used in unit testing.
- In this testing, the entire test is based on how the control is executed during the code.
- The complete information of all the software's features and logic is necessary to execute the control flow testing.

Slice-based testing

- It was initially created and established to keep the software.
- The basic idea is to sort the complete code into small chunks and then evaluate each portion carefully.
- The slice-based testing is very beneficial for the maintenance of the software along with fixing the application too.

Structural Testing Tools

Like other testing has their tools, structural testing also contains some open-source and commercial tools that have their functionality.

Some of the most commonly used tools for structural testing are as follows:

- **Cucumber**
- **JBehave**
- **Cfix**
- **JUnit**

Notes

Let understand them one by one in details:

Cucumber

It is a widely used tool for behaviour-driven Development as it delivers an easily understandable testing script for **system acceptance** and automation testing.

It is a software tool used by the test engineer to develop test cases for the testing of the software's behaviour.

JBehave

It is a framework for BDD (**Behaviour-Driven Development**). The BDD is a development of **TDD (test-driven Development)** and acceptance-test-driven design. It is planned to create these practices more accessible and spontaneous to beginners and professionals alike.

Features of JBehave

Some of the commonly used features of JBehave are as follows:

- It is purely executing in Java programming language, which plays well with Java-based enterprises.
- In this, we can write the user stories in **JBehave syntax/ Gherkin syntax**.
- The user stories can be implemented as **JUnit**
- It provides the Annotation-based configuration and steps class specifications.
- It allows stories to be executed through Ant task.

Cfix

Another structural testing tool is **Cfix**, an XUnit testing framework supported by the C/C++ programming language.

In this tool, the unit tests are compiled and linked into a DLL. It has been designed to work well in combination with **Windows Debuggers** such as **Visual Studio** and **WinDBG**.

Characteristic of Cfix

Following are the commonly used features of Cfix:

- It supports development of both user and kernel mode unit tests.
- The failing test case can be highly customized in case of implementation.

JUnit

JUnit

JUnit is one of the essential tools of structural testing. It is an open-source unit testing framework, which was written in Java language.

It will help us to enhance the developer's efficiency, which provides the consistency of the development code and reduces the time of the debugging.

Notes

Advantages and Disadvantages of Structural Testing

Below are the benefits and drawback of structural testing:

Advantages of Structural Testing

The benefits of Structural testing are as follows:

- Structural testing does not require a lot of manual work as it is an automated process.
- Structural testing is not a time-consuming process.
- All the early defects can easily be identified
- It removes the dead code (extra code) or statements easily.
- It provides easy coding and implementation.
- It delivers detailed testing of the software.

Disadvantages of Structural Testing

The **drawback** of the structural testing are as follows:

- To perform structural testing, in-depth knowledge of programming languages is required.
- Even though structural testing is automatic, it might turn out very difficult because it involves training in the tool used for testing.
- It is expensive in respect of money because sometimes resources are necessary to efficiently perform structural testing.
- There is also a small chance that some commands, statements or branches could be missed unintentionally.

Overview

In this tutorial, we have understood structural testing, the type of structural testing, advantages and disadvantages. After learning all the specific topics, we can easily conclude that Structural testing, which is also called **White-box testing, glass box testing, and clear box testing** is used to verify the structure of code. And apart from that, we can say that while executing the different types of software testing, there is no guarantee of 100% efficiency of the product. Therefore, it is always helpful if we associate different categories of testing and methods.

The various structural testing types, which we have looked at, such as data flow testing, mutation testing, slice-based testing, and control flow testing, could be copied back to errors such as:

- Mutation testing (using the wrong operator).
- Data flow testing (referencing a variable before using it).

In case someone is looking to use the structural testing methods, they need to consider both the benefits and drawbacks of the structural testing.

Furthermore, they need to take care of the fact that structural testing is implemented successfully.

Notes**Error:**

Error handling testing is a type of software testing that is performed to check whether the system is capable of or able to handle the errors that may happen in future. This type of testing is basically performed with the help of both developers and the testers. Error handling testing not only focuses on the determination of error but also focuses on the exception handling.

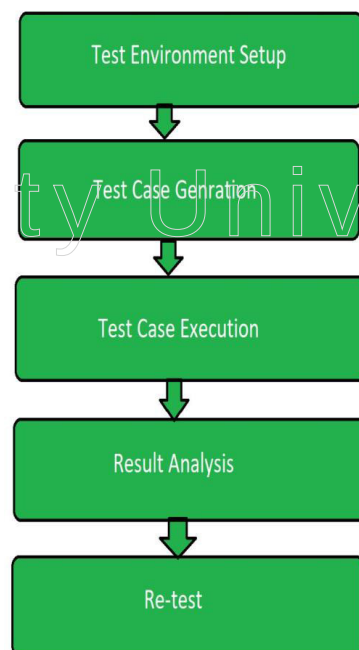
Objective of Error Handling Testing:

The objective of error handling testing is:

- To check the system ability to handle errors.
- To check the system highest soak point.
- To make sure errors can be handles properly by the system in the future.
- To make system capable of exception handling also.

Steps involved in the Error Handling testing:

Following are the steps involved in the error handling testing:

**1. Test Environment Set Up:**

Test environment is set according to the software testing technique so that the testing process can run smoothly. This step includes planning for the testing. System which is going to be tested is made sure have less significant data as there might be crash problem in the system during testing.

2. Test Case Generation:

In this software testing test case generation is nothing but making different test cases which may cause error. Suppose a software operates on fractions then setting the denominator of the fractions as zero. Test case generation is associated with the developing team as without knowing the internal code, test cases can't be designed.

Notes

3. Test Case Execution:

After the test case generation, real testing process begins. This is the most prominent part of the testing process. It includes the running the program over the test case generated.

4. Result and Analysis:

After the execution of the test case, its result is analyzed. It includes the checking of the inconsistency in the expected output for the generated test case. There might be a chance of the program going into an infinite loop which may lead up to software failure.

5. Re-test:

If the testing is failed then after the analysis once more all the above steps are performed to test the system. It also includes the testing of the system under new test cases generated recently.

Advantages of Error handling testing:

- It helps in construction of an error handling powered software.
- It makes the software ready for all circumstances.
- It develops the exception handling technique in the software.
- It helps in maintenance of the software.

Disadvantages of the Error handling testing

- It is costly as both the developing and testing team is involved.
- It takes lot of time to perform the testing operations.

5.5.2 Static Testing Tools

What is Static Testing?

Static Testing is a type of software testing in which software application is tested without code execution. Manual or automated reviews of code, requirement documents and document design are done in order to find the errors. The main objective of static testing is to improve the quality of software applications by finding errors in early stages of software development process.

Static testing involves manual or automated reviews of the documents. This review is done during an initial phase of testing to catch Defect early in STLC. It examines work documents and provides review comments. It is also called Non-execution testing or verification testing.

Examples of Work documents-

- Requirement specifications
- Design document
- Source Code
- Test Plans

- Test Cases
- Test Scripts
- Help or User document
- Web Page content

Static Testing Techniques:

- **Informal Reviews:** This is one of the type of review which doesn't follow any process to find errors in the document. Under this technique, you just review the document and give informal comments on it.
- **Technical Reviews:** A team consisting of your peers, review the technical specification of the software product and checks whether it is suitable for the project. They try to find any discrepancies in the specifications and standards followed. This review concentrates mainly on the technical documentation related to the software such as Test Strategy, Test Plan and requirement specification documents.
- **Walkthrough:** The author of the work product explains the product to his team. Participants can ask questions if any. A meeting is led by the author. Scribe makes note of review comments
- **Inspection:** The main purpose is to find defects and meeting is led by a trained moderator. This review is a formal type of review where it follows a strict process to find the defects. Reviewers have a checklist to review the work products. They record the defect and inform the participants to rectify those errors.
- **Static code Review:** This is a systematic review of the software source code without executing the code. It checks the syntax of the code, coding standards, code optimization, etc. This is also termed as white box testing. This review can be done at any point during development.

5.5.3 Dynamic Testing Tools**What is Dynamic Testing?**

Under **Dynamic Testing**, a code is executed. It checks for functional behavior of software system, memory/cpu usage and overall performance of the system. Hence the name "Dynamic"

The main objective of this testing is to confirm that the software product works in conformance with the business requirements. This testing is also called an Execution technique or validation testing. Dynamic testing executes the software and validates the output with the expected outcome. Dynamic testing is performed at all levels of testing and it can be either black or white box testing.

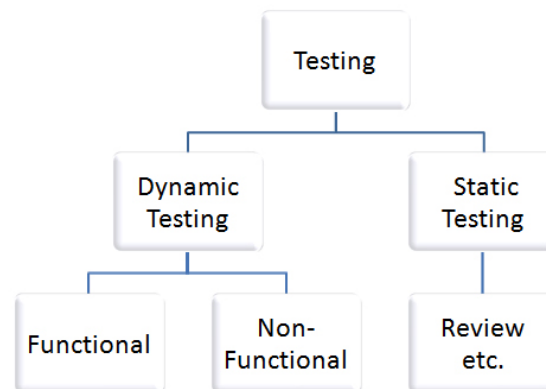
Key Difference

- Static testing was done without executing the program whereas Dynamic testing is done by executing the program.
- Static testing checks the code, requirement documents, and design documents to find errors whereas Dynamic testing checks the functional behavior of software

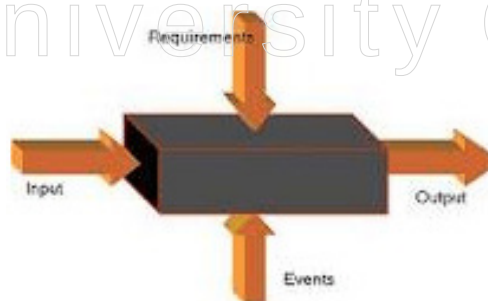
Notes

system, memory/CPU usage and overall performance of the system.

- Static testing is about the prevention of defects whereas Dynamic testing is about finding and fixing the defects.
- Static testing does the verification process while Dynamic testing does the validation process.
- Static testing is performed before compilation whereas Dynamic testing is performed after compilation.
- Static testing techniques are structural and statement coverage while Dynamic testing techniques are Boundary Value Analysis & Equivalence Partitioning.



Dynamic Testing Techniques:



- **Unit Testing:** Under Unit Testing, individual units or modules are tested by the developers. It involves testing of source code by developers.
- **Integration Testing:** Individual modules are grouped together and tested by the developers. The purpose is to determine what modules are working as expected once they are integrated.
- **System Testing:** System Testing is performed on the whole system by checking whether the system or application meets the requirement specification document. Also, Non-functional testing like performance, Security Testing fall under the category of dynamic testing.

Difference between Static and Dynamic Testing:

Notes



Static Testing	Dynamic Testing
Testing was done without executing the program	Testing is done by executing the program
This testing does the verification process	Dynamic testing does the validation process
Static testing is about prevention of defects	Dynamic testing is about finding and fixing the defects
Static testing gives an assessment of code and documentation	Dynamic testing gives bugs/bottlenecks in the software system.
Static testing involves a checklist and process to be followed	Dynamic testing involves test cases for execution
This testing can be performed before compilation	Dynamic testing is performed after compilation
Static testing covers the structural and statement coverage testing	Dynamic testing techniques are Boundary Value Analysis & Equivalence Partitioning.
Cost of finding defects and fixing is less	Cost of finding and fixing defects is high
Return on investment will be high as this process involved at an early stage	Return on investment will be low as this process involves after the development phase
More reviews comments are highly recommended for good quality	More defects are highly recommended for good quality.
Requires loads of meetings	Comparatively requires lesser meetings

5.5.4 Characteristics of Modern Testing Tools

Following are characteristics of modern testing tools

- It should use one or more testing strategy for performing testing on host as well as on target platform.
- It should support GUI based test preparation.
- It should provide complete code coverage and create test documentation in various formats (HTML/DOC/RTF...).
- These tools should able to adopt the underlying hardware.
- It should be easy to use. 6. Finally it should provide a clear report on test

Notes

case, steps, test case status (PASS/FAIL).

- They have an improved speed.
- They are more efficient and accurate.
- They are easy to use.
- They can be automated.

Check your Understanding

1. Which one is not a software quality model?
 - a) ISO 9000
 - b) McCall model
 - c) Boehm model
 - d) ISO 9126
2. How many levels are present in CMM?
 - a) three
 - b) four
 - c) five
 - d) six
3. Which level of CMM is for process management?
 - a) Initial
 - b) Repeatable
 - c) Defined
 - d) Optimizing
4. In ISO 9126, time behavior and resource utilization are a part of
 - a) maintainability
 - b) portability
 - c) efficiency
 - d) usability
5. Which of the following is not a Probabilistic Model?
 - a) Error seeding
 - b) NHPP
 - c) Input domain
 - d) Halstead's software metric
6. Software reliability is defined with respect to
 - a) time
 - b) bugs
 - c) failures

Notes

- d) quality
7. Failure In Time (FIT) is another way of reporting
- a) MTTR
 - b) MTTF
 - c) MTSF
 - d) MTBF
8. MTTF stands for
- a) Minimum time to failure
 - b) Mean time to failure
 - c) Maximum time to failure
 - d) None of the mentioned
9. Mean Time To Repair (MTTR) is the time needed to repair a failed hardware module.
- a) True
 - b) False
10. IMC Networks is a leading _____ certified manufacturer of optical networking and LAN/WAN connectivity solutions for enterprise, telecommunications and service provider applications.
- a) Telco Systems
 - b) D-Link
 - c) Arista Networks
 - d) ISO 9001

Summary

- SQA is an umbrella activity which is employed throughout the software lifecycle.
- Software quality assurance is very important for your software product or service to succeed in the market and survive up to the customer's expectations.
- There are various activities, standards, and techniques that you need to follow to assure that the deliverable software is of high quality and aligns closely with the business needs.
- SQ: Software quality is defined as **a field of study and practice that describes the desirable attributes of software products**. There are two main approaches to software quality: defect management and quality attributes.
- SQA: Software quality assurance (SQA) is **a means and practice of monitoring the software engineering processes and methods used in a project to ensure proper quality of the software**. It may include ensuring conformance to standards or models, such as ISO/IEC 9126 (now superseded by ISO 25010), SPICE or CMMI.
- Quality control plans are often viewed as a set of instructions that should be followed. They **document the planning, implementation, and assessment**

Notes

procedures for a project, as well as any QA or QC activities. Some areas may be more detailed than others, based on the project, process, or organization's needs.

- **Quality planning** is the task of determining what factors are important to a project and figuring out how to meet those factors. Such factors often include the resources that will be used, the steps needed to complete the project and any other specifications. So for Molly, this means she needs to plan what resources the project will need, determine the cost of those resources, plan a timeline for completing the project, outline the steps she and her team will take, and she will need to assign the tasks and responsibilities to each person.
- 820.3(a) Verification means confirmation by examination and provision of objective evidence that specified requirements have been fulfilled.
- “Documented procedures, performed in the user environment, for obtaining, recording, and interpreting the results required to establish that predetermined specifications have been met” (AAMI).
- 820.3(z) Validation means confirmation by examination and provision of objective evidence that the particular requirements for a specific intended use can be consistently fulfilled.
- Process Validation means establishing by objective evidence that a process consistently produces a result or product meeting its predetermined specifications.
- Design Validation means establishing by objective evidence that device specifications conform with user needs and intended use(s).
- “Documented procedure for obtaining, recording, and interpreting the results required to establish that a process will consistently yield product complying with predetermined specifications” (AAMI).

Activity

1. “Software Reliability Allocation Models” this is a vital aspect of software testing. Comment.

Questions & Exercises

1. What is Software Reliability Allocation Models?
2. Explain Criteria for Model Evaluation.
3. Describe Optimal Reliability Allocation.
4. What is Quality Planning and Control?
5. Discuss Quality Improvement Process.
6. Explain Evolution of Software Quality Assurance (SQA) also describe Major SQA Activities and Major SQA Issues
7. What is Verification and Validation?
8. Explain Evolutionary Nature of Verification and Validation
9. Discuss the Impracticality of Testing all Data and Paths
10. Explain Proof of Correctness, Software Testing, Functional,

Notes

11. Describe the Structural and Error-Oriented Analysis & Testing,
12. Describe the Static and Dynamic Testing Tools also explain Characteristics of Modern Testing Tools

Glossary

- **SQ: Software quality** is defined as a field of study and practice that describes the desirable attributes of software products. There are two main approaches to software quality: defect management and quality attributes.
- **SQA: Software quality assurance (SQA)** is a means and practice of monitoring the software engineering processes and methods used in a project to ensure proper quality of the software. It may include ensuring conformance to standards or models, such as ISO/IEC 9126 (now superseded by ISO 25010), SPICE or CMMI.
- **Quality control** plans are often viewed as a set of instructions that should be followed. They document the planning, implementation, and assessment procedures for a project, as well as any QA or QC activities. Some areas may be more detailed than others, based on the project, process, or organization's needs.
- **Quality planning** is the task of determining what factors are important to a project and figuring out how to meet those factors. Such factors often include the resources that will be used, the steps needed to complete the project and any other specifications. So for Molly, this means she needs to plan what resources the project will need, determine the cost of those resources, plan a timeline for completing the project, outline the steps she and her team will take, and she will need to assign the tasks and responsibilities to each person.
- **Verification:** is the process of ensuring that the software product accomplishes its aim without including any errors. It is the process of determining whether or not the product that has been developed is correct. It determines whether or not the generated product meets the client's specifications.
- **Validation:** is the process of determining whether or not a software product is up to par, or in other words, whether or not the product meets high-level specifications.
- **Debugging:** The process in which developers identify, diagnose, and fix errors found. See also bug and defect.
- **Decision table:** A test design and requirements specification technique. A decision table describes the logical conditions and rules for a system. Testers use the table as the basis for creating test cases.
- **Defect:** A flaw in a component or system that can cause the component or system to fail to perform its required function. A defect, if encountered during execution, may cause a failure of the component or system.

Further Readings

1. Software Engineering, Sixth Edition, 2001, Ian Sommerville; Pearson Education.
2. Software Engineering – A Practitioner's Approach, Roger S. Pressman;

Notes

McGraw-Hill International Edition.

3. An Integrated approach to Software Engineering, Pankaj Jalote; Narcosis Publishing House.
4. Software Quality, 1997, Mordechai Ben-Menachem and Garry S.Marliss; Thomson Learning.
5. Emden, D. and Blakeman, K. (1990). Going Automated. London: ASLIB.
6. Gates, H. (1985). Directory of Library and Information Retrieval Software for Microcomputers. Brook-field: Gower Pub. Co.
7. Levert, V.M. (1985). Library Specific Microcomputer Software. Library Software Review. 4 (May-June), 3, 5-38.
8. Nolan, J.M. (1984). Micro Software Evaluation. Torrance CA: Nolan Information Services. Poole, B.J. (1995). Education for an Information Age. Wisconsin: WCB Brown & Benchmark.
9. Rowley, Jennifer (1998). The Electronic Library, 4th Edition of Computers for Libraries. London: Library Association Publishing.
10. Tenopir, Carol (1984), Identification and Evaluation of Software for Micro-computerbased in-house Databases. Information Technology, and Libraries. March , 21-34.

Check your Understanding – Answers

- 1) a
- 2) c
- 3) d
- 4) c
- 5) d
- 6) d
- 7) a
- 8) b
- 9) a
- 10) a